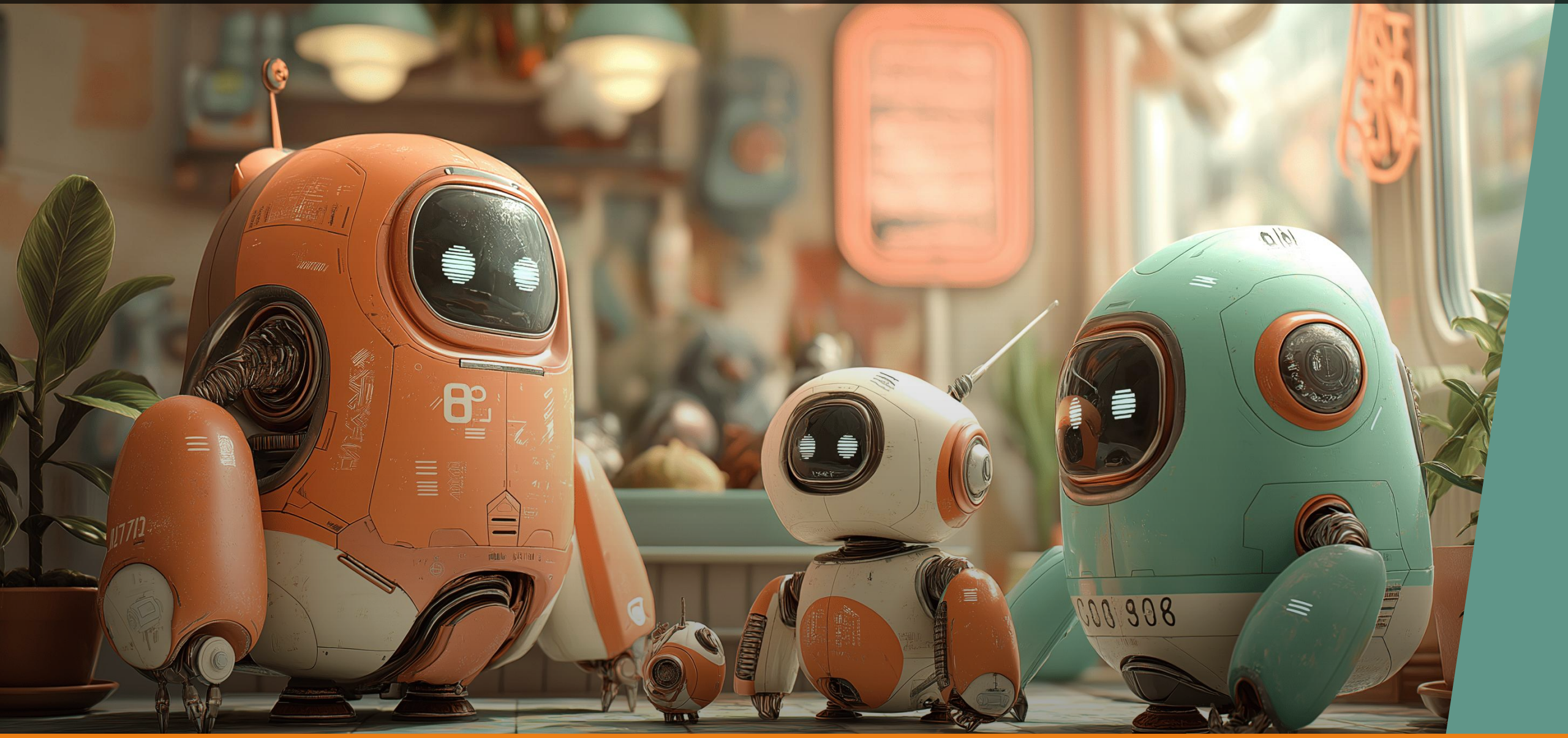
DEVOXX FRANCE 2026

G1, ZGC, SHENANDOAH…
Avec tous ces GCs dans Java, je choisis lequel ?

DEVOXX FRANCE 2026

G1, ZGC, SHENANDOAH…
Avec tous ces GCs dans Java, je choisis lequel ?

GCS DANS JAVA EN 2026



GCS DANS JAVA EN 2026

G1

ZGC

Serial

Parallel

Shenandoah

GCS DANS JAVA EN 2026

G1

ZGC

Serial

Parallel

Shenandoah

Comprendre les GCs

Comprendre les GCs

(Garbage Collectors)

Comprendre les GCs

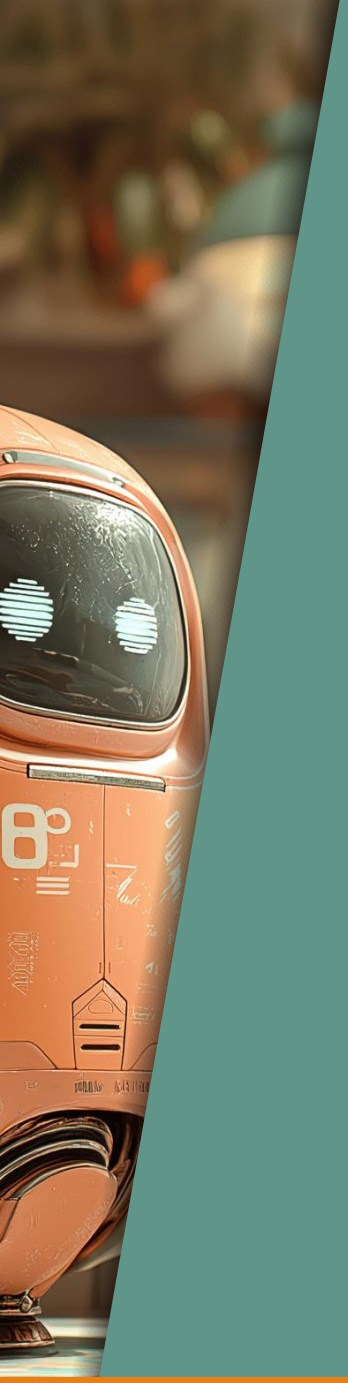
(Garbage Collectors)

(en Java)

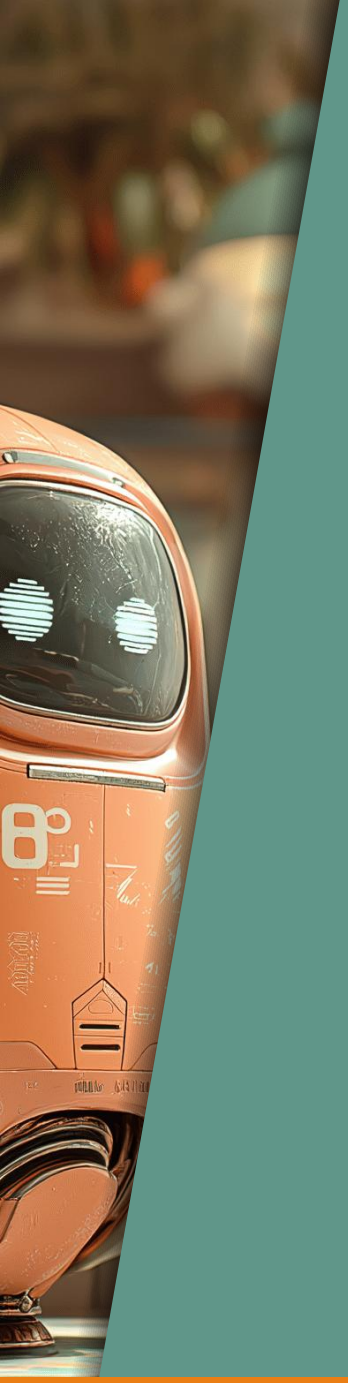
Antoine DESSAIGNE

axway

74Software

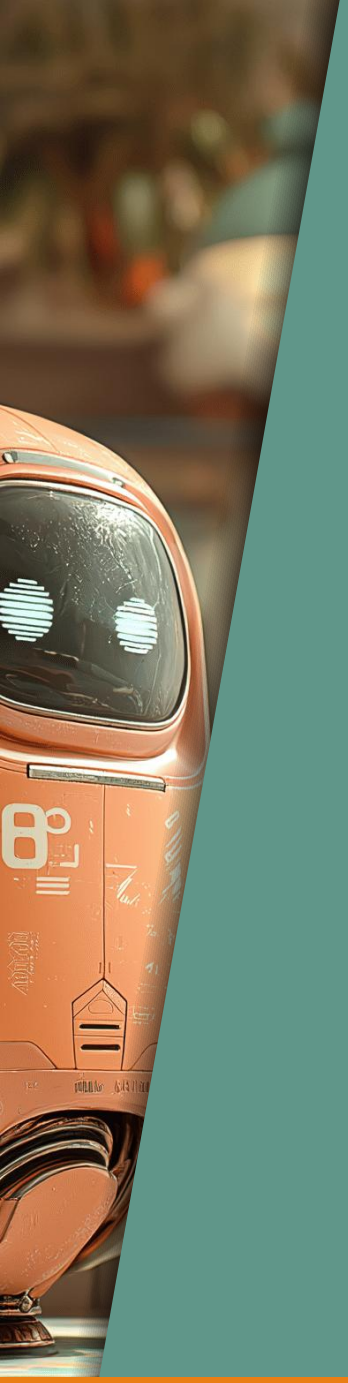


A quoi sert un GC ?



A quoi sert un GC ?

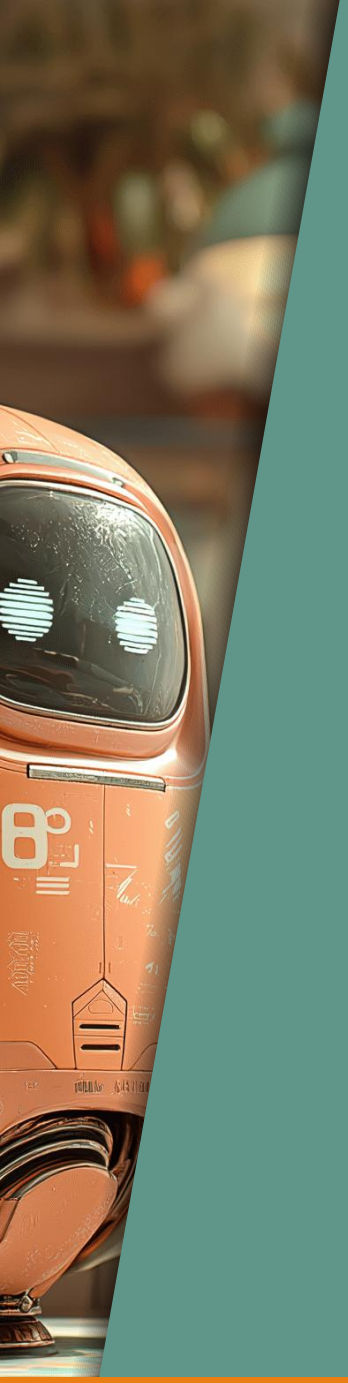
Libérer
la mémoire



A quoi sert un GC ?

Allouer
la mémoire

Libérer
la mémoire



A quoi sert un GC ?

Allouer
des objets

Libérer
des objets

La mémoire ? Les objets ?

```
var conf = new Conference("Devoxx", 2026);
```

La mémoire ? Les objets ?

```
var conf = new Conference("Devoxx", 2026);
```

conf

La mémoire ? Les objets ?

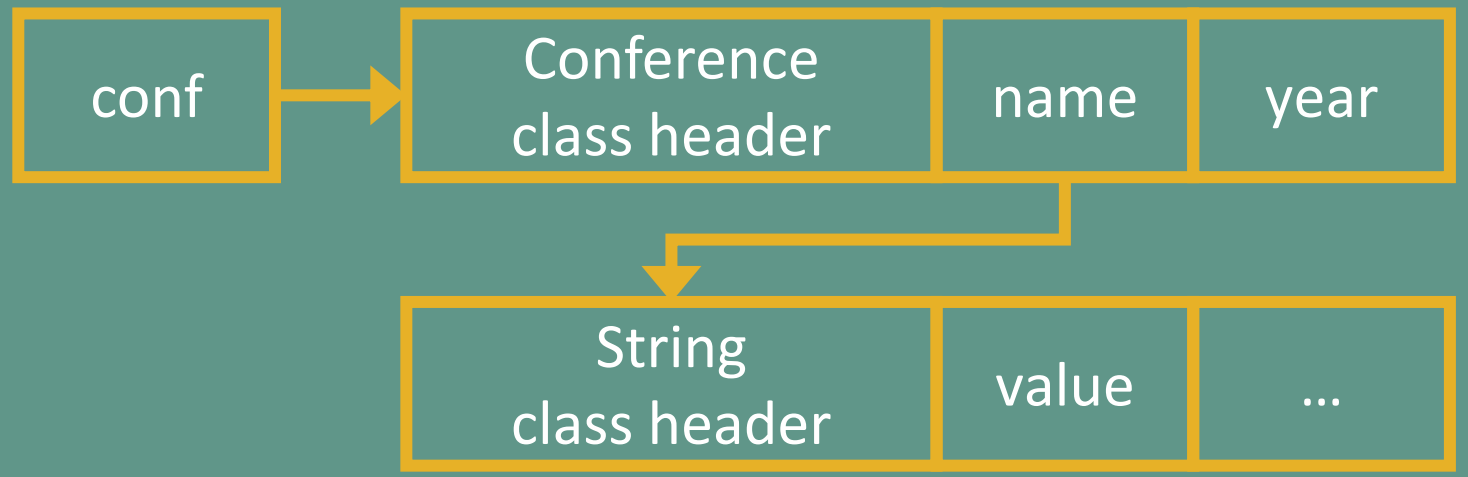
```
var conf = new Conference("Devoxx", 2026);
```

La mémoire ? Les objets ?

```
var conf = new Conference("Devoxx", 2026);
```

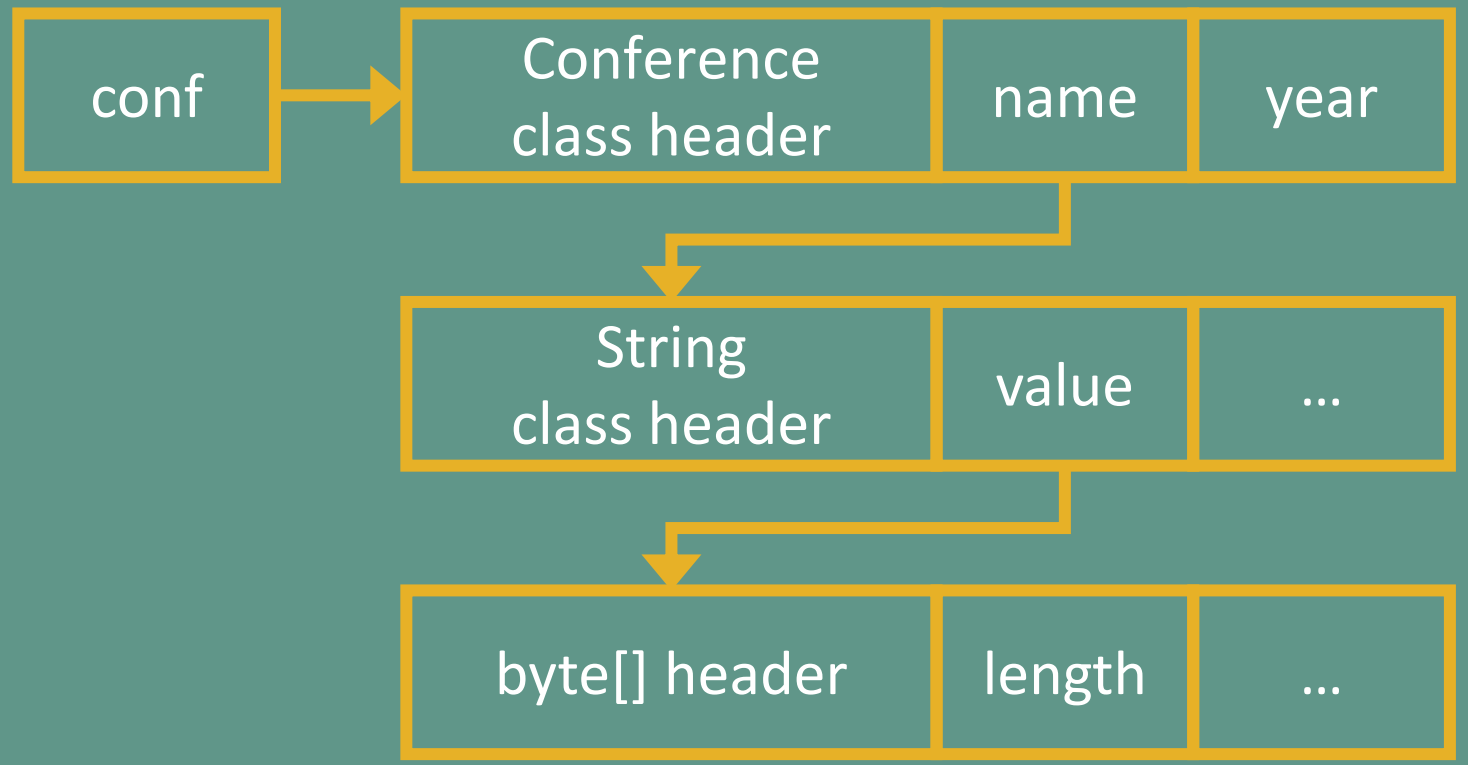
La mémoire ? Les objets ?

```
var conf = new Conference("Devoxx", 2026);
```



La mémoire ? Les objets ?

```
var conf = new Conference("Devoxx", 2026);
```



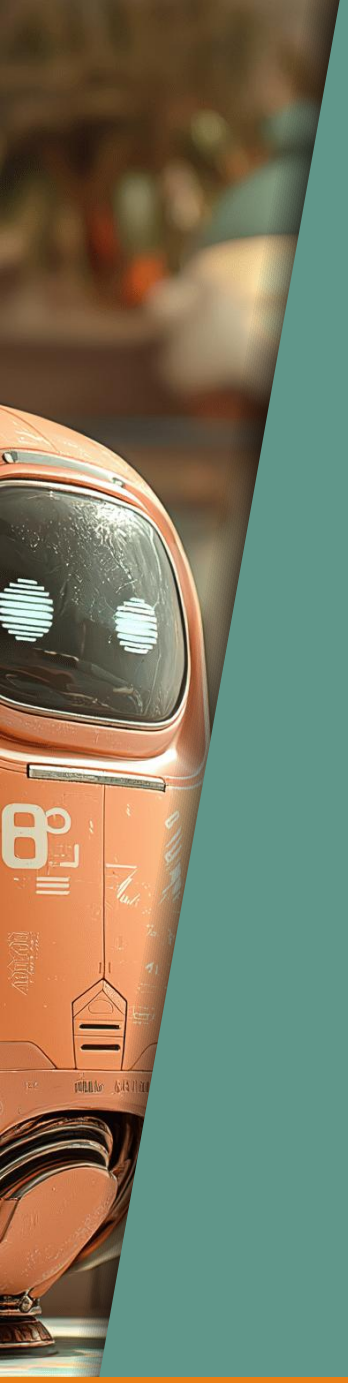
La mémoire dans Java

Mémoire

La mémoire dans Java

-Xms8G -Xmx16G

Mémoire

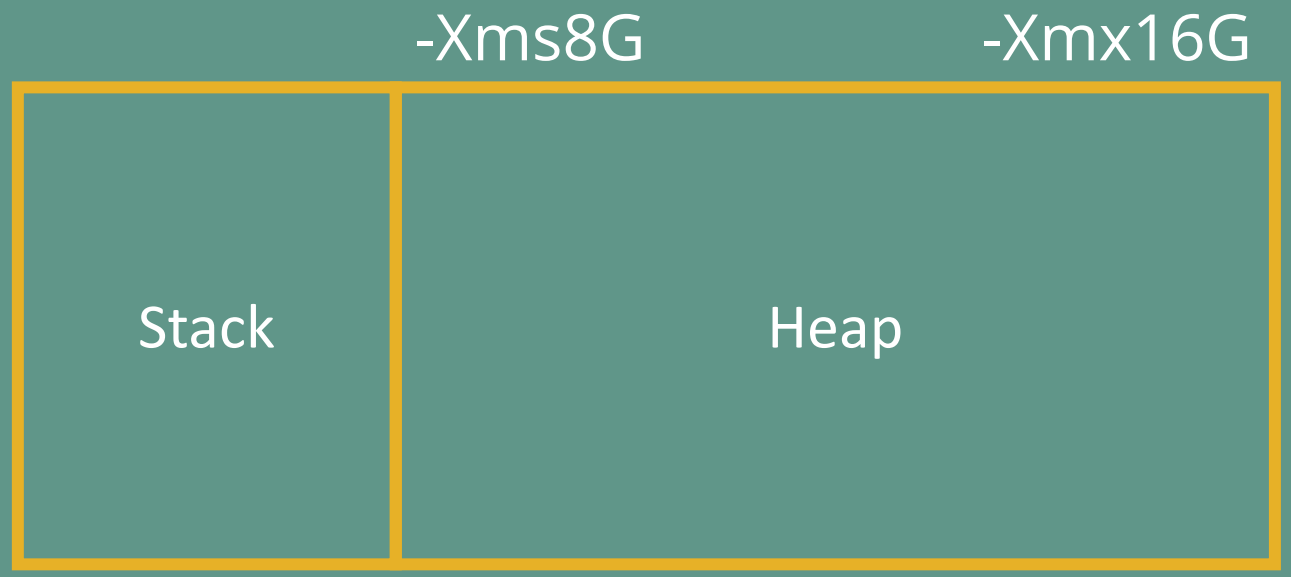


La mémoire dans Java

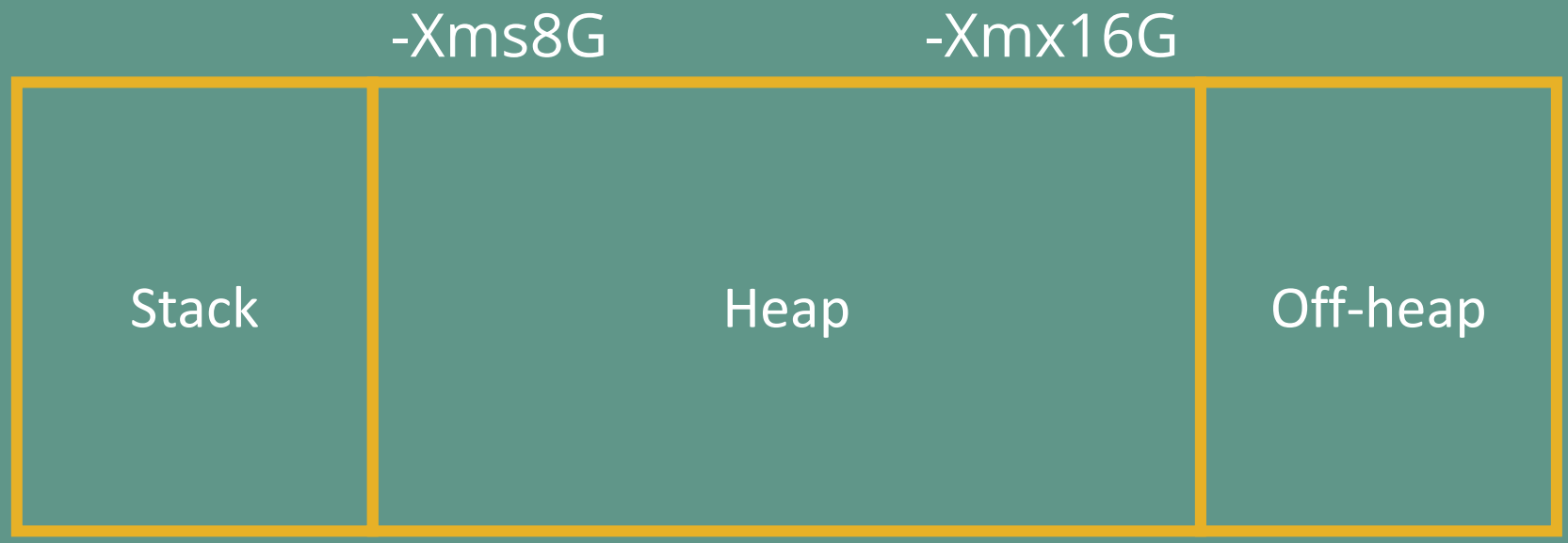
-Xms8G -Xmx16G

Heap

La mémoire dans Java



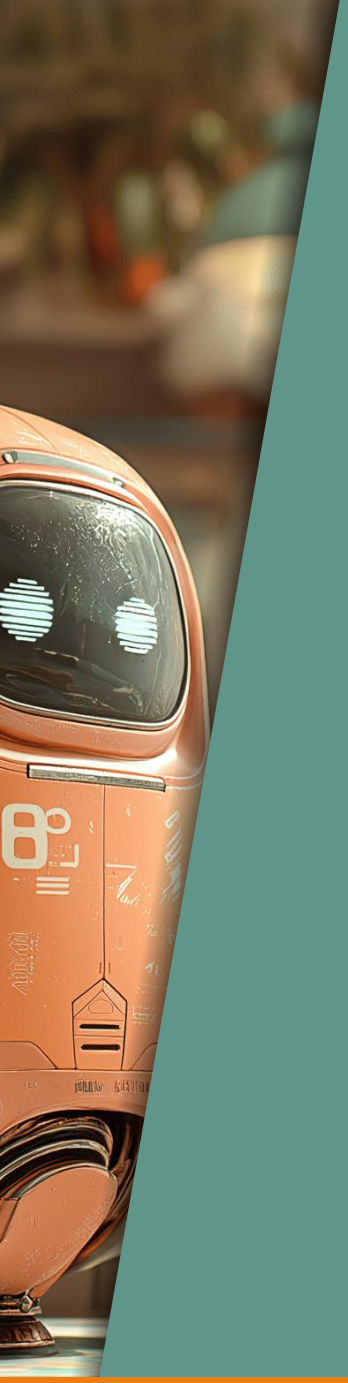
La mémoire dans Java



Au commencement…

Au commencement...

Java 1



- Processeur :100 à 166 Mhz

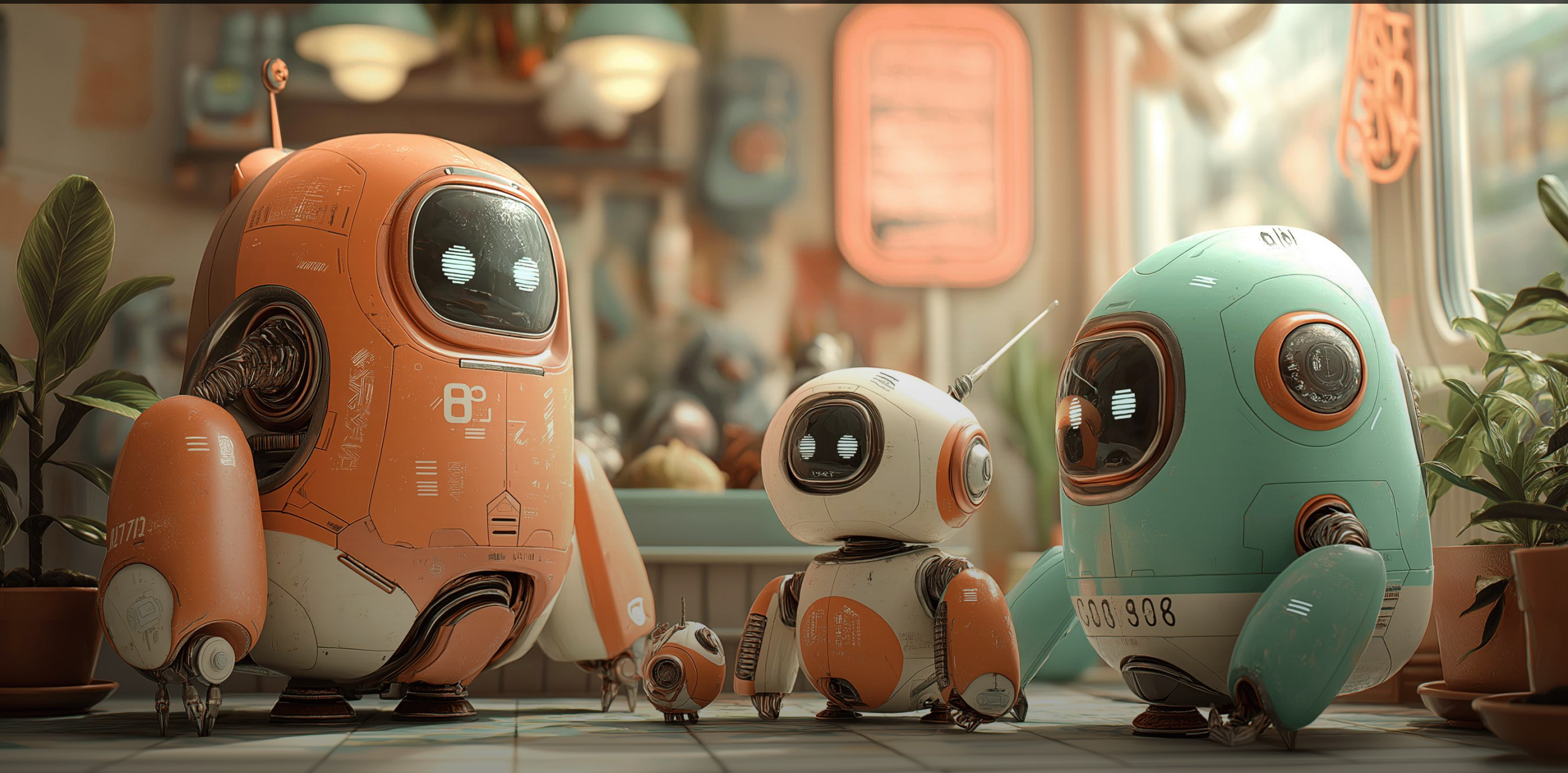
- Processeur :100 à 166 Mhz
- Mémoire : 8 à 16 Mo

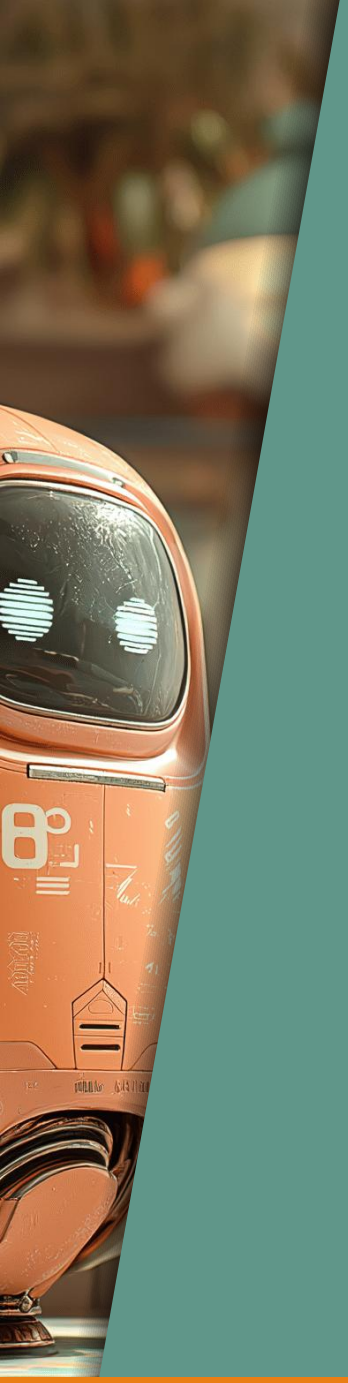
- Processeur :100 à 166 Mhz
- Mémoire : 8 à 16 Mo
- Stockage : 500 à 1200 Mo

Top qualité

- Processeur :100 à 166 Mhz
- Mémoire : 8 à 16 Mo
- Stockage : 500 à 1200 Mo

BREF...





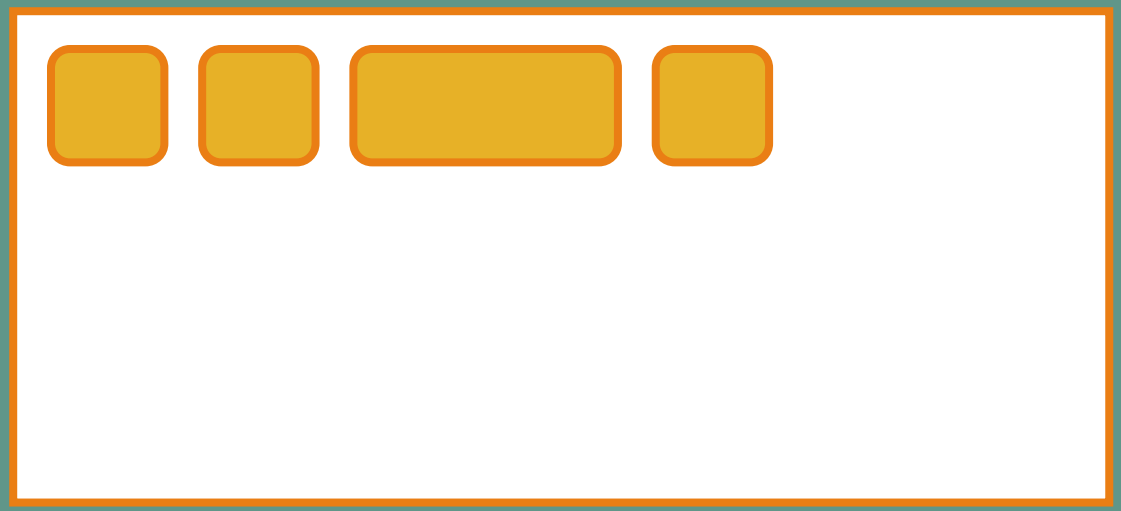
Java 1

Java 1

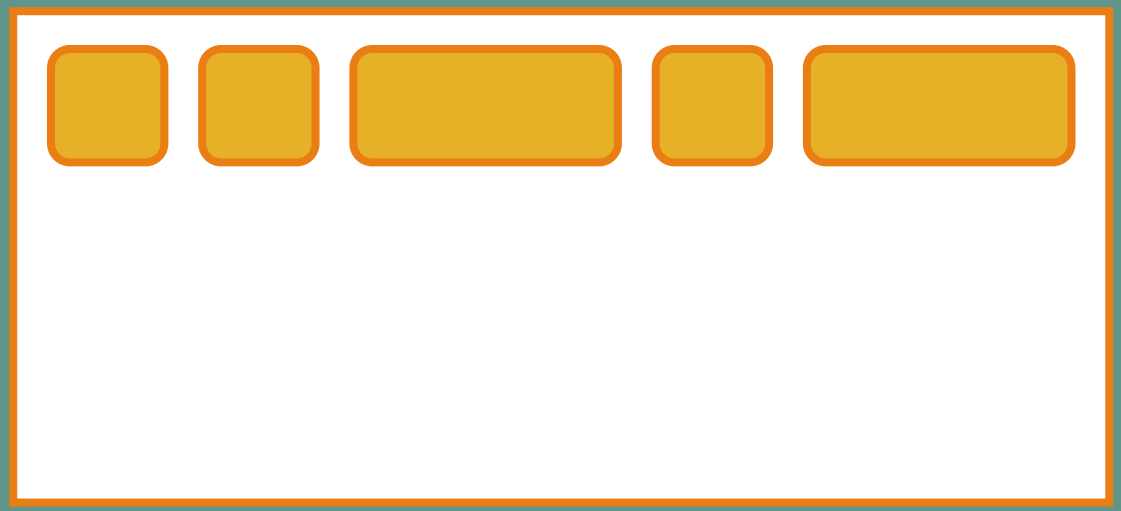
Java 1

Java 1

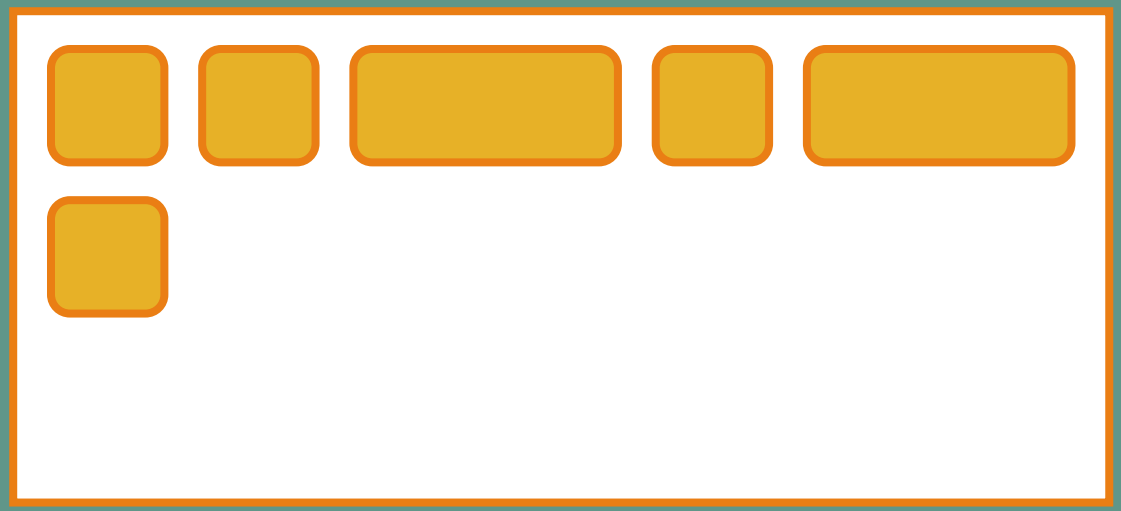
Java 1



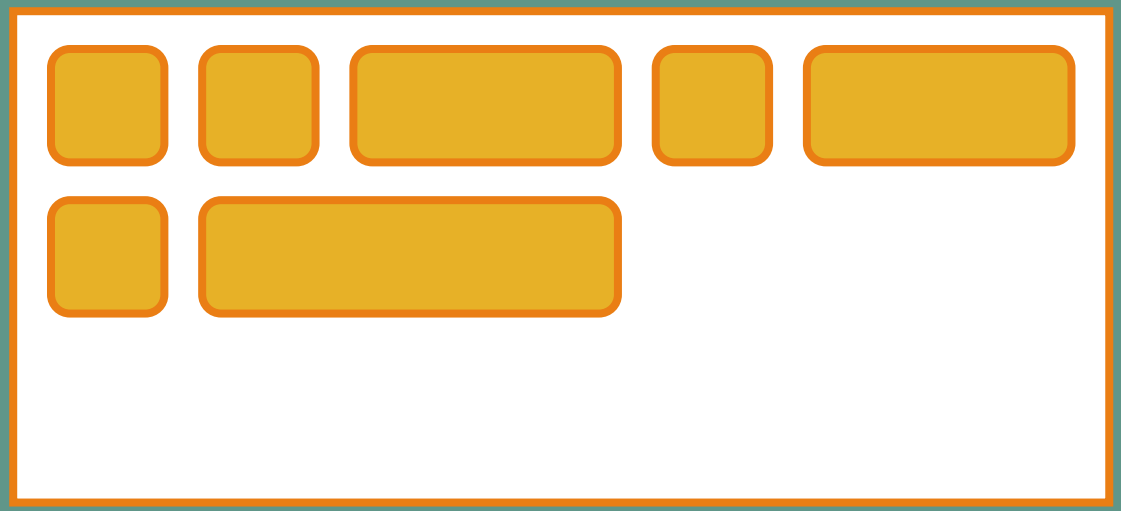
Java 1



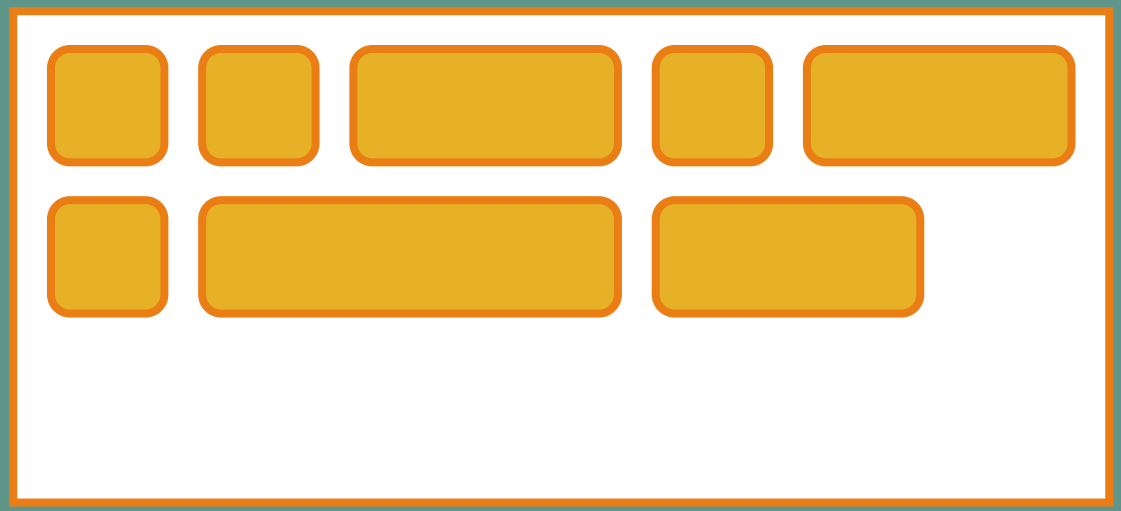
Java 1



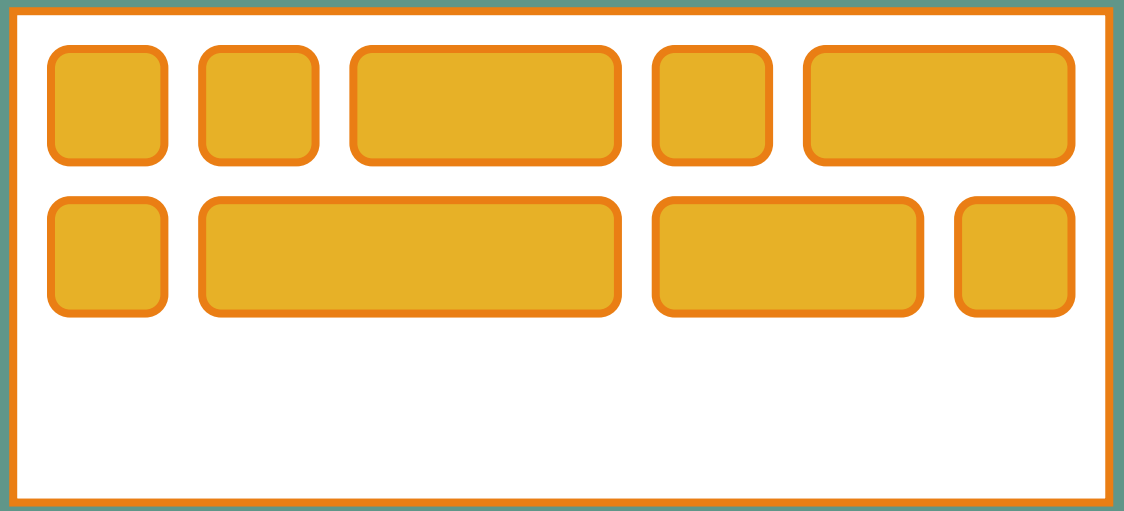
Java 1



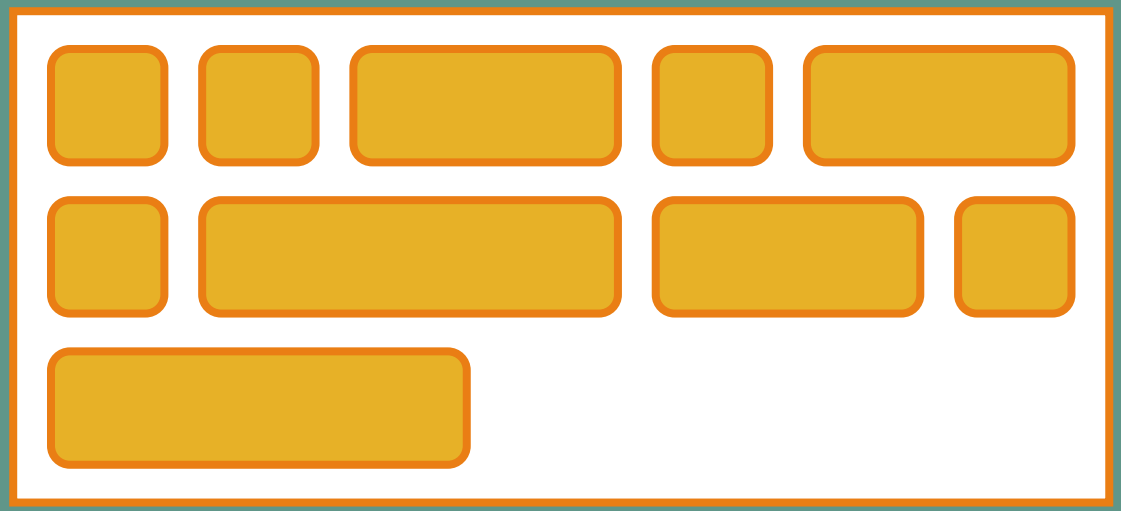
Java 1

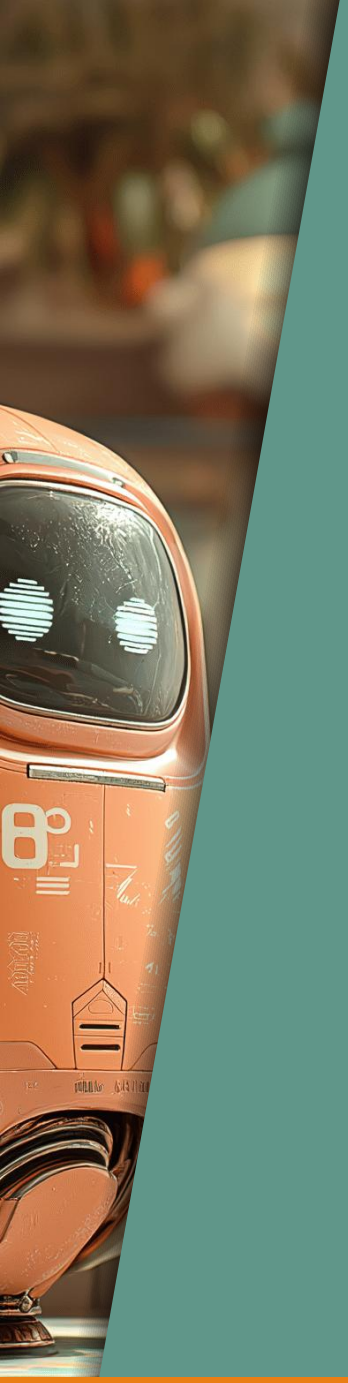


Java 1

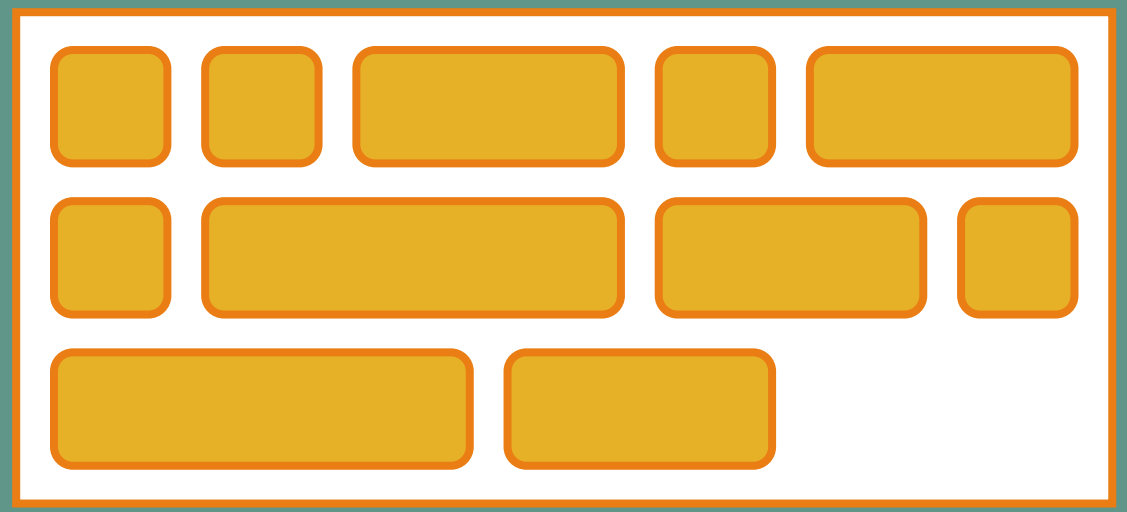


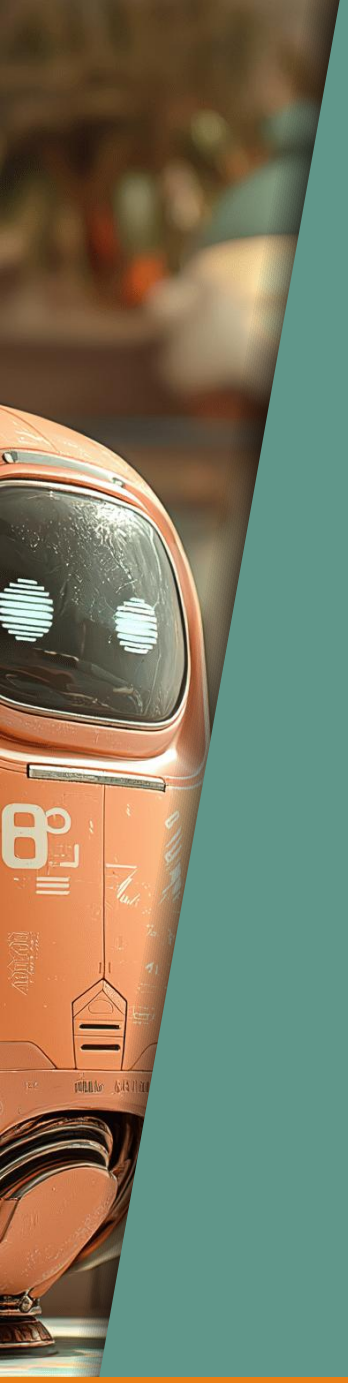
Java 1



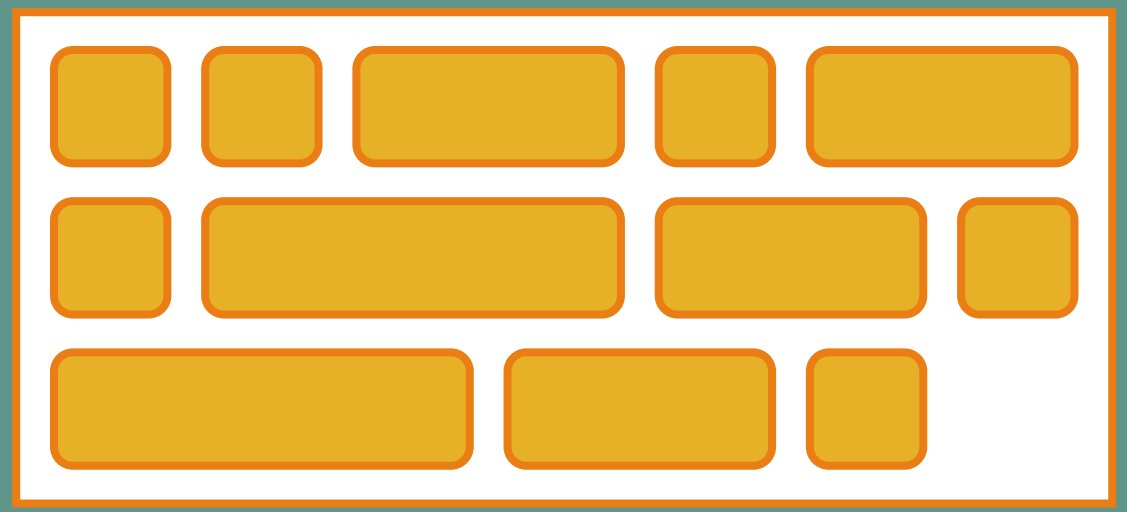


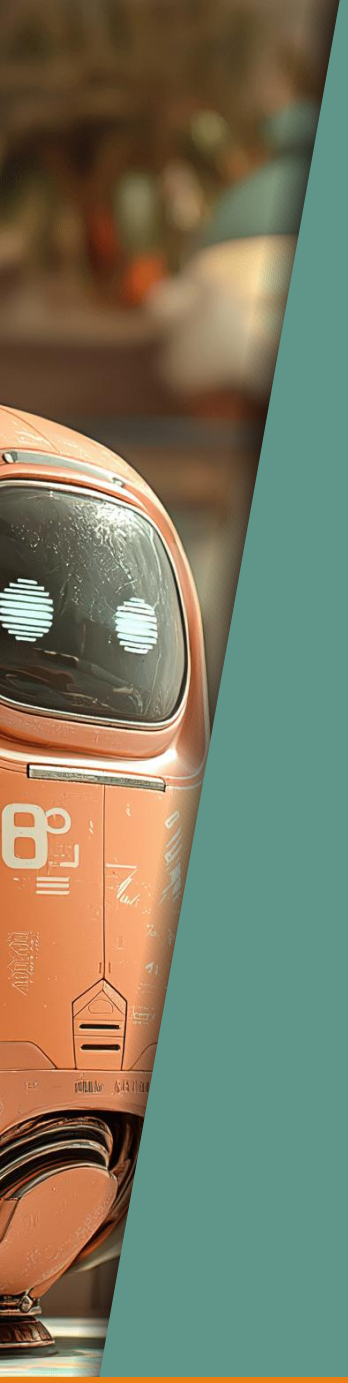
Java 1



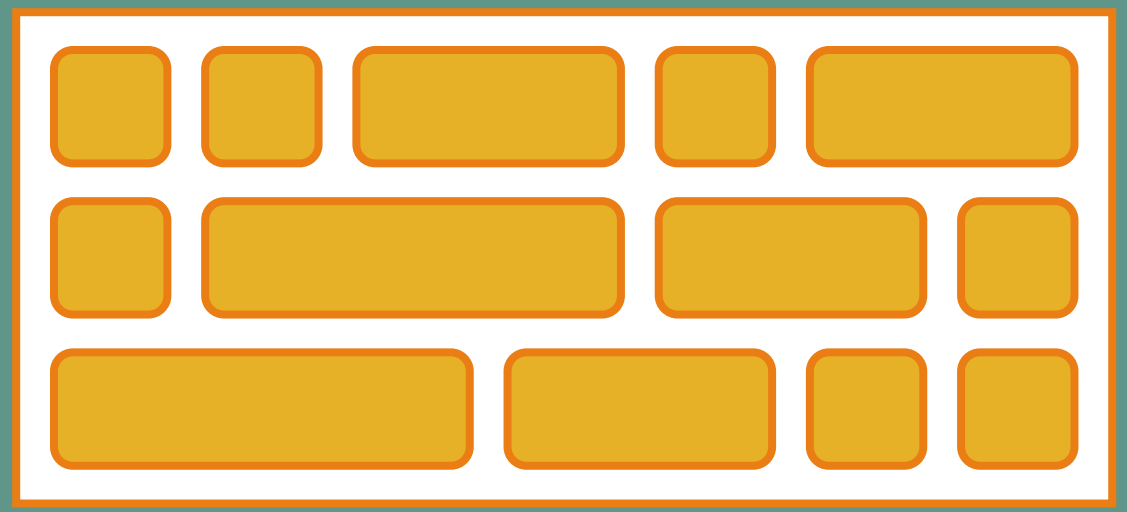


Java 1

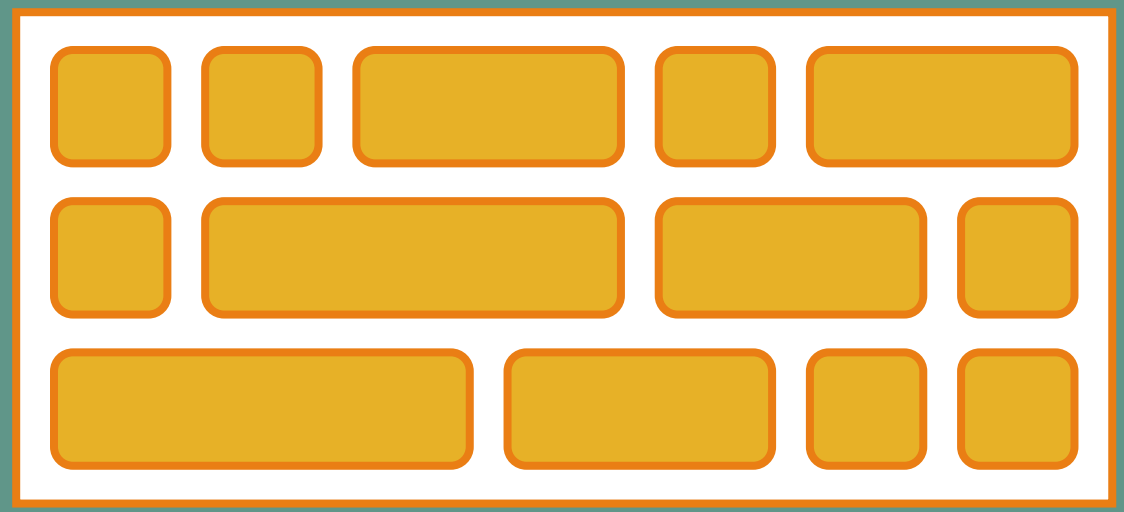


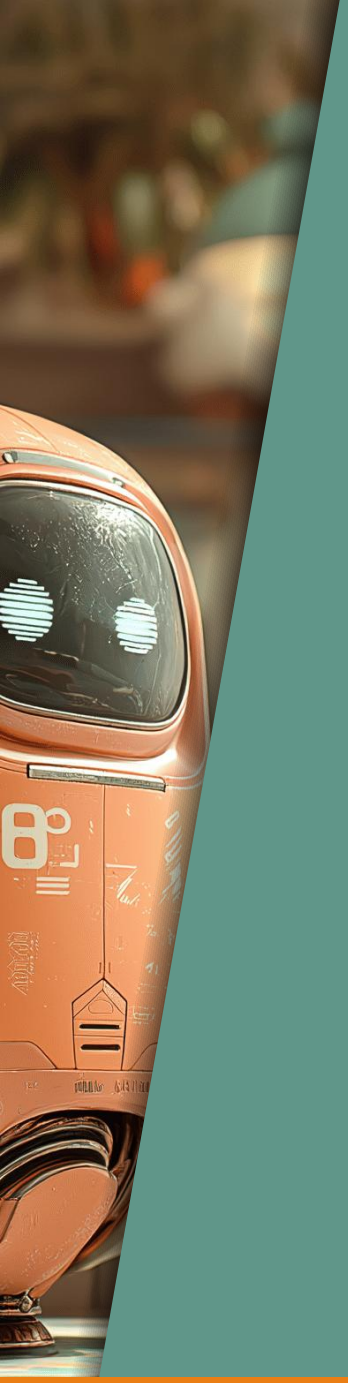


Java 1

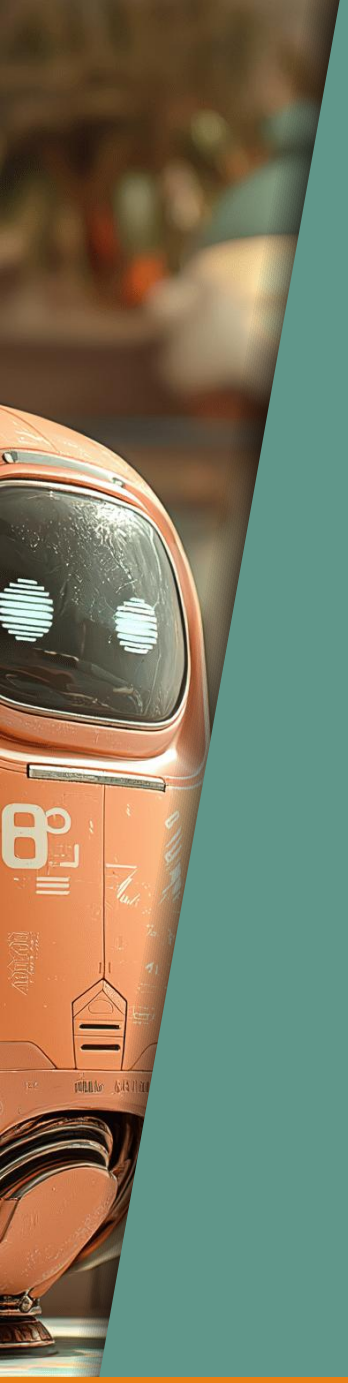


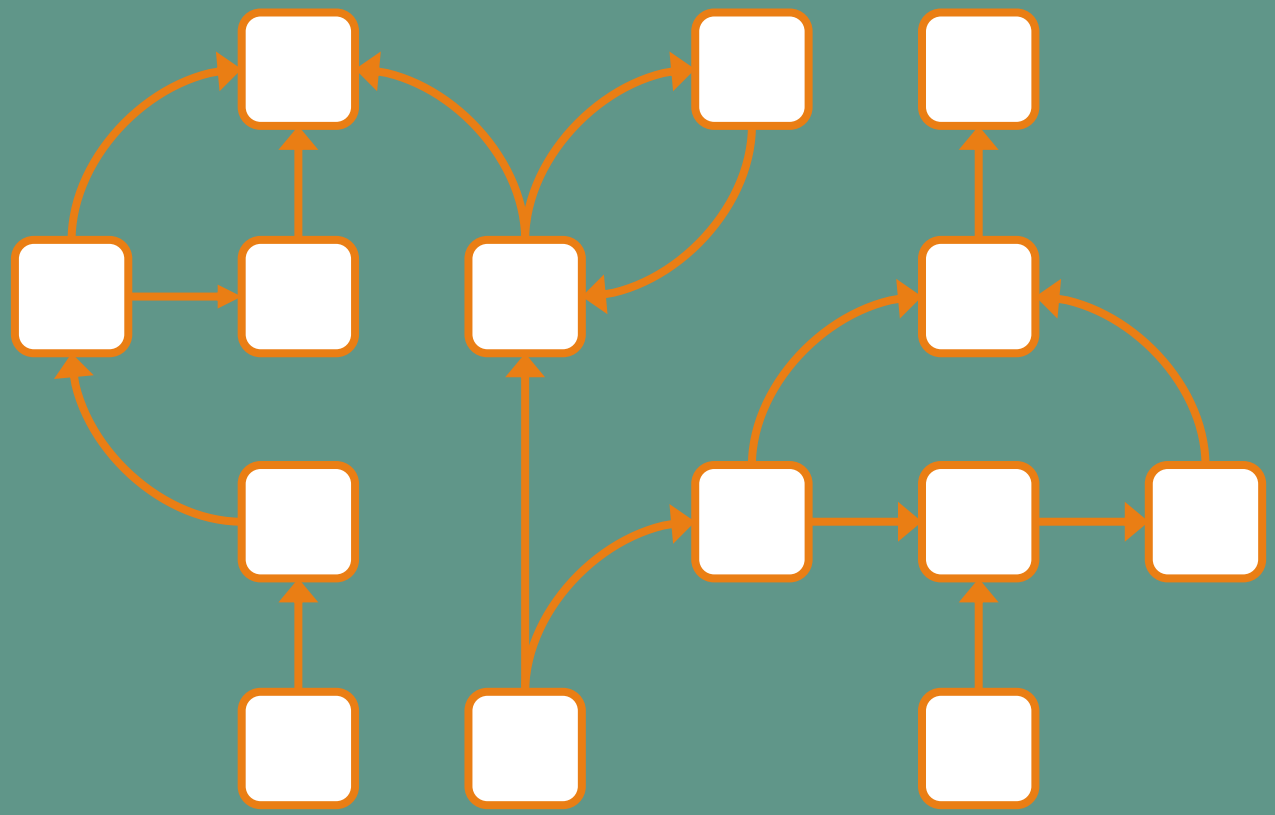
Java 1

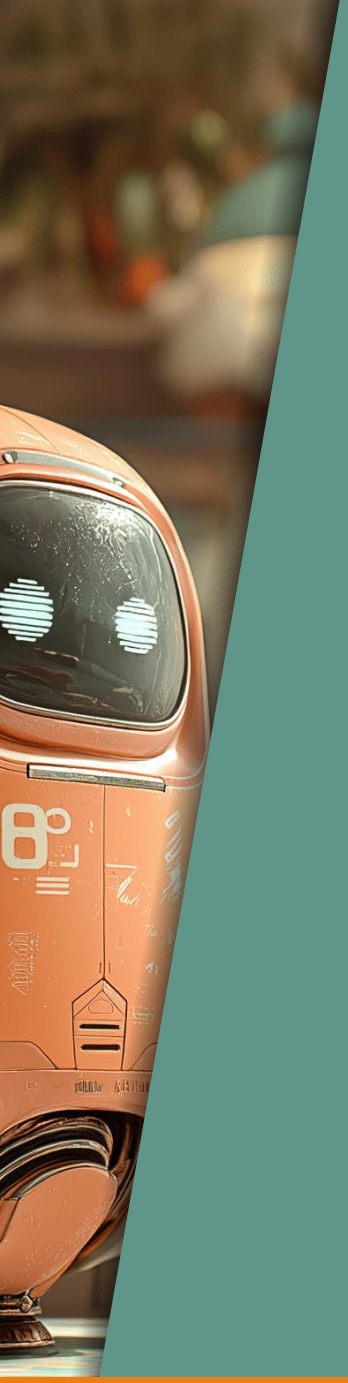


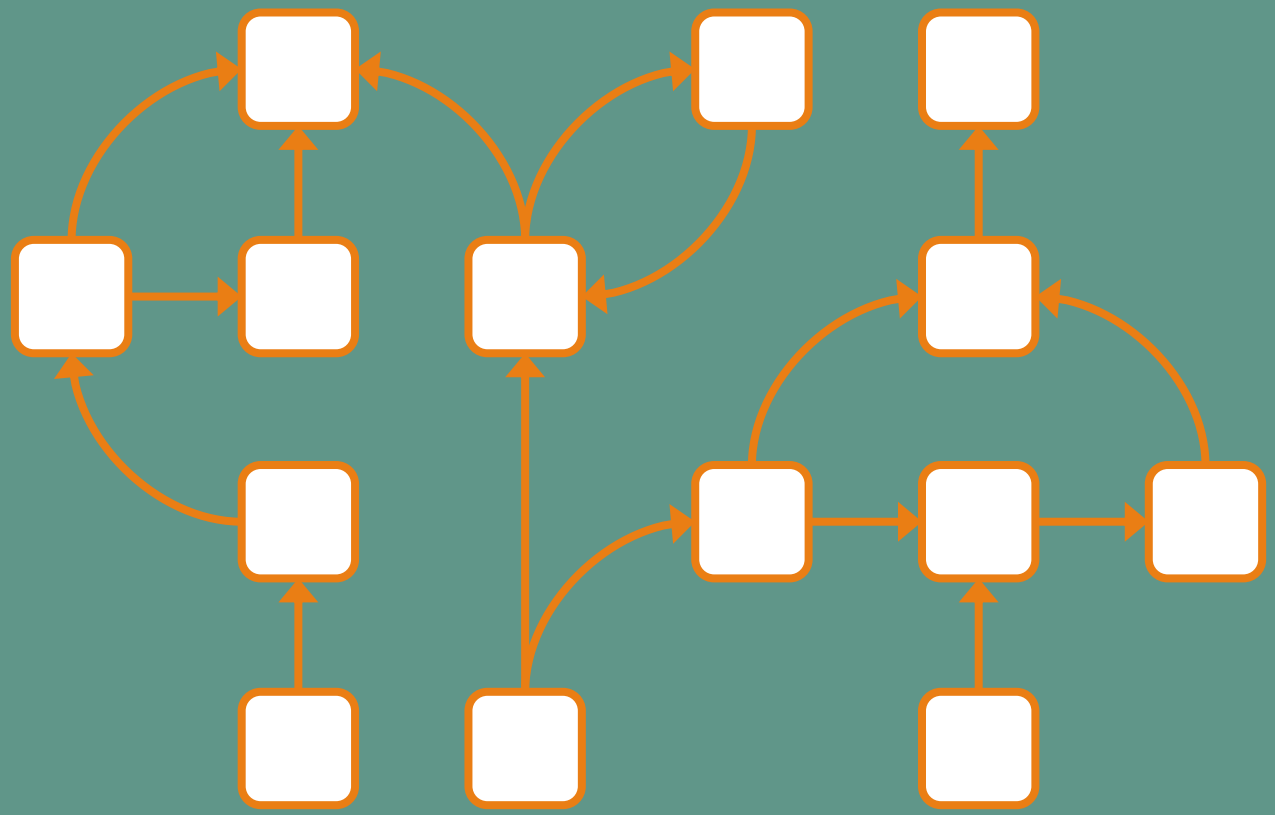


Comment on sait si un objet est toujours utilisé ou non ?

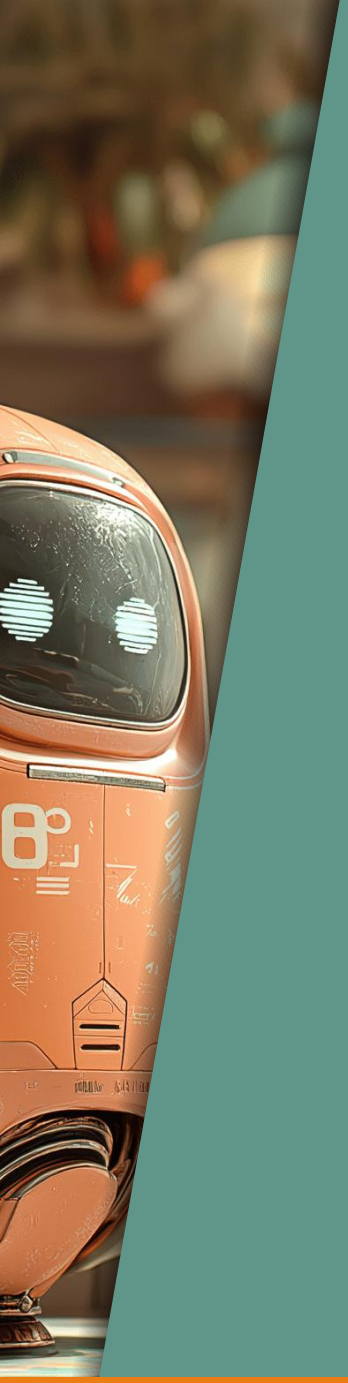


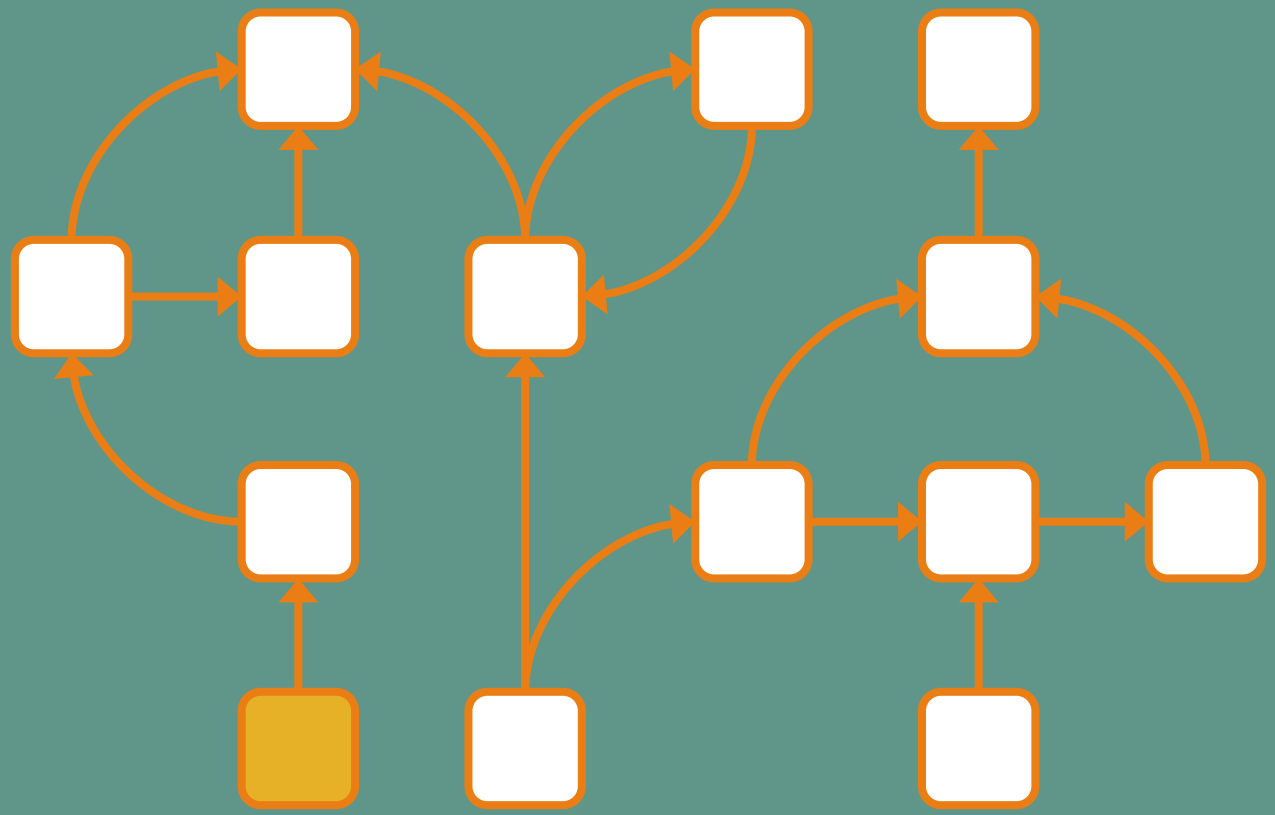




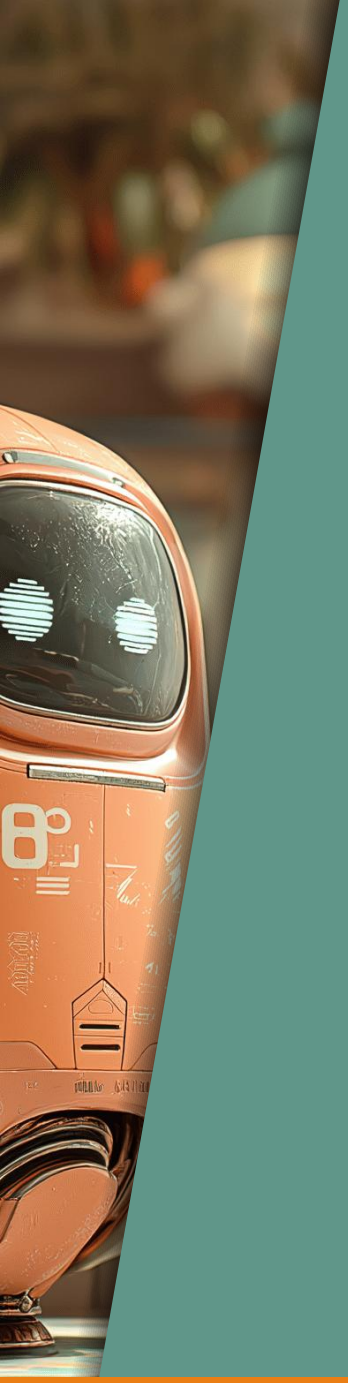


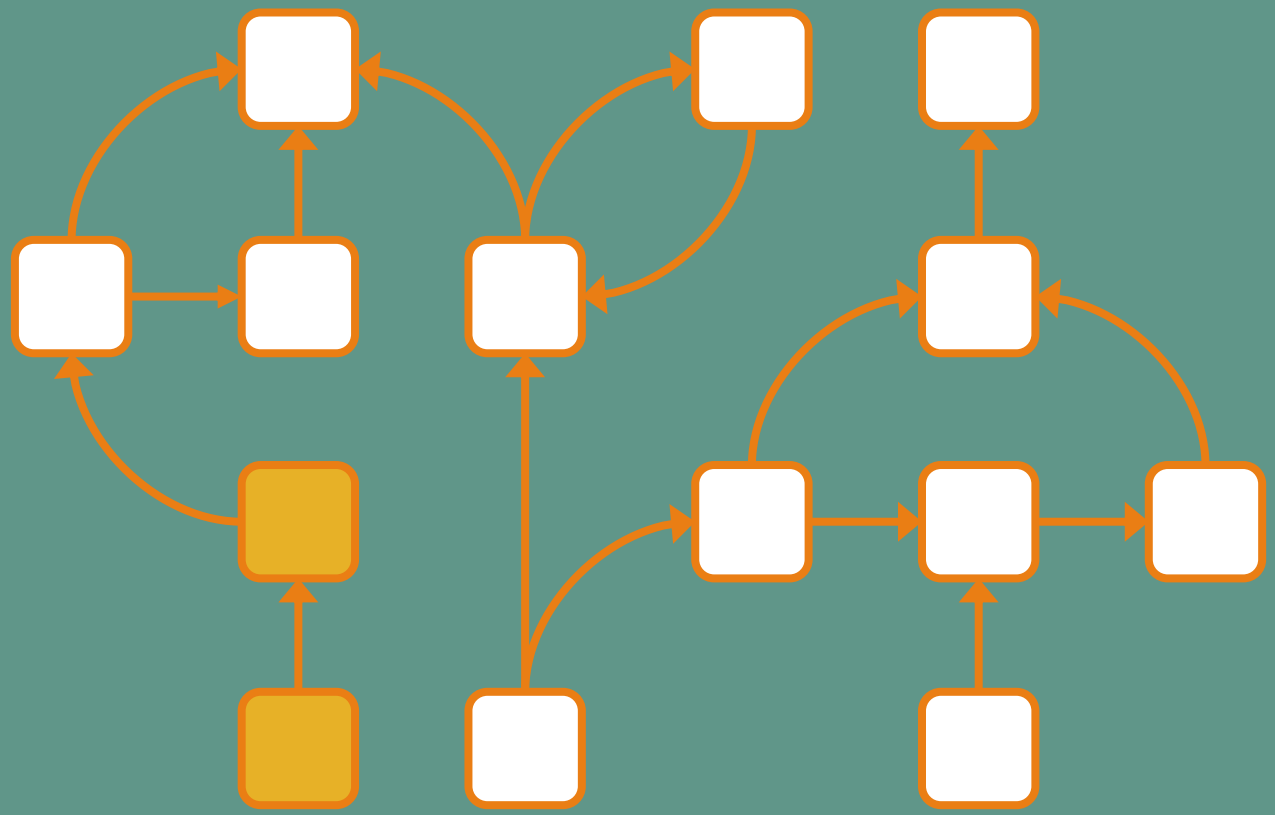
Local variables



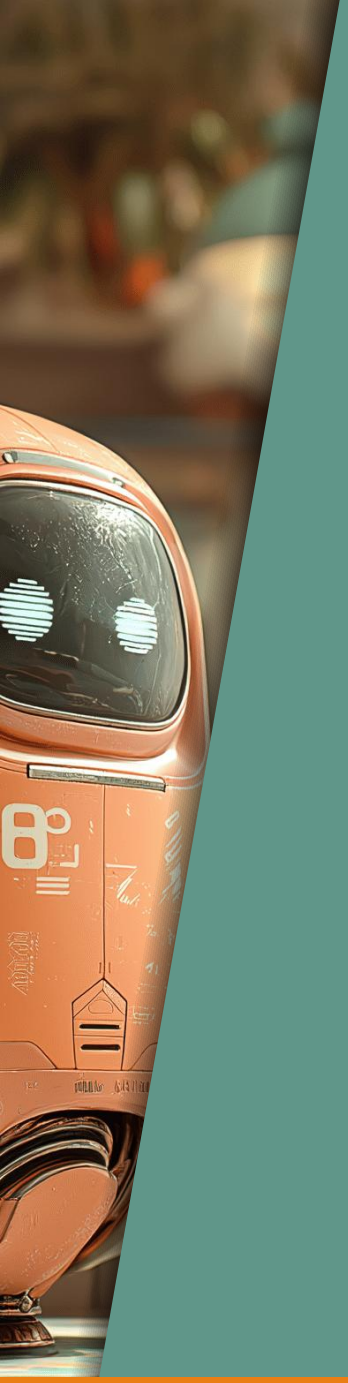


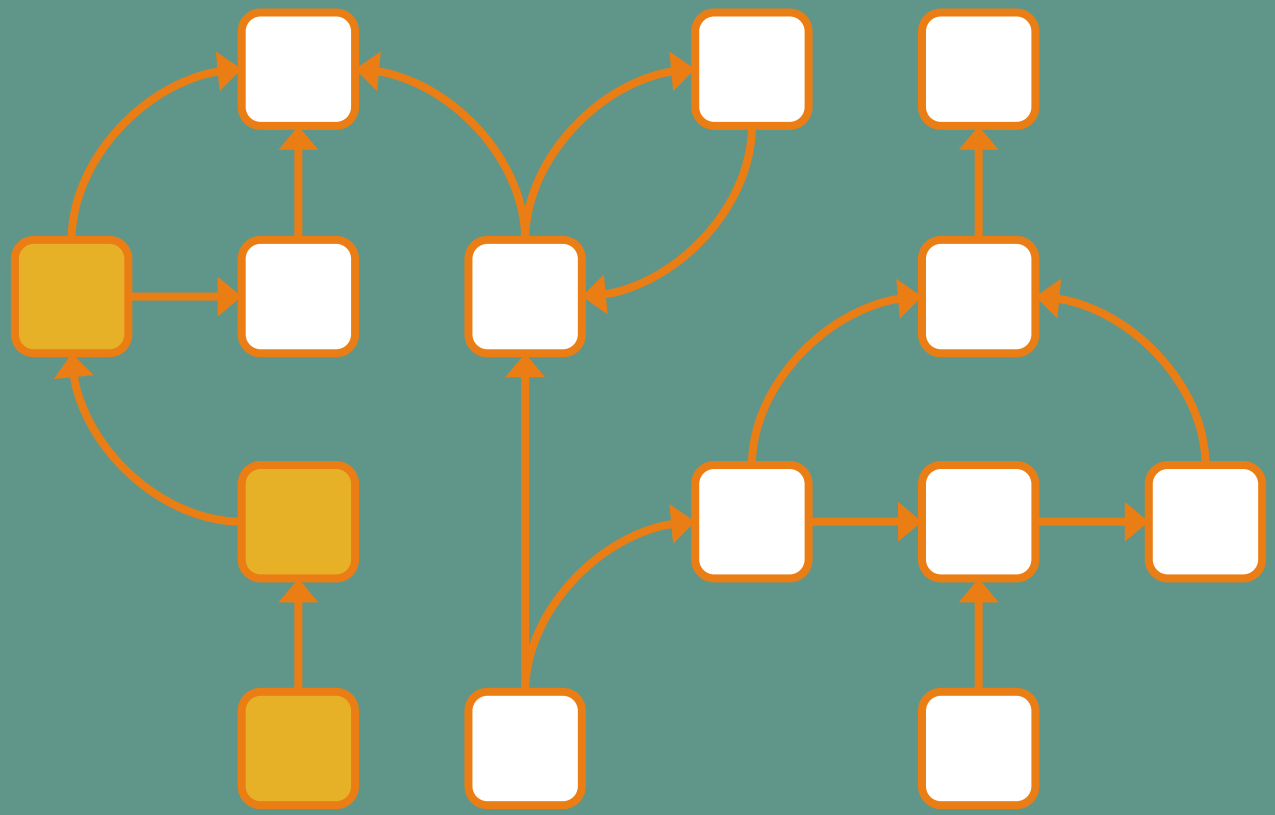
Local variables



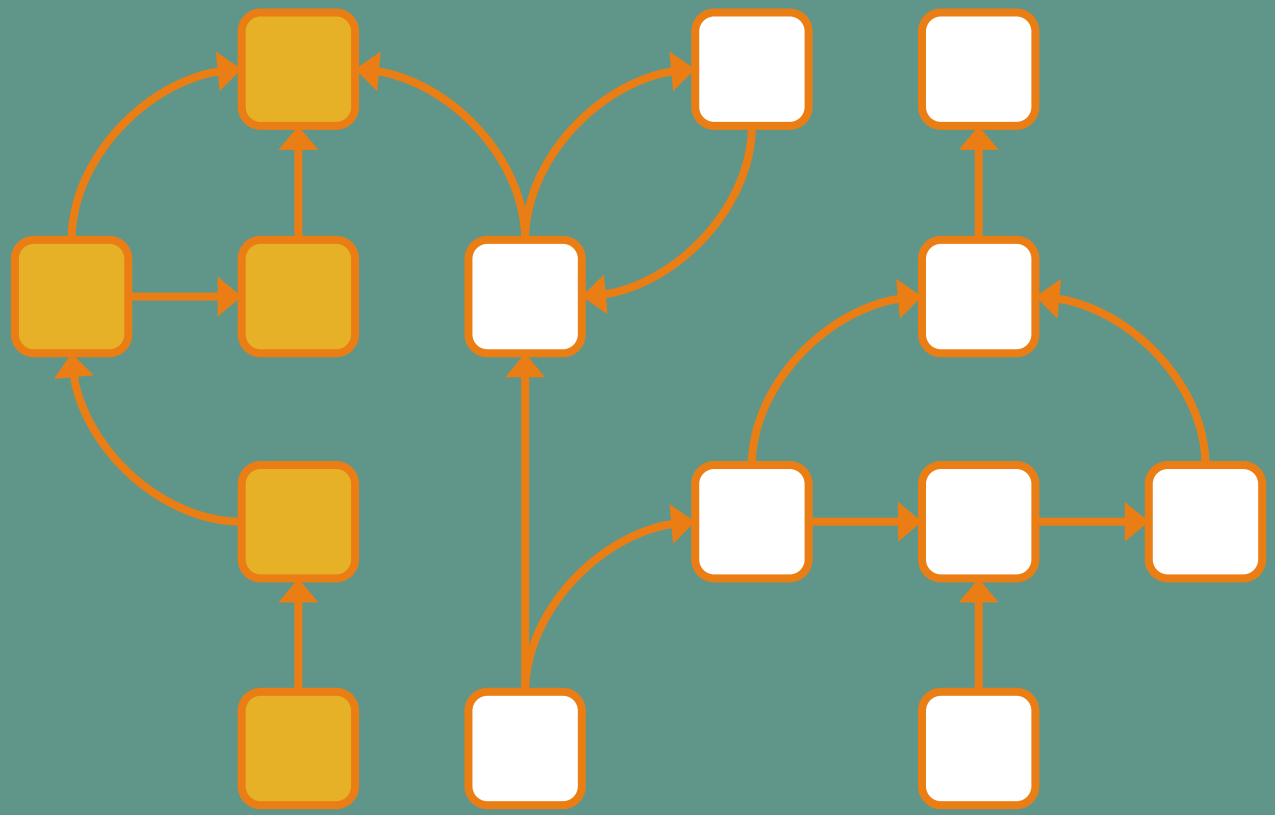


Local variables

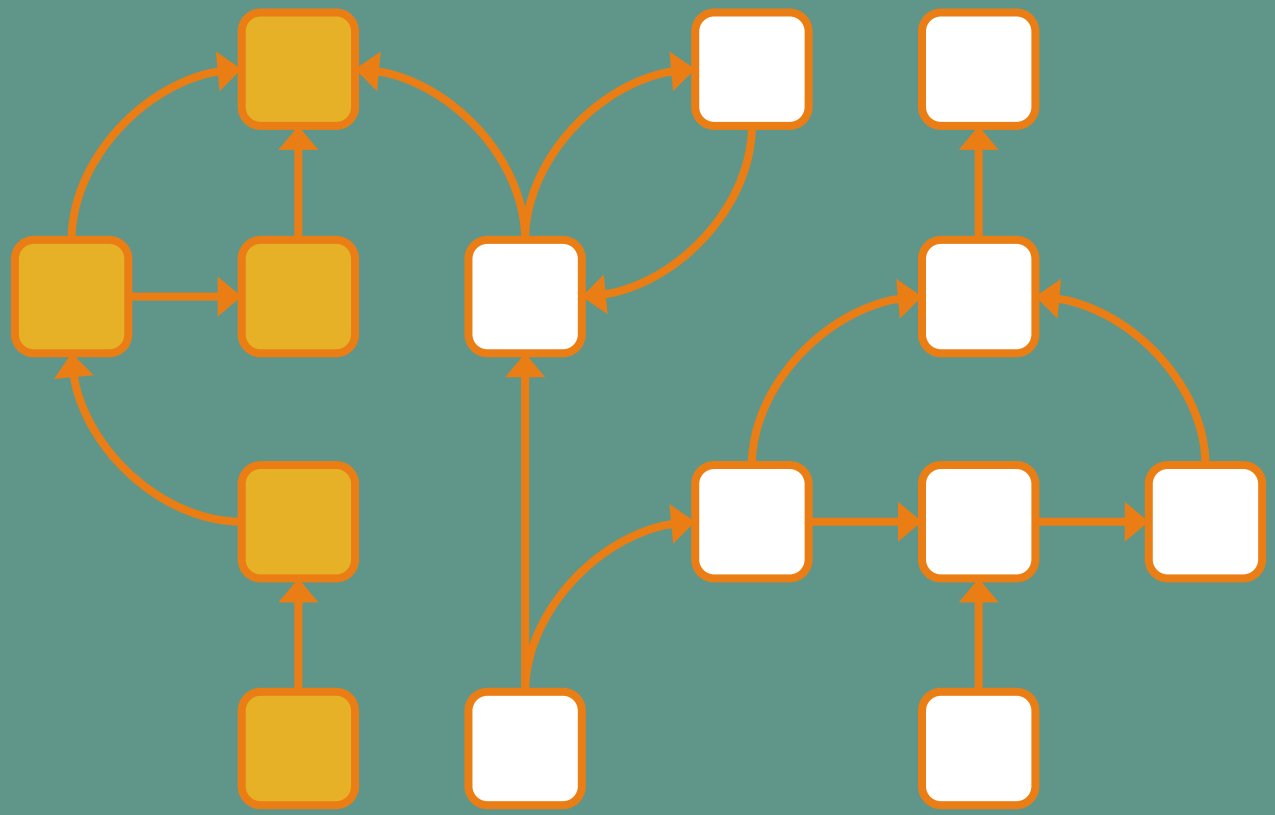




Local variables

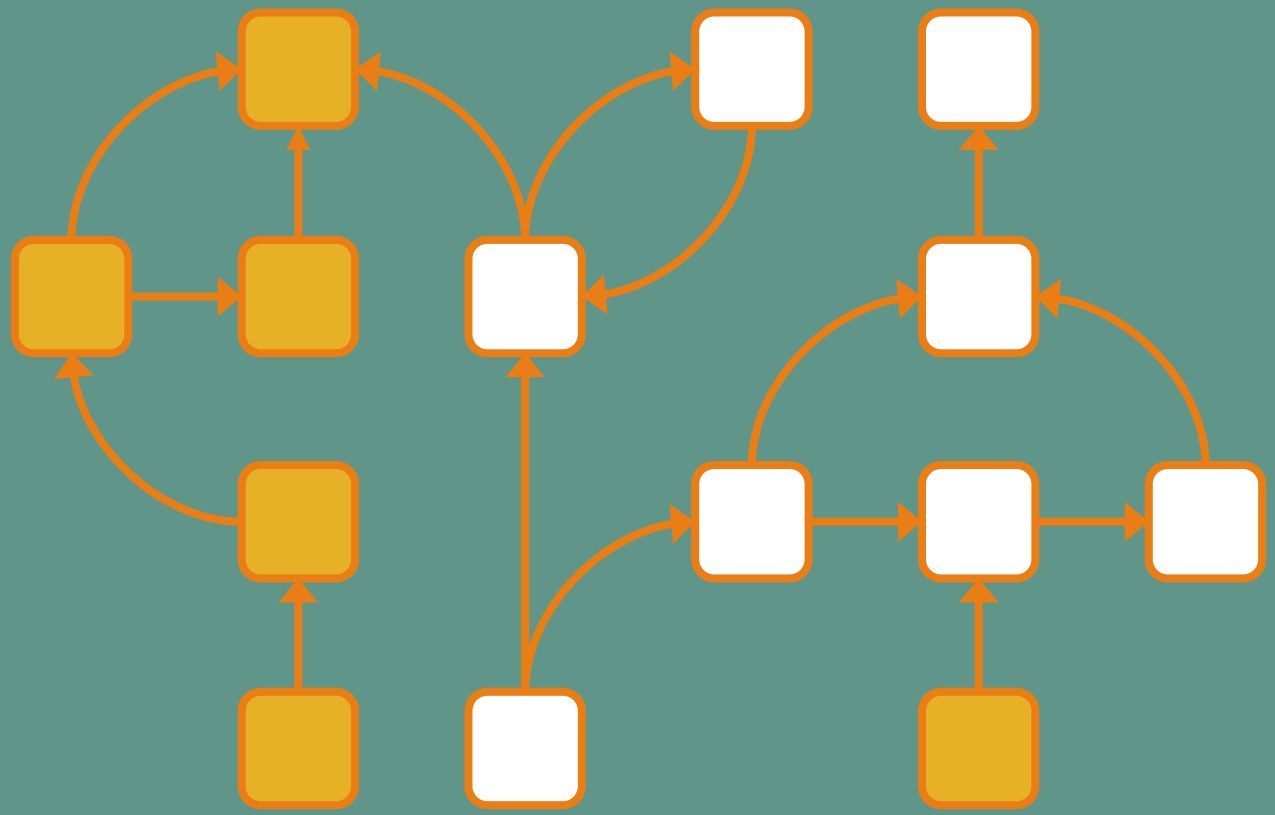


Local variables



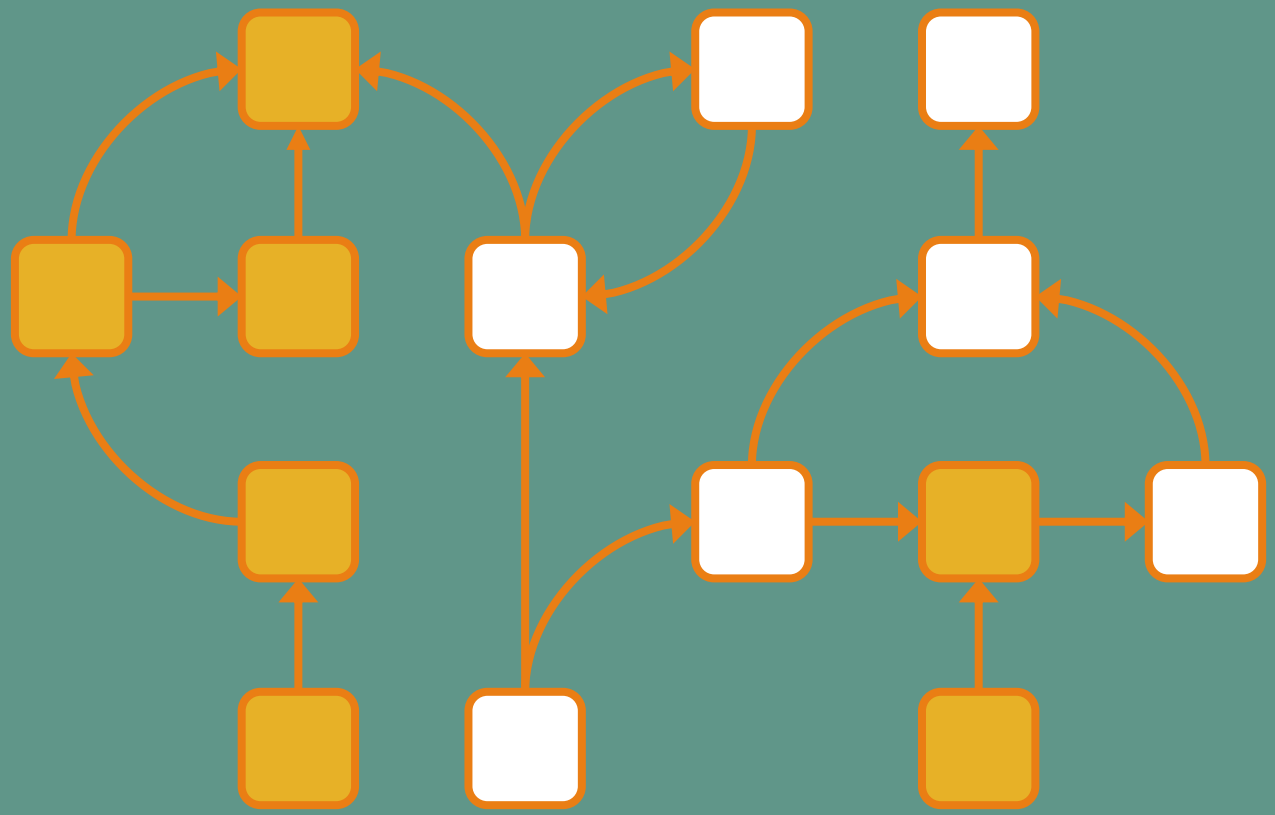
Local variables

Static fields



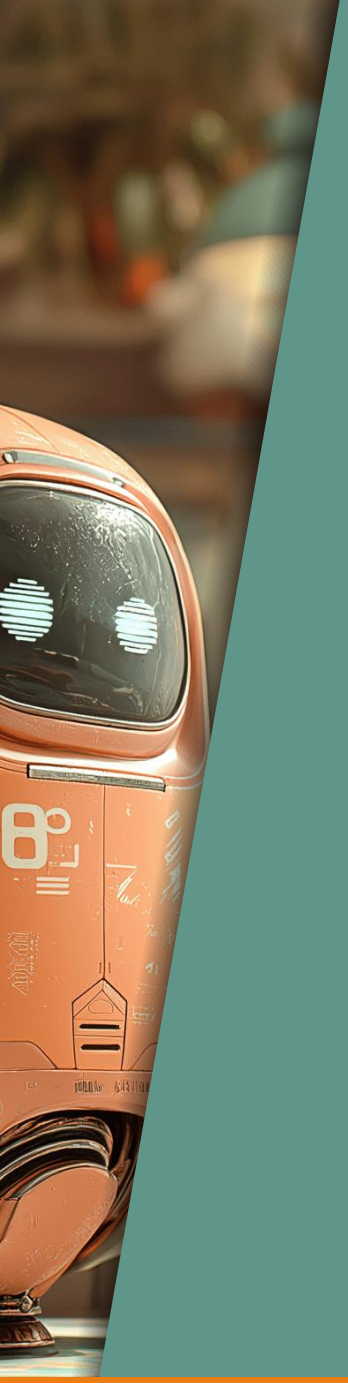
Local variables

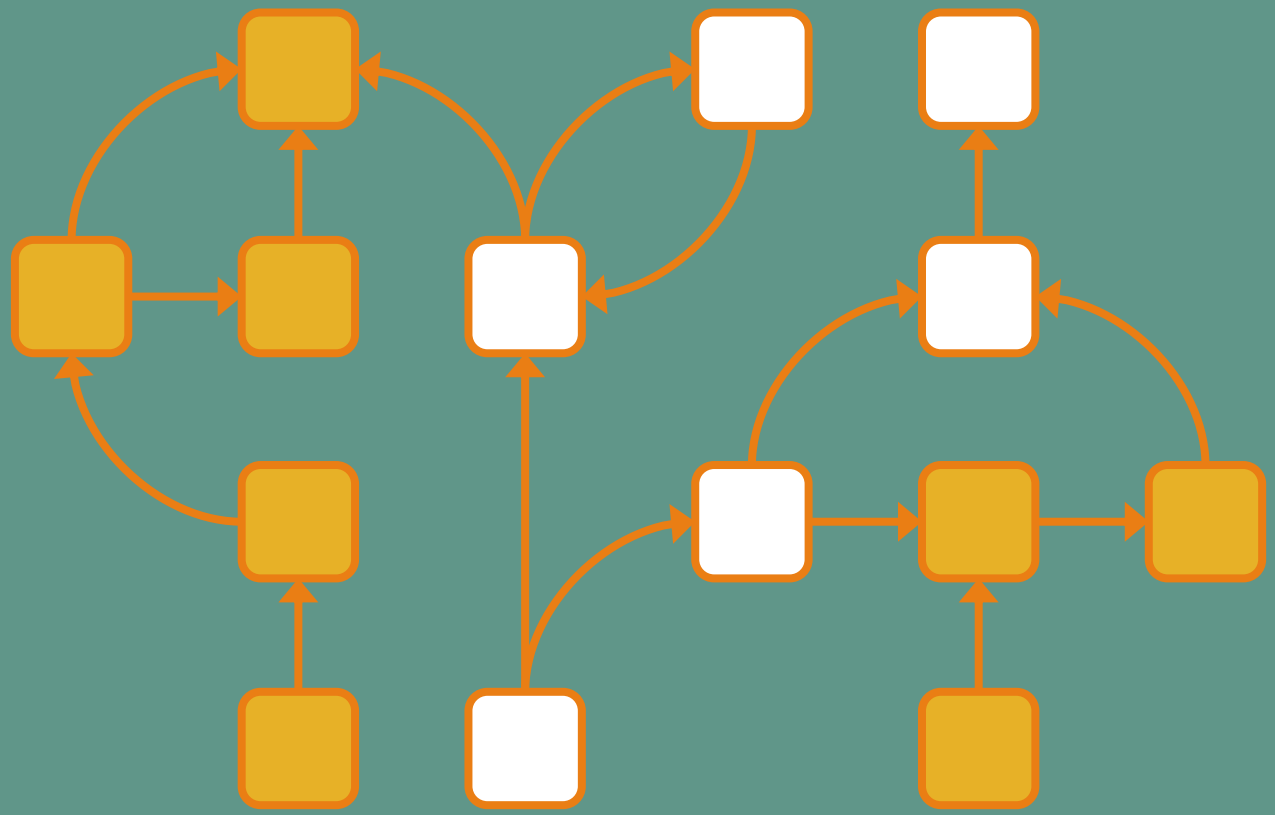
Static fields



Local variables

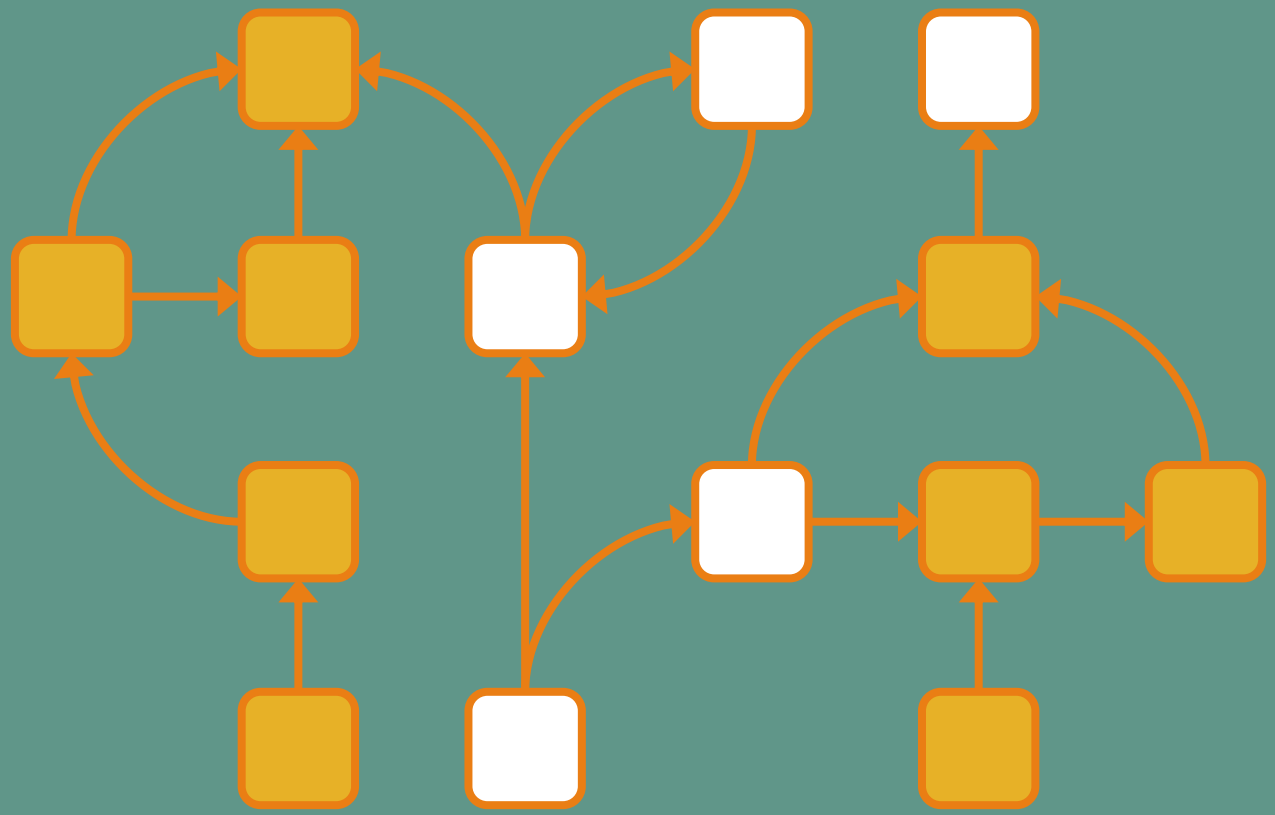
Static fields





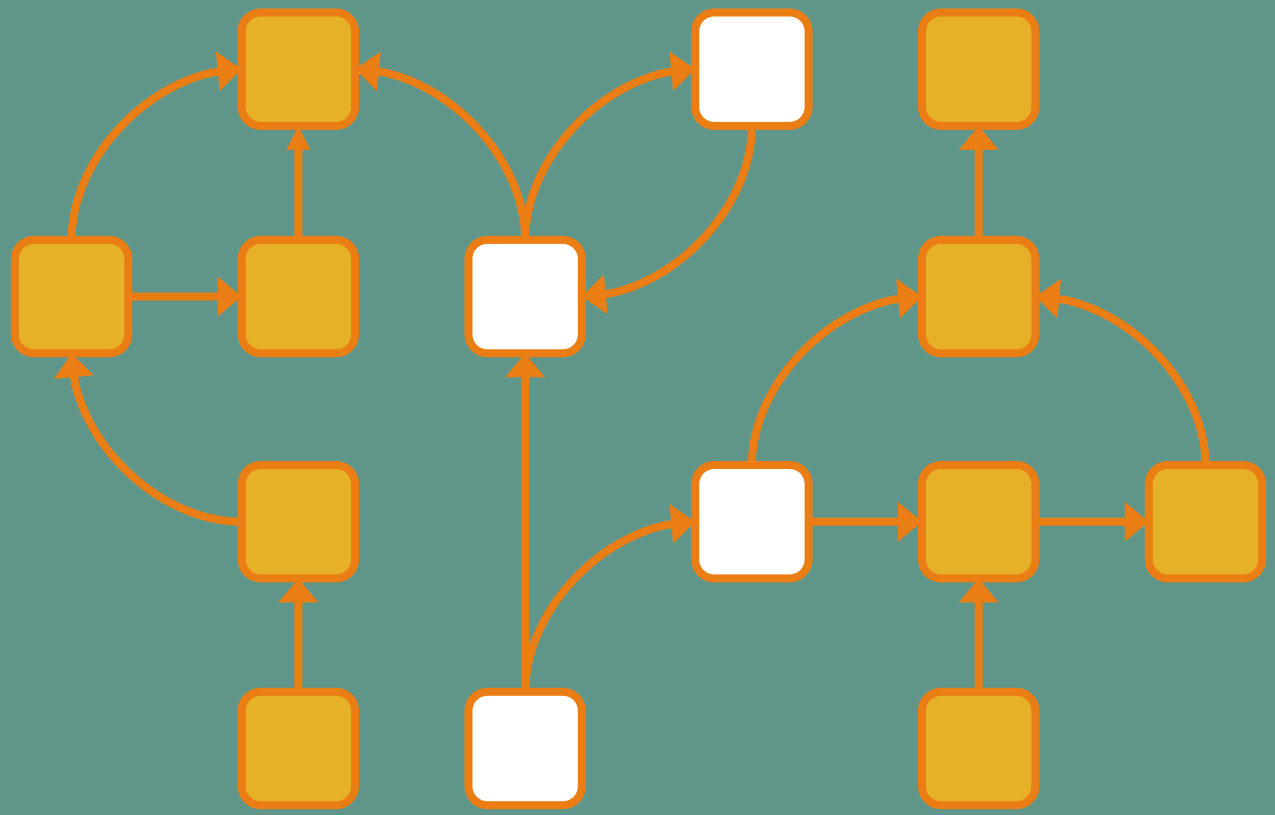
Local variables

Static fields



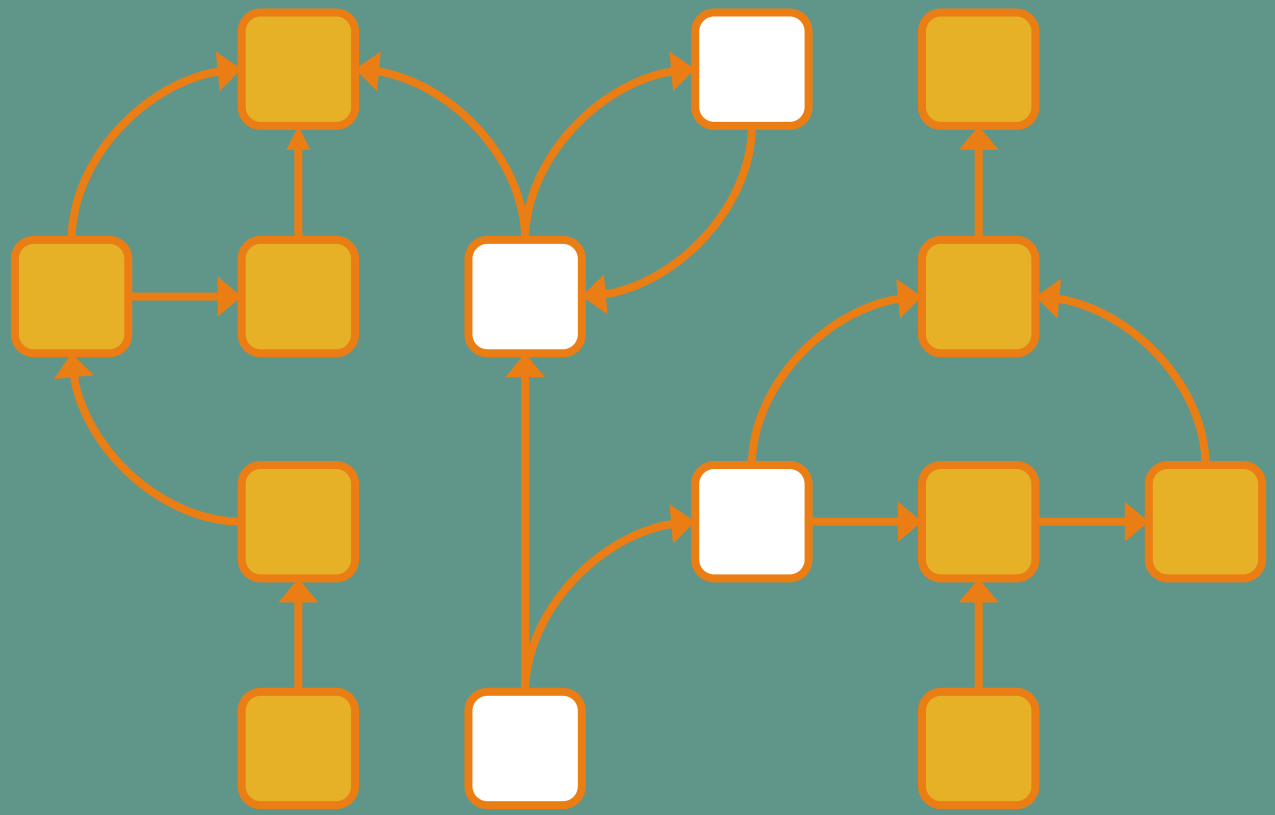
Local variables

Static fields

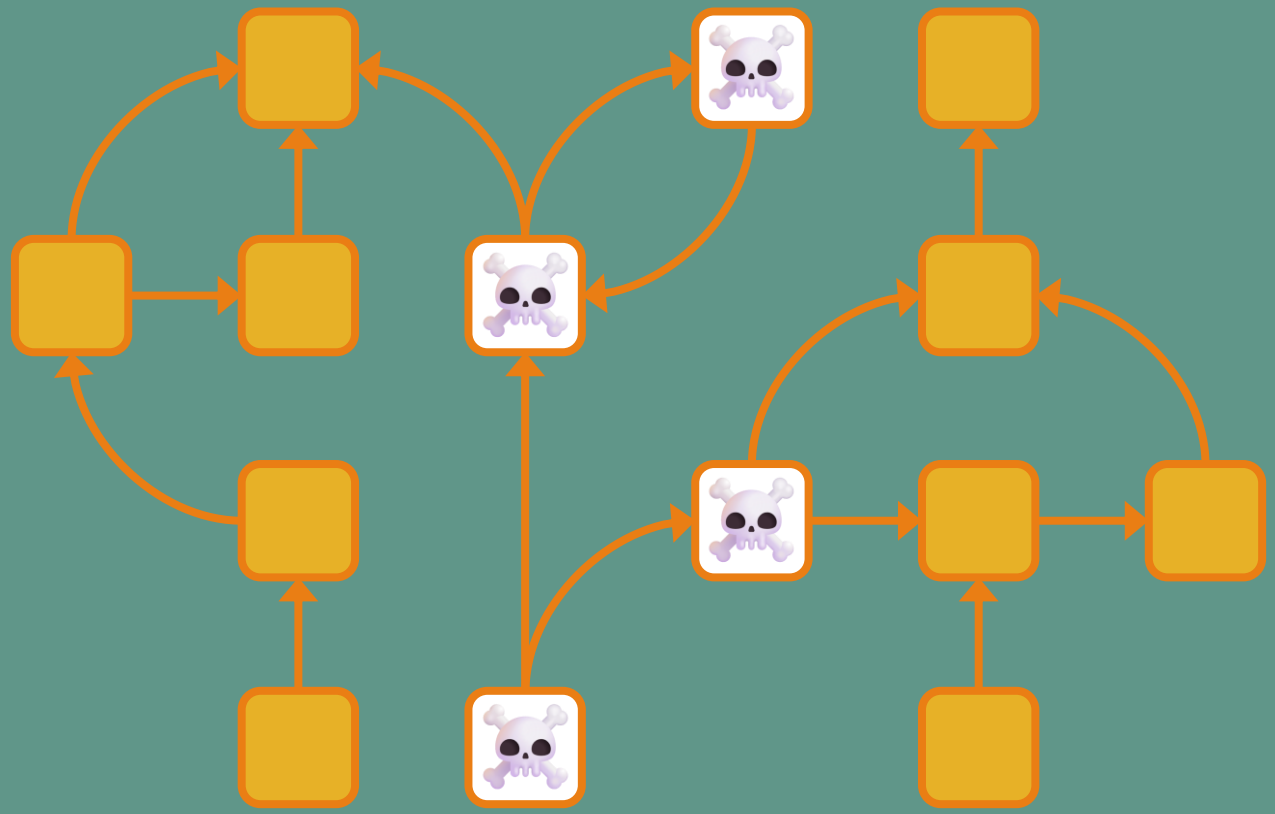


Local variables

Static fields

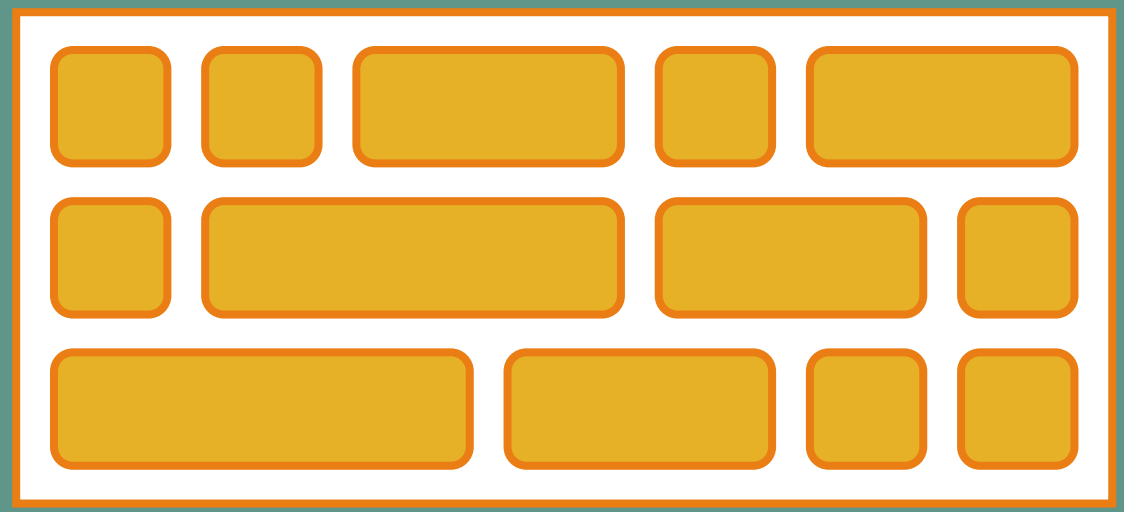


GC roots = Local variables + Static fields

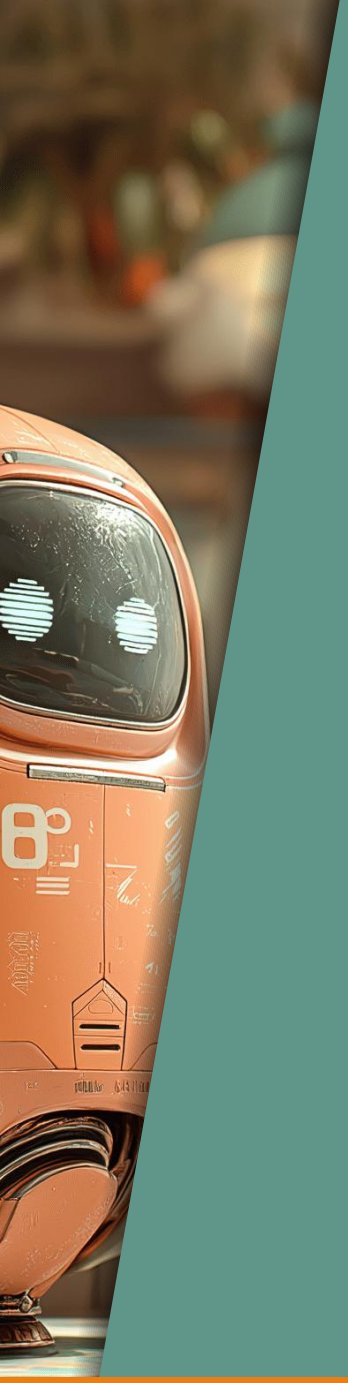


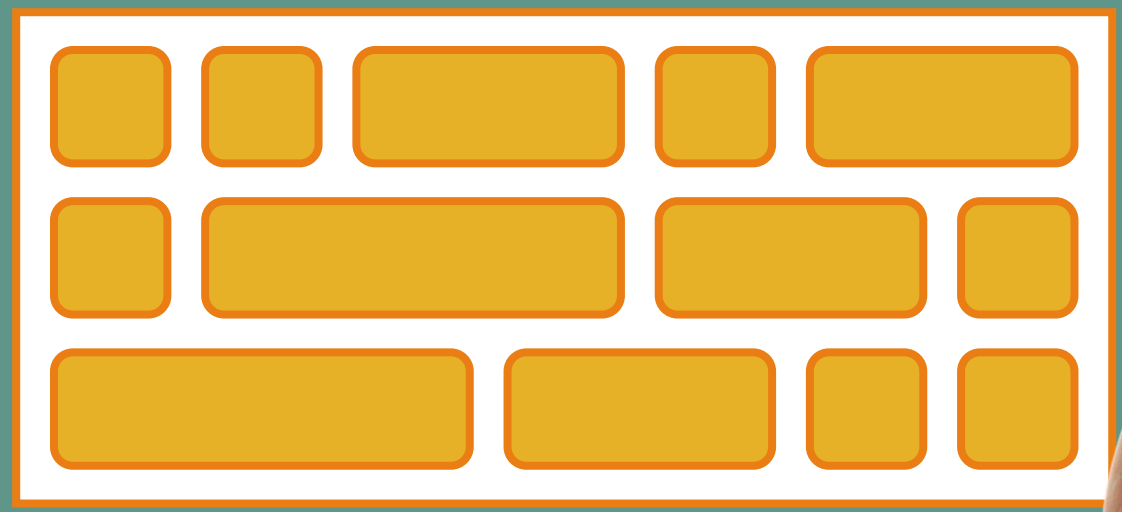
GC roots = Local variables + Static fields

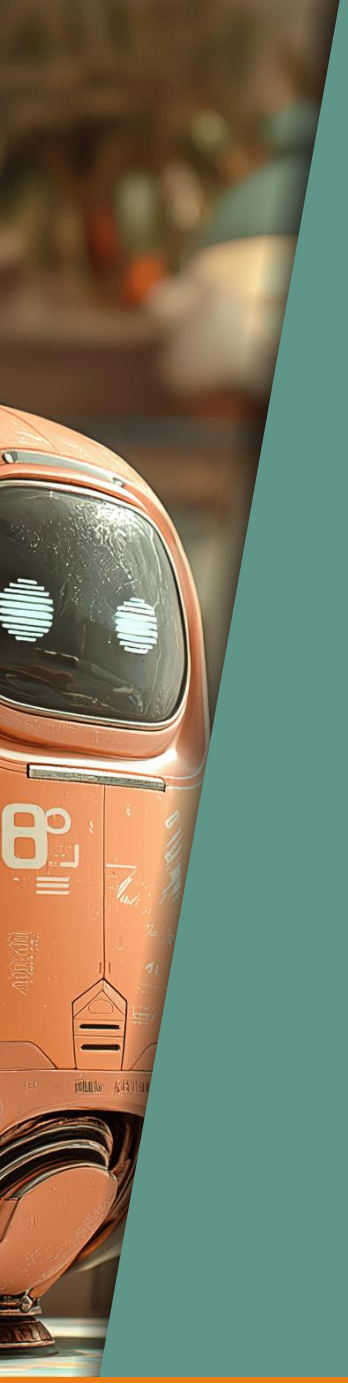
Java 1



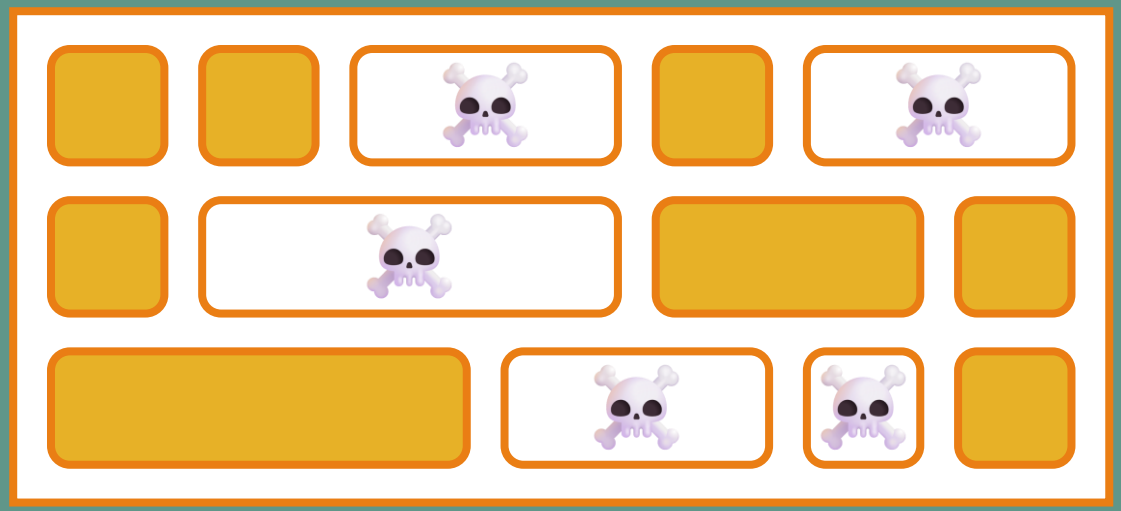
Java 1



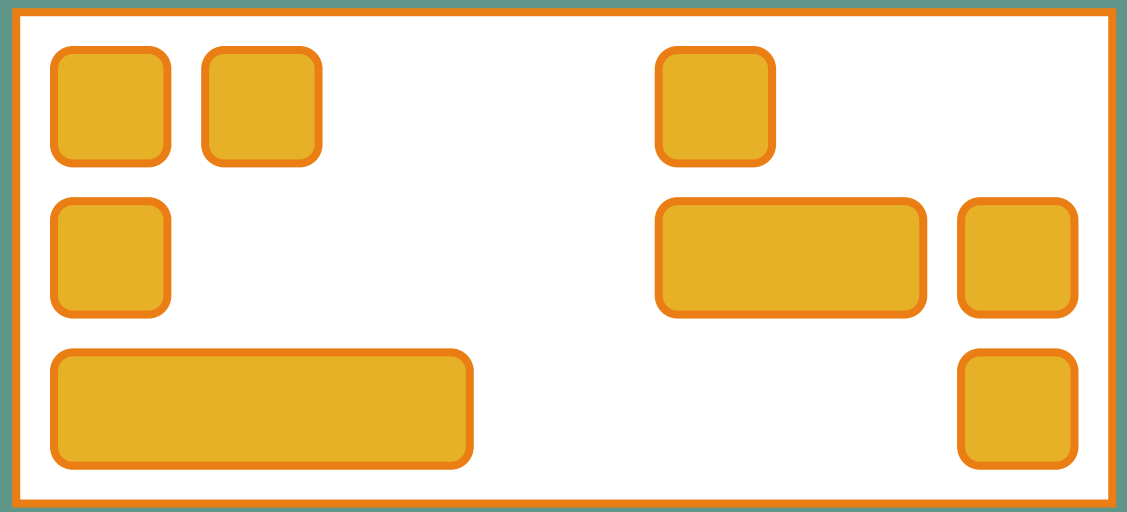


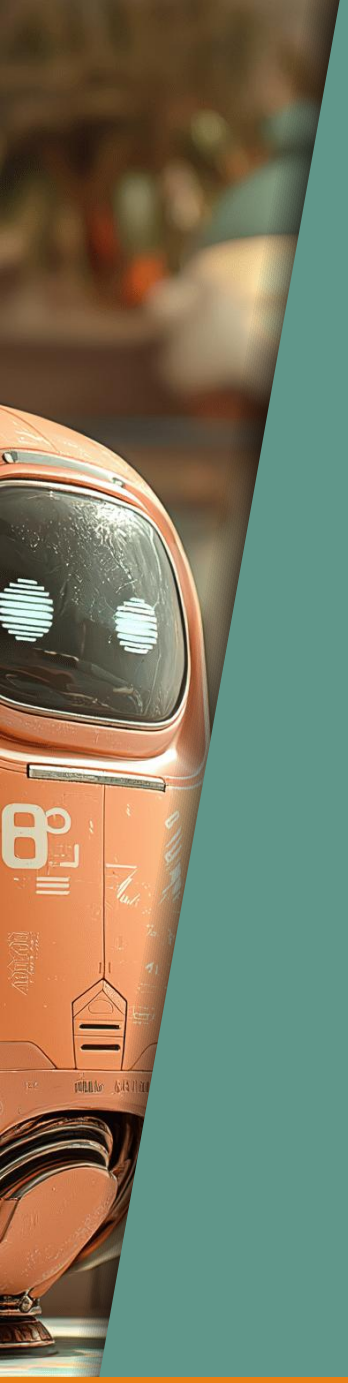


Java 1



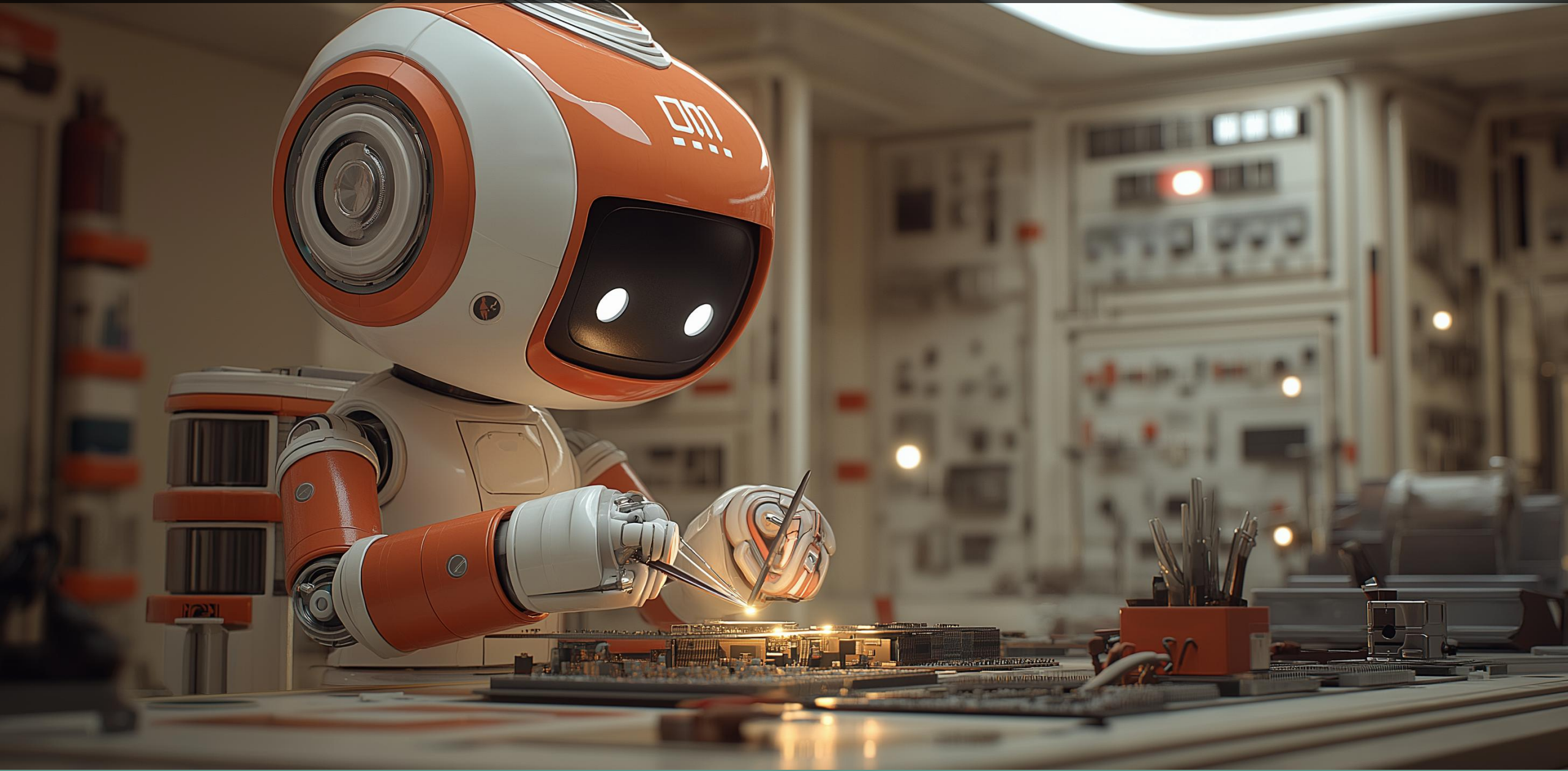
Java 1





Ce serait bien d’éviter de parcourir tous les objets à chaque fois…

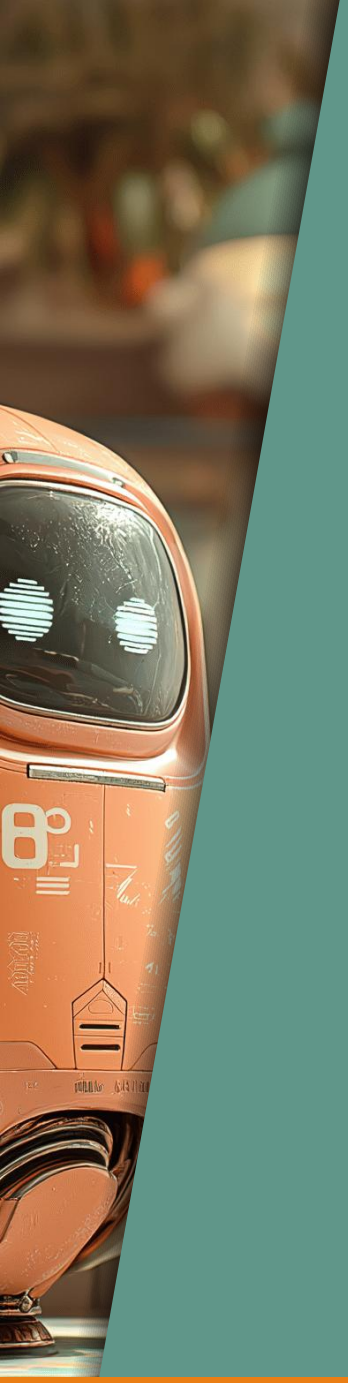
L'HYPOTHÈSE GÉNÉRATIONNELLE



L’hypothèse générationnelle

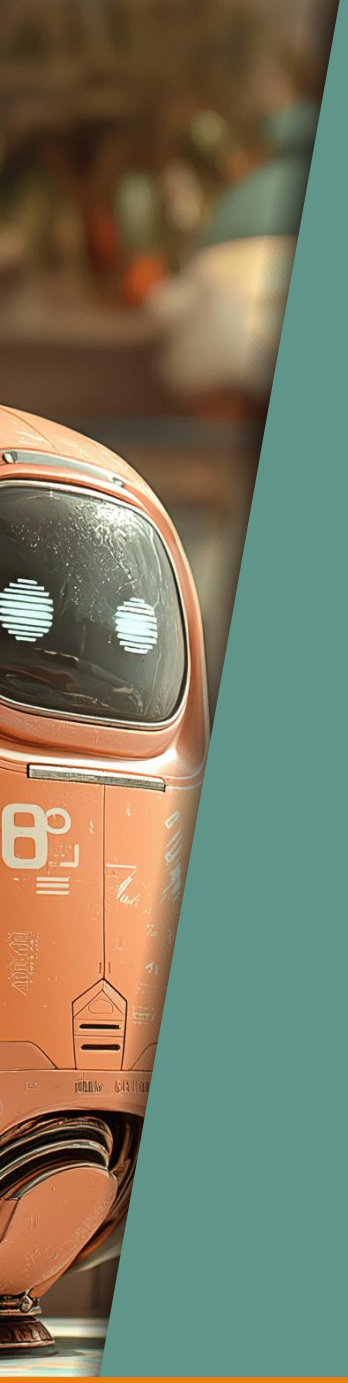
L’hypothèse générationnelle

- Soit les objets meurent rapidement



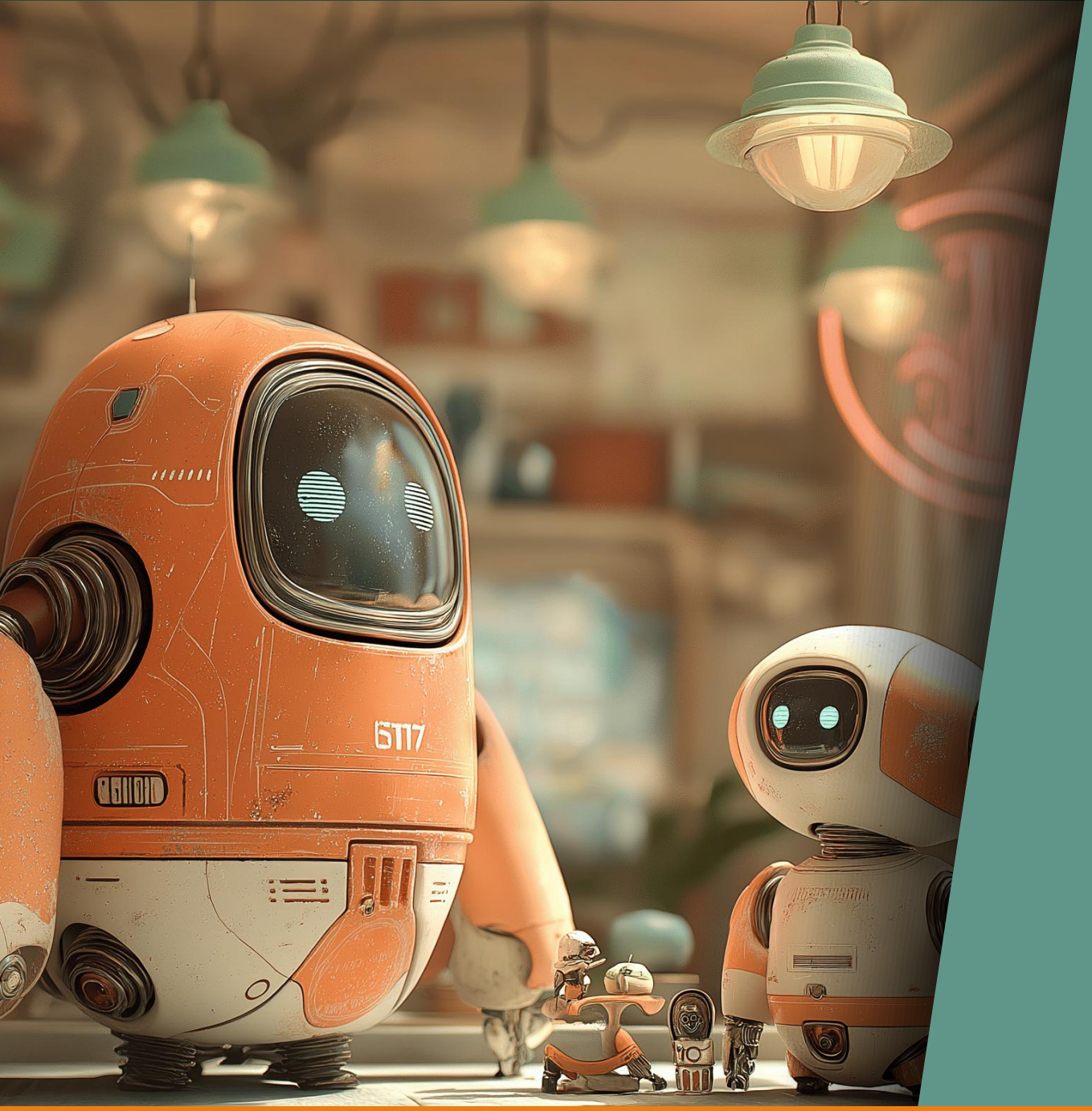
L’hypothèse générationnelle

- Soit les objets meurent rapidement
- Soit les objets sont là pour longtemps (voire toujours)



L’hypothèse générationnelle

- Soit les objets meurent rapidement
- Soit les objets sont là pour longtemps (voire toujours)

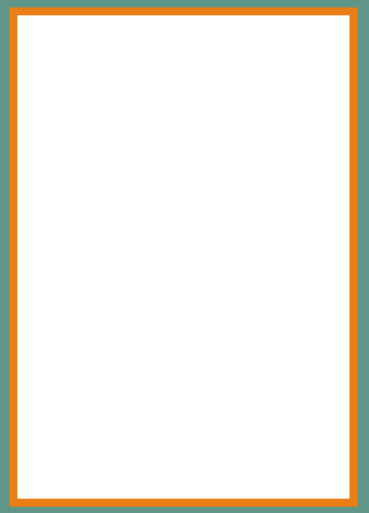


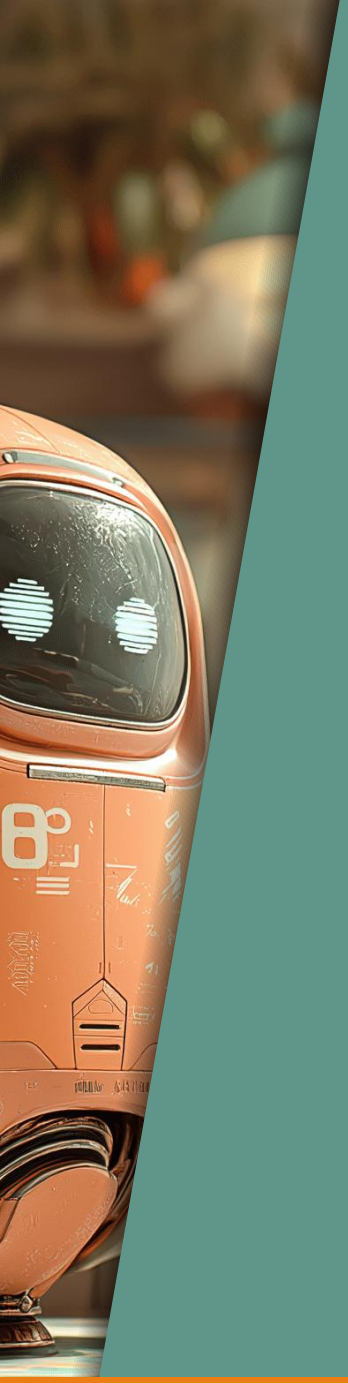
Serial GC

Java 1.2 (1998)

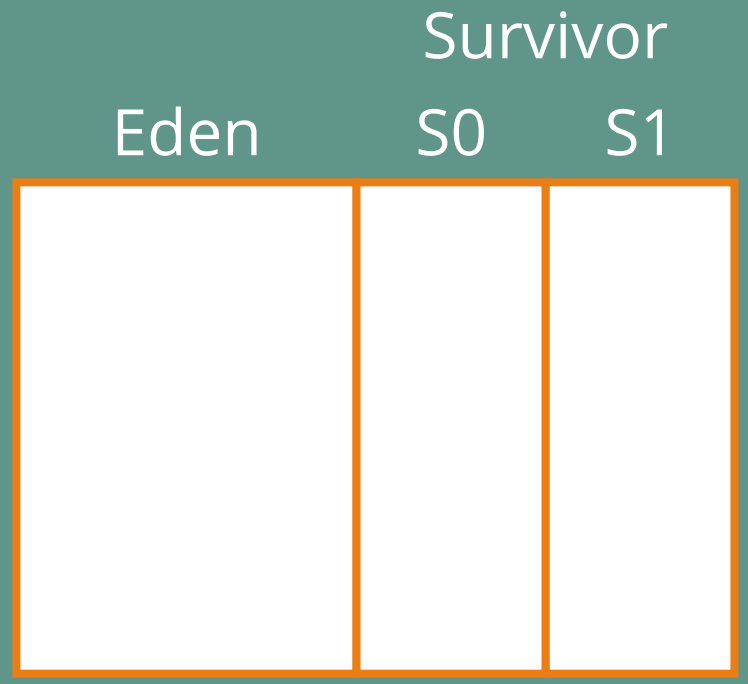
Serial GC

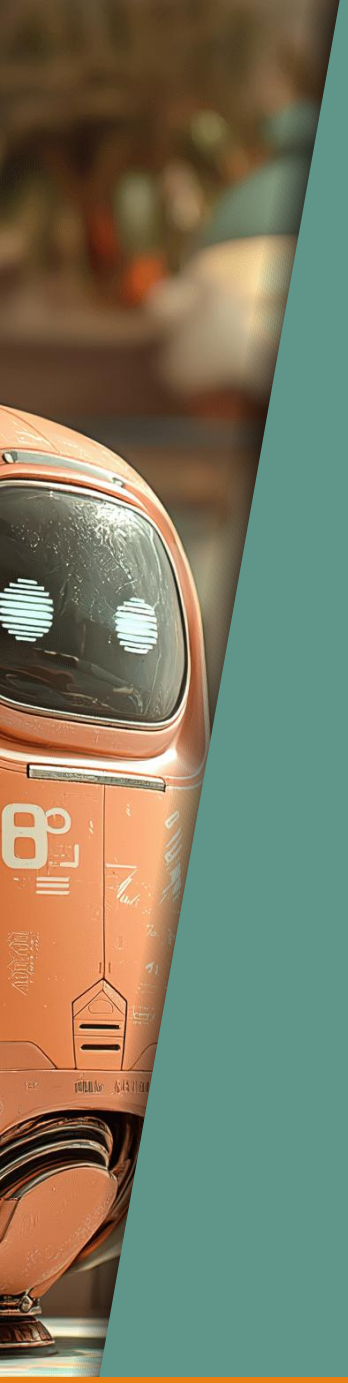
Eden



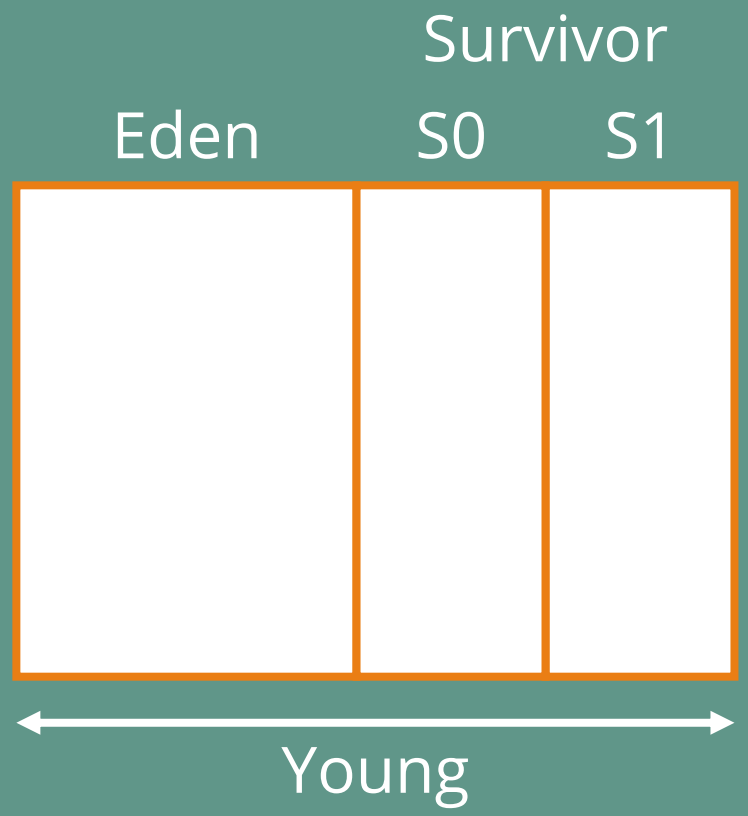


Serial GC

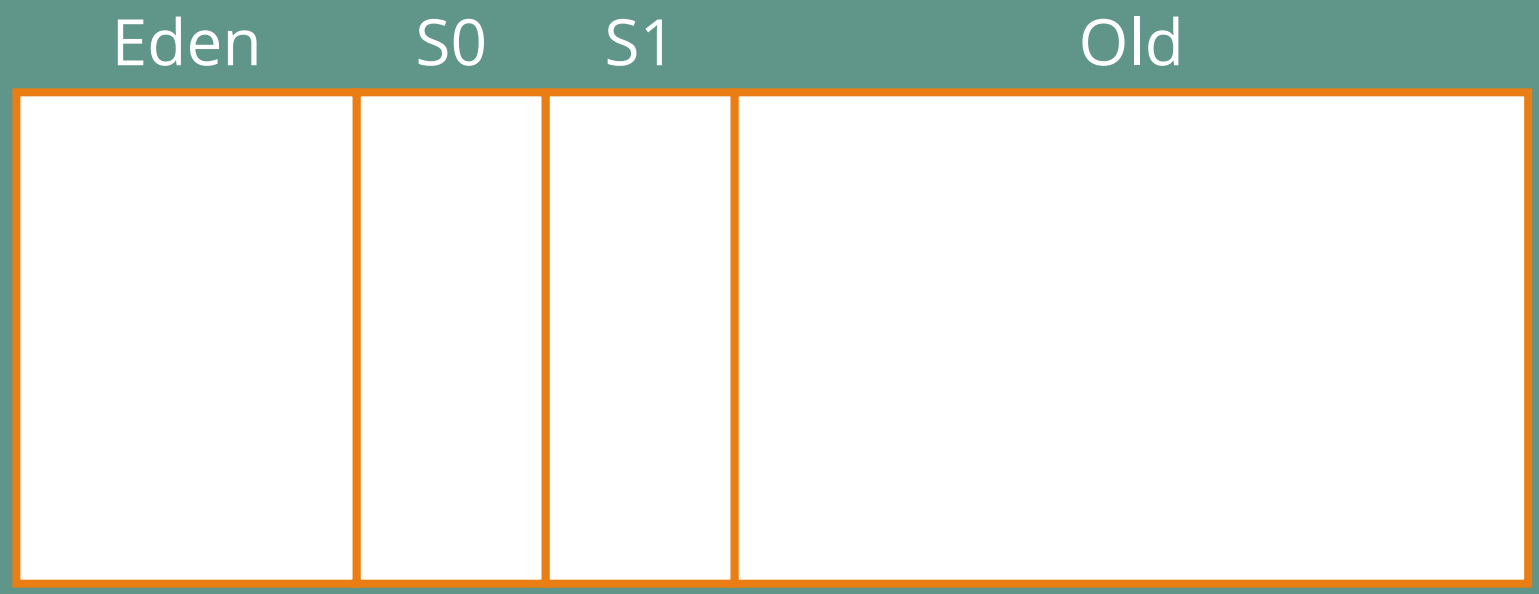


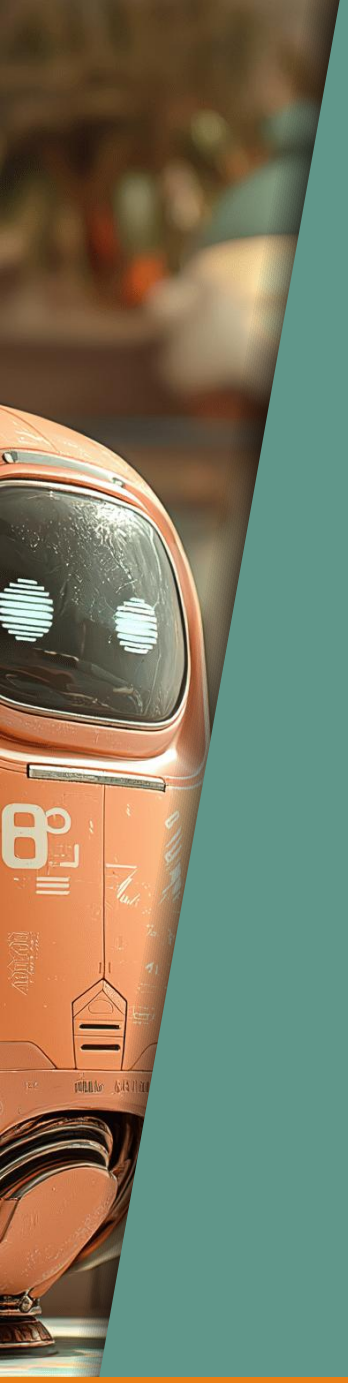


Serial GC

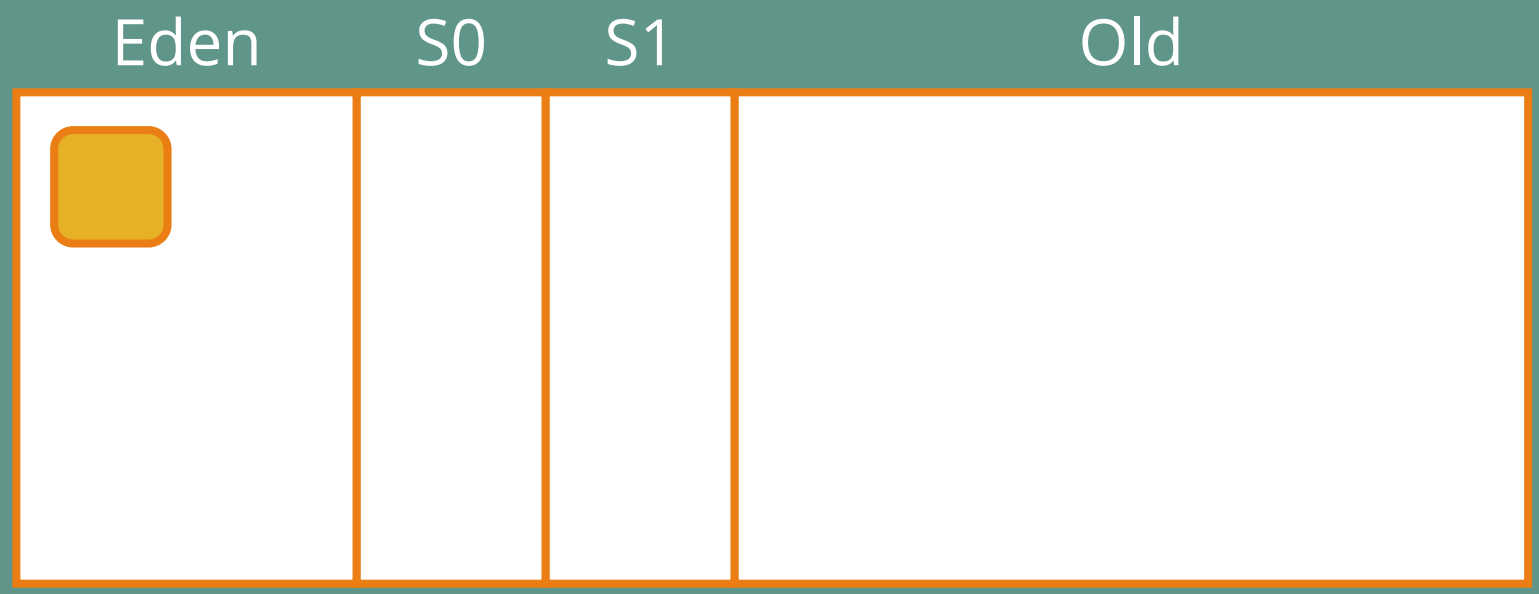


Serial GC

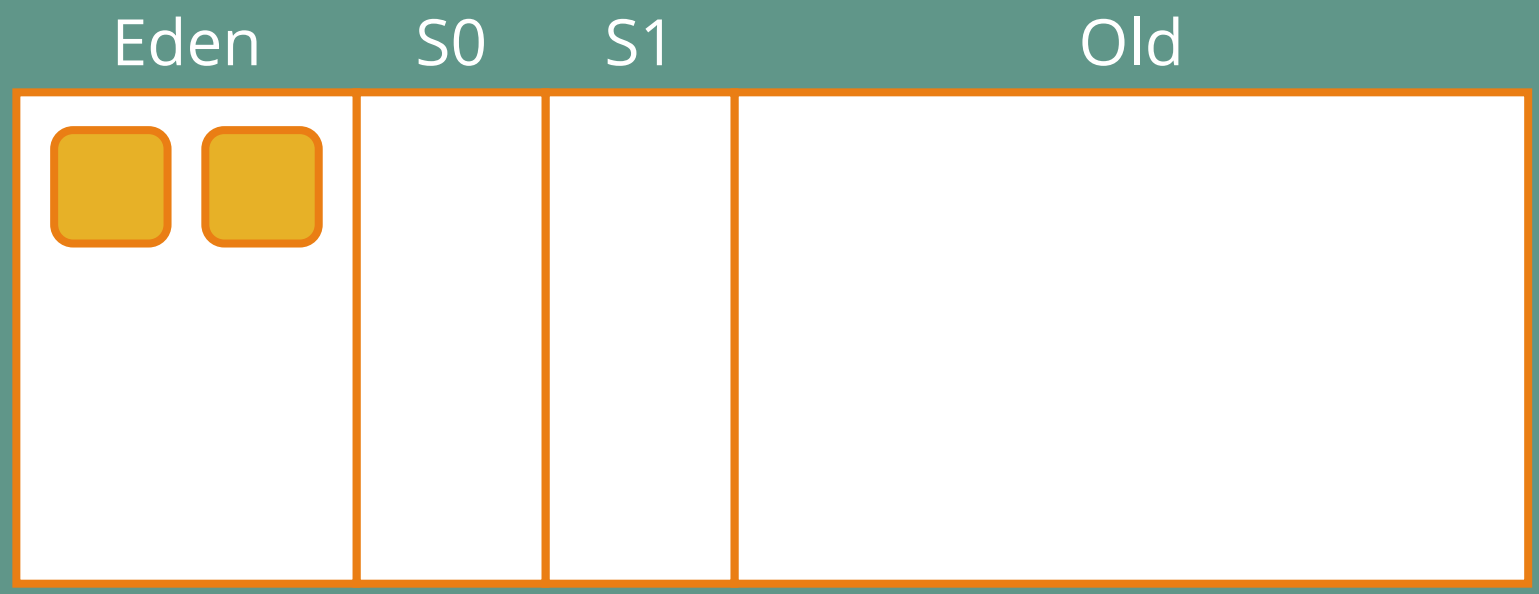




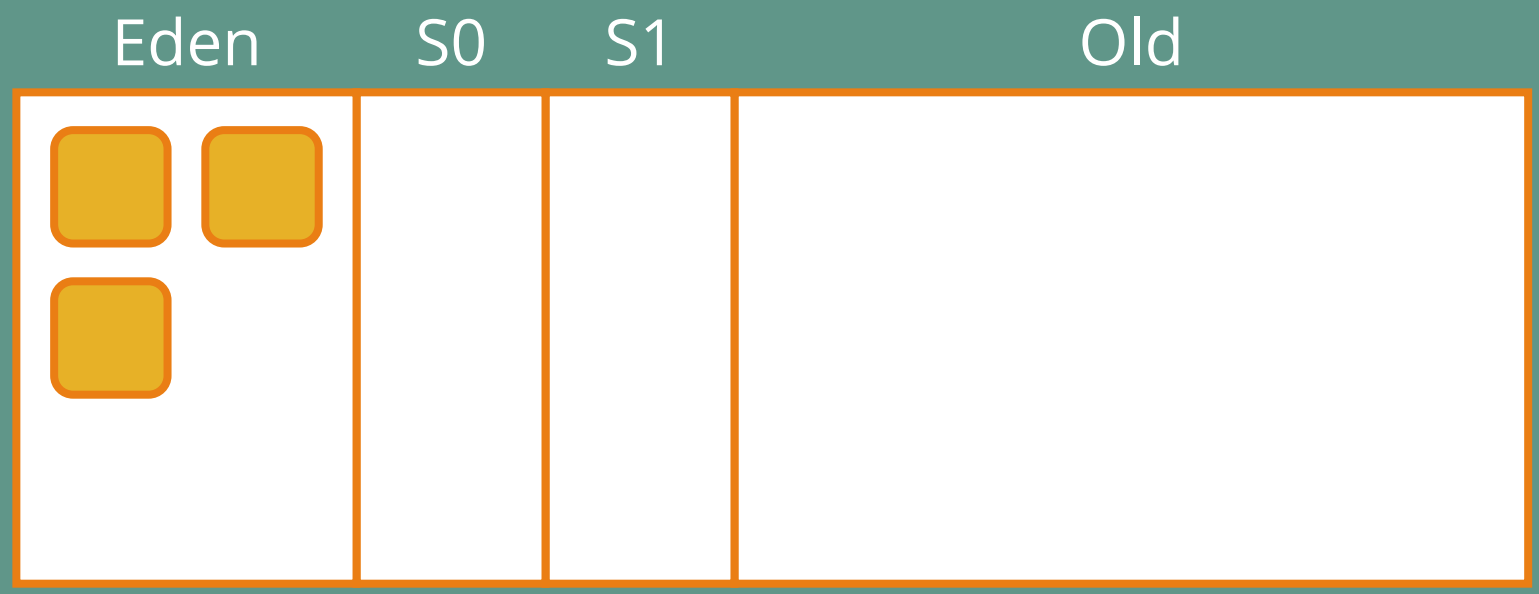
Serial GC



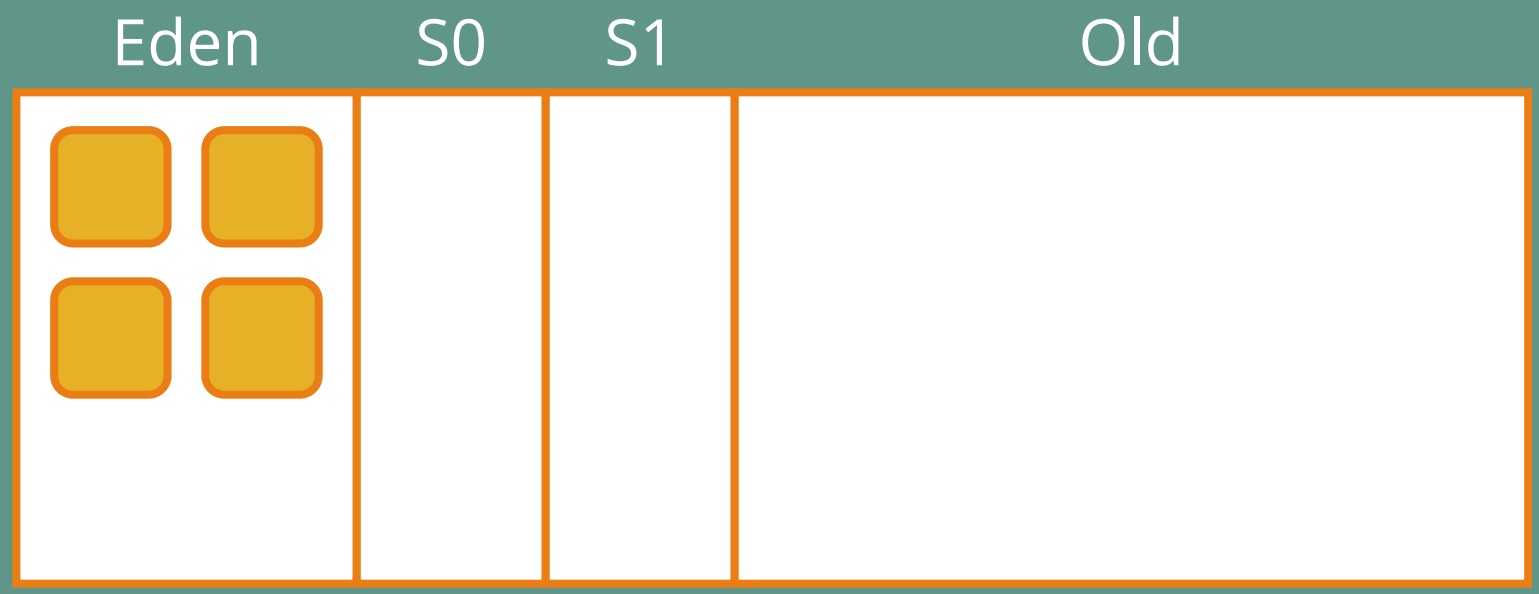
Serial GC



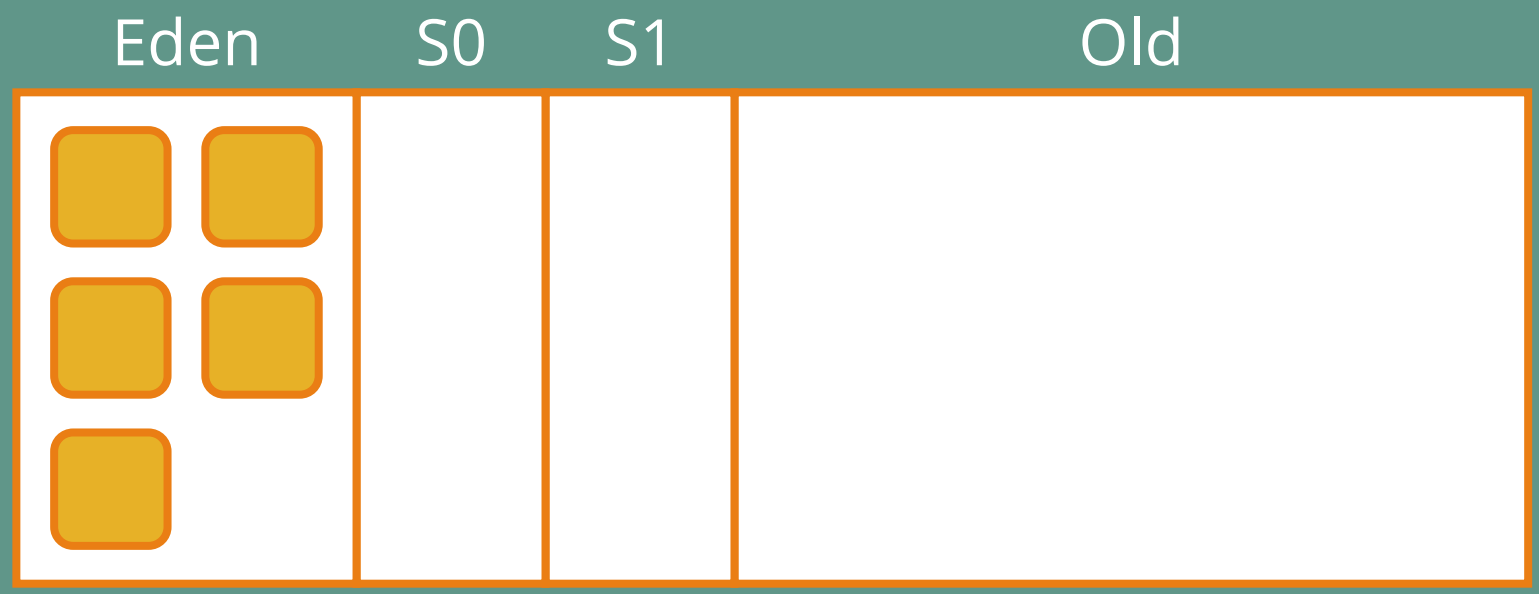
Serial GC



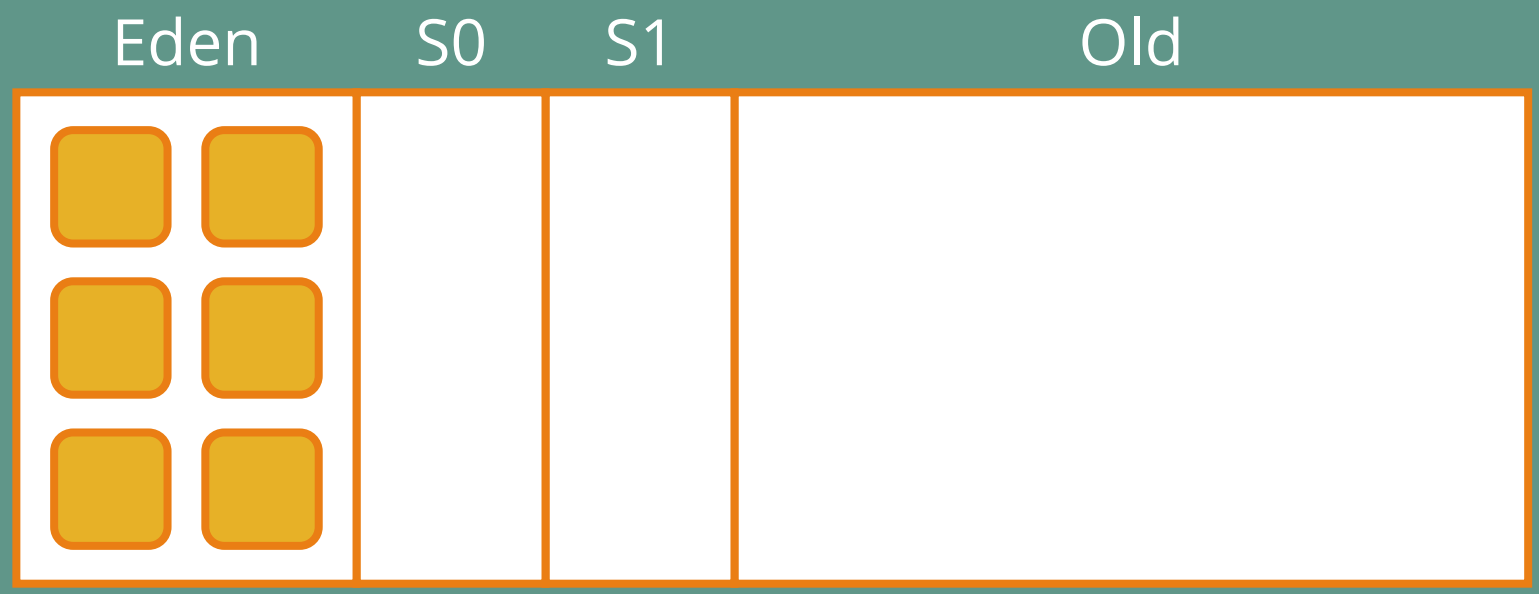
Serial GC



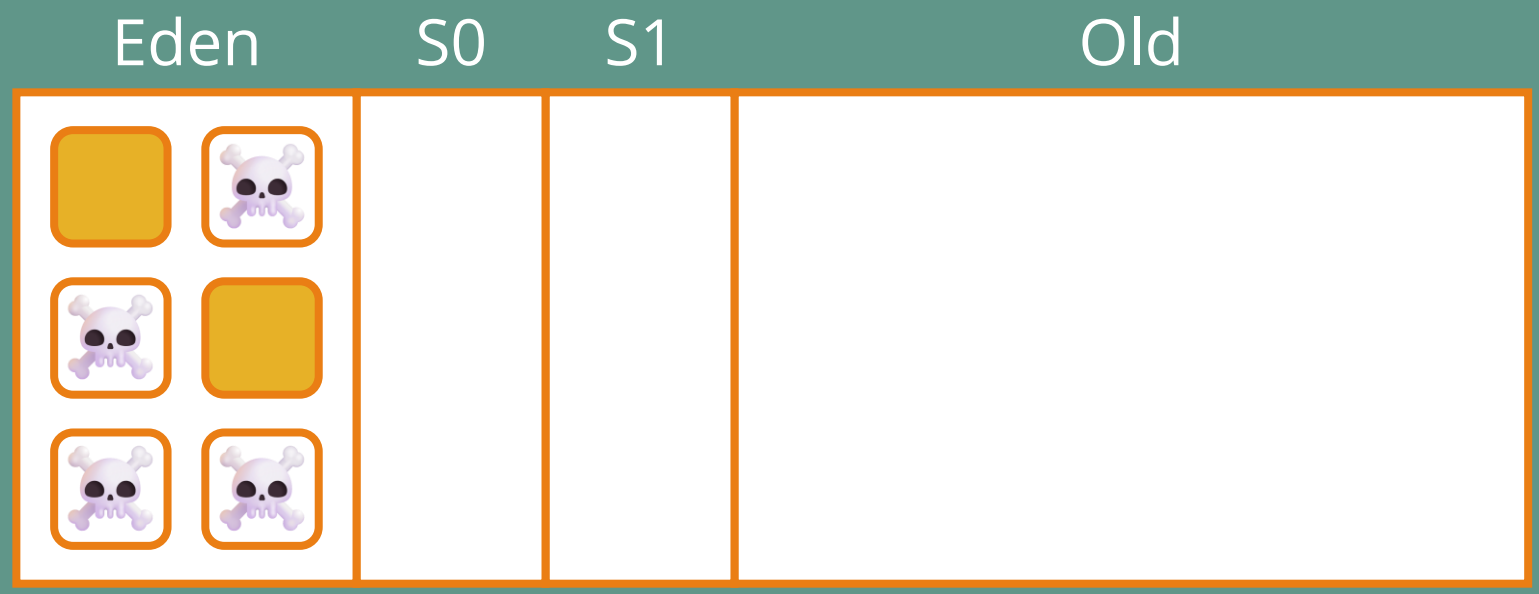
Serial GC



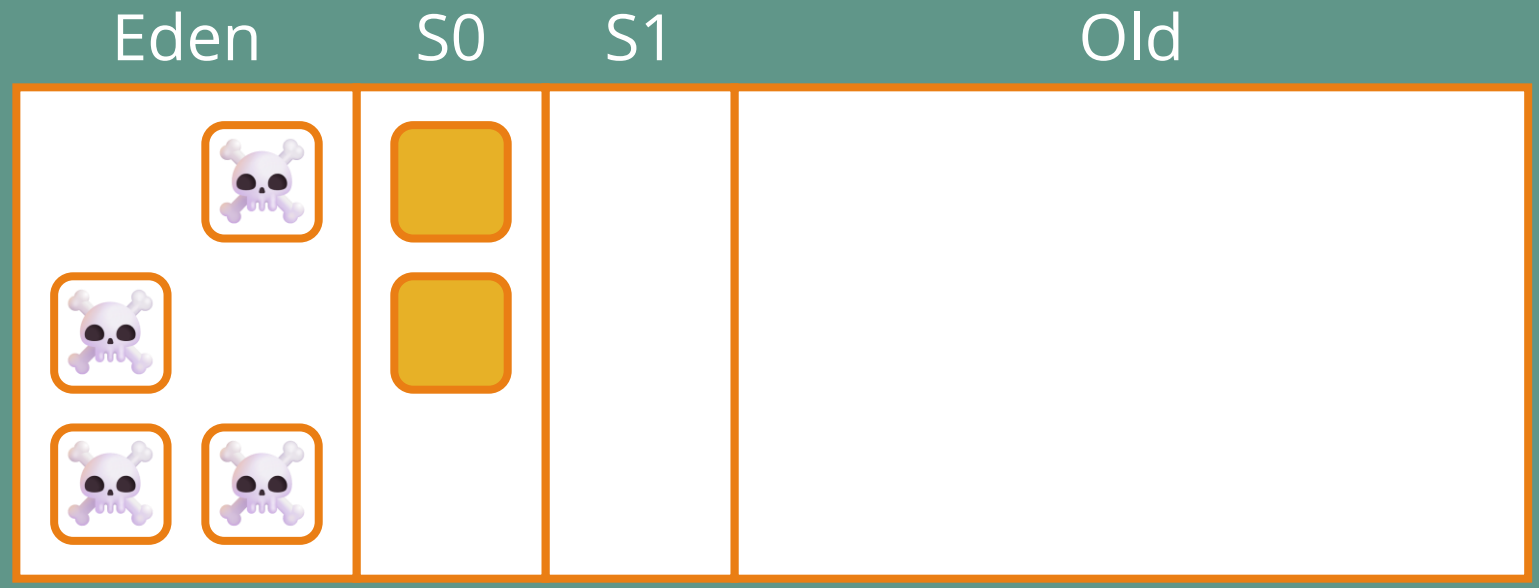
Serial GC



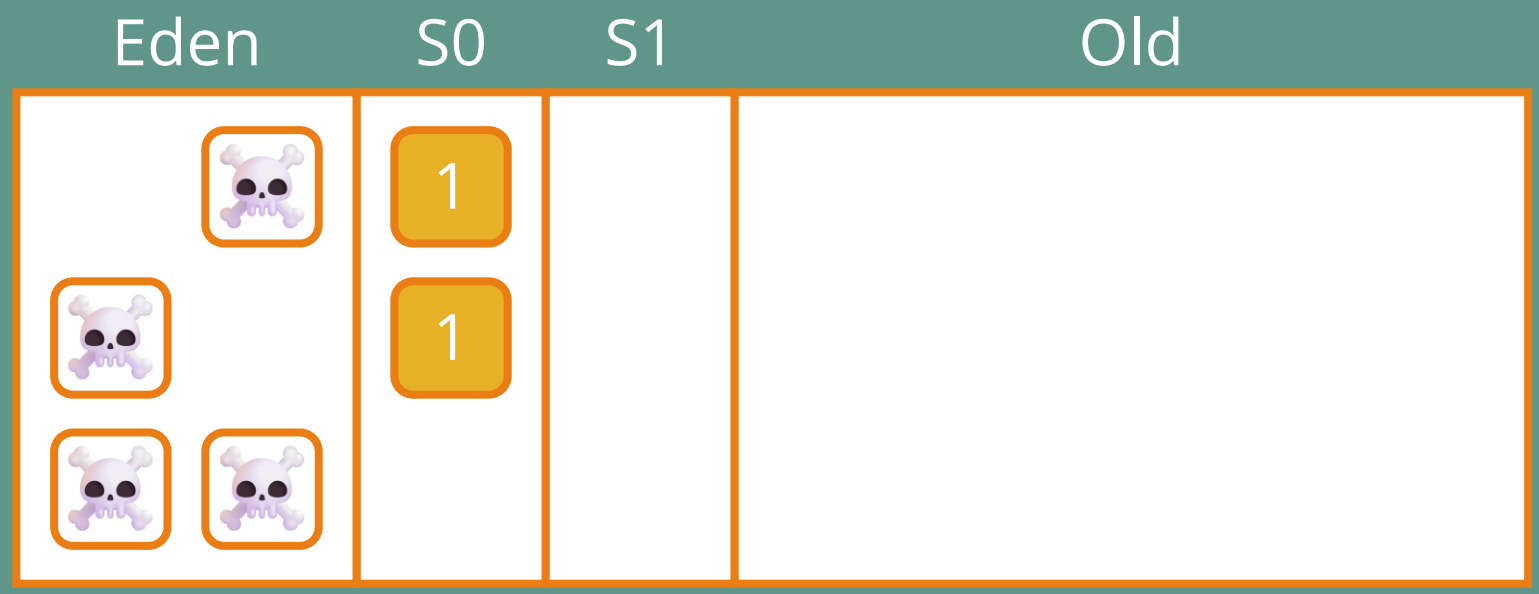
Serial GC



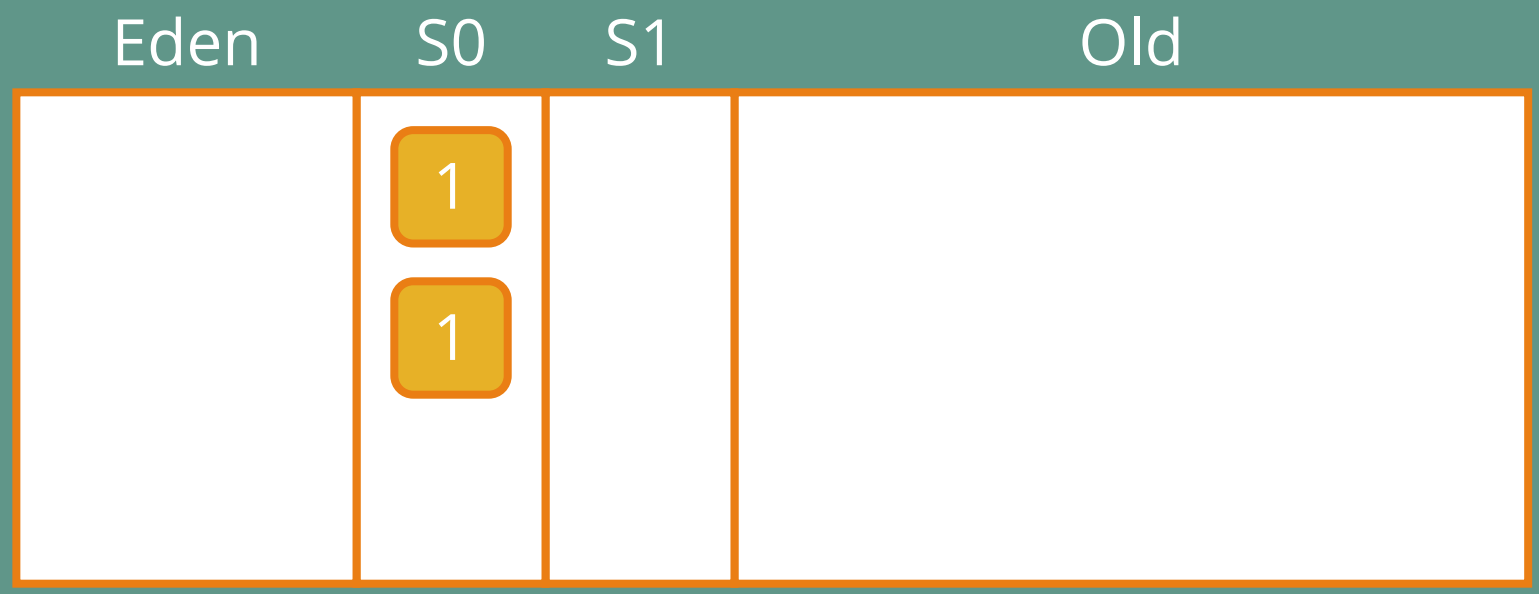
Serial GC



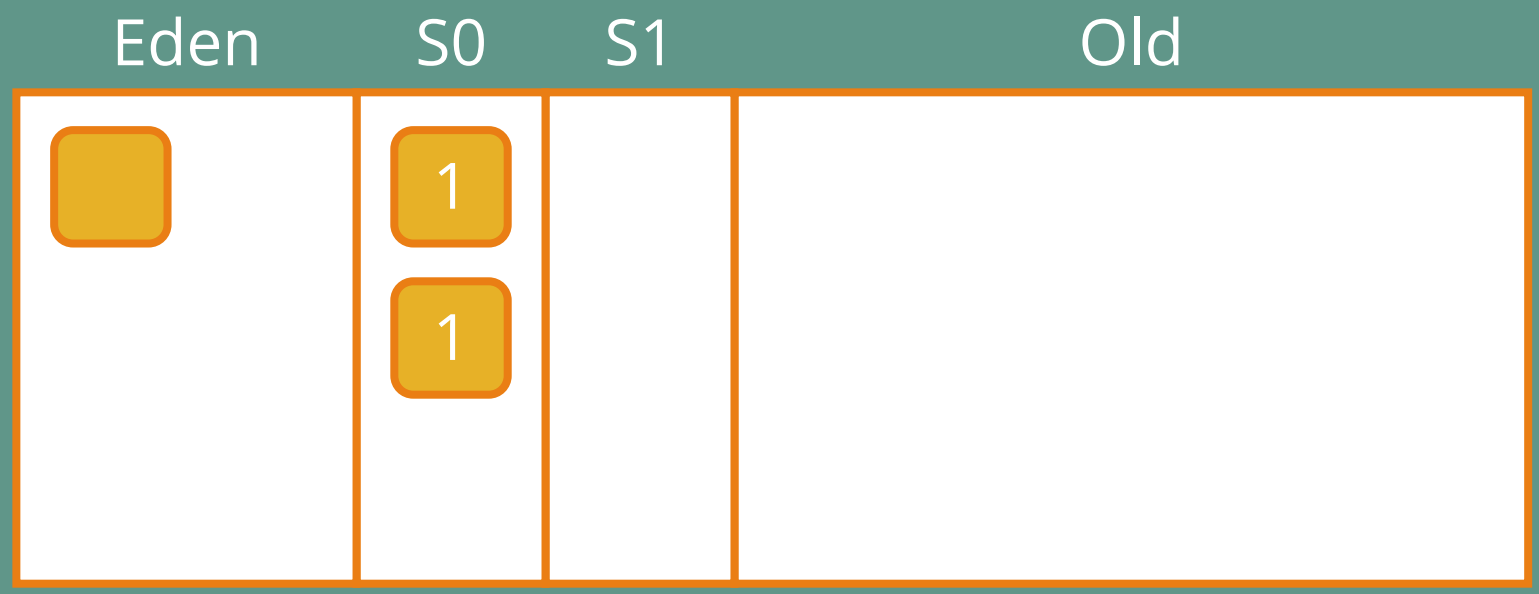
Serial GC



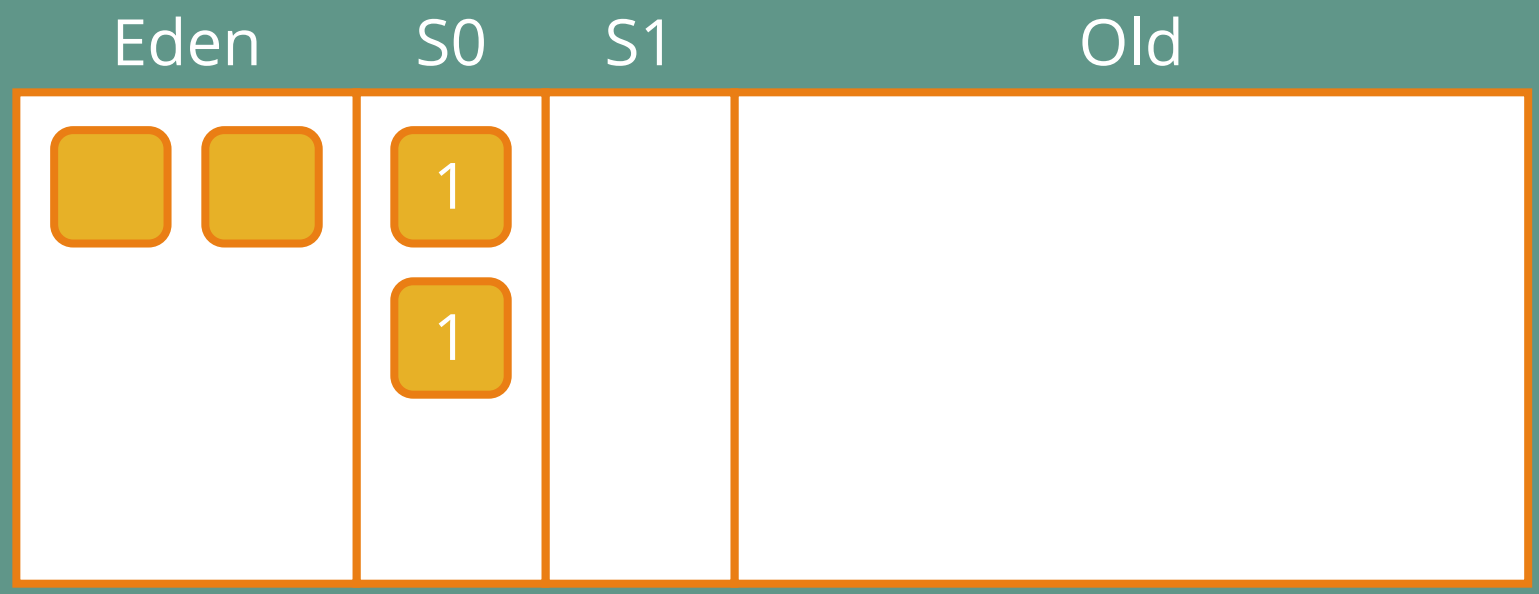
Serial GC



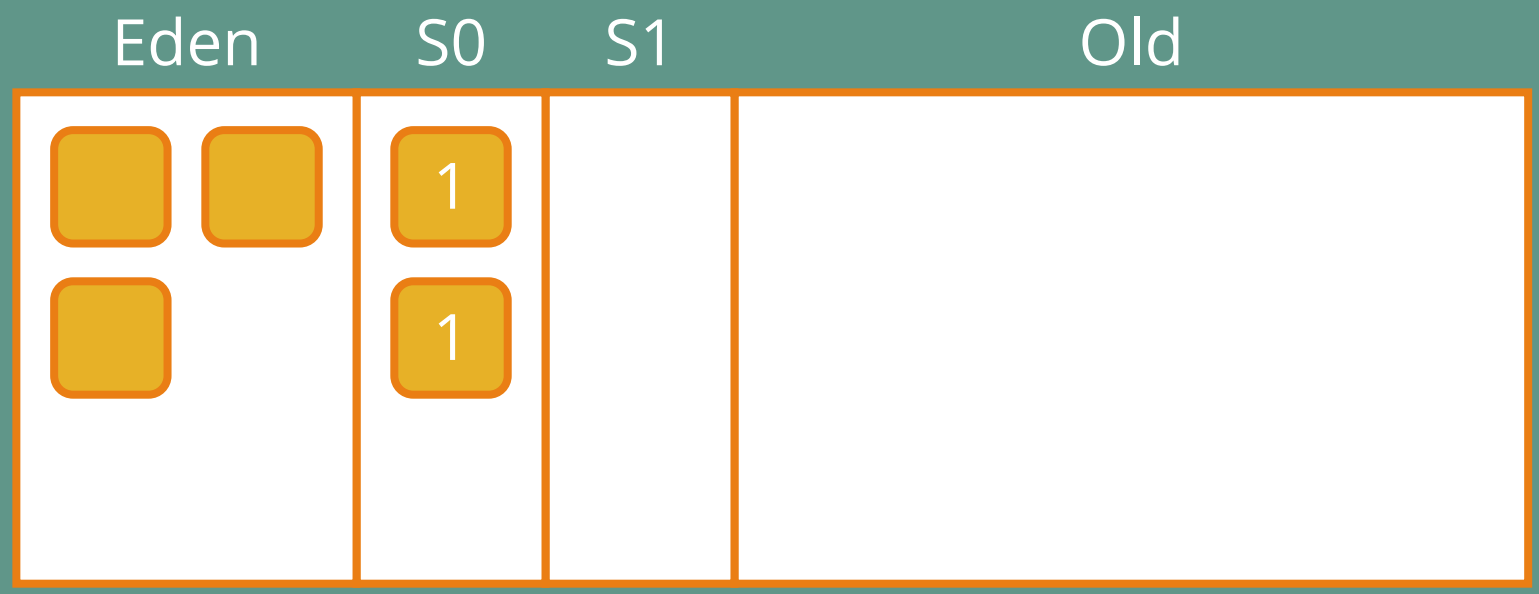
Serial GC



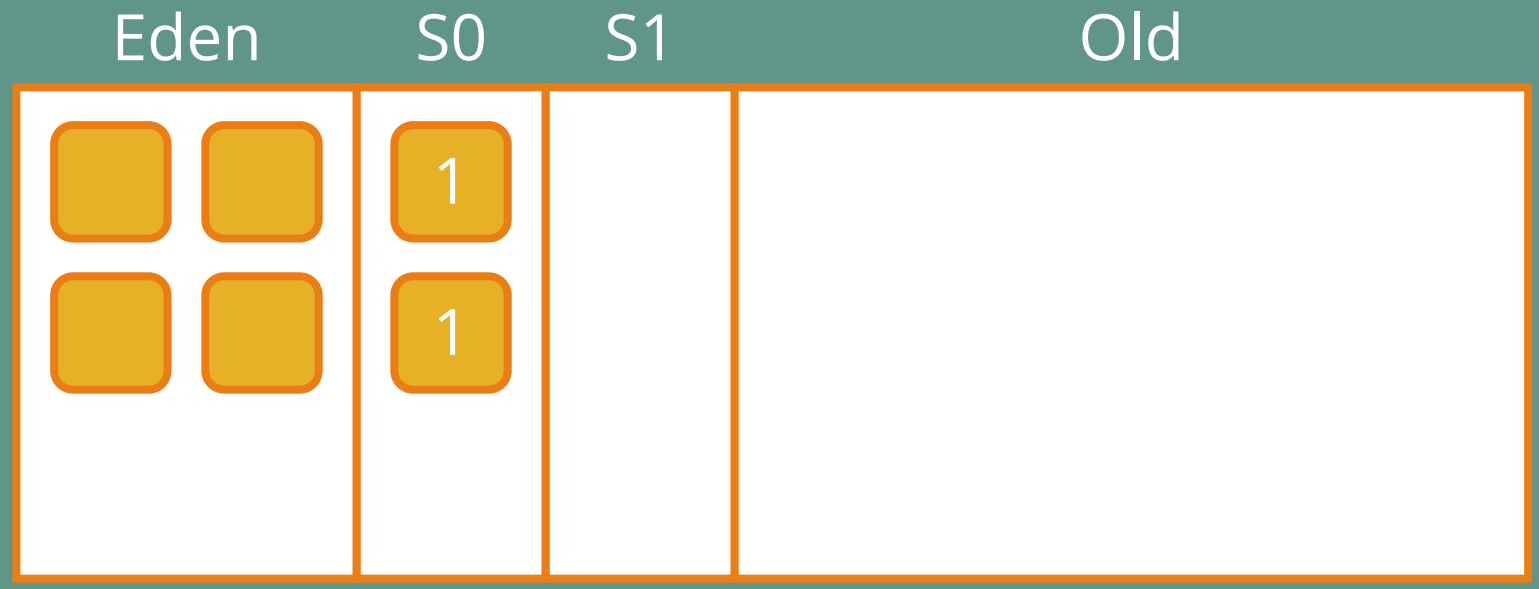
Serial GC



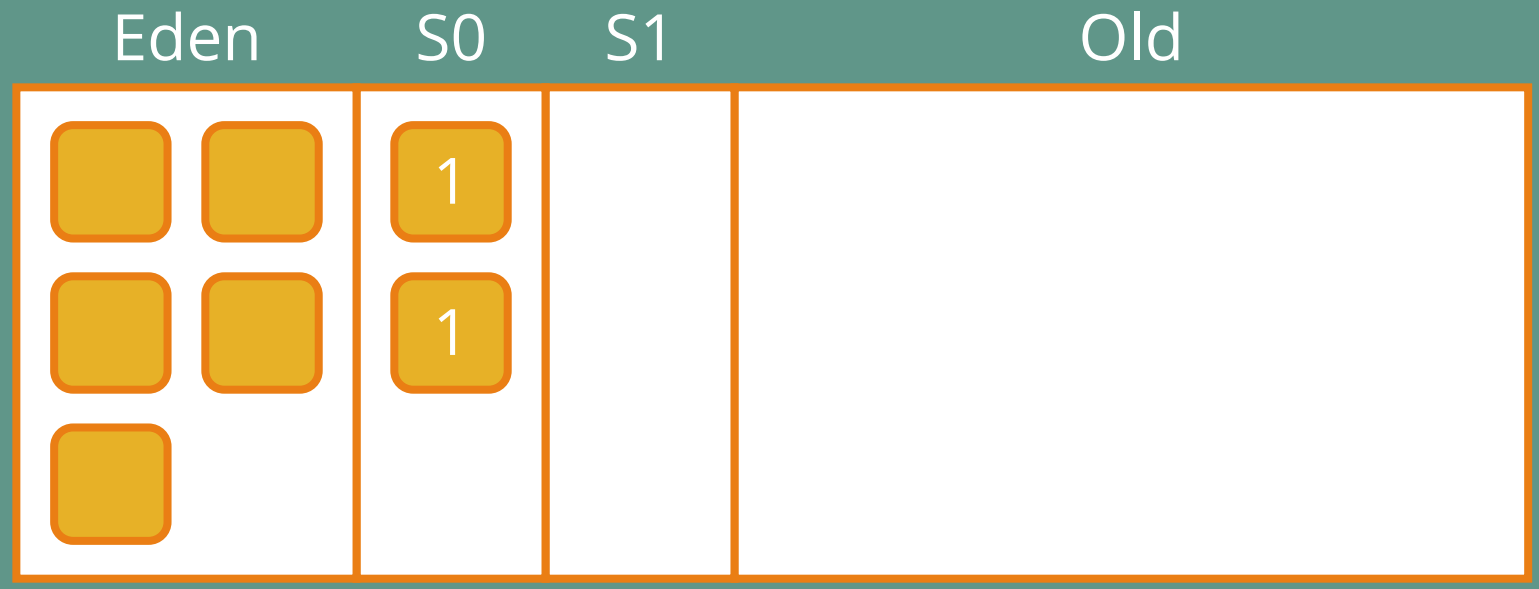
Serial GC



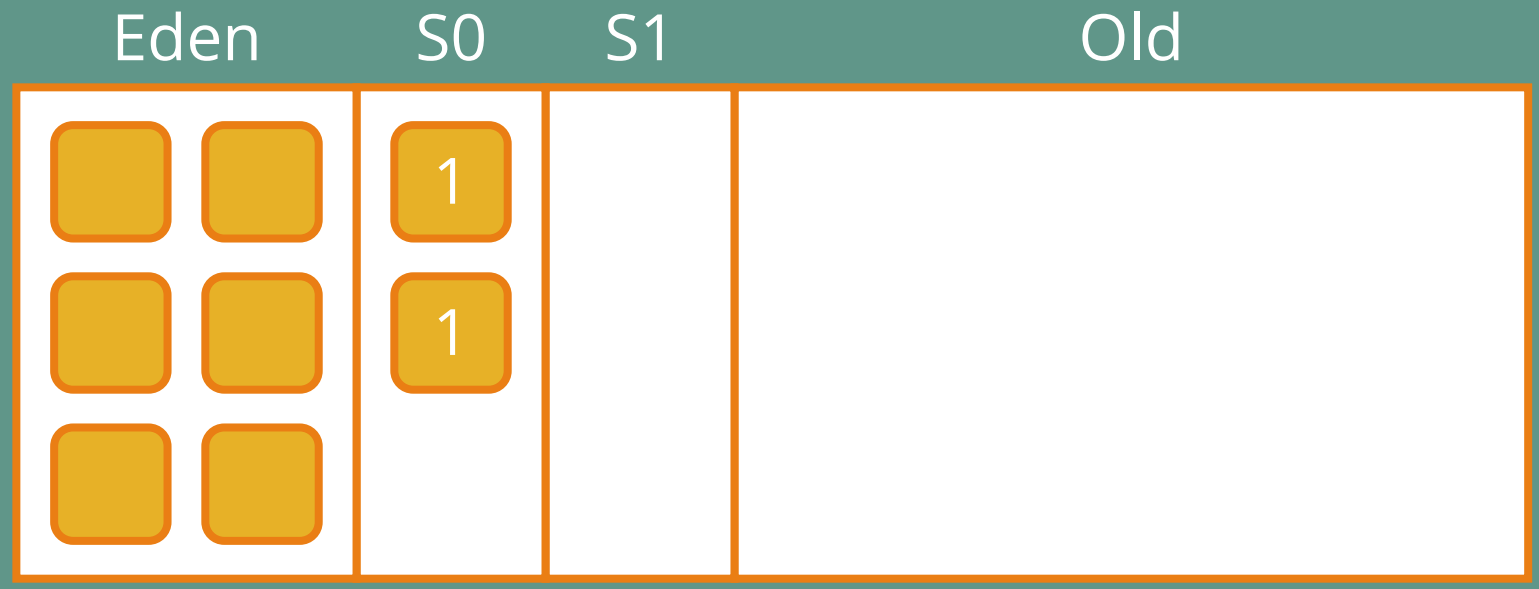
Serial GC



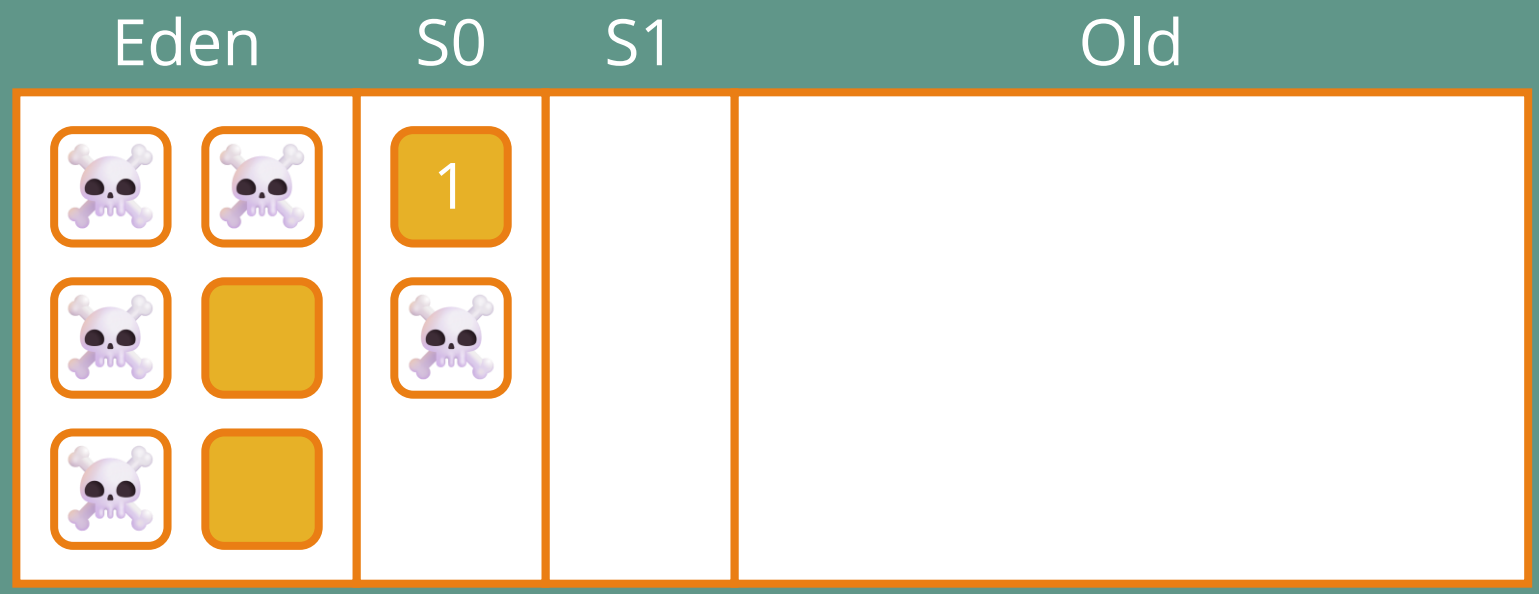
Serial GC



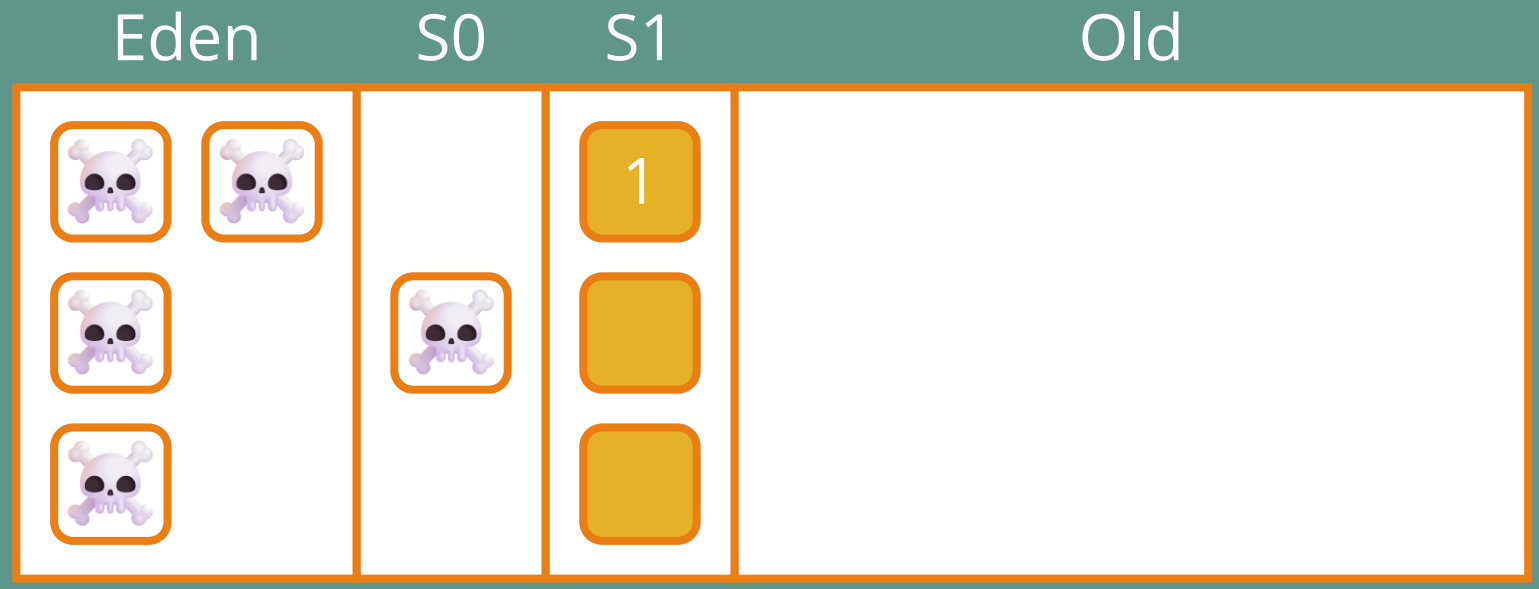
Serial GC

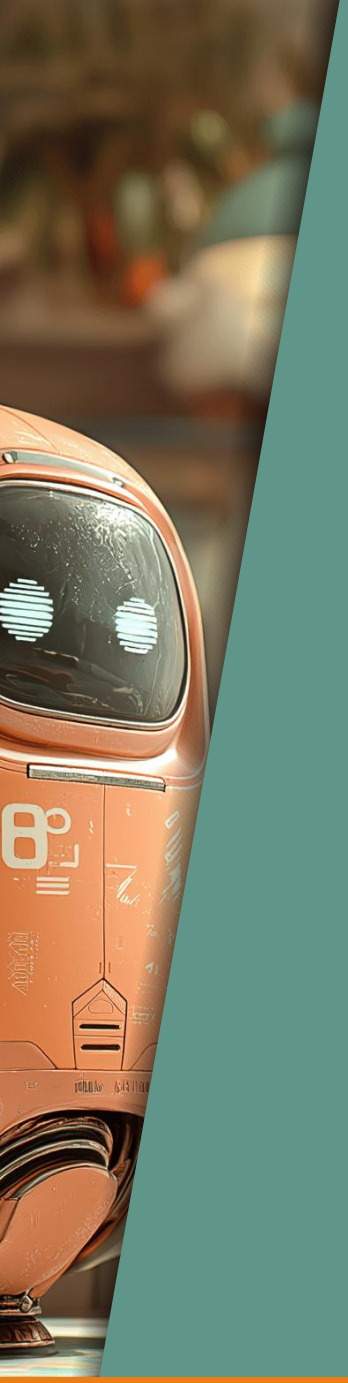


Serial GC

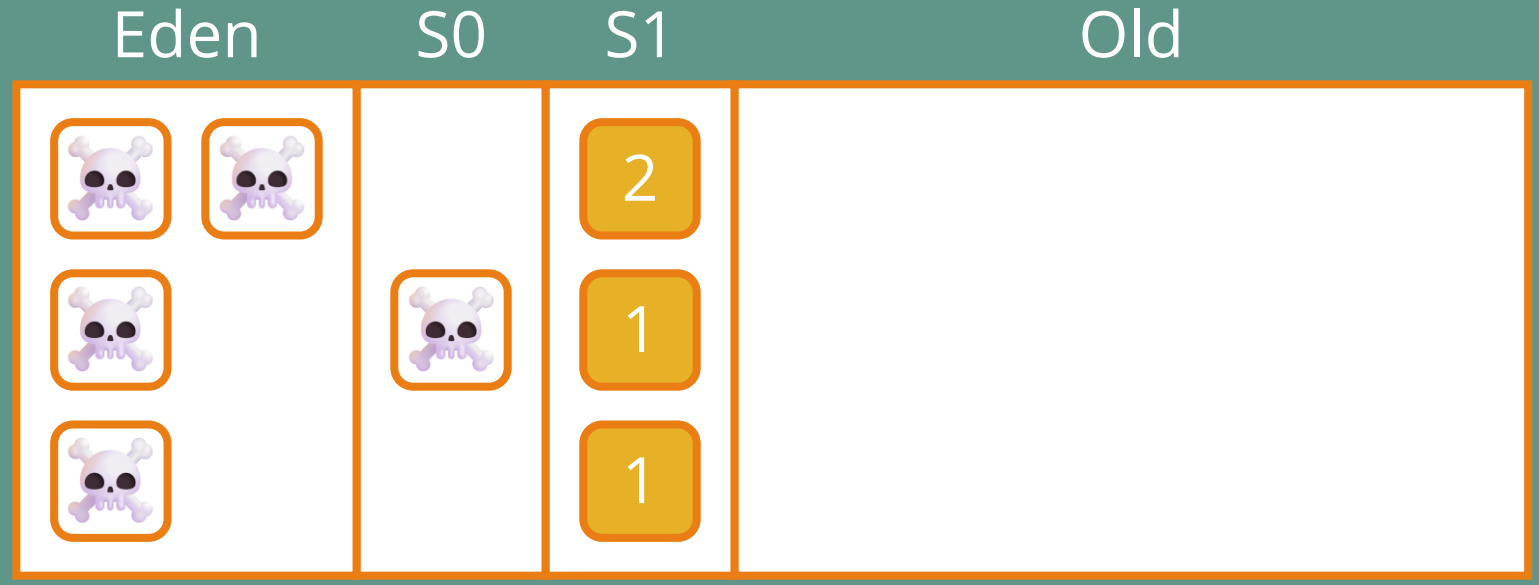


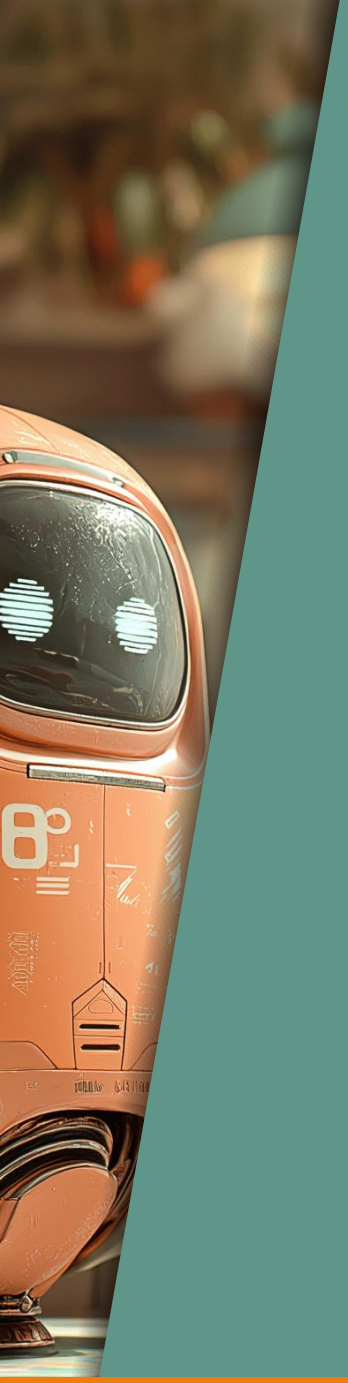
Serial GC



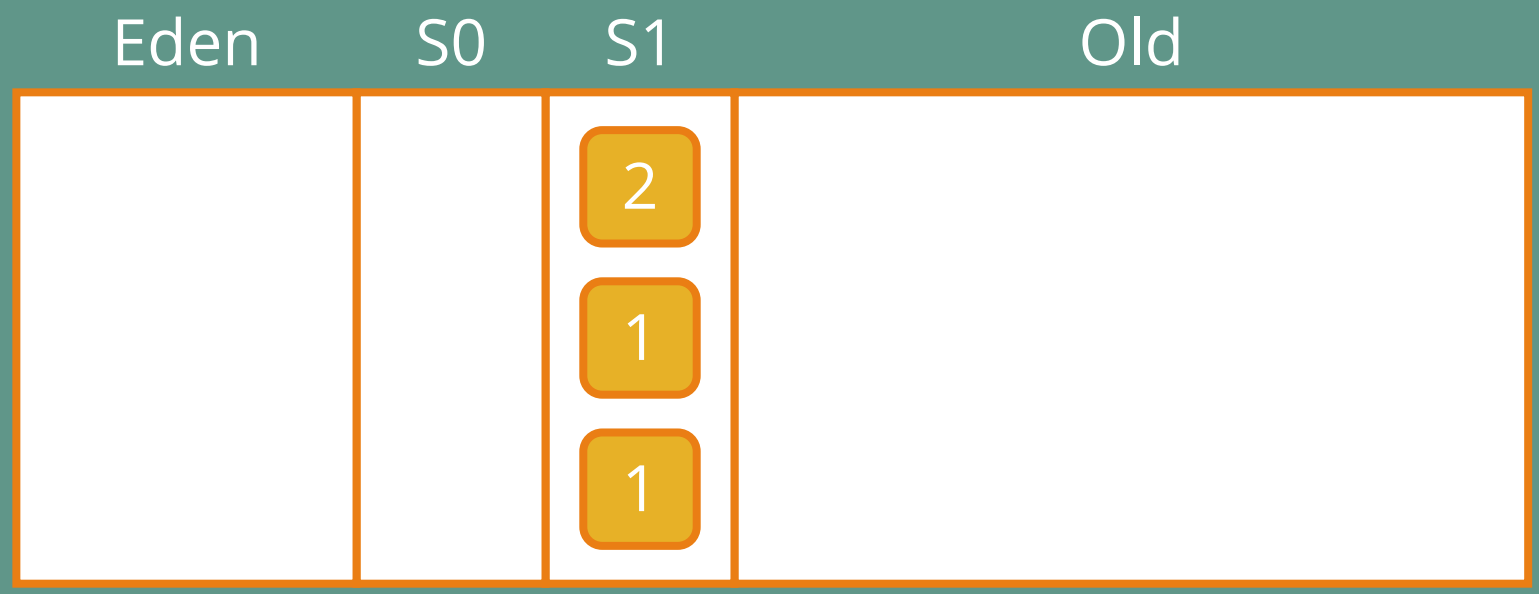


Serial GC

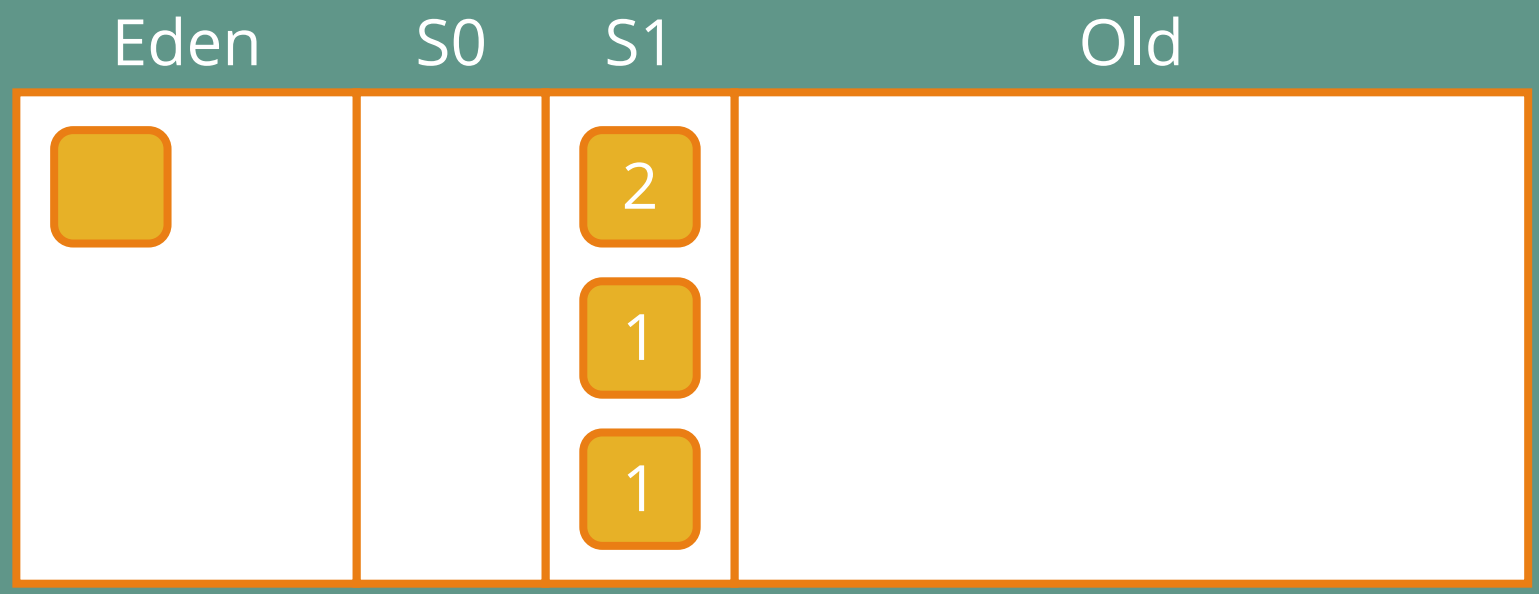




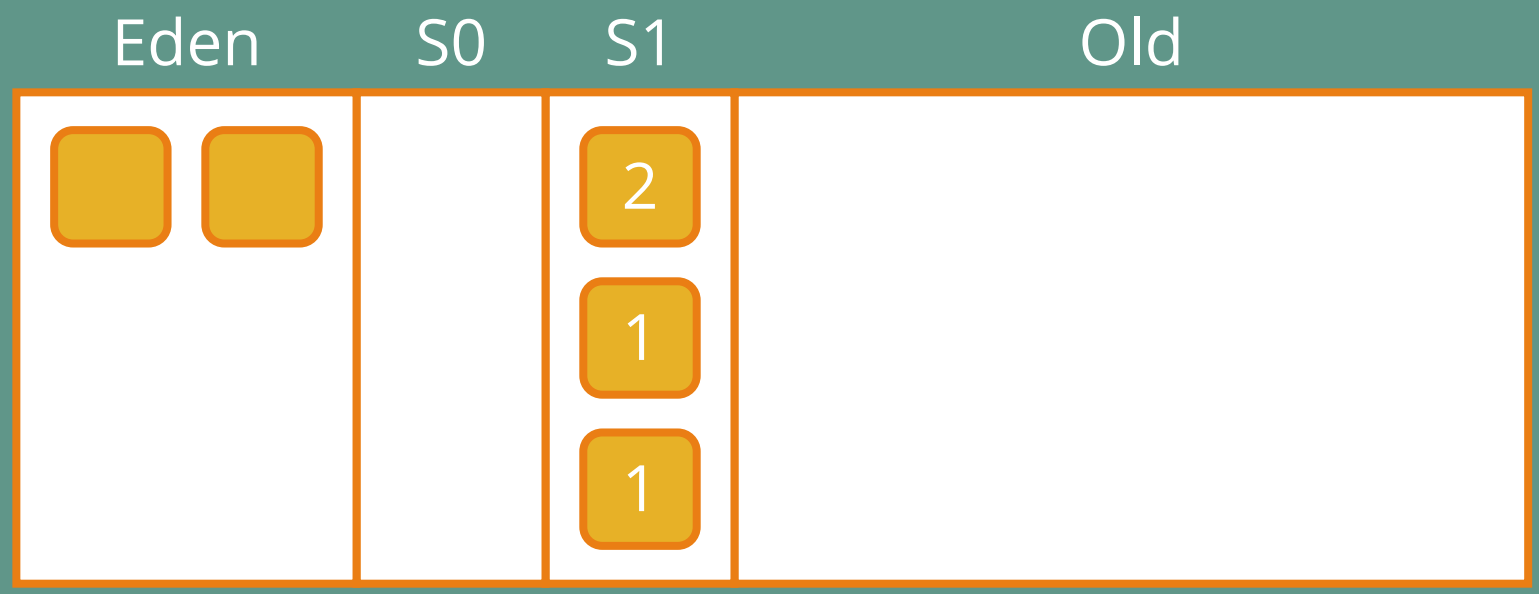
Serial GC



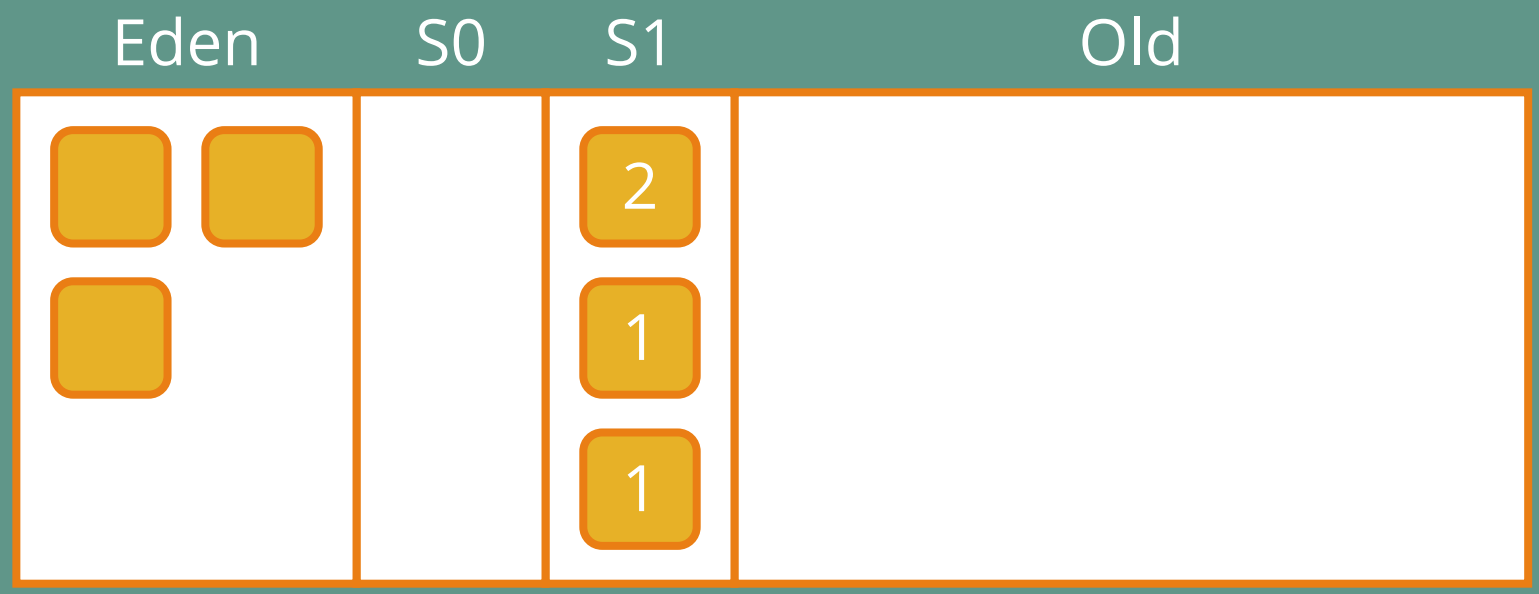
Serial GC

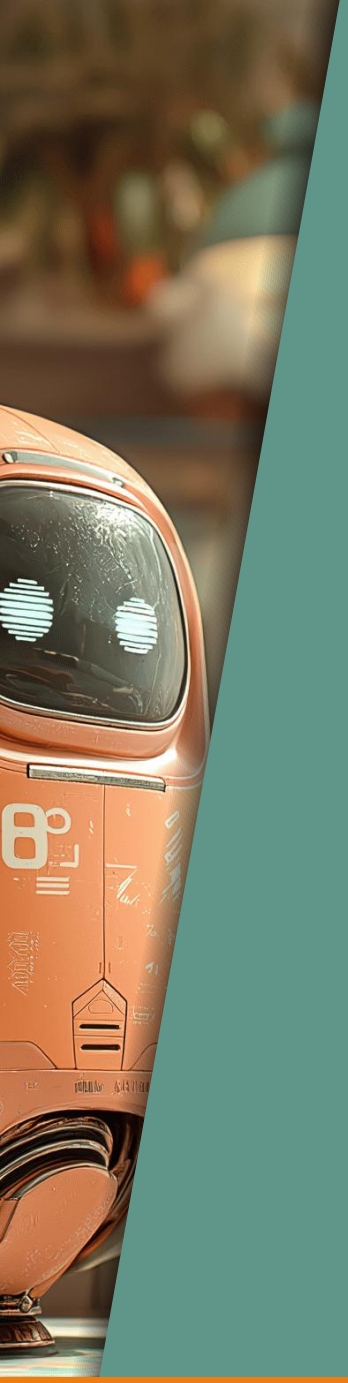


Serial GC

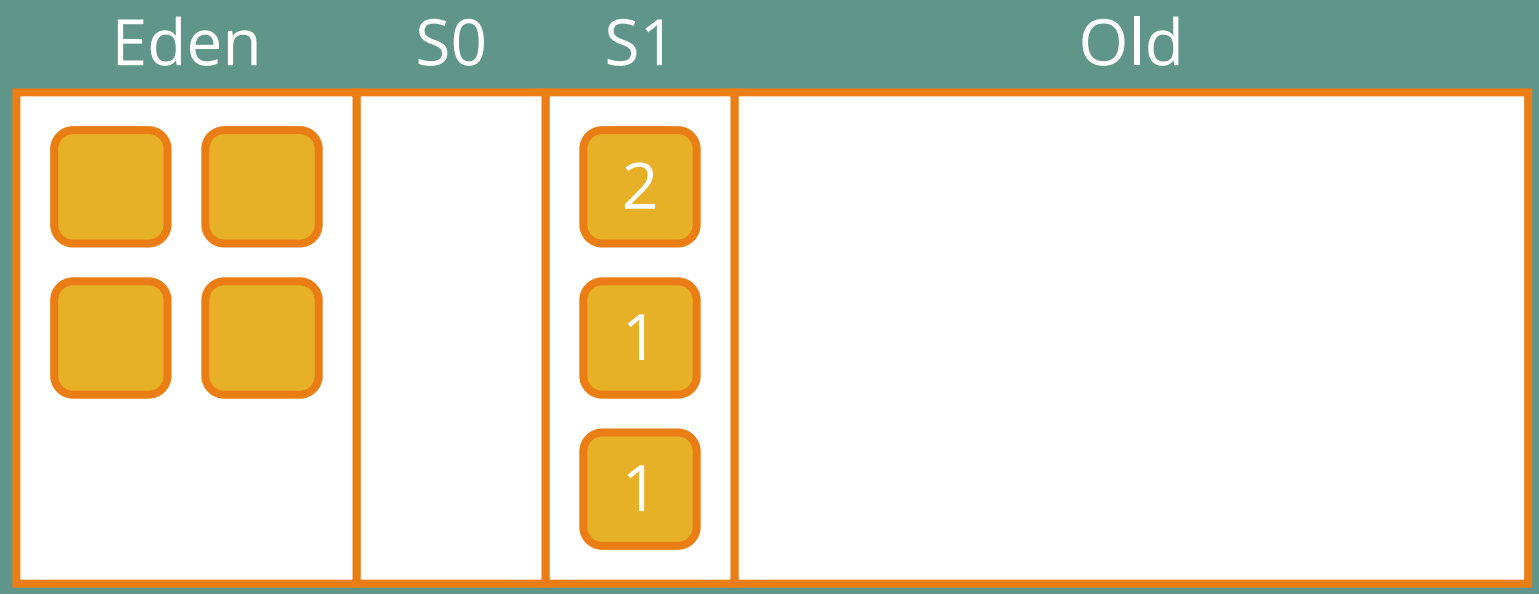


Serial GC

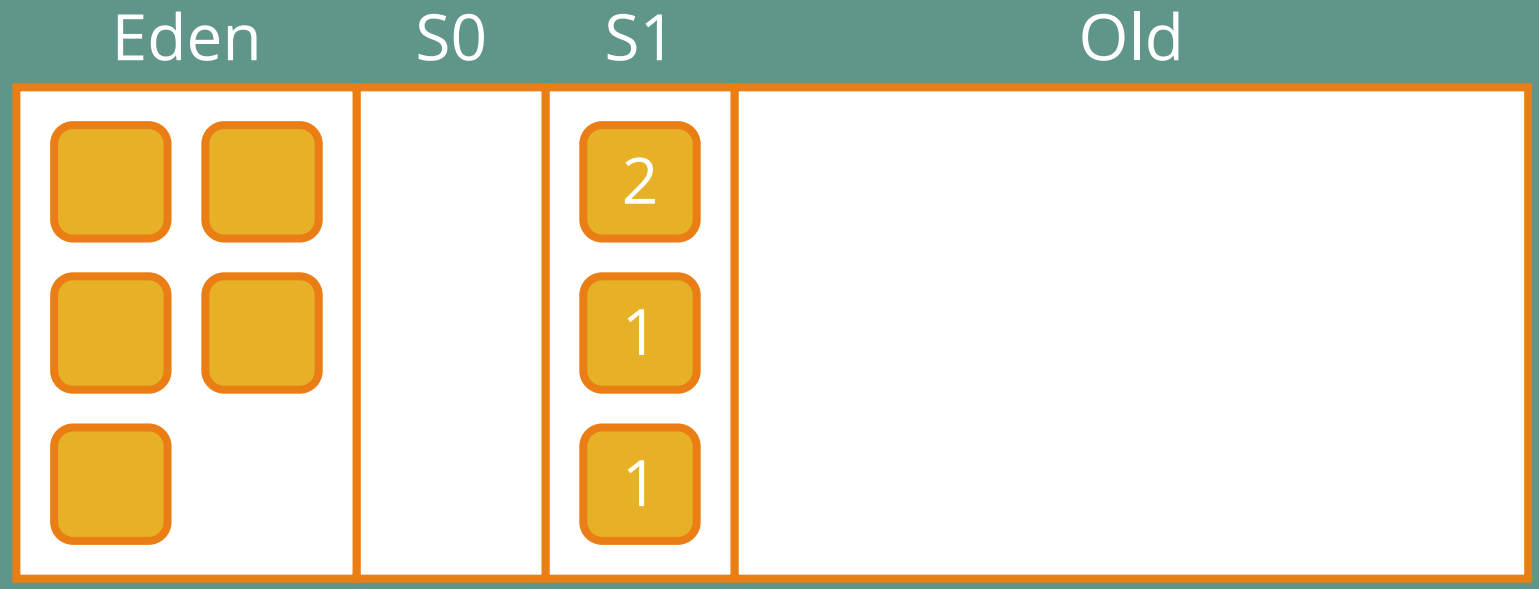




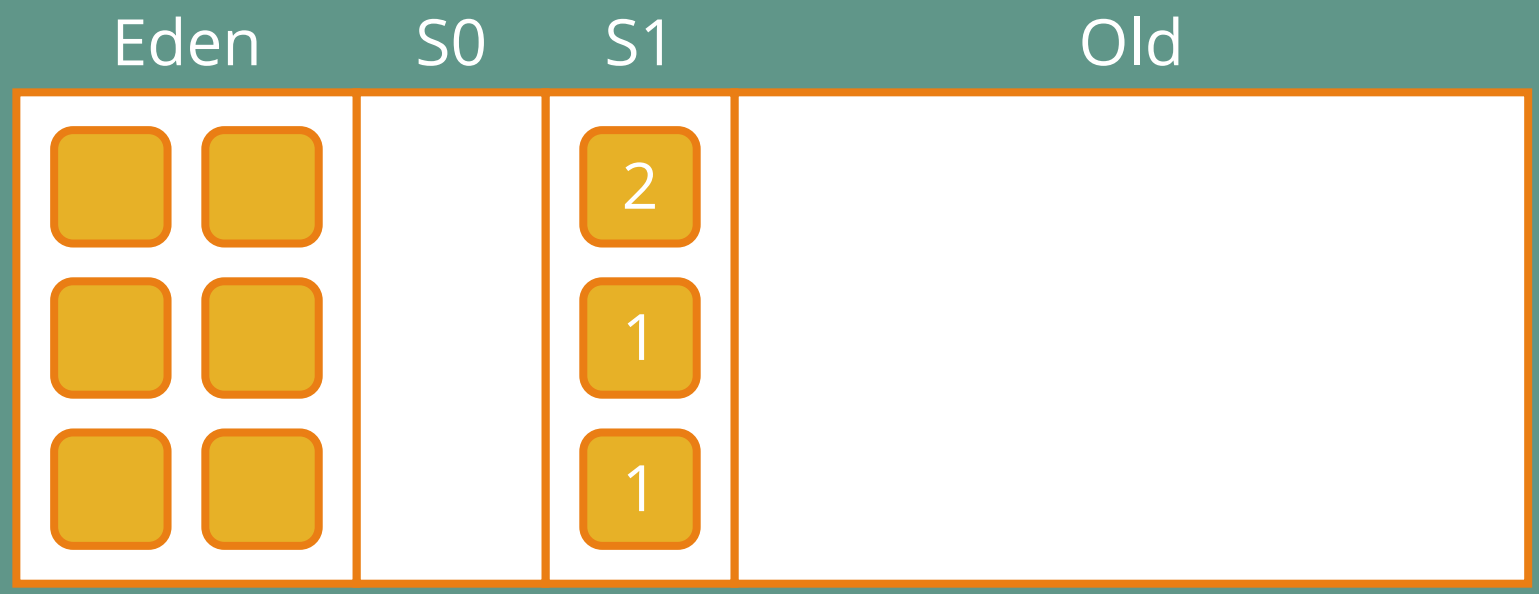
Serial GC



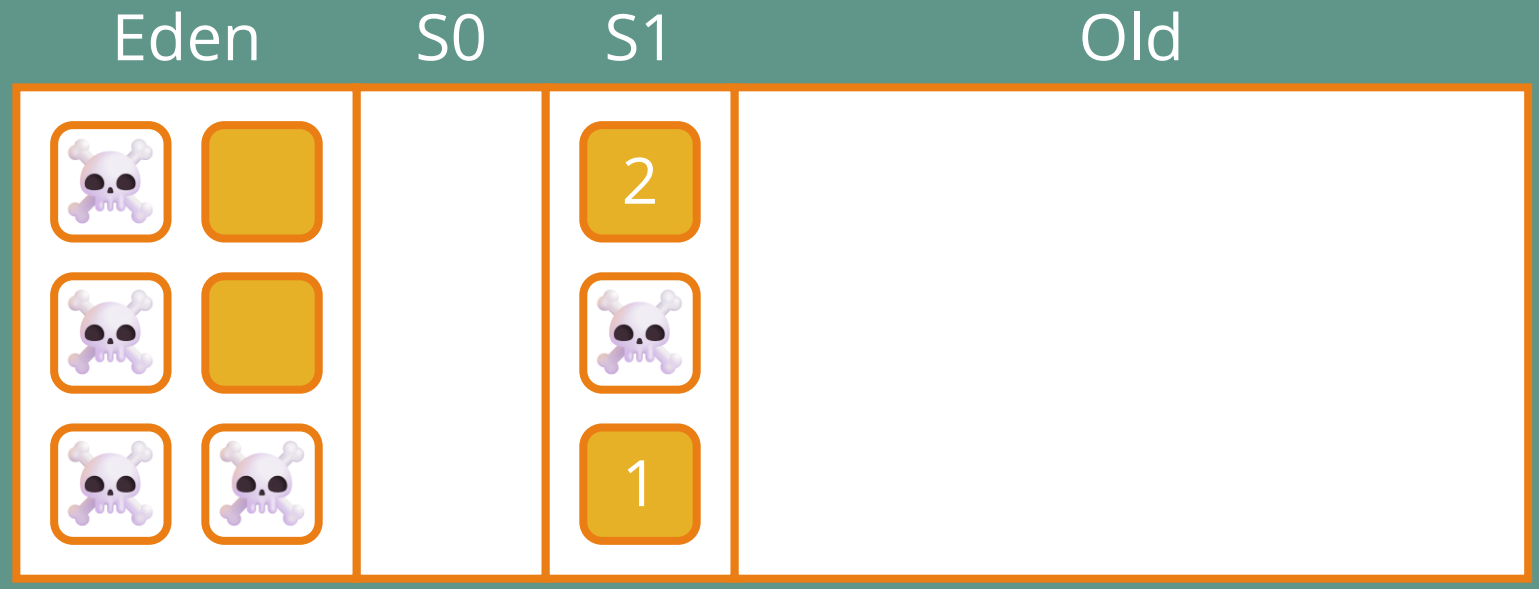
Serial GC



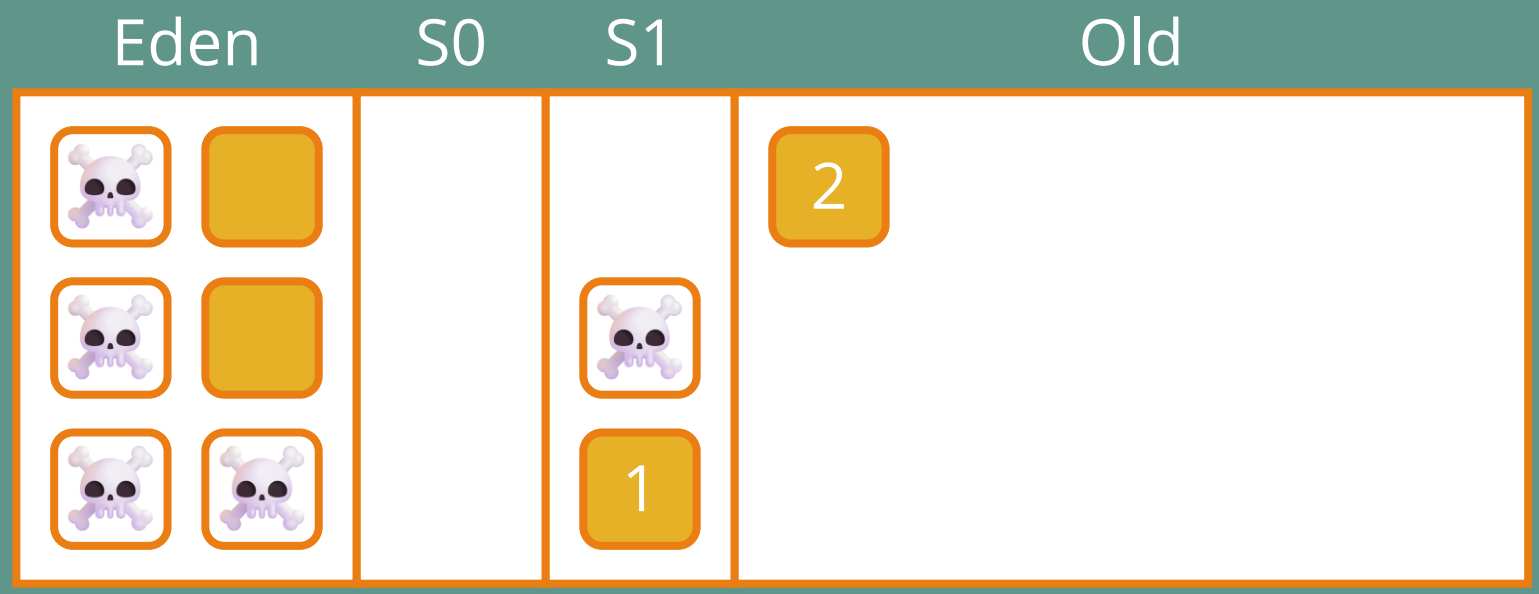
Serial GC



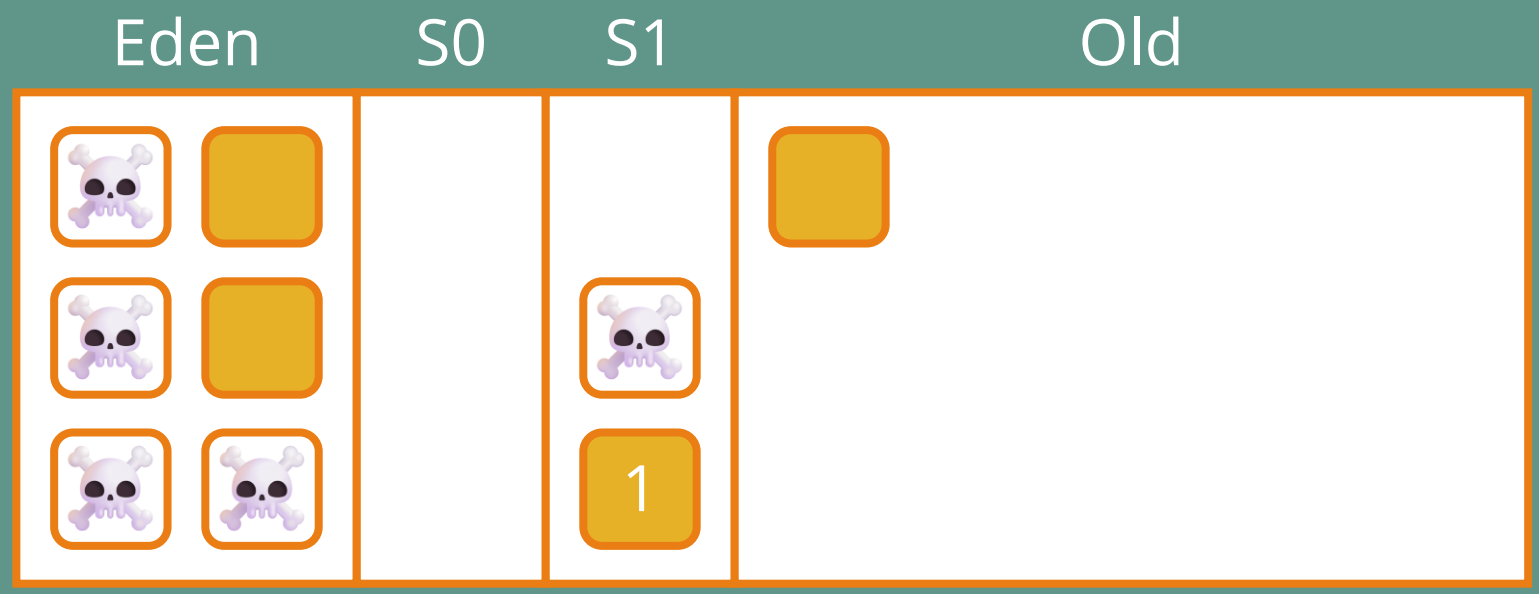
Serial GC



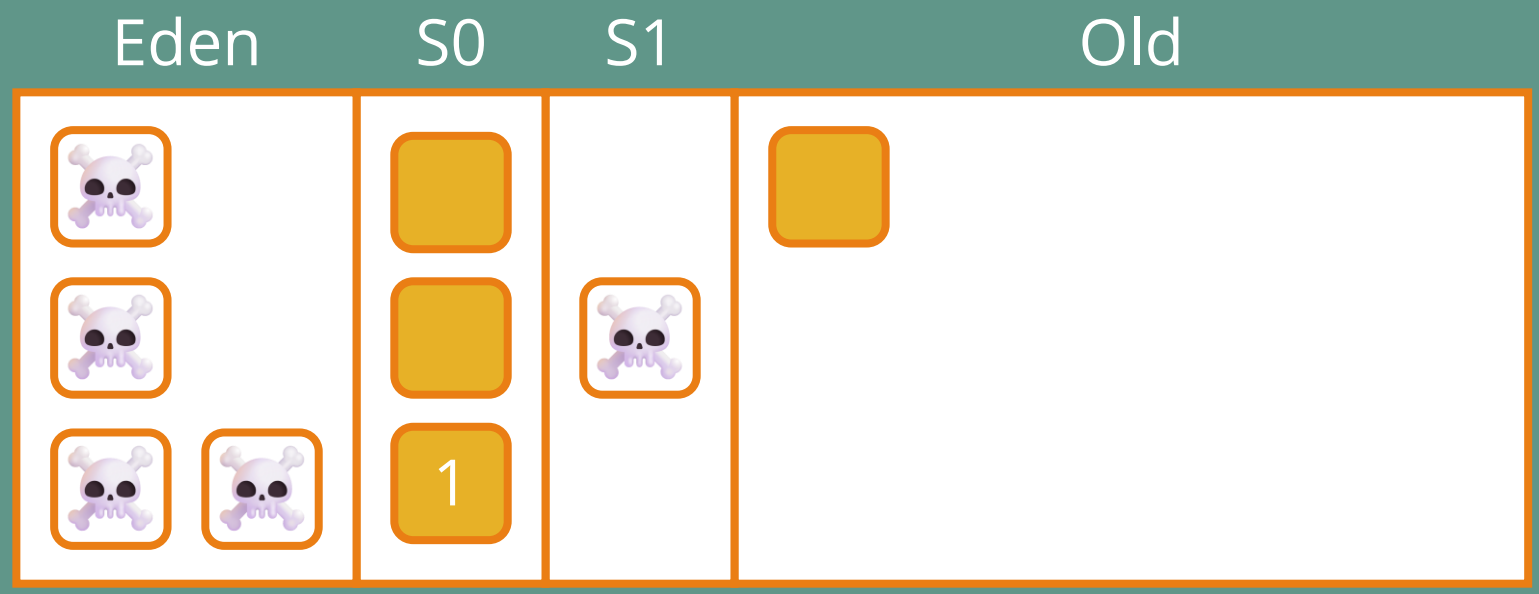
Serial GC



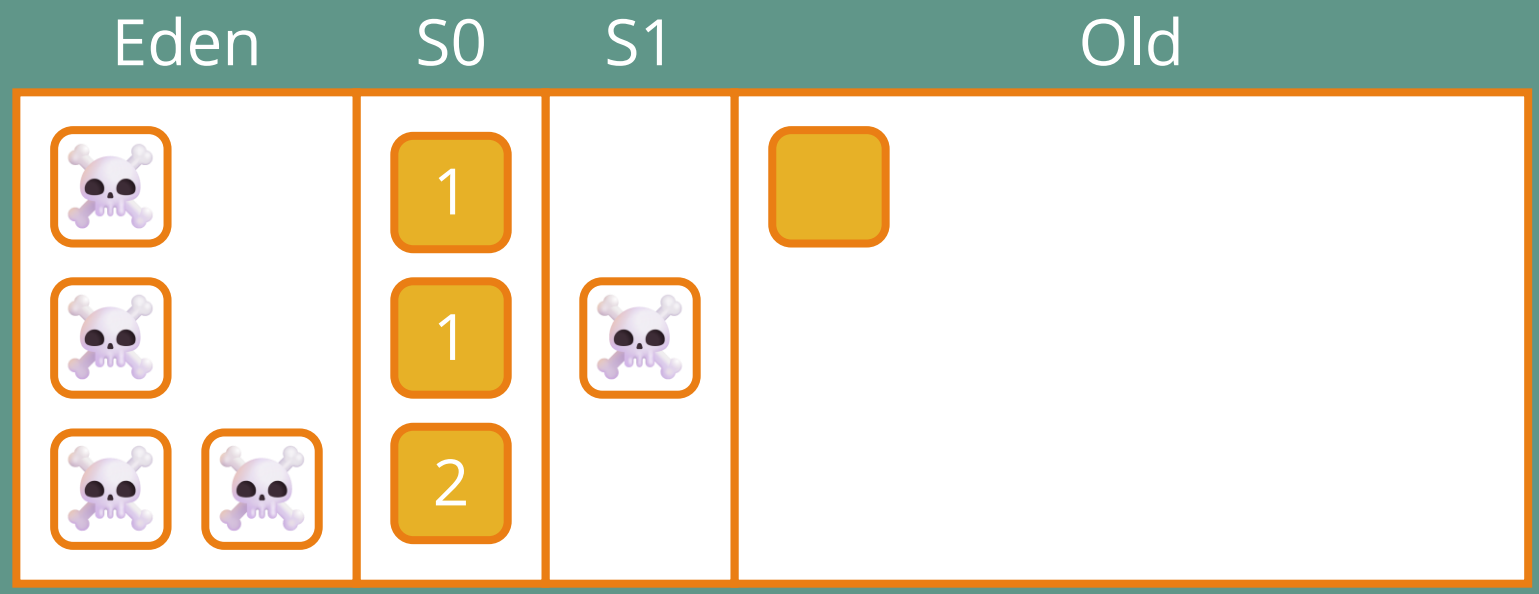
Serial GC

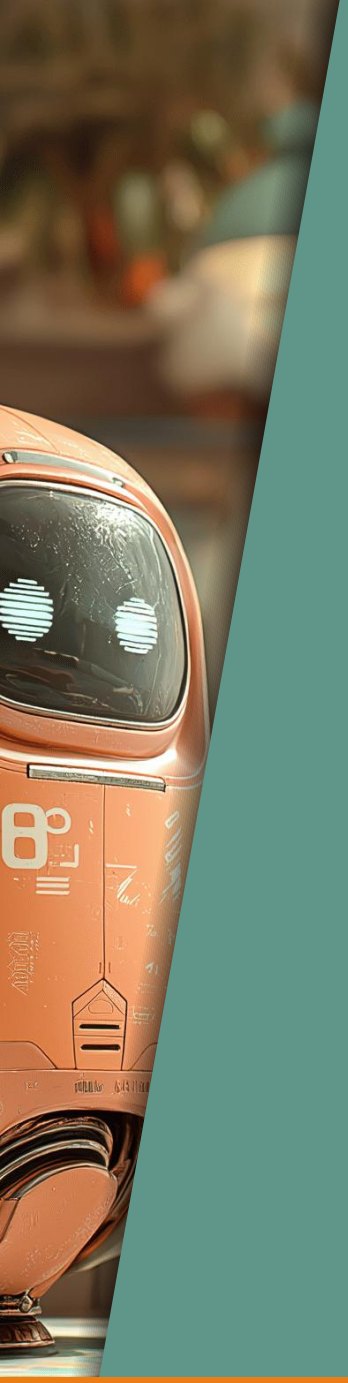


Serial GC

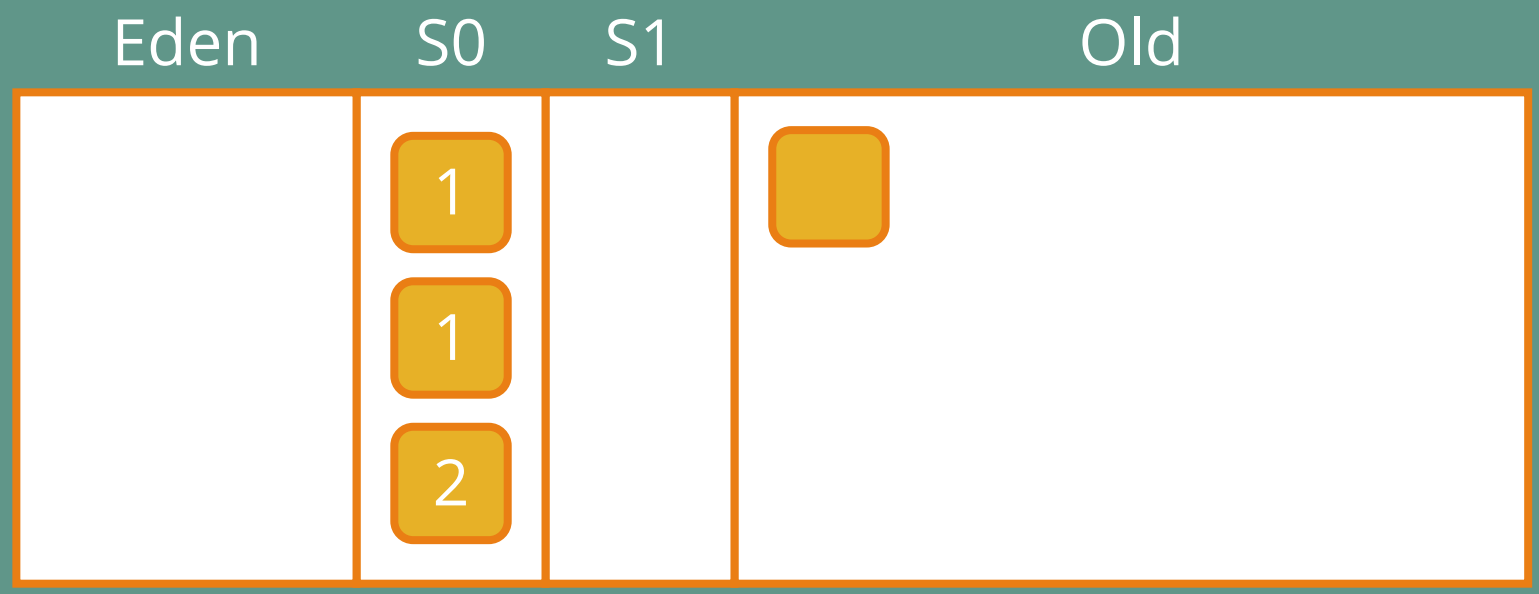


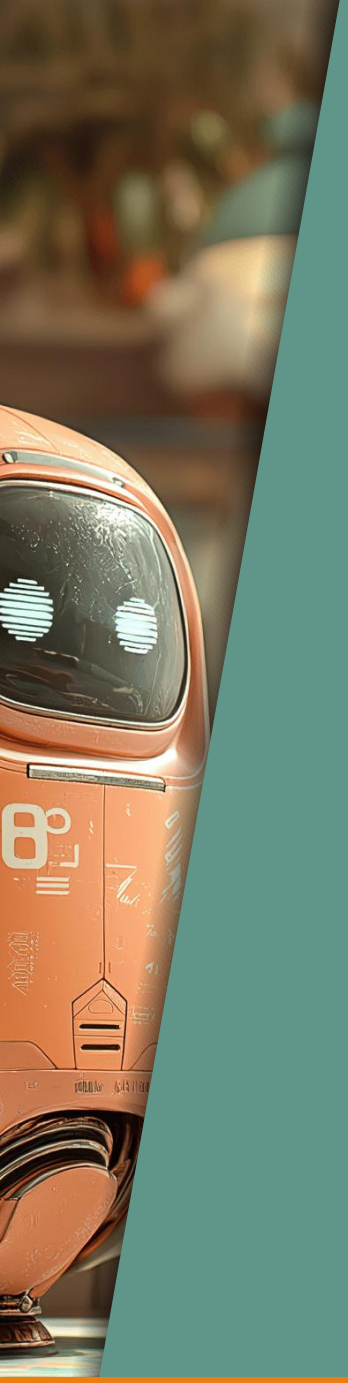
Serial GC



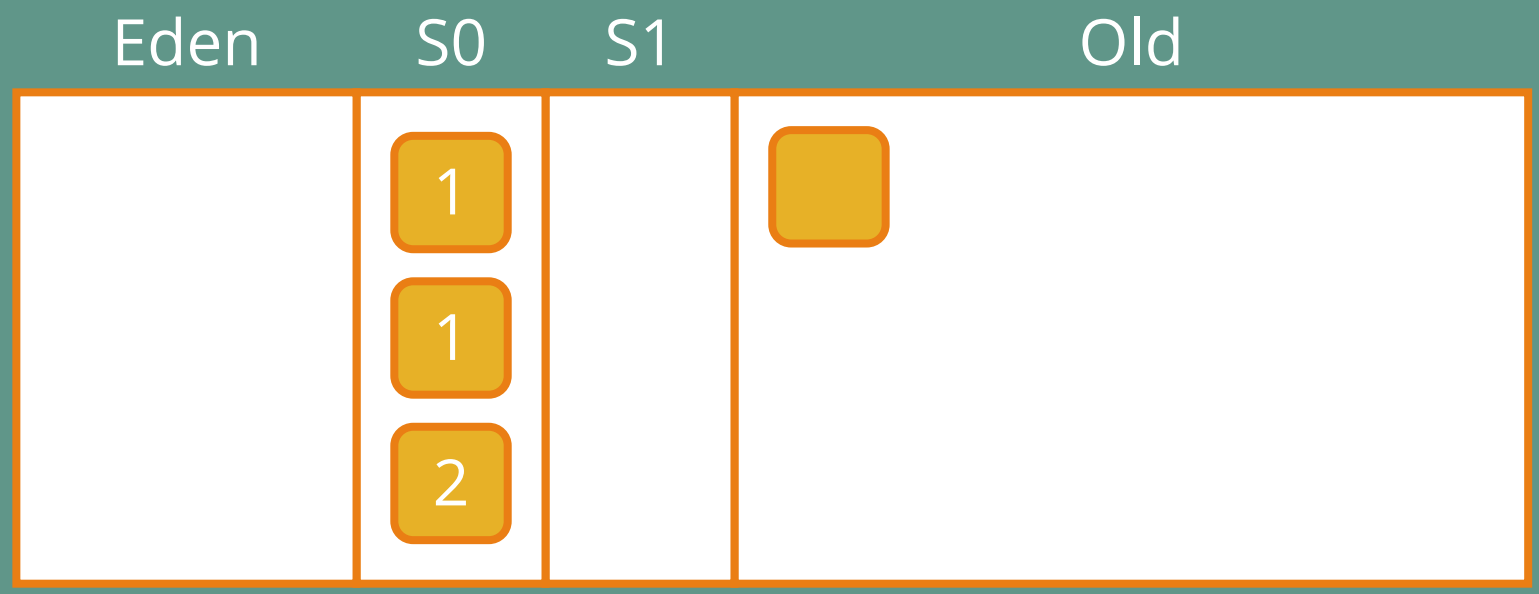


Serial GC



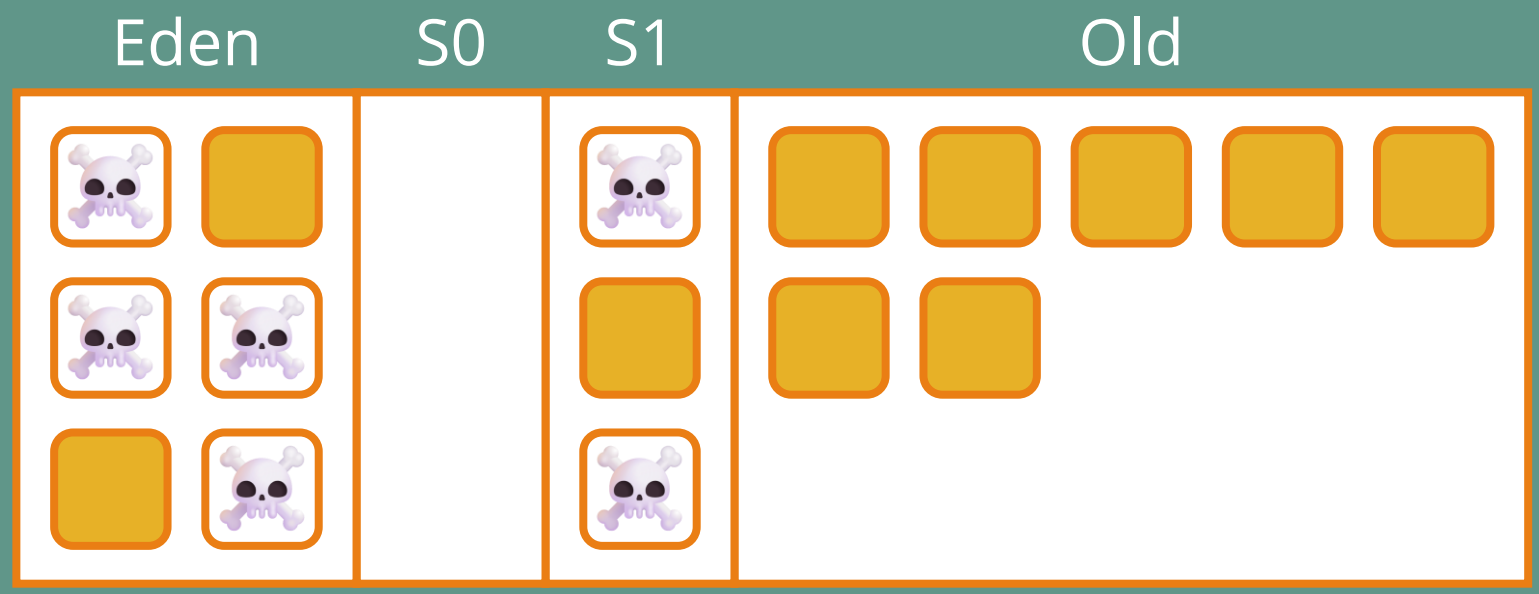


Serial GC

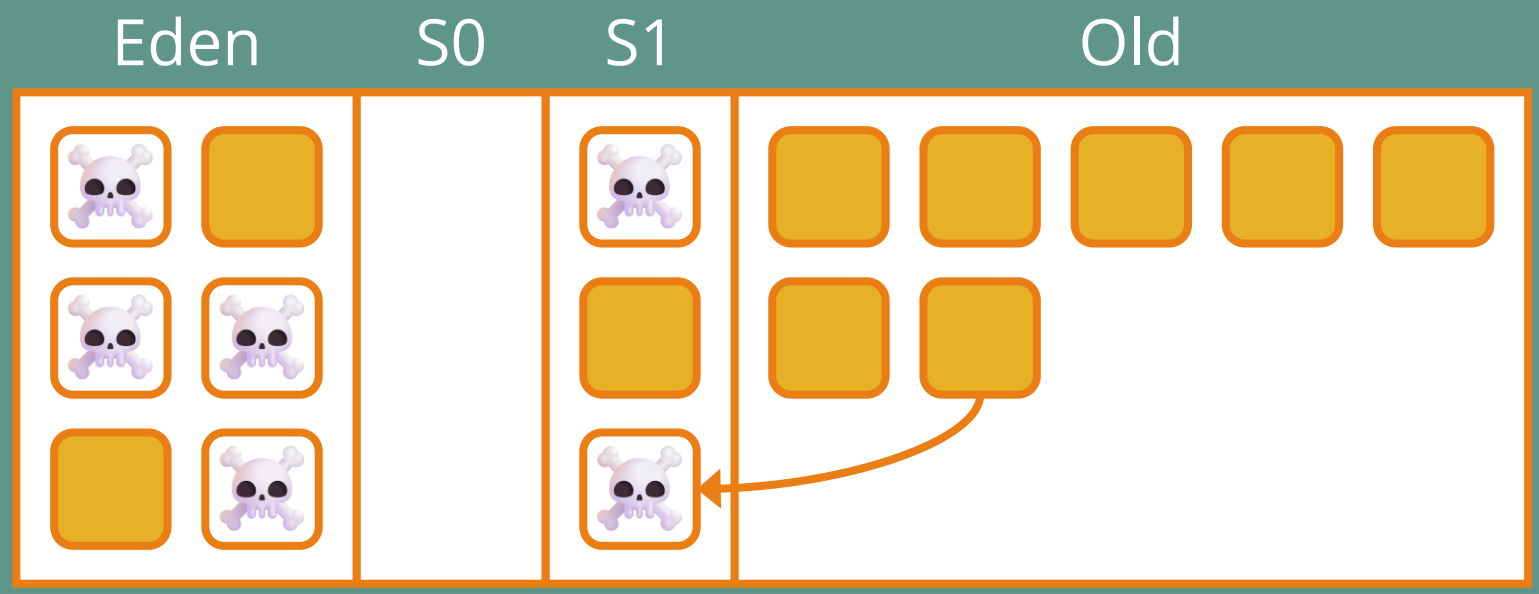


(et ça continue...)

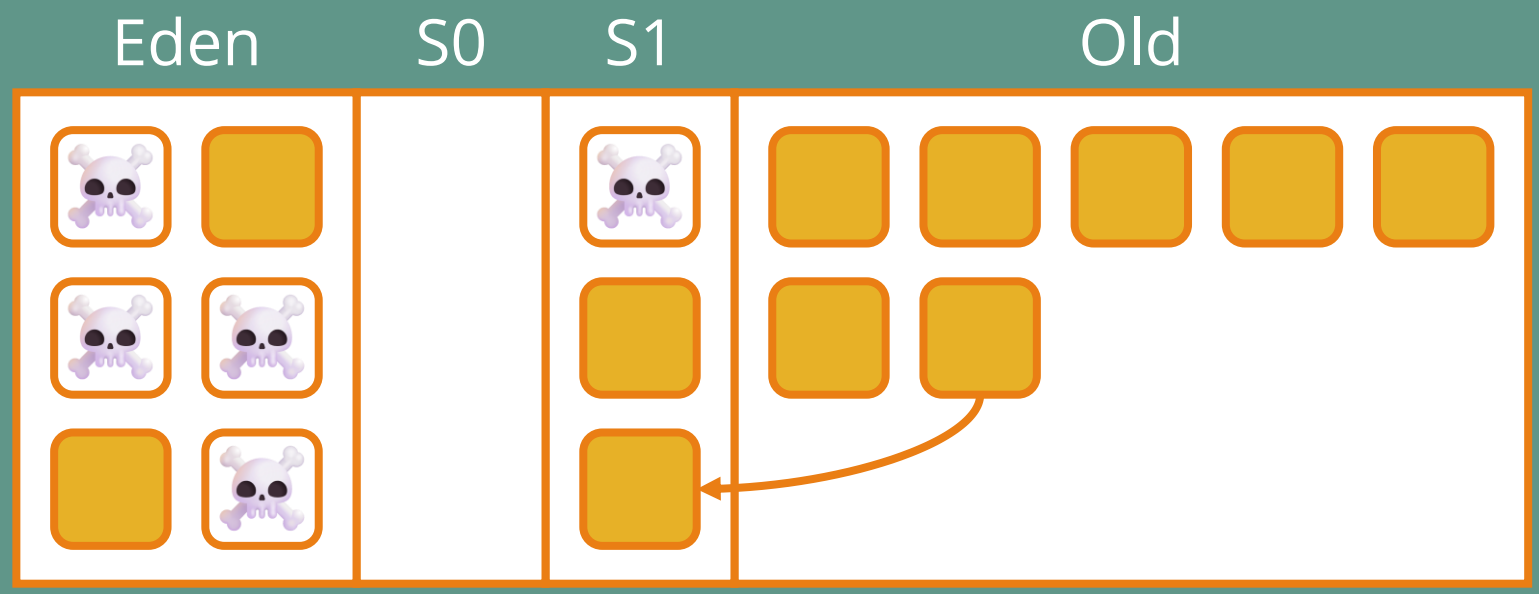
Serial GC



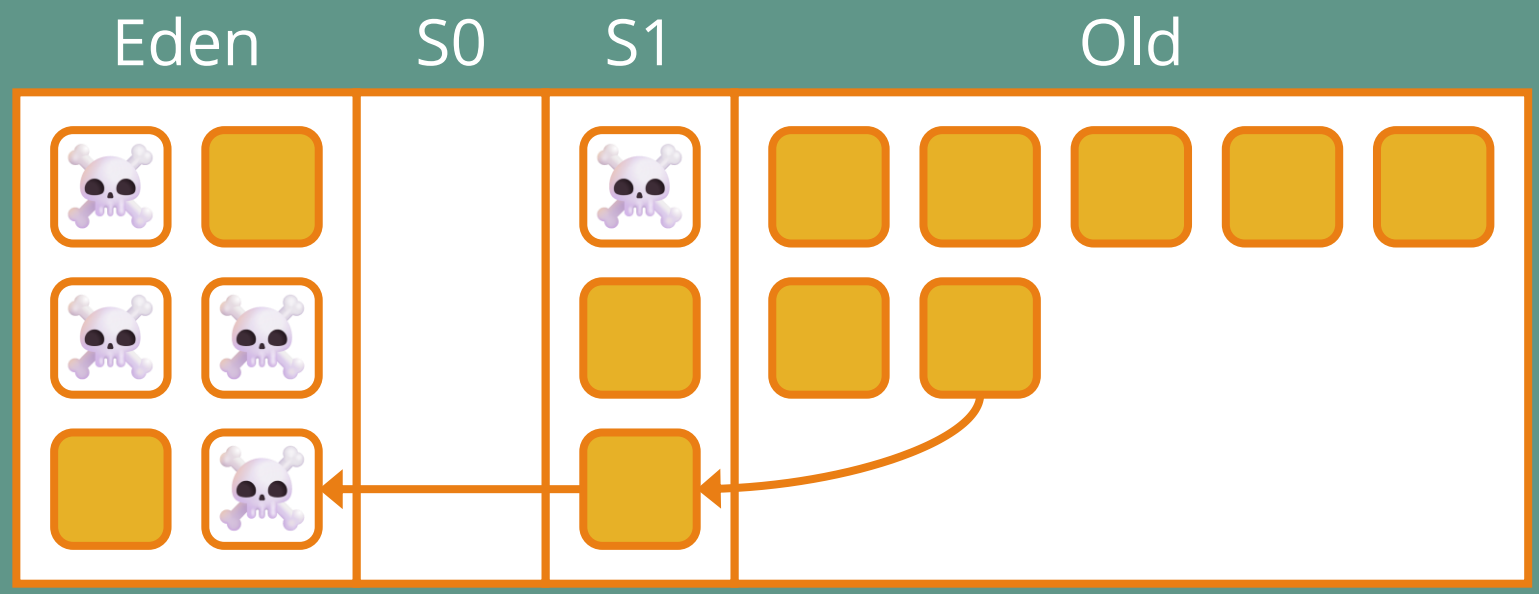
Serial GC



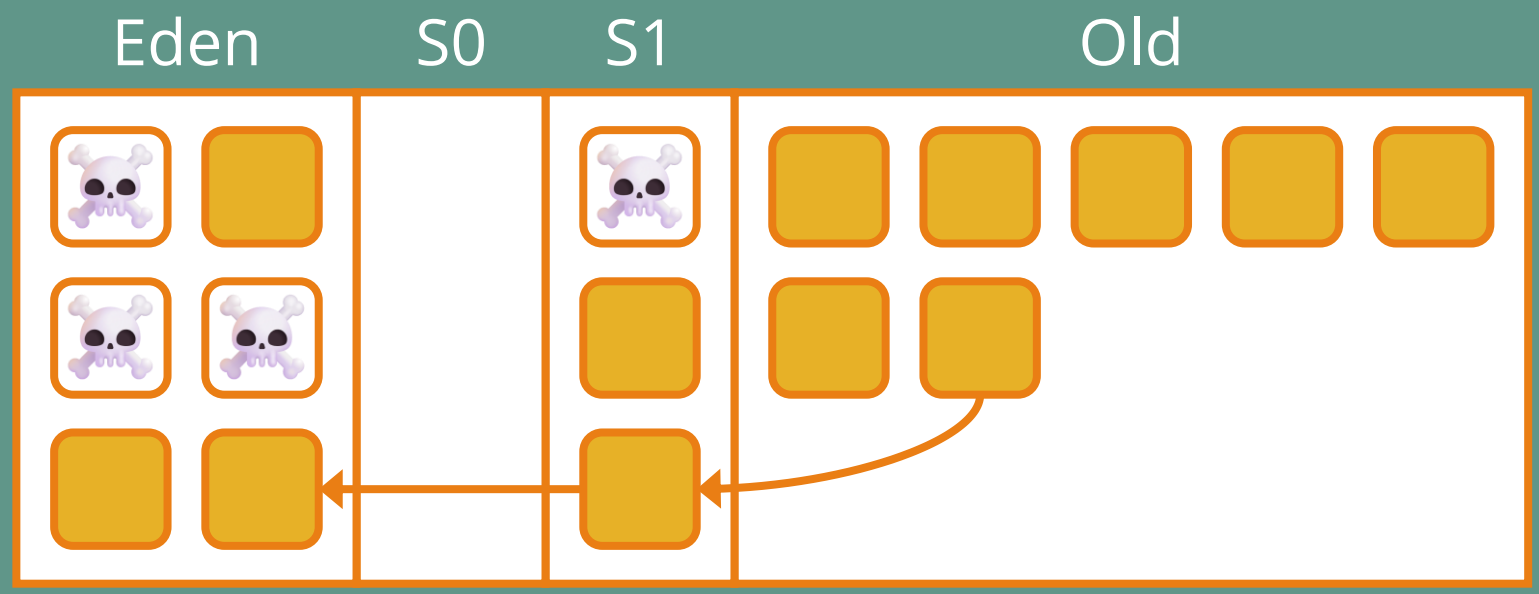
Serial GC



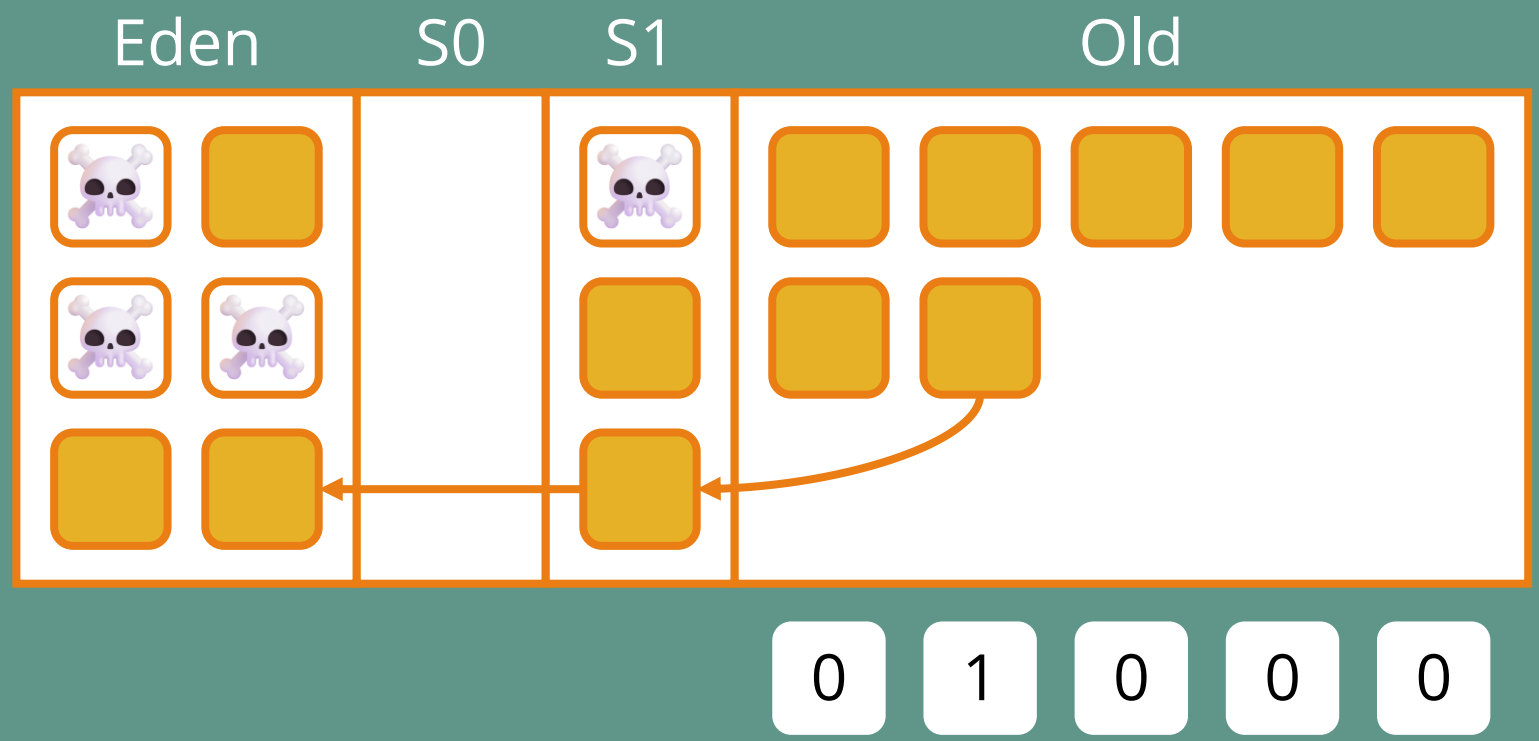
Serial GC

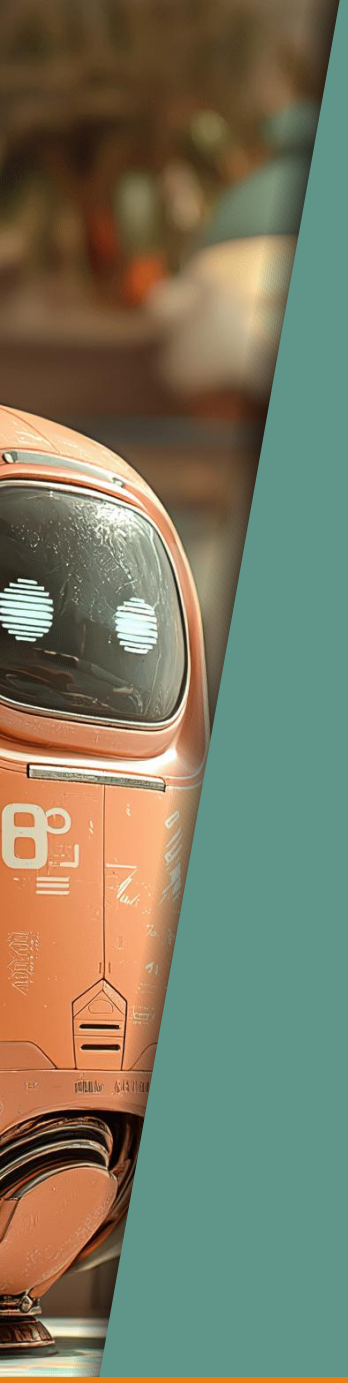


Serial GC



Serial GC





Serial GC – Write barrier

```
cache.addValue(newValue);
```

Serial GC – Write barrier

```
cache.addValue(newValue);

if (isOldObject(cache) && !isOldObject(newValue)) {

}
```

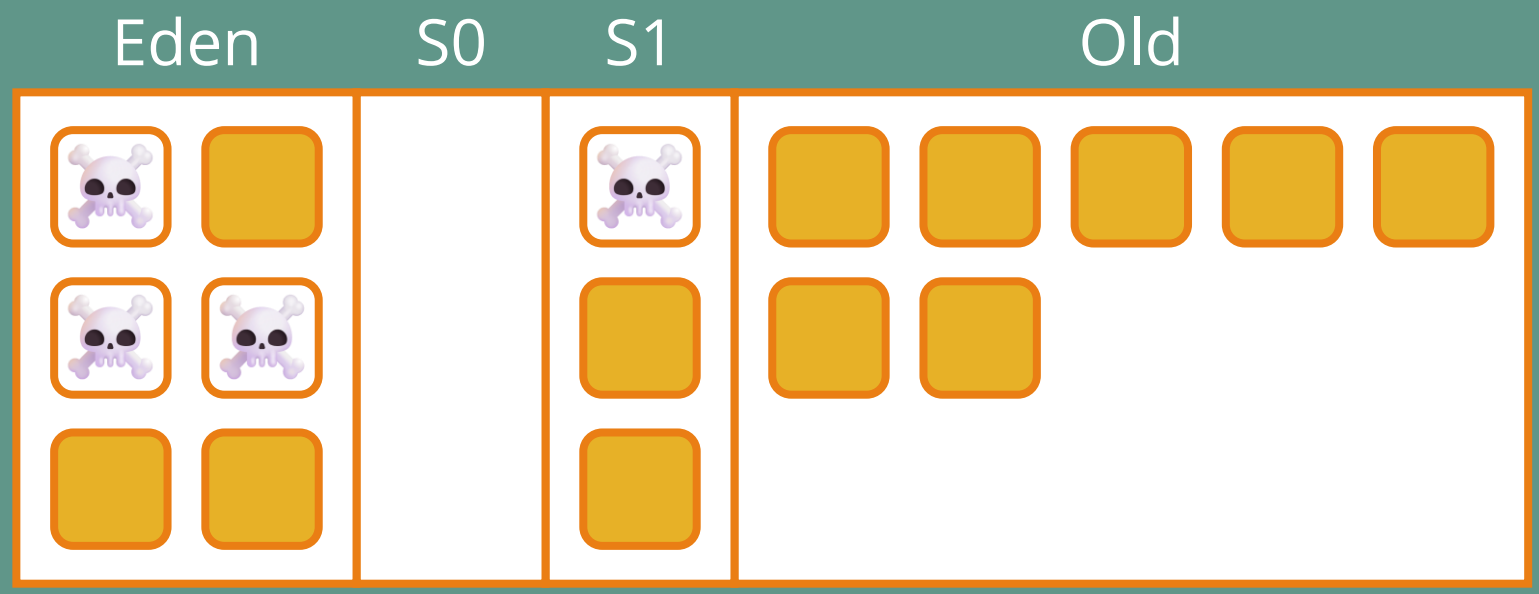
Serial GC – Write barrier

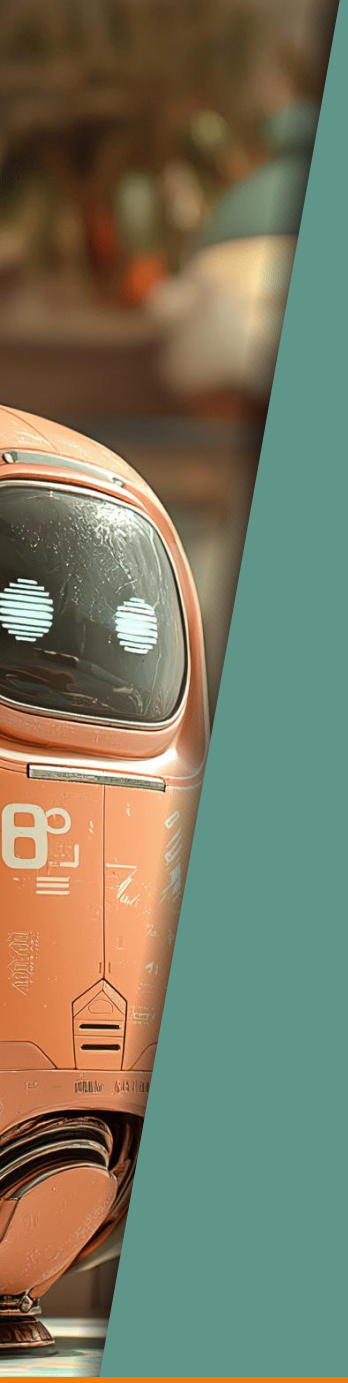
```
cache.addValue(newValue);


if (isOldObject(cache) && !isOldObject(newValue)) {
        int index = getMemoryAddress(cache) / 512;

}
```

Serial GC – Write barrier

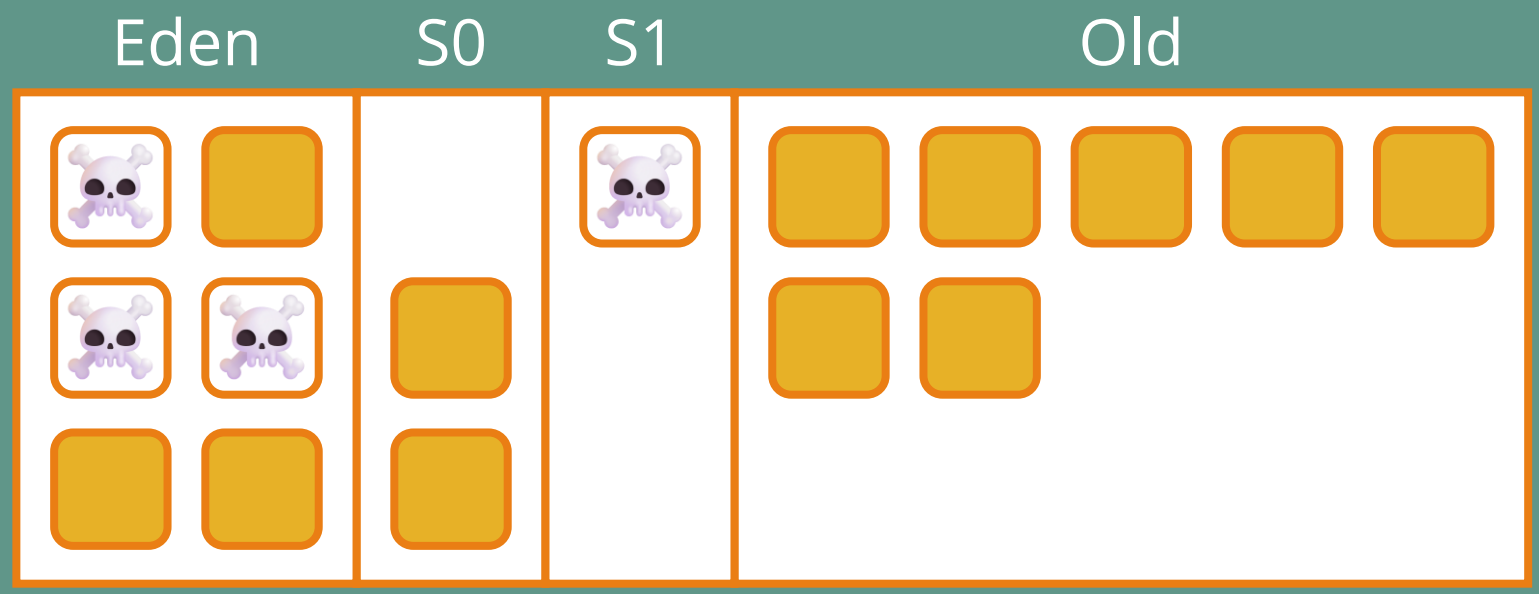
```
cache.addValue(newValue);


if (isOldObject(cache) && !isOldObject(newValue)) {
        int index = getMemoryAddress(cache) / 512;
        cardTable[index] = 1;
}
```

Serial GC

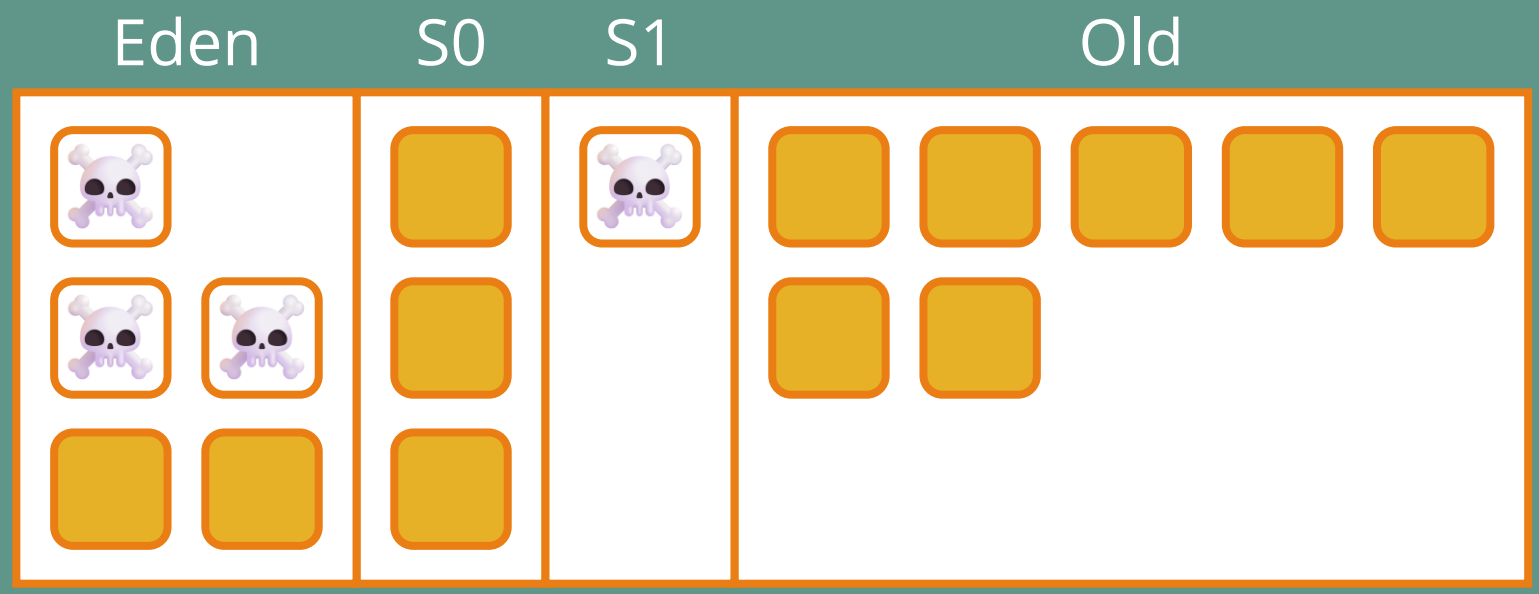


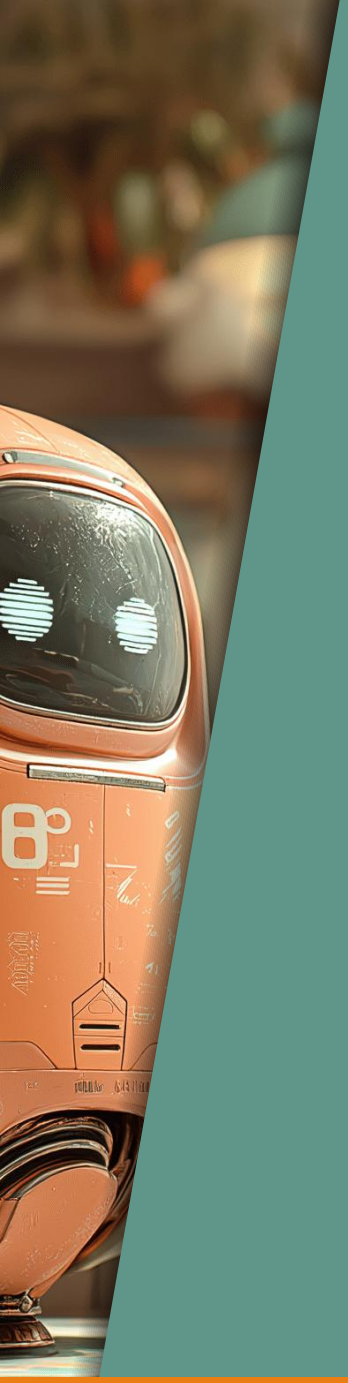


Serial GC

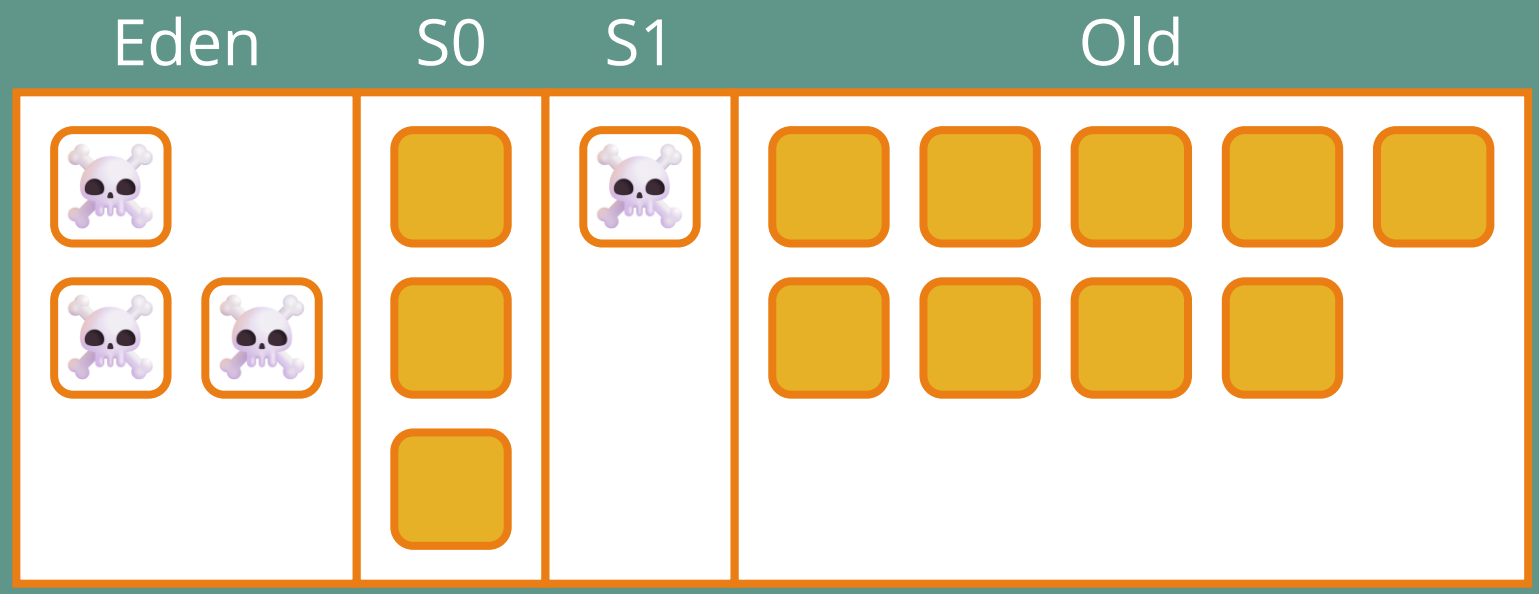


Serial GC

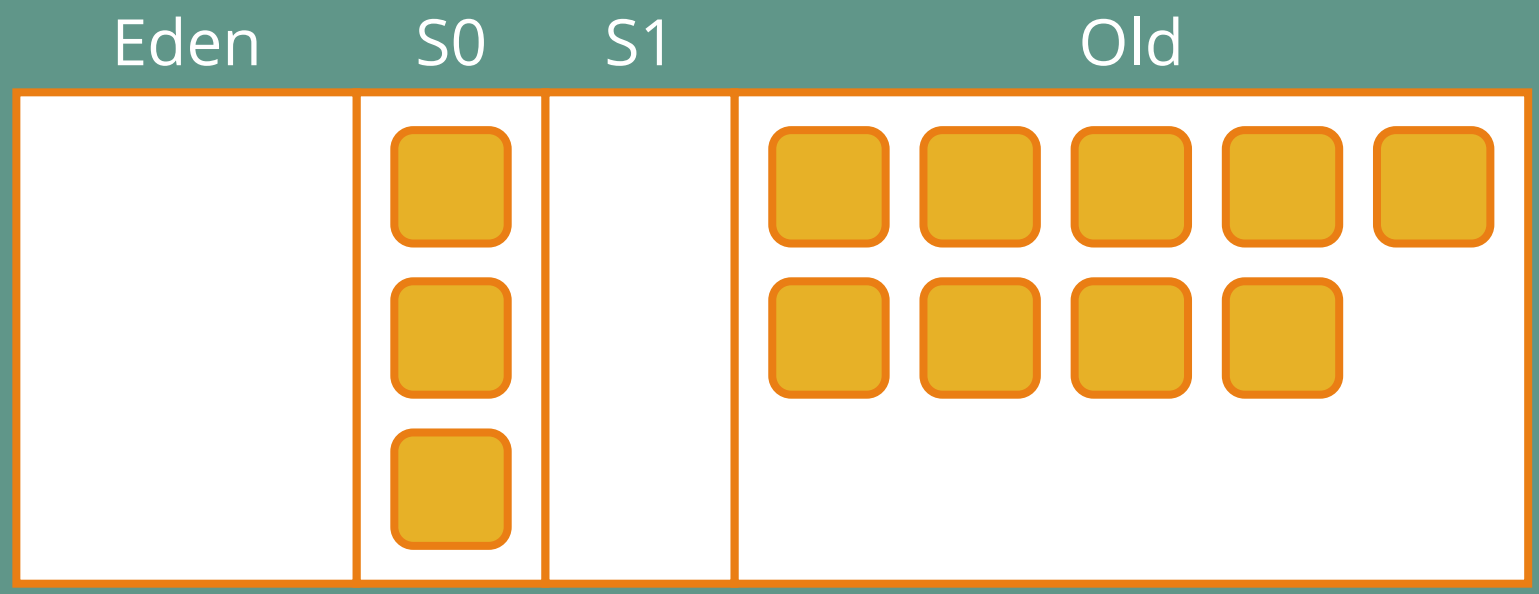


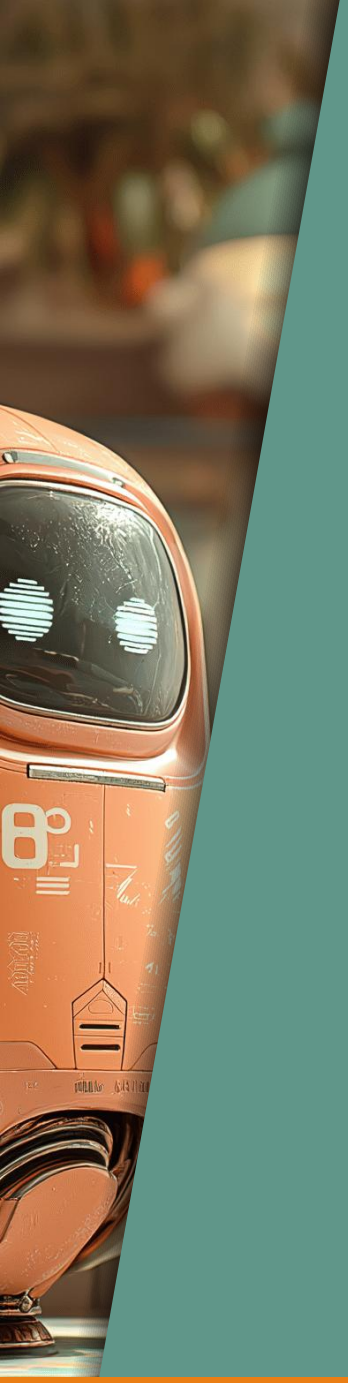


Serial GC

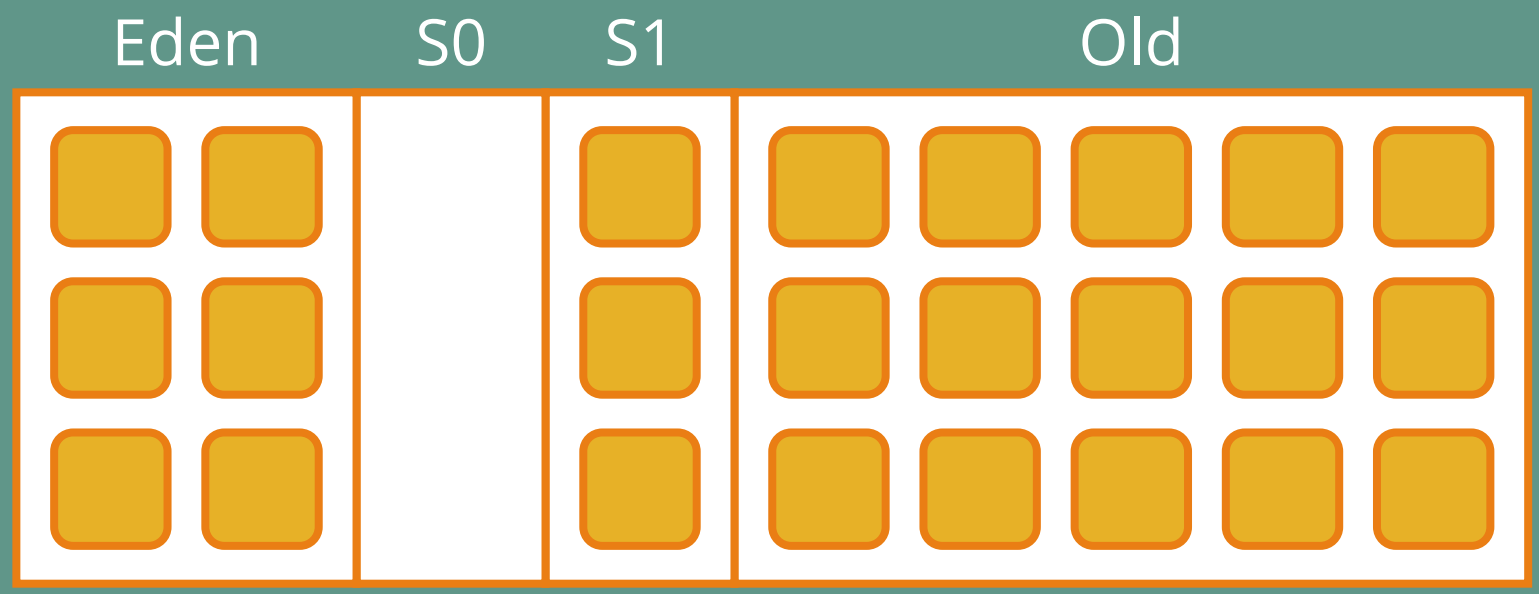


Serial GC

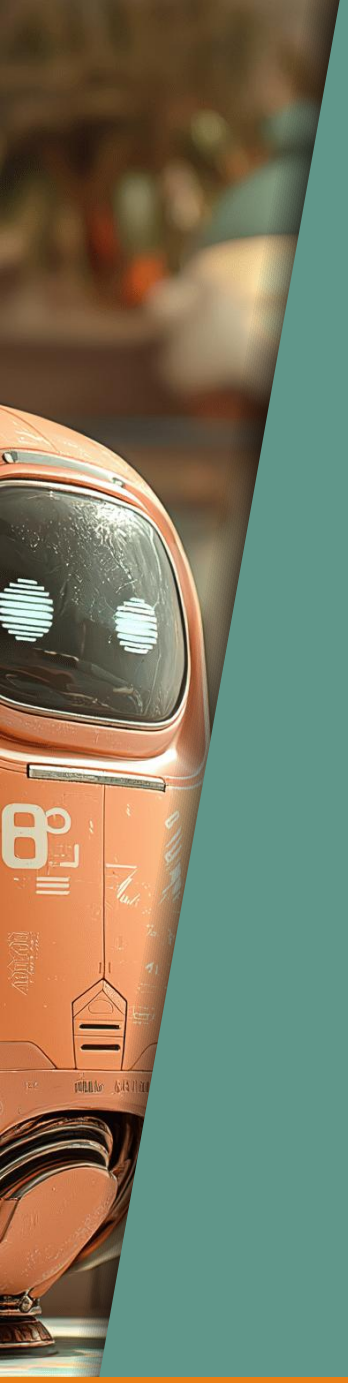




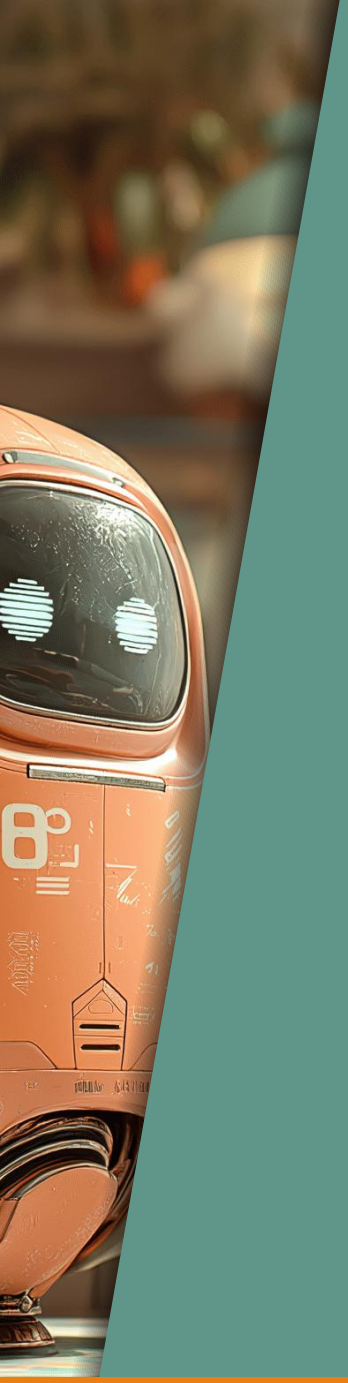
Serial GC – Full GC



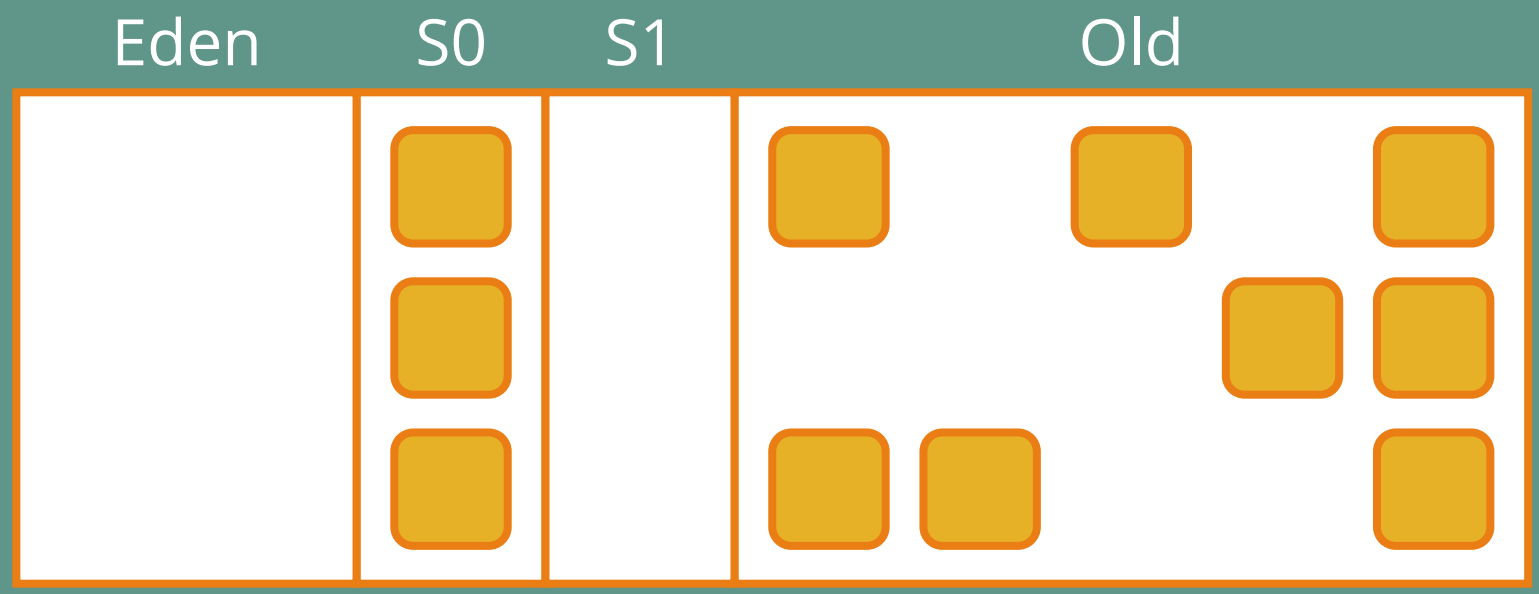
Serial GC – Full GC



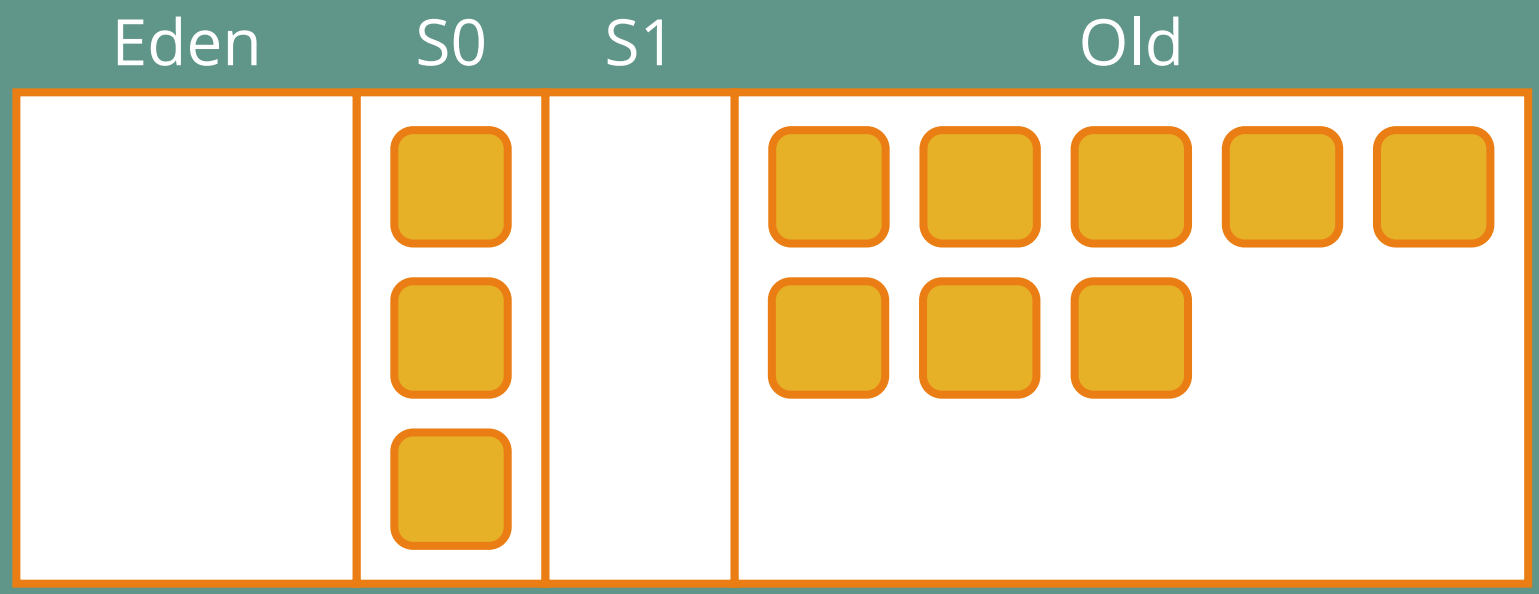
Serial GC – Full GC



Serial GC – Full GC



Serial GC – Full GC



Serial GC

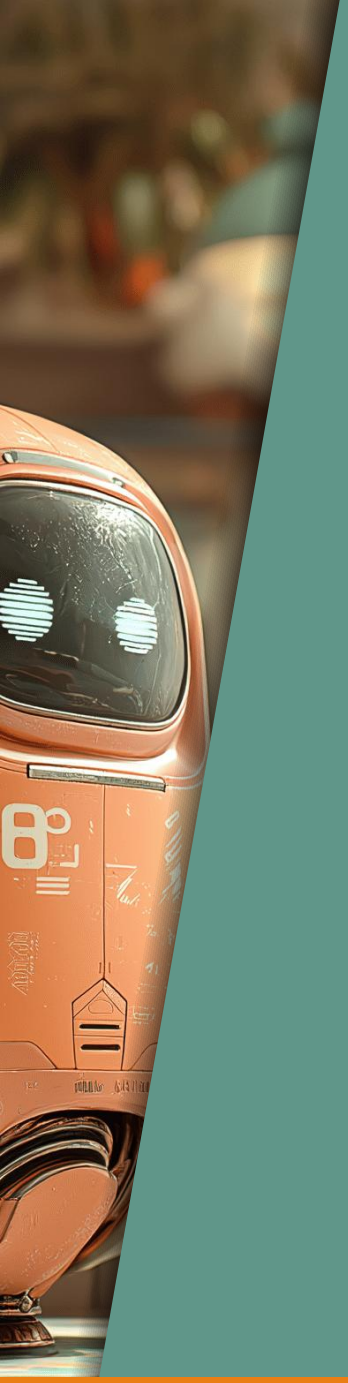
- 💓 Toujours présent et maintenu

Serial GC

- Toujours présent et maintenu
- ⛔ Stop-the-world

Serial GC

- Toujours présent et maintenu
- Stop-the-world
- ☝ Un seul thread



Serial GC

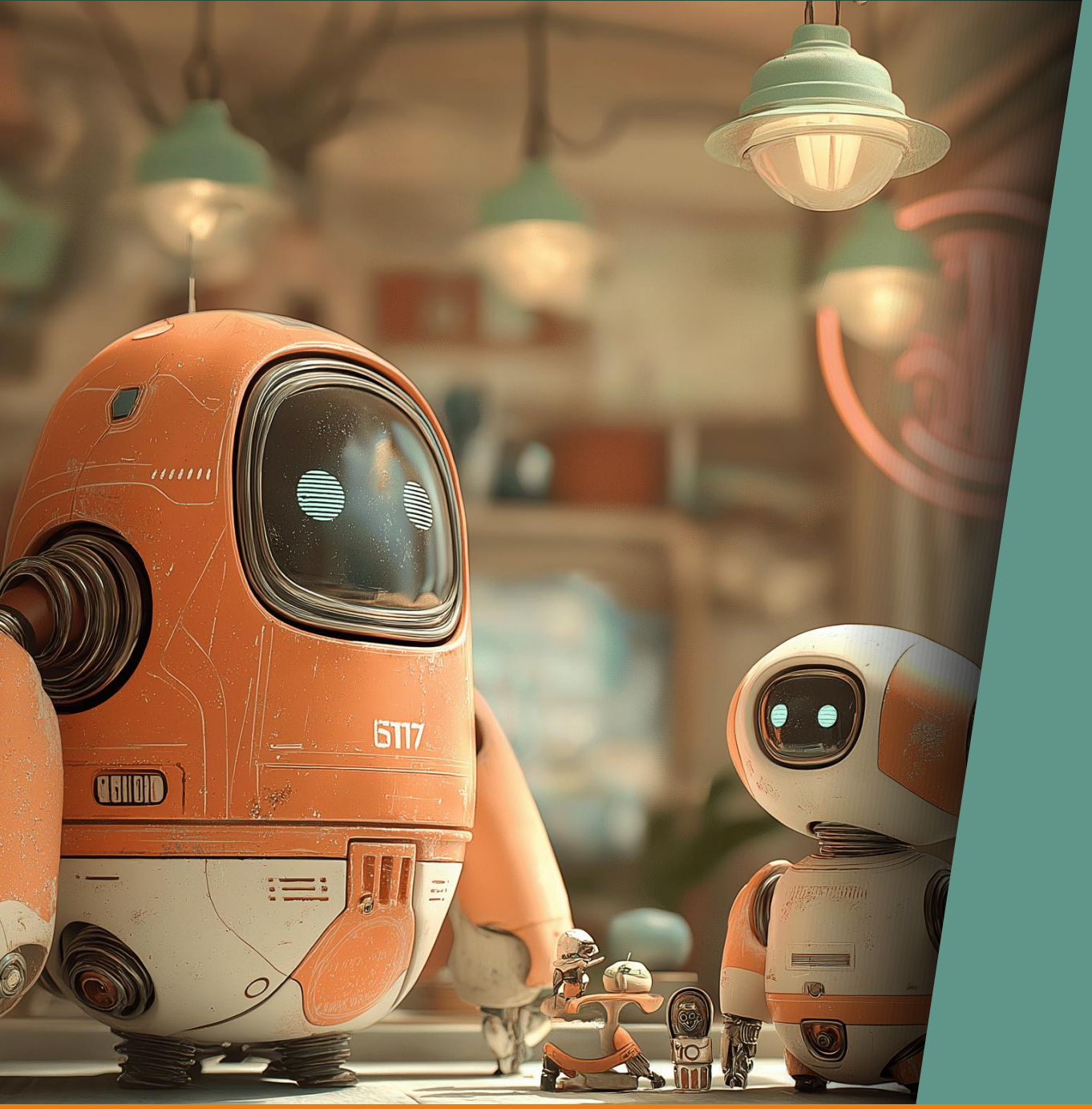
- Toujours présent et maintenu
- Stop-the-world
- Un seul thread
- 🎯 Le GC avec le moins d’overhead

Serial GC

- Toujours présent et maintenu
- Stop-the-world
- Un seul thread
- Le GC avec le moins d’overhead
- ✊ Par défaut si < 2 Go de RAM ou < 2 CPUs

Serial GC

- Toujours présent et maintenu
- Stop-the-world
- Un seul thread
- Le GC avec le moins d’overhead
- Par défaut si < 2 Go de RAM ou < 2 CPUs
- Idéal pour les micro-services



Parallel GC

Java 5 (2004)

Parallel GC

- La même chose mais en parallèle

Parallel GC

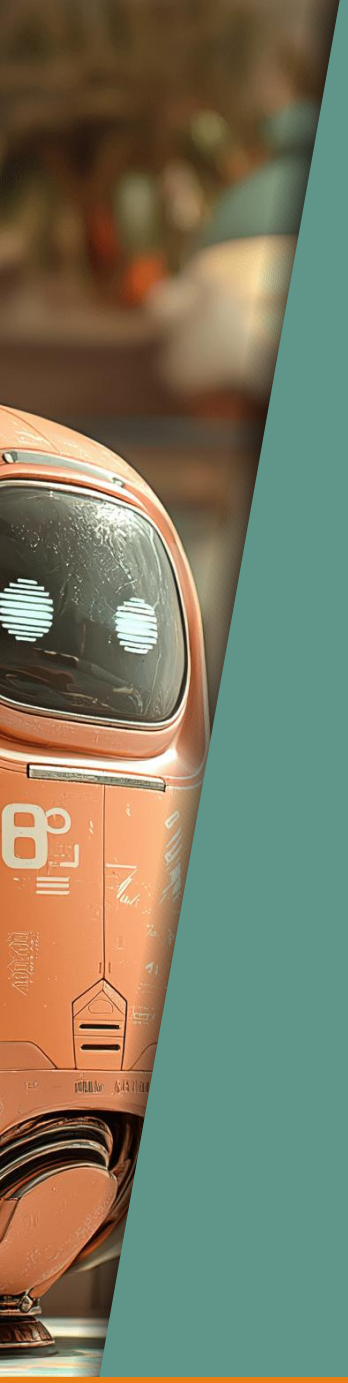
- La même chose mais en parallèle
- ⛔ Toujours stop-the-world

Parallel GC

- La même chose mais en parallèle
- Toujours stop-the-world
- 🚀 Le GC avec le plus de throughput

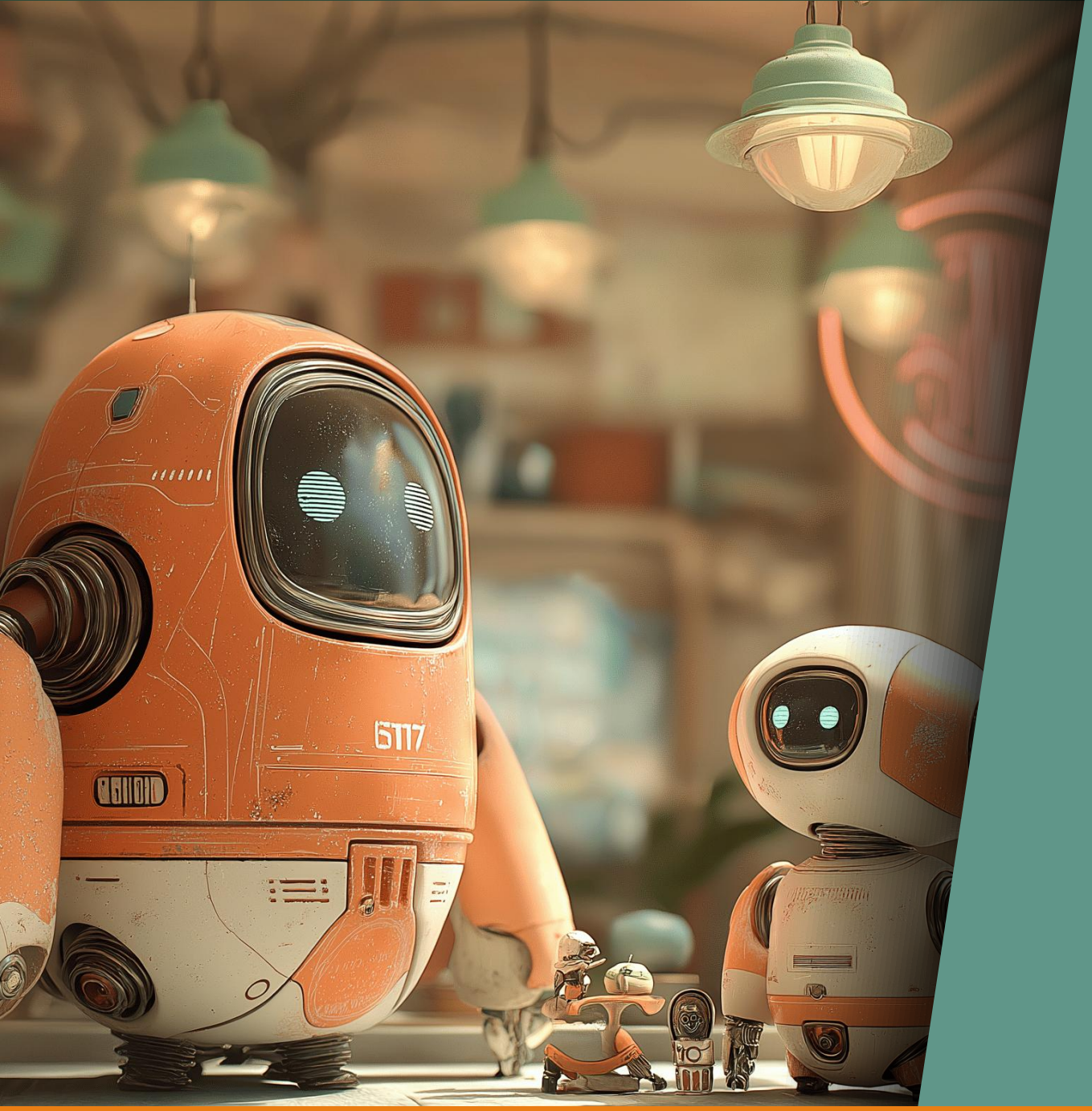
Parallel GC

- La même chose mais en parallèle
- Toujours stop-the-world
- Le GC avec le plus de throughput
- ⏳ Jusqu’à plusieurs secondes sur grosses heaps



Parallel GC

- La même chose mais en parallèle
- Toujours stop-the-world
- Le GC avec le plus de throughput
- Jusqu’à plusieurs secondes sur grosses heaps
- Idéal pour le traitement de données, batchs, …



Latence ?

Et la latence ?

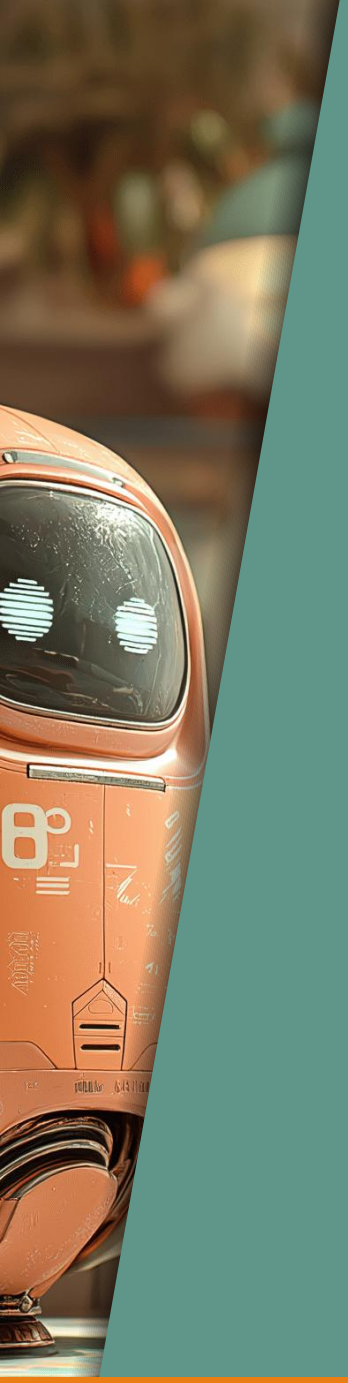
- 🕓 Le GC peut bloquer l’application plusieurs secondes

Et la latence ?

- Le GC peut bloquer l’application plusieurs secondes
- 🚂 Essai raté : Train GC

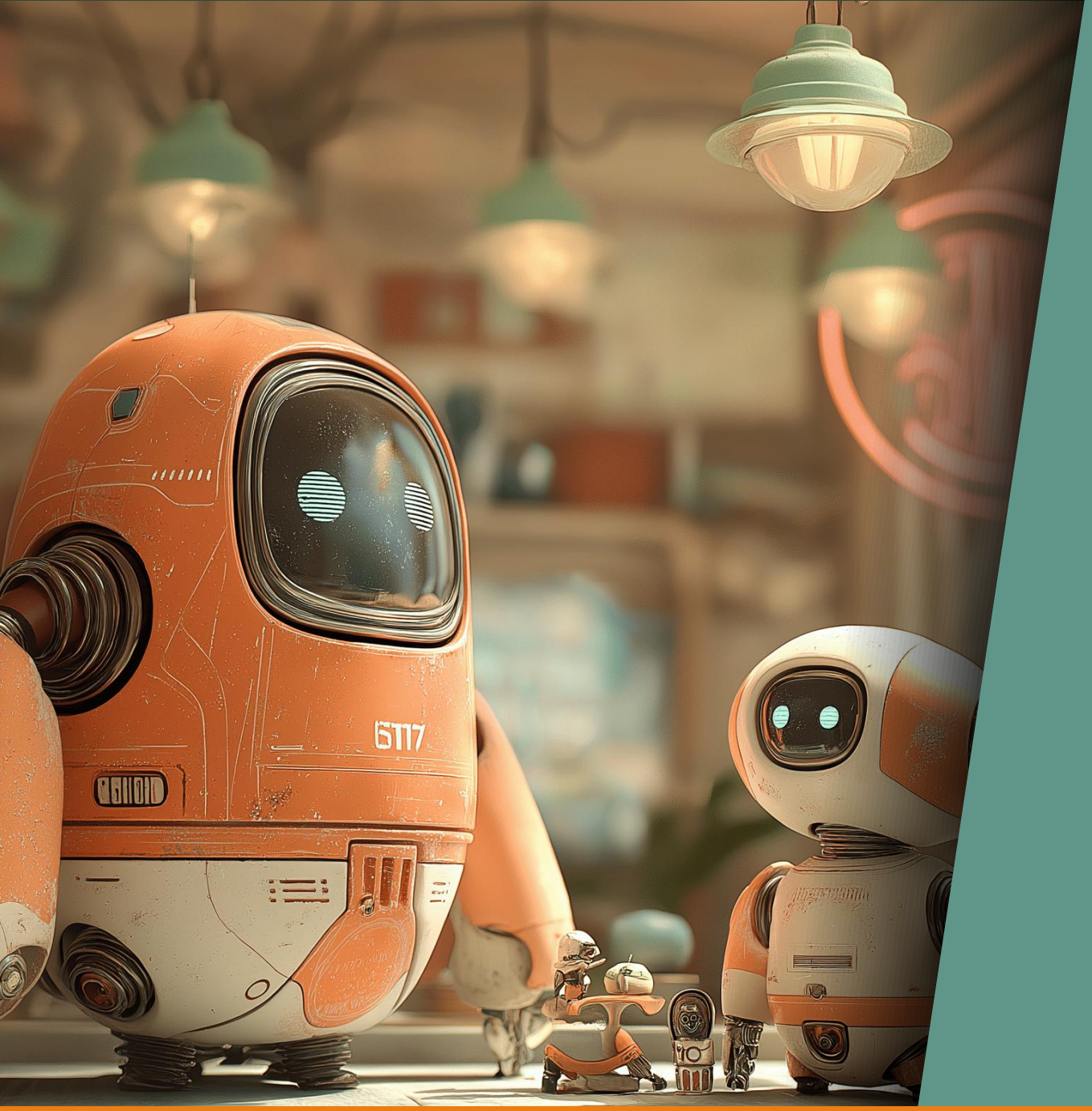
Et la latence ?

- Le GC peut bloquer l’application plusieurs secondes
- Essai raté : Train GC
- 🧀 Essai semi-raté : CMS GC



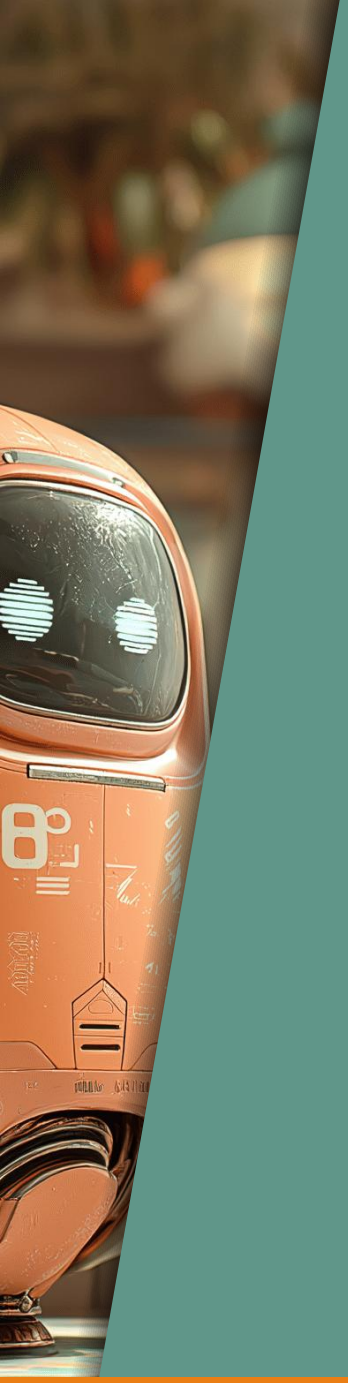
Et la latence ?

- Le GC peut bloquer l’application plusieurs secondes
- Essai raté : Train GC
- Essai semi-raté : CMS GC
- 🗑️ Essai réussi : G1 GC en Java 7 (par défaut à partir de Java 9)

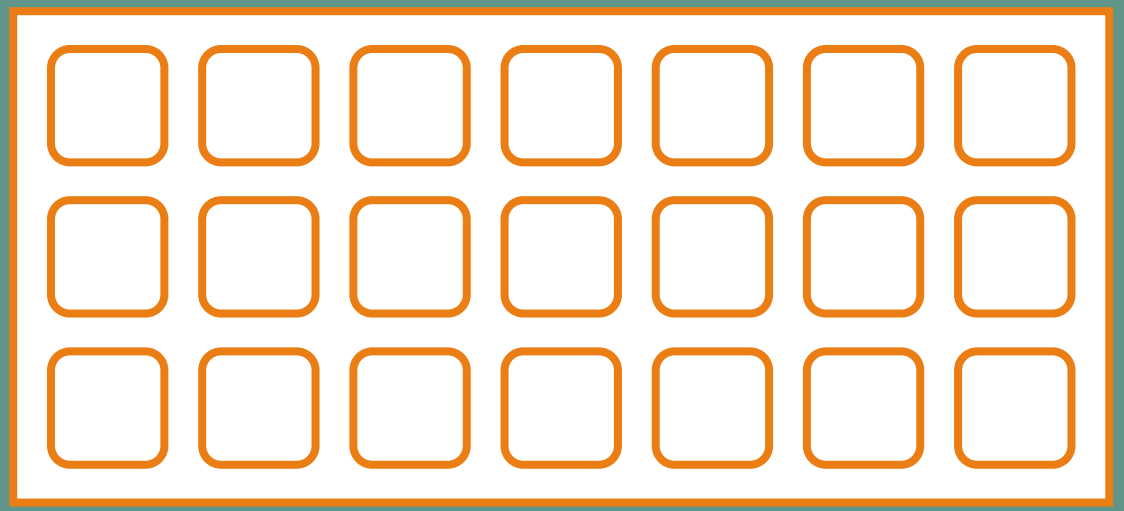


G1 GC

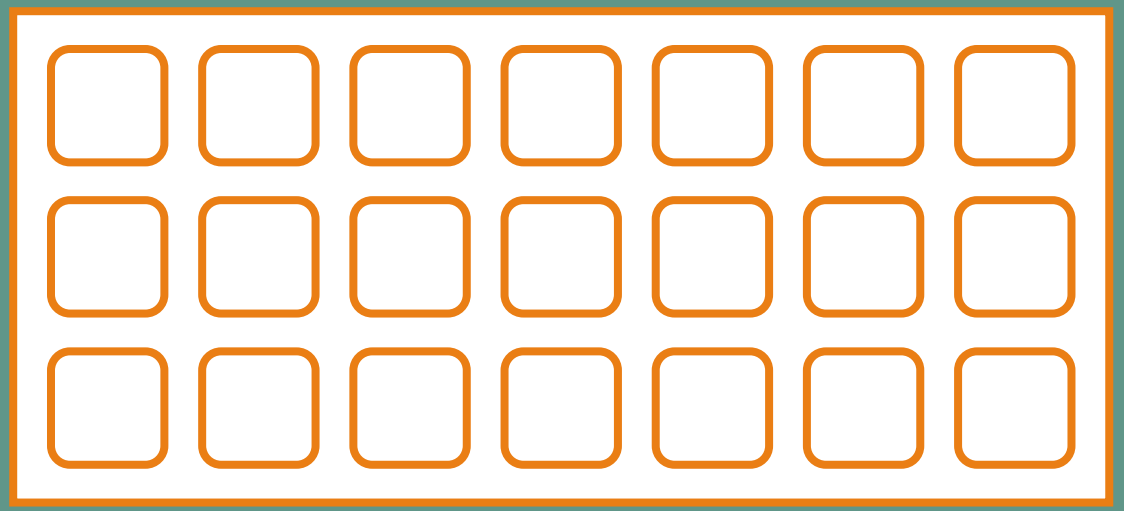
Java 7 (2011)



G1 GC

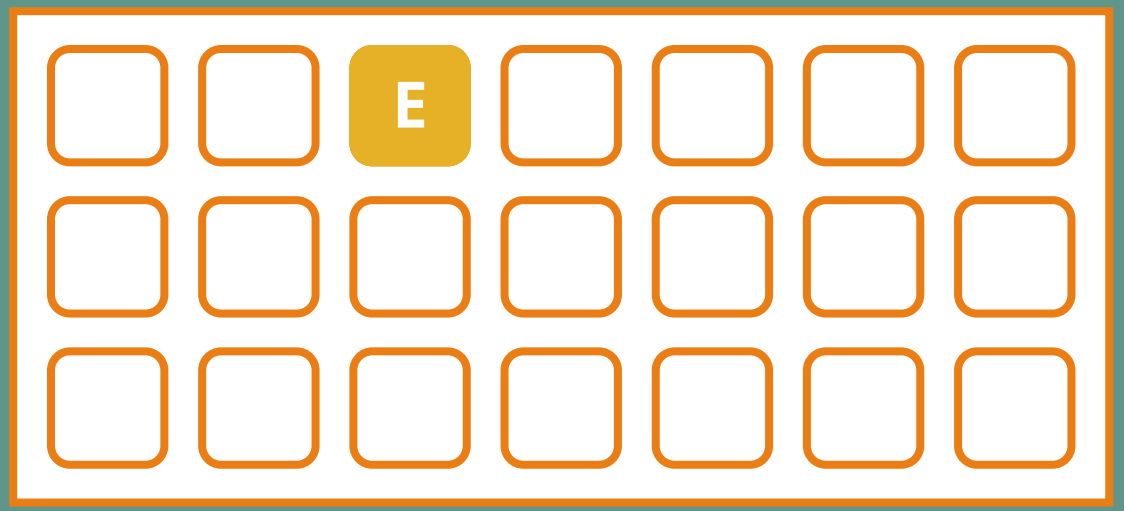


G1 GC

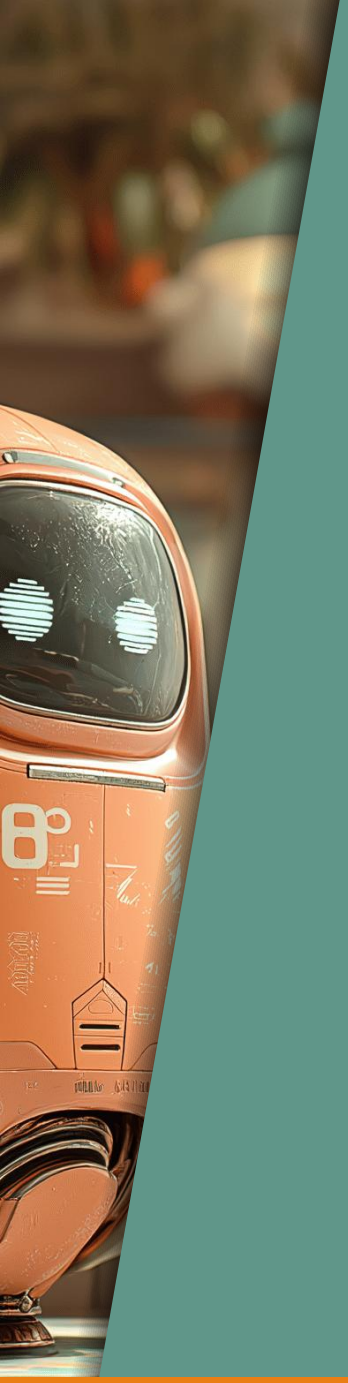


E Eden

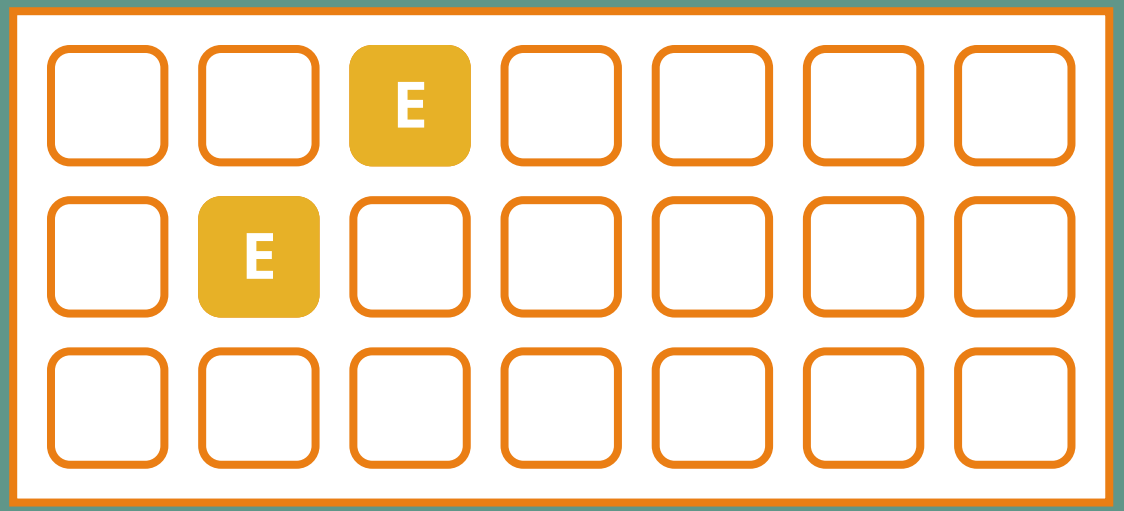
G1 GC



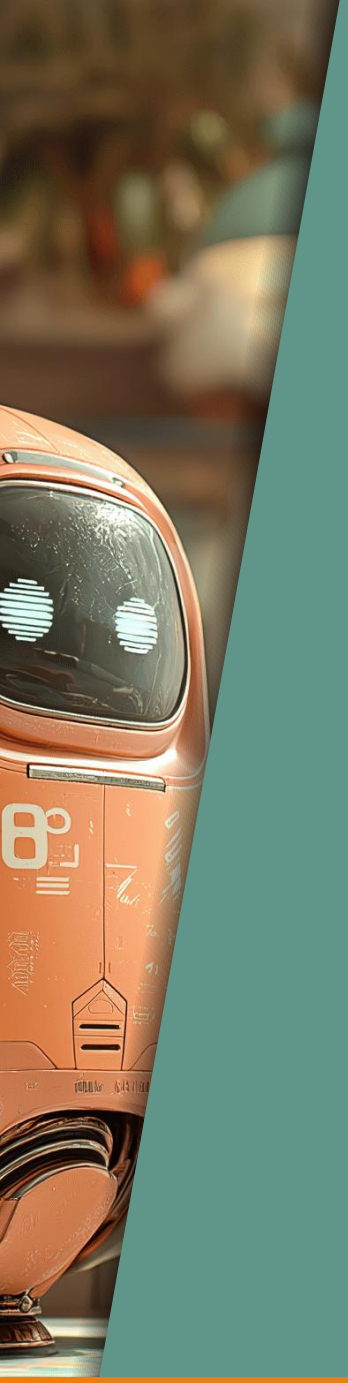
E Eden



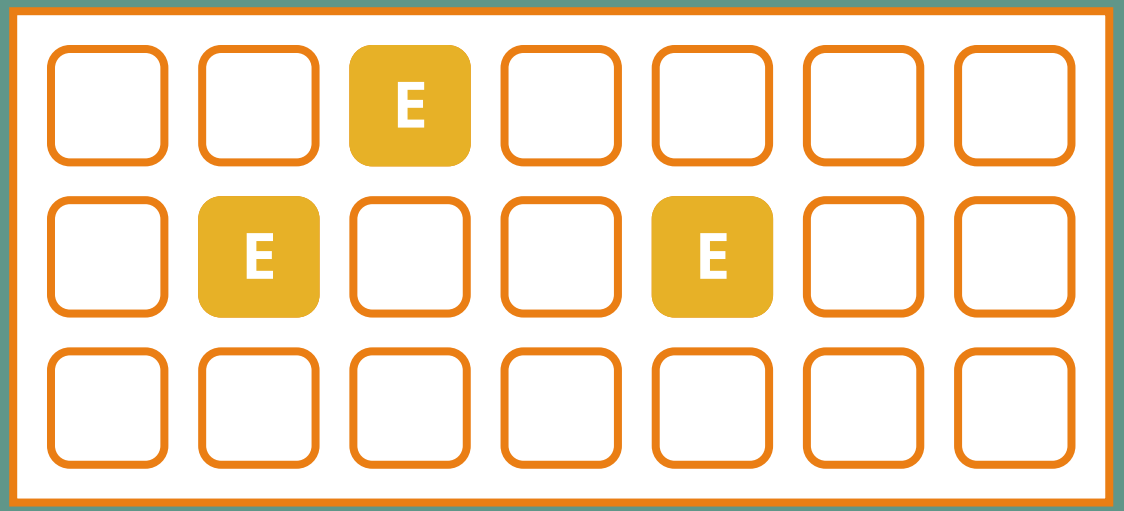
G1 GC



E Eden

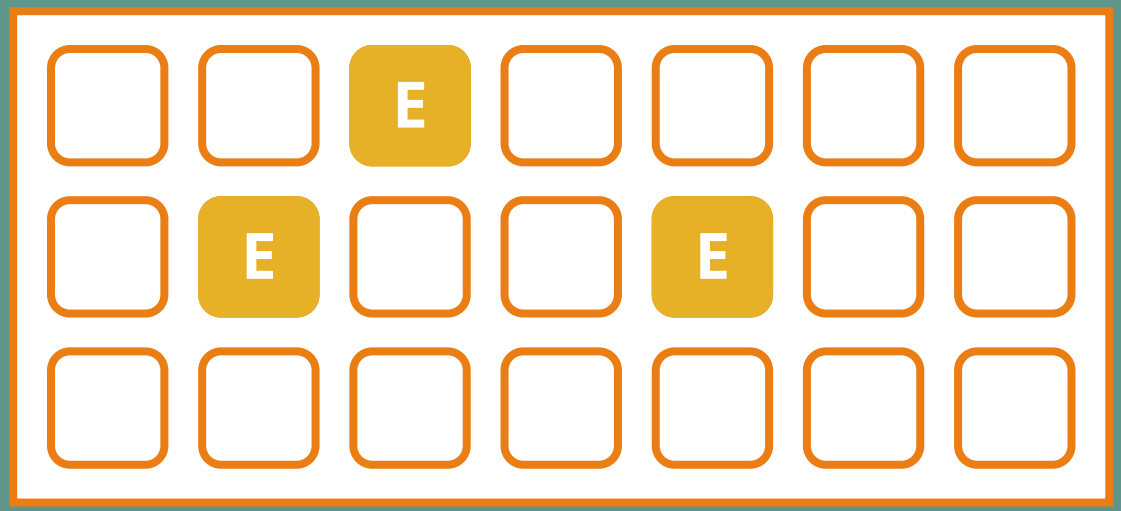


G1 GC



E Eden

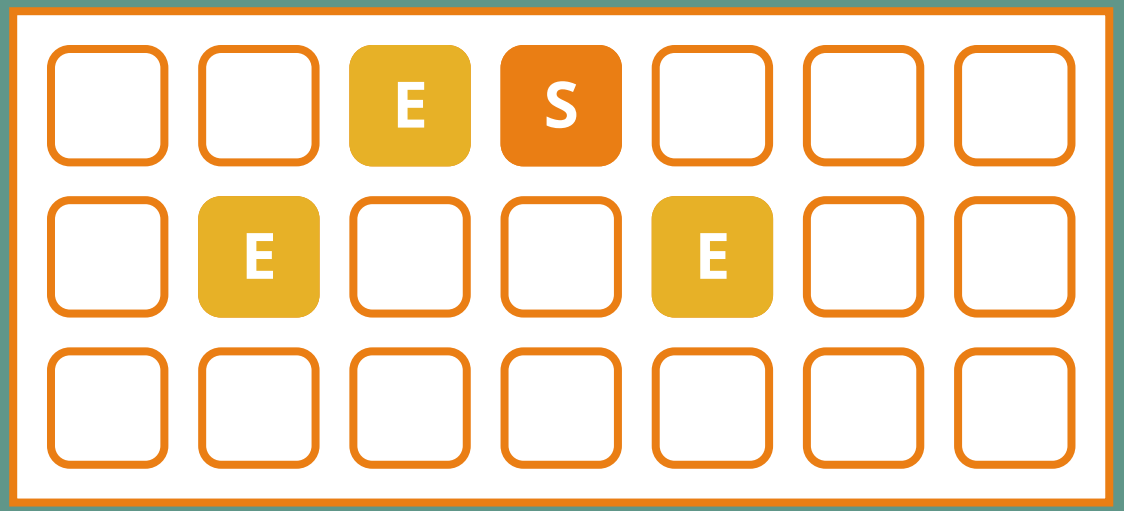
G1 GC



E Eden

S Survivor

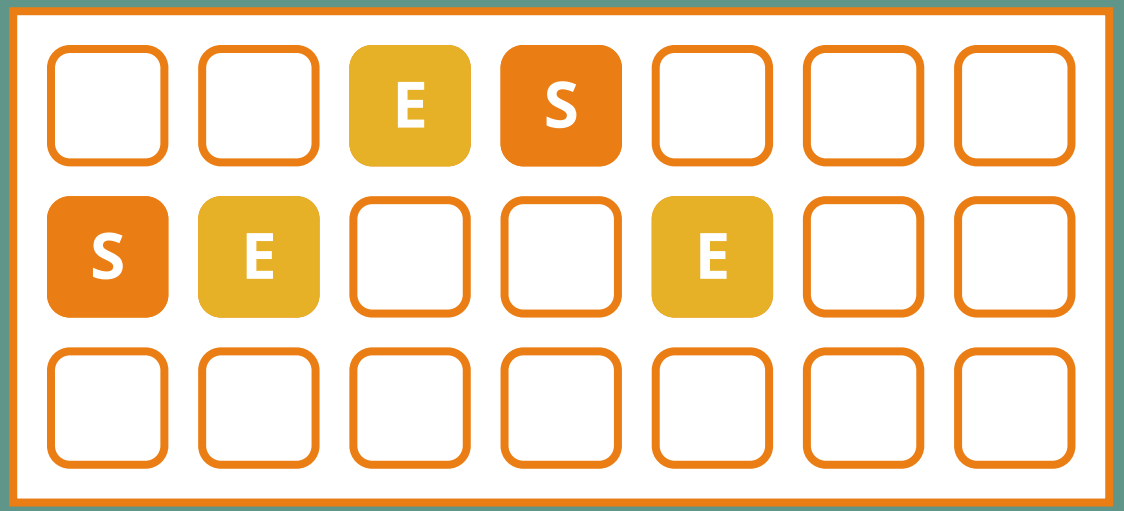
G1 GC



E Eden

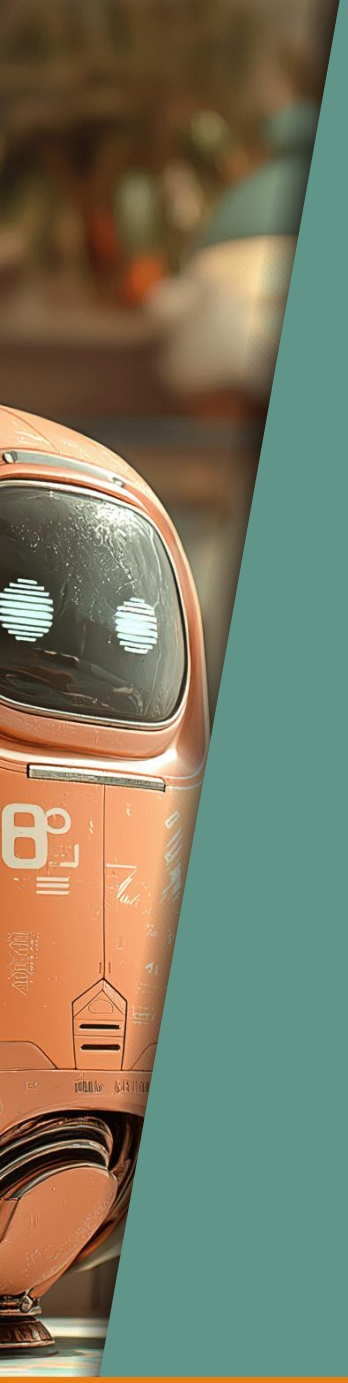
S Survivor

G1 GC

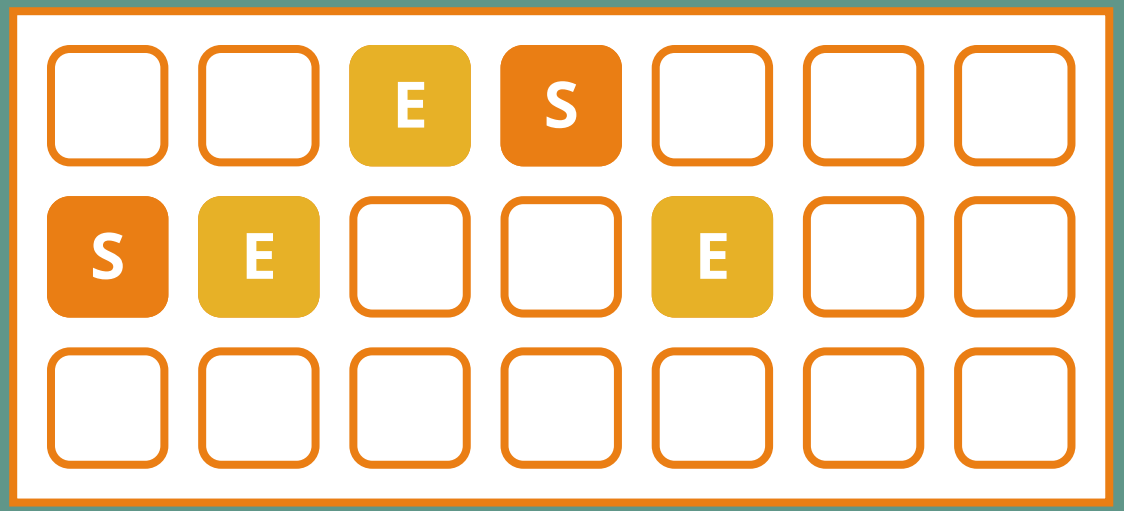


E Eden

S Survivor



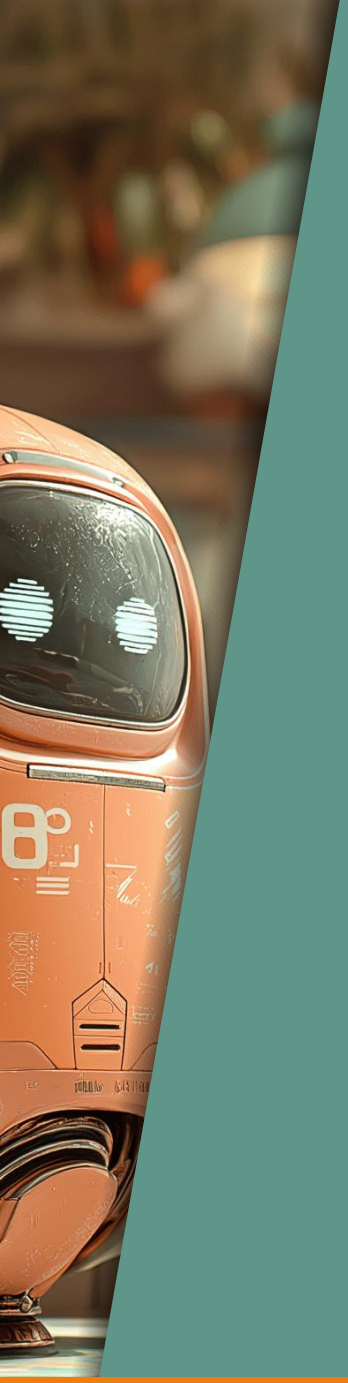
G1 GC



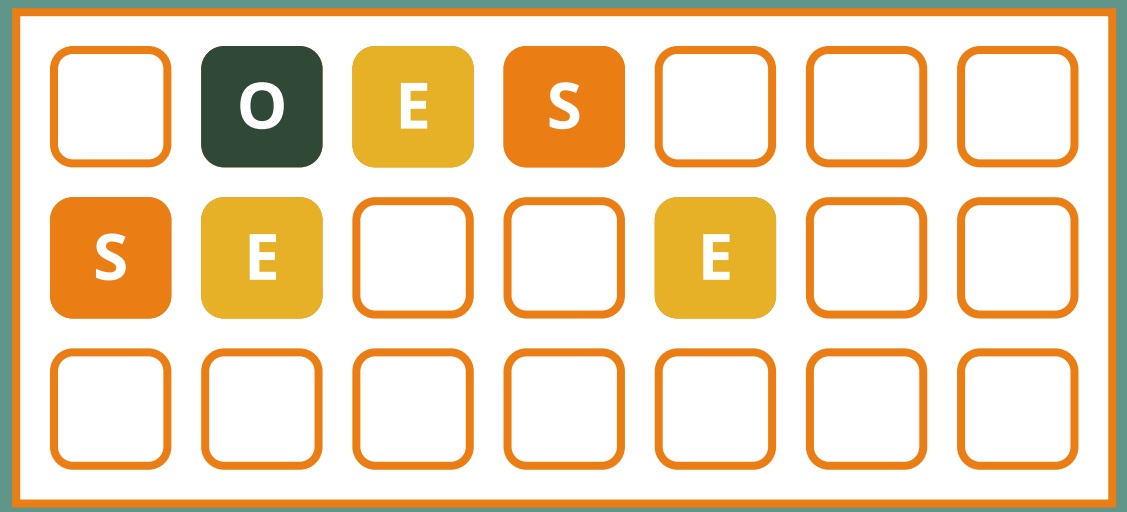
E Eden

S Survivor

O Old



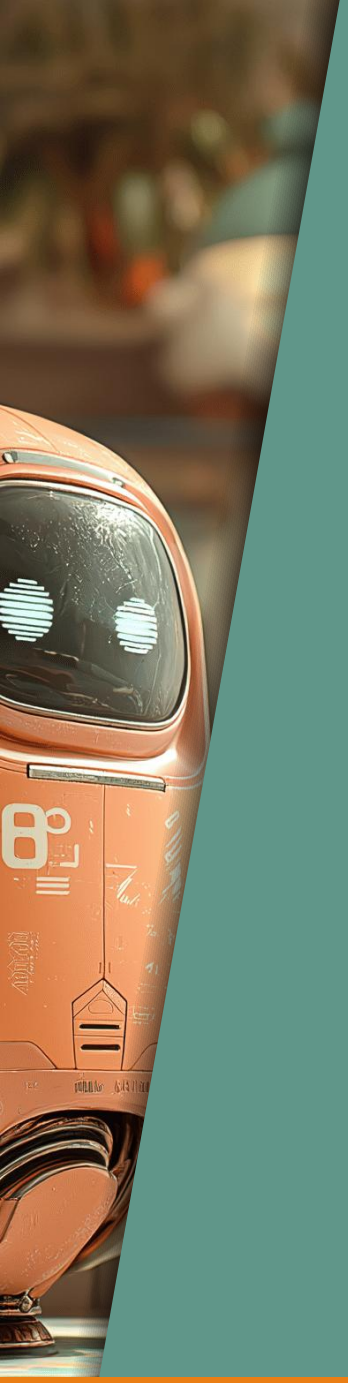
G1 GC



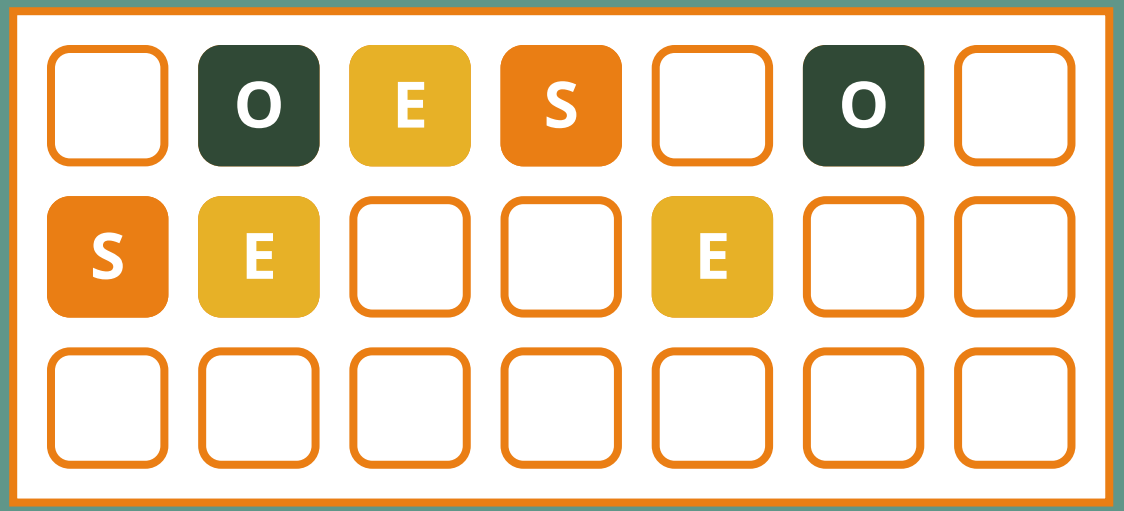
E Eden

S Survivor

O Old



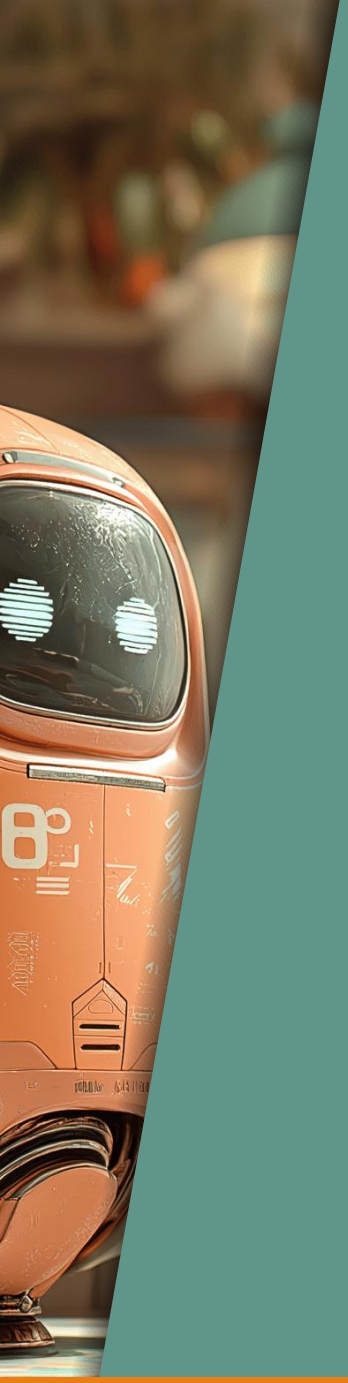
G1 GC



E Eden

S Survivor

O Old

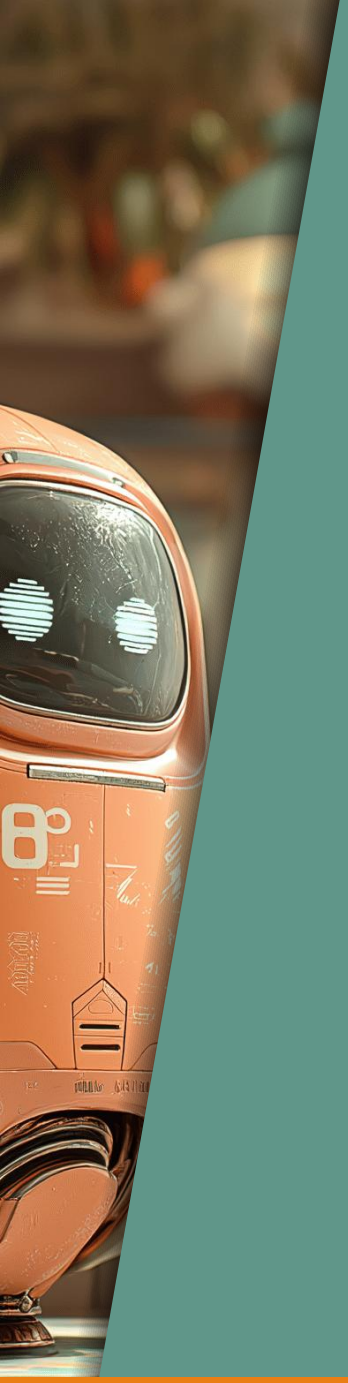


G1 GC

E Eden

S Survivor

O Old

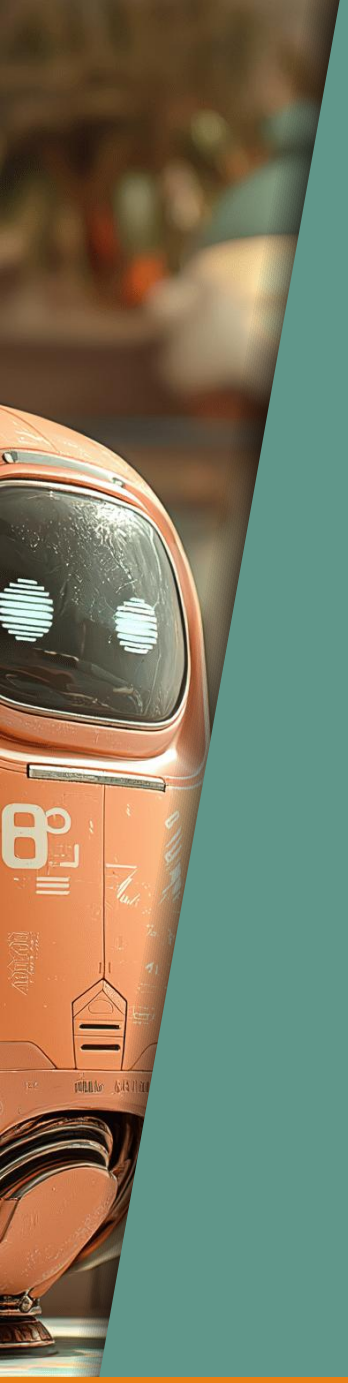


G1 GC

E Eden

S Survivor

O Old



G1 GC

E Eden

S Survivor

O Old

H Humongous

G1 GC

E Eden

S Survivor

O Old

H Humongous

G1 GC

E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection

E Eden

S Survivor

O Old

H Humongous

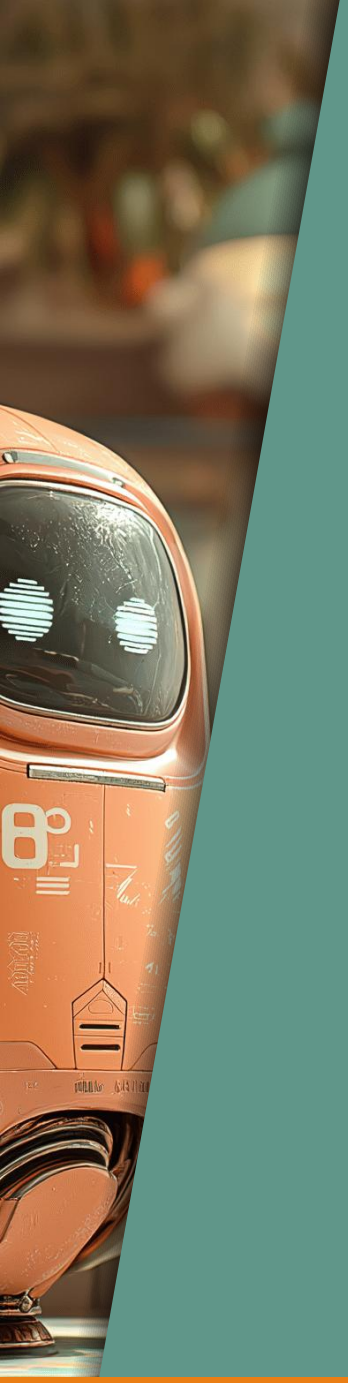
G1 GC – Young collection

E Eden

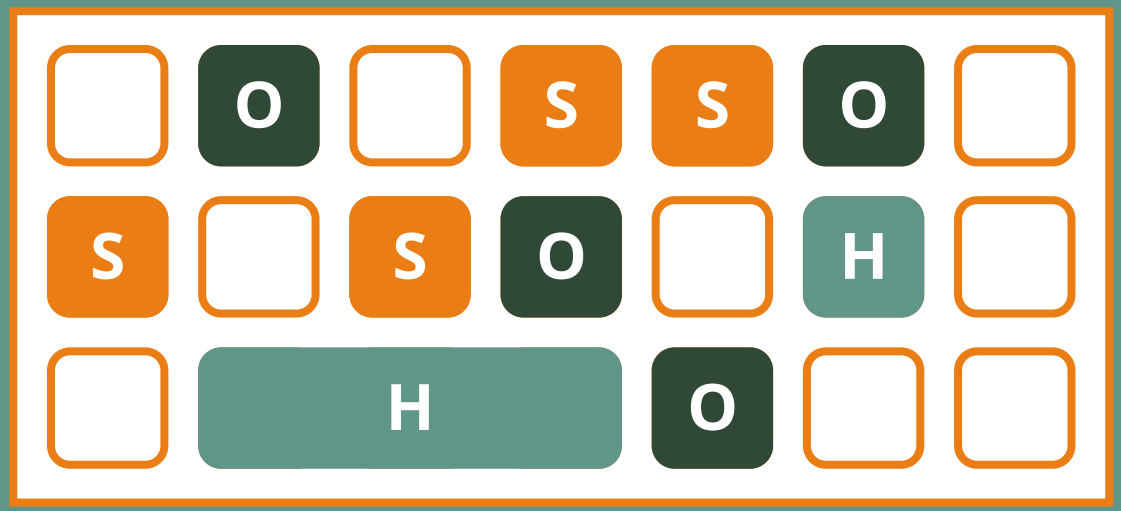
S Survivor

O Old

H Humongous



G1 GC – Young collection



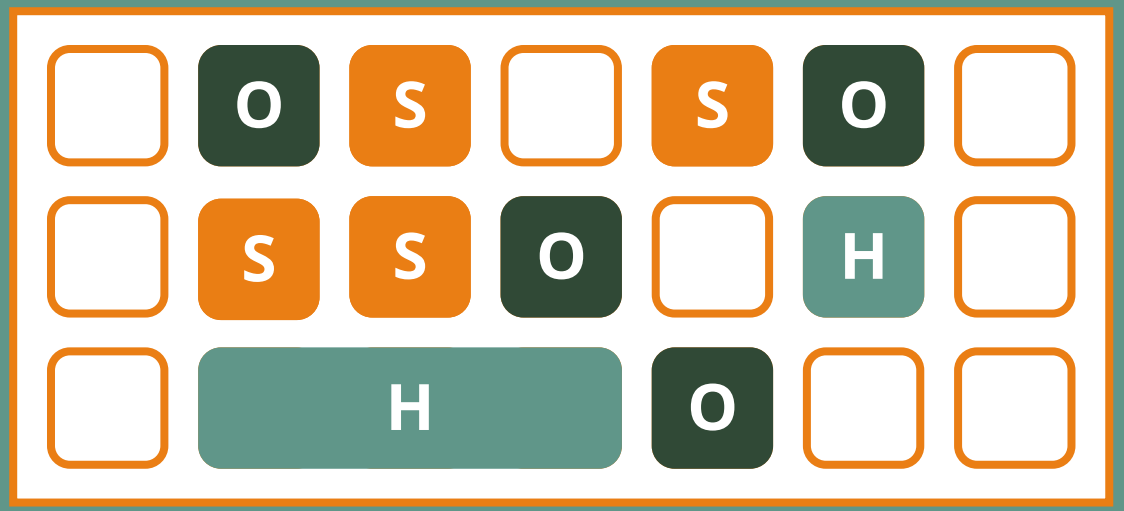
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection



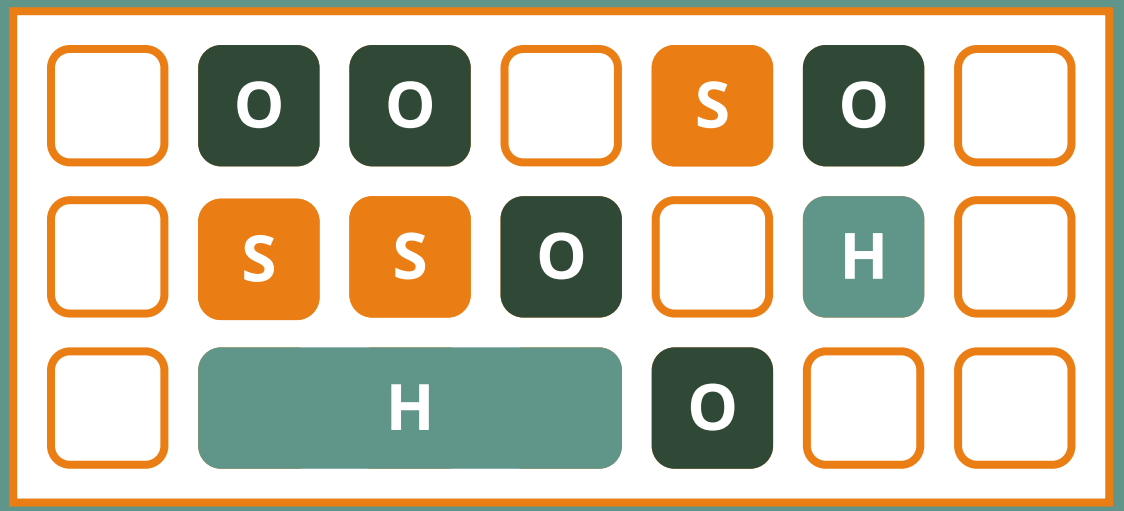
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection

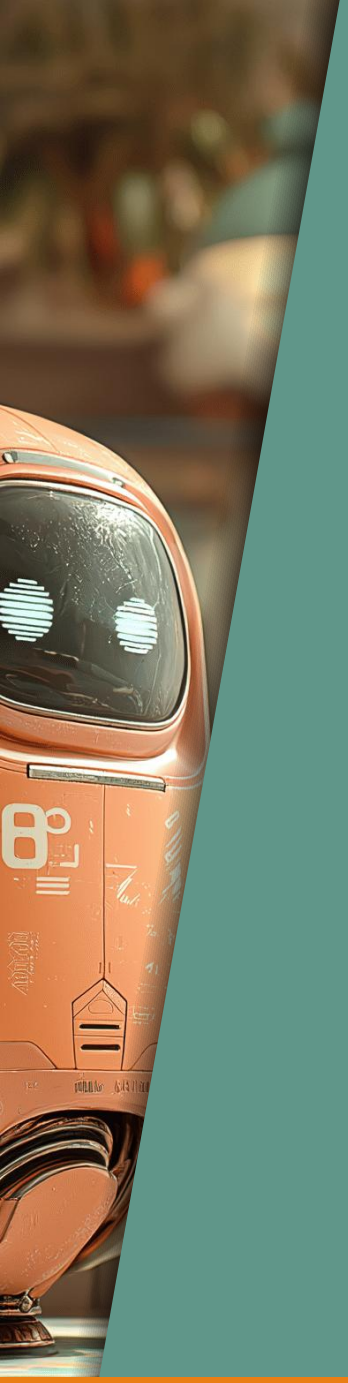


E Eden

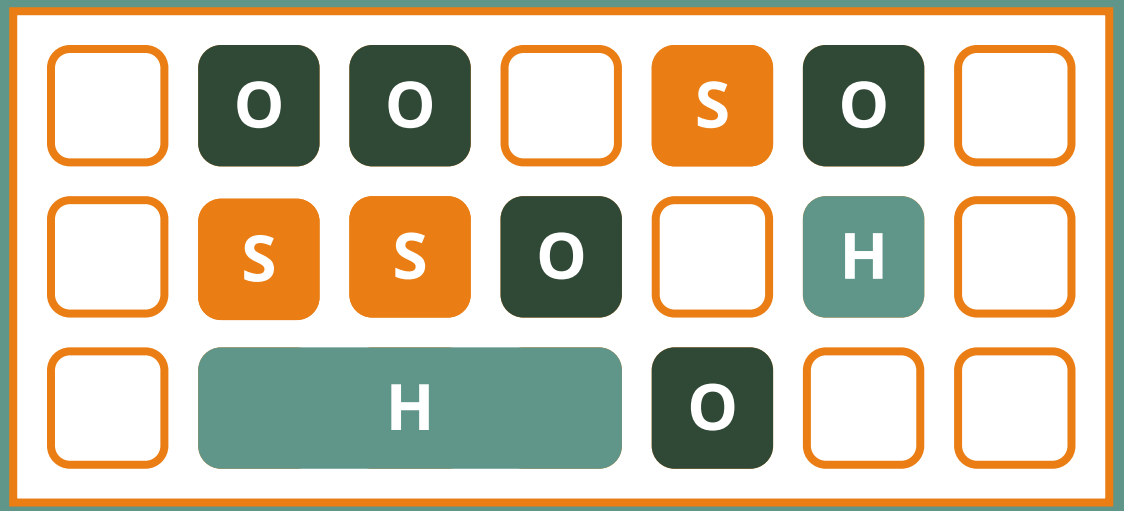
S Survivor

O Old

H Humongous



G1 GC – Mixed collection



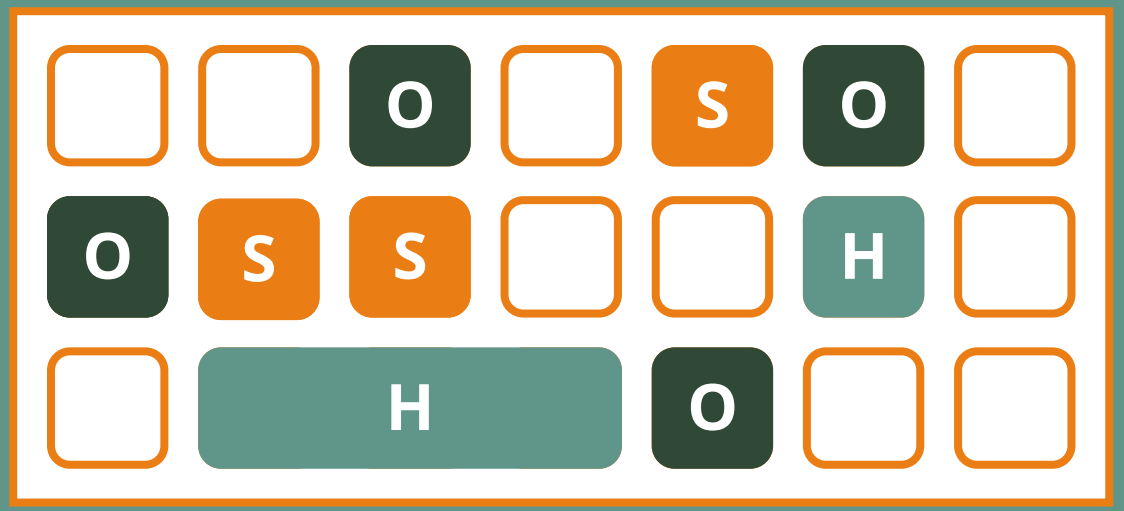
E Eden

S Survivor

O Old

H Humongous

G1 GC – Mixed collection



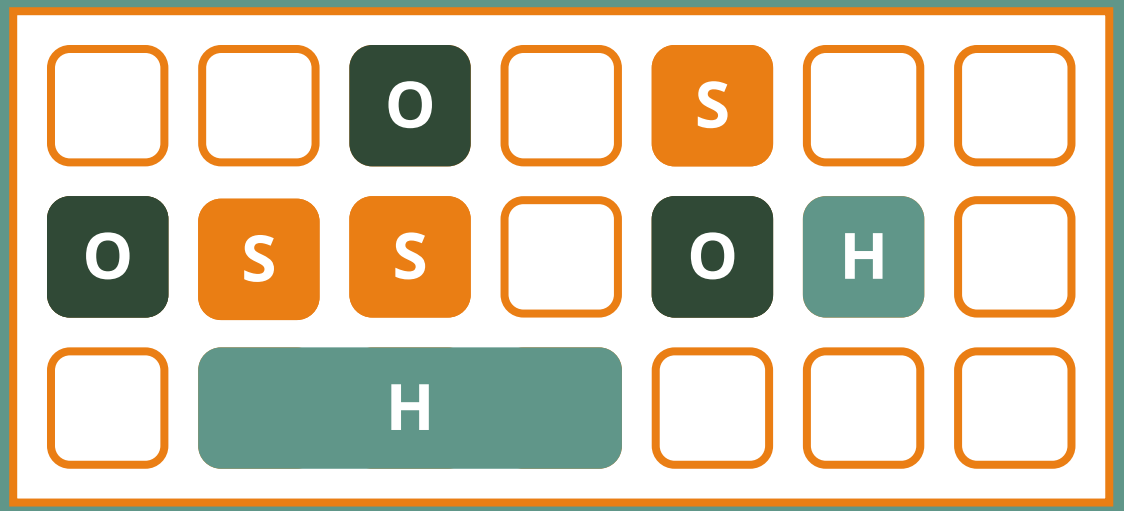
E Eden

S Survivor

O Old

H Humongous

G1 GC – Mixed collection



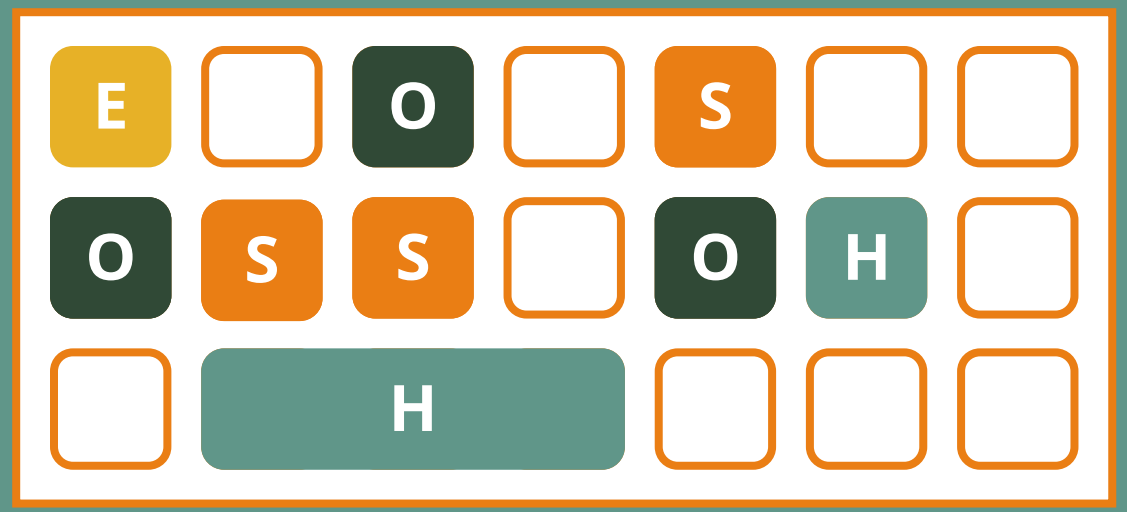
E Eden

S Survivor

O Old

H Humongous

G1 GC



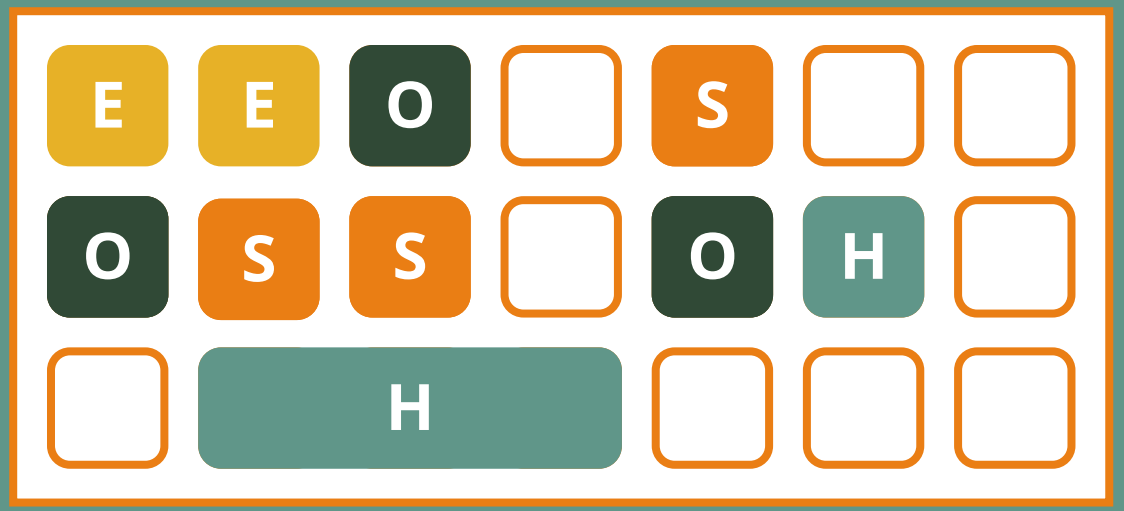
E Eden

S Survivor

O Old

H Humongous

G1 GC



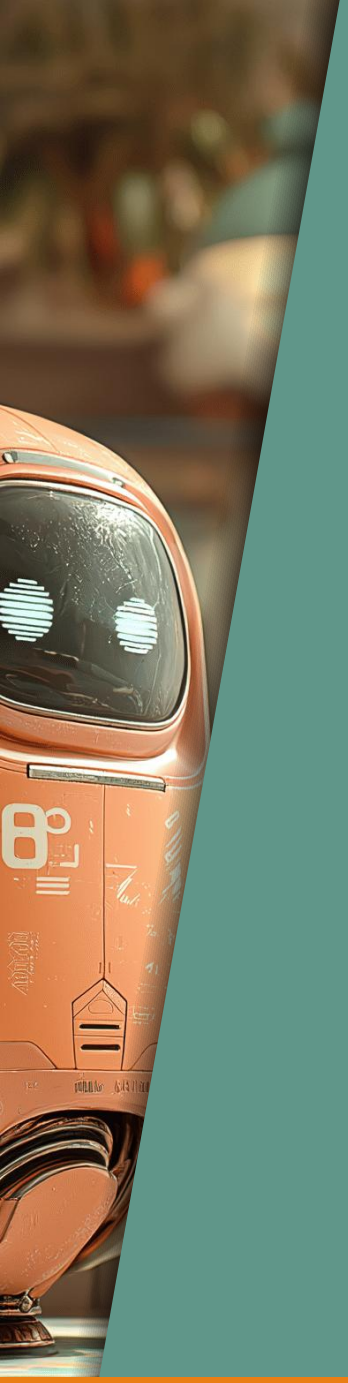
E Eden

S Survivor

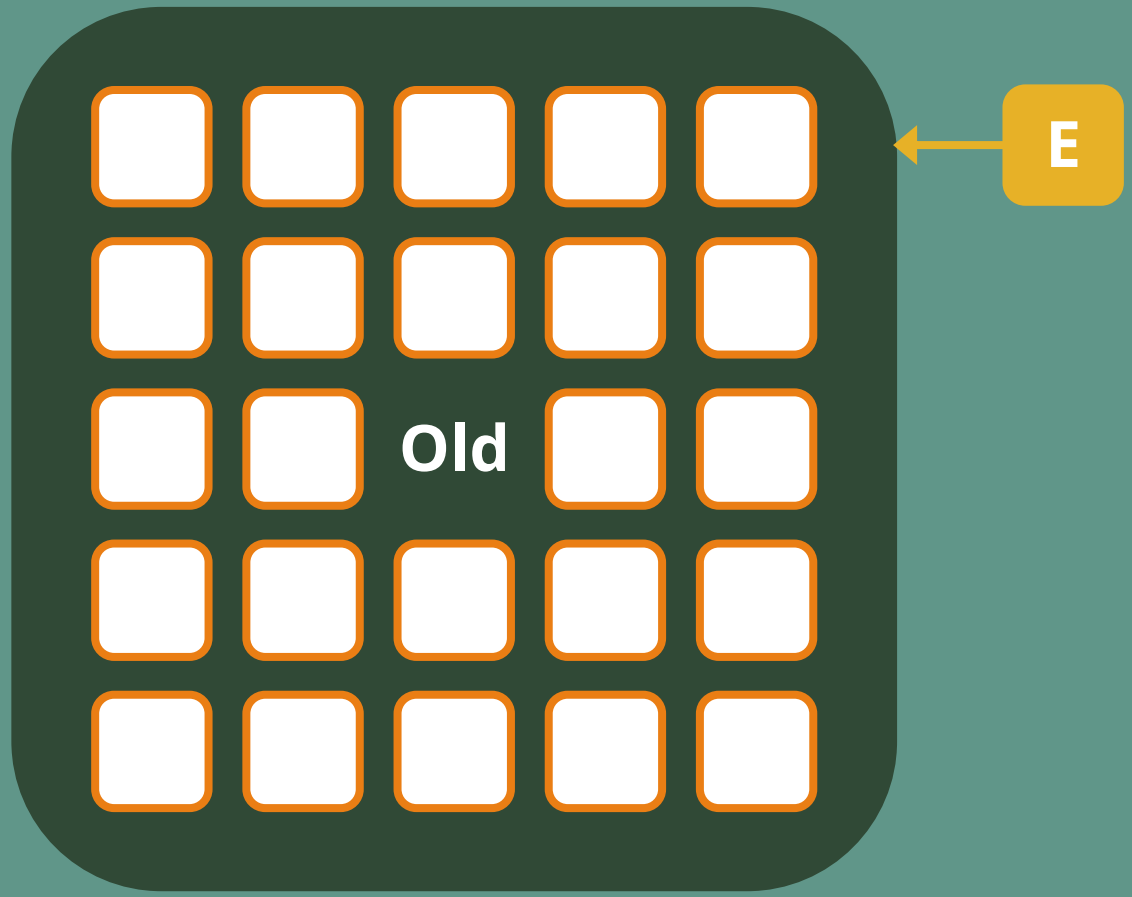
O Old

H Humongous

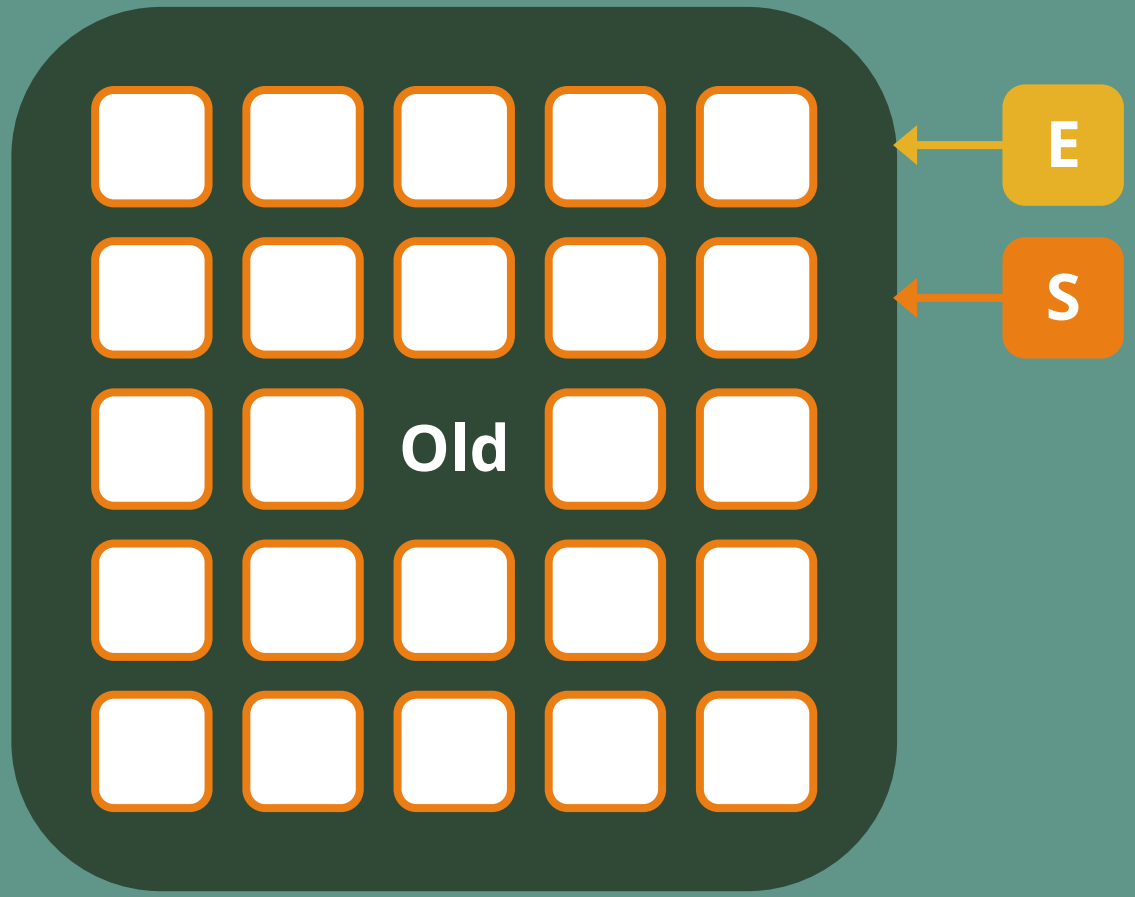
G1 GC – Région

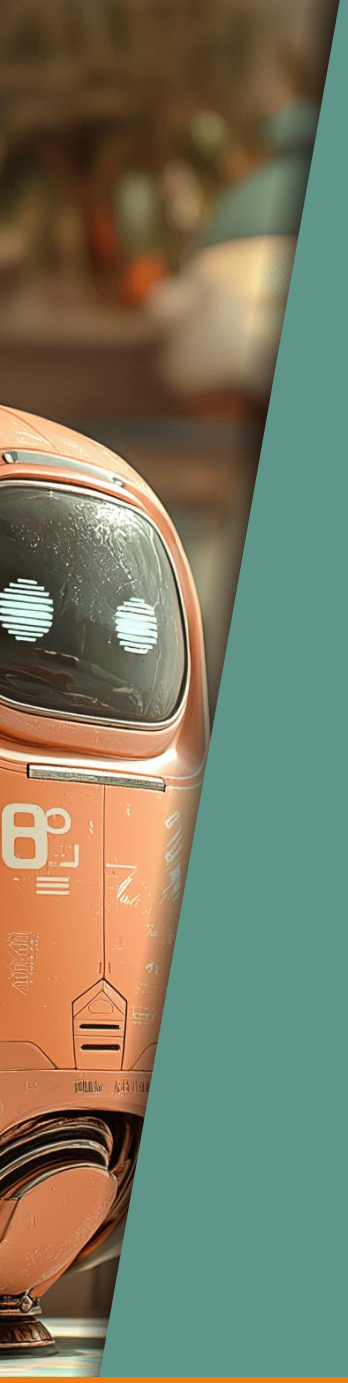


G1 GC – Région

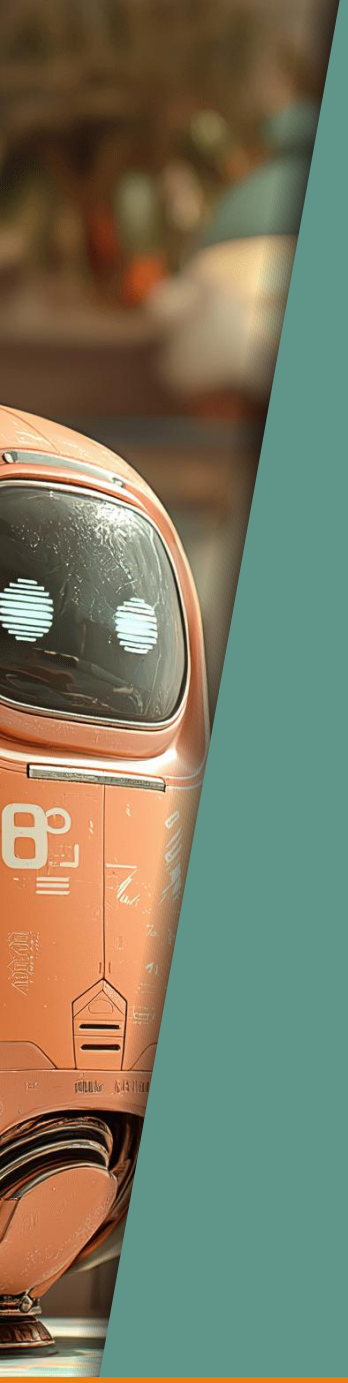


G1 GC – Région

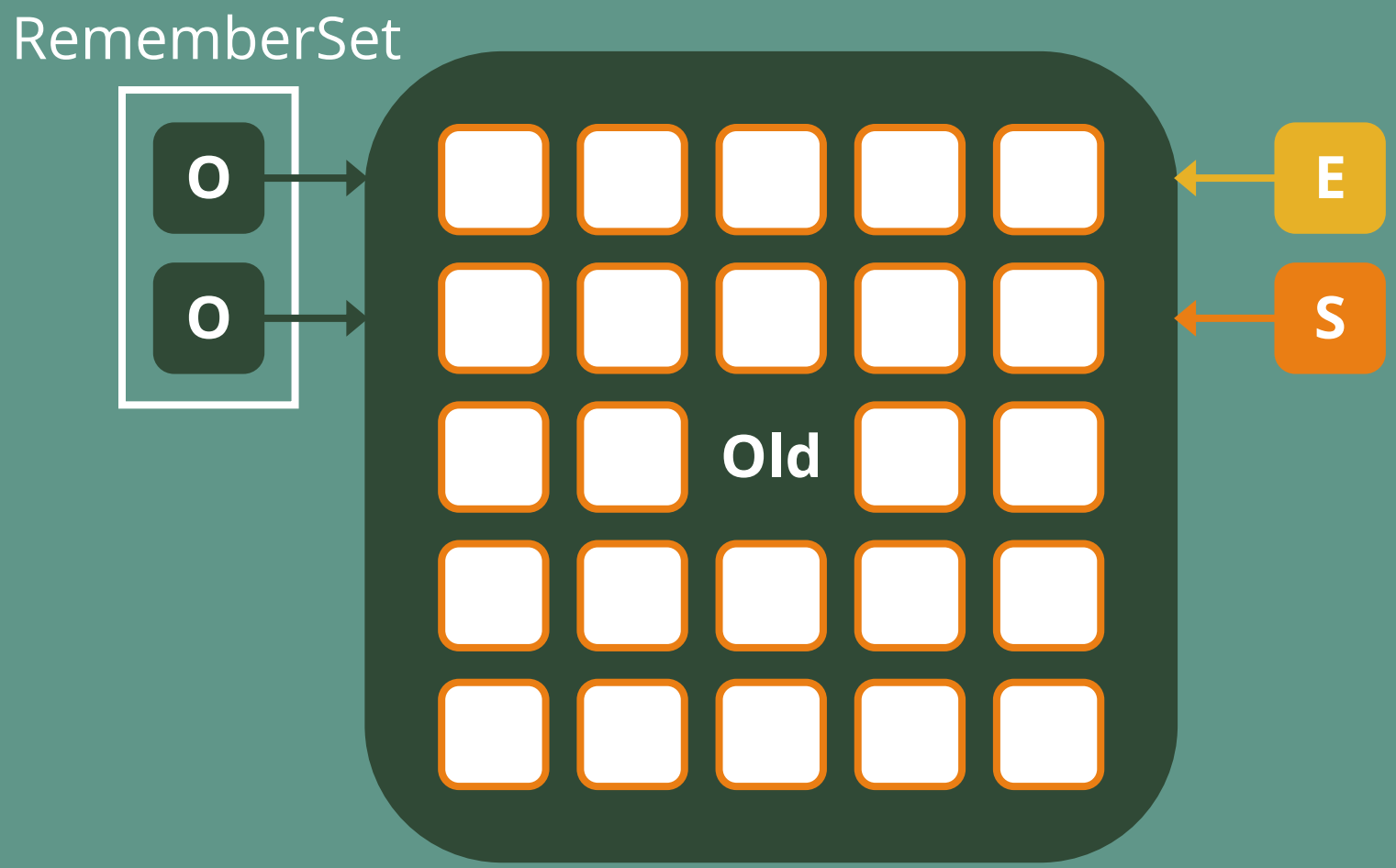


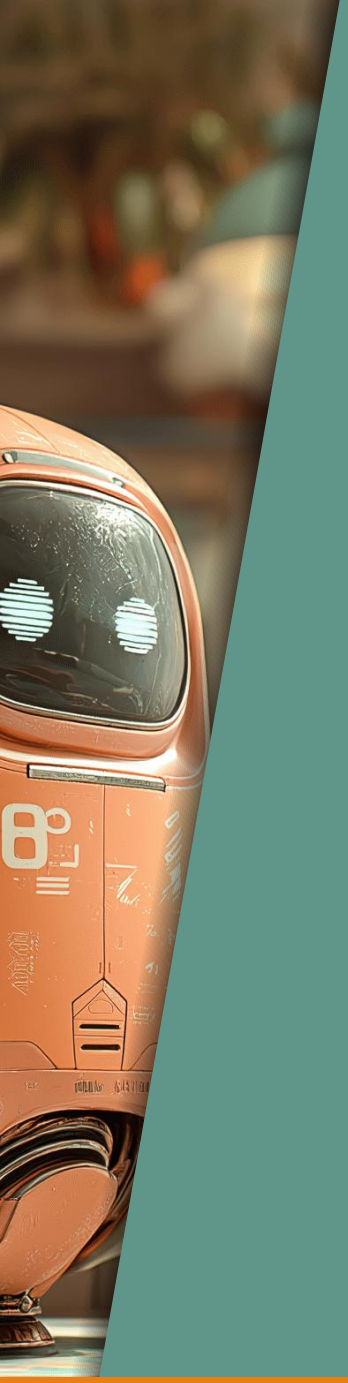


G1 GC – Région

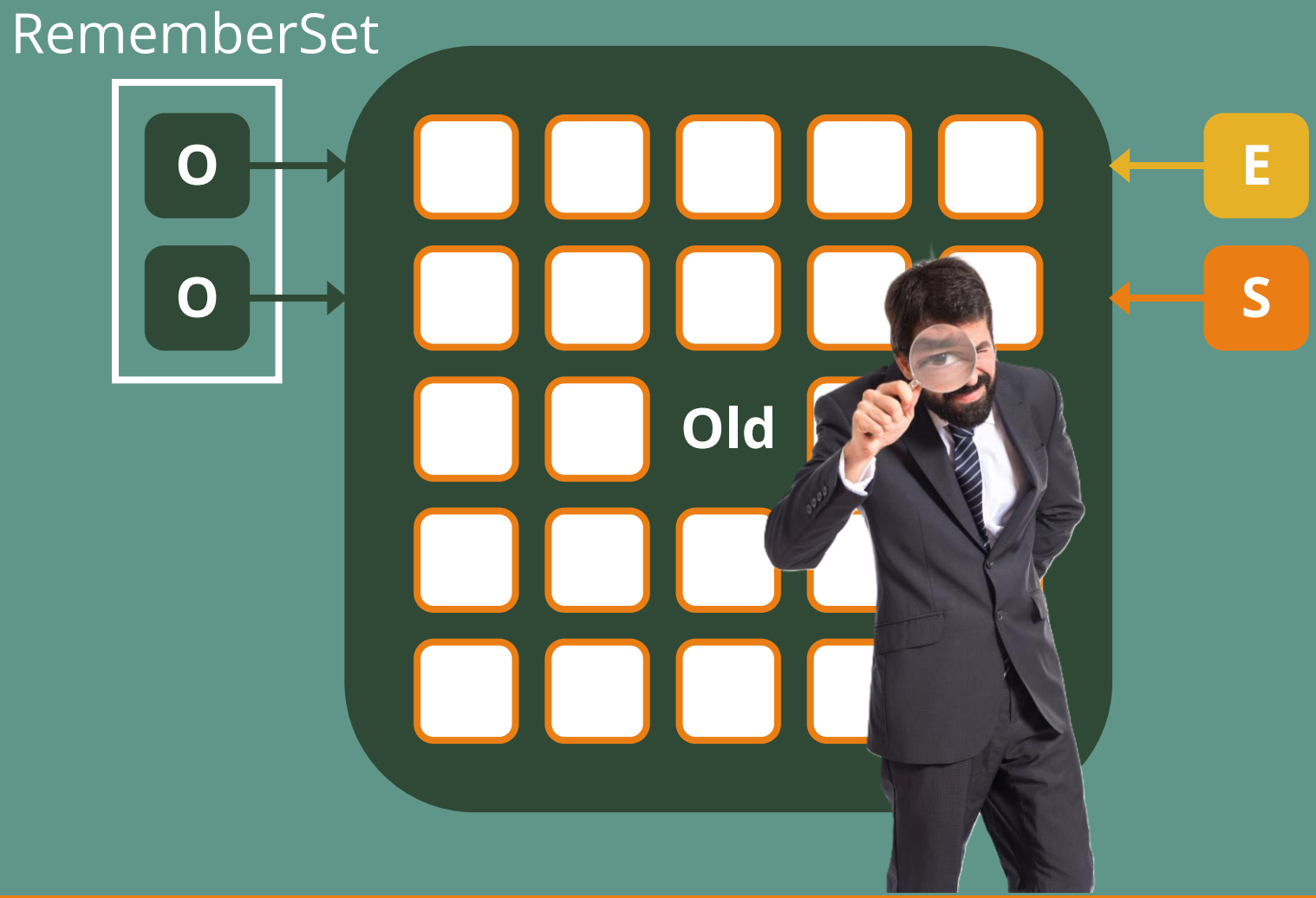


G1 GC – Région





G1 GC – Région



G1 GC

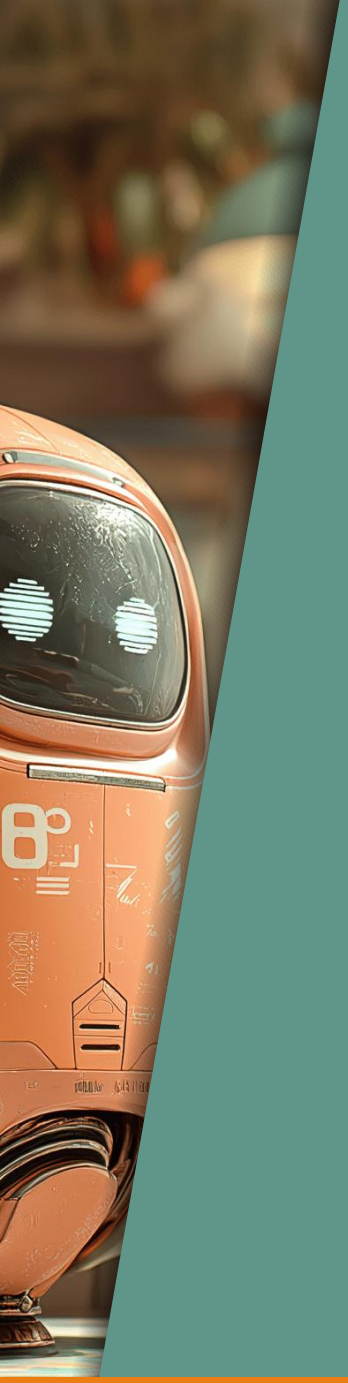
- Il commence par les régions avec le plus de déchets

G1 GC

- Il commence par les régions avec le plus de déchets
- 📋 Il copie les objets vivants dans d’autres régions

G1 GC

- Il commence par les régions avec le plus de déchets
- Il copie les objets vivants dans d’autres régions
- ⚠ Partiellement stop-the-world

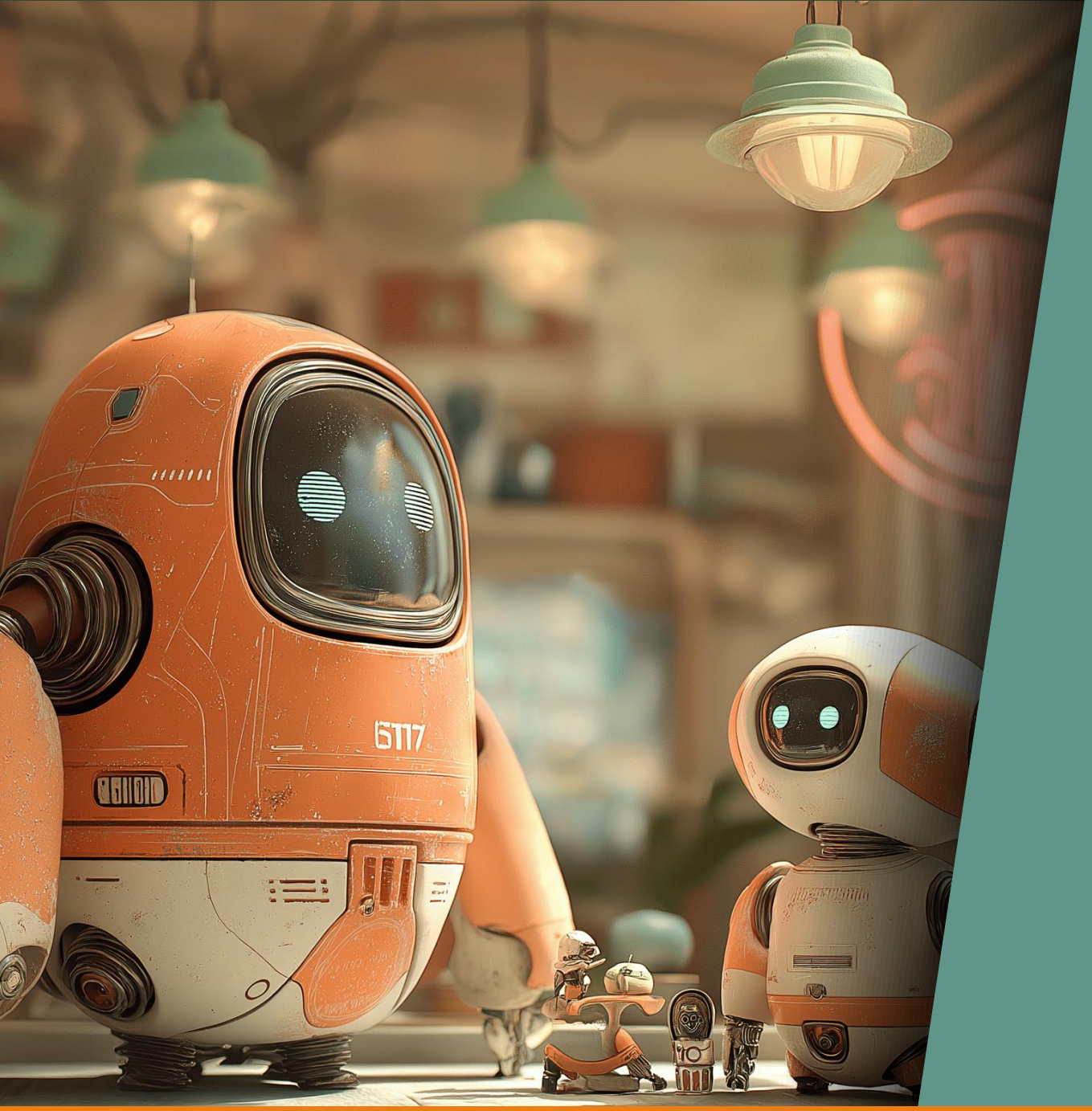


G1 GC

- Il commence par les régions avec le plus de déchets
- Il copie les objets vivants dans d’autres régions
- Partiellement stop-the-world
- ⏱️ Il s’arrête après 200 ms

G1 GC

- Il commence par les régions avec le plus de déchets
- Il copie les objets vivants dans d’autres régions
- Partiellement stop-the-world
- Il s’arrête après 200 ms
- 👌 C’est le GC généraliste



Latence ?

Latence

- ⏱️ Pause maximum : 200 ms

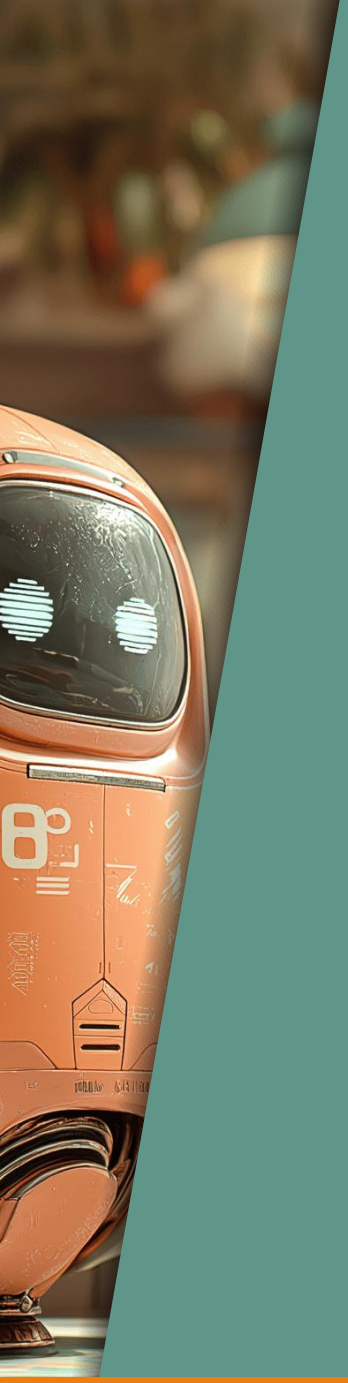
Latence

- ⏱️ Pause maximum : 200 ms *

Latence

Pause maximum : 200 ms *

⏳ * Il peut faire un full GC



Latence

Pause maximum : 200 ms **

* Il peut faire un full GC

Latence

Pause maximum : 200 ms **

* Il peut faire un full GC

🤯 ** C’est par JVM, on peut enchainer les GCs sur les micro-services

Ultra low latency

ZGC & Shenandoah GC

ZGC

- 💡 Idée: stocker des informations dans les références

ZGC

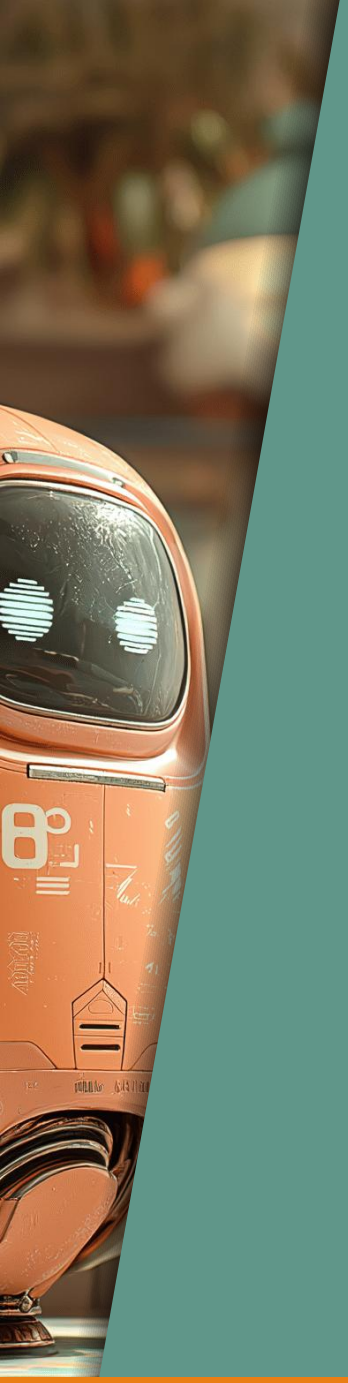
Idée: stocker des informations dans les références

```
var conf = new Conference("Devoxx", 2026);
```

ZGC

Idée: stocker des informations dans les références

```
var conf = new Conference("Devoxx", 2026);
```



ZGC – Référence

ZGC – Référence

	Adresse (44 bits)

Adresse : jusqu’à 16 To

ZGC – Référence

	F		Adresse (44 bits)

Adresse : jusqu’à 16 To

Flag Finalizer

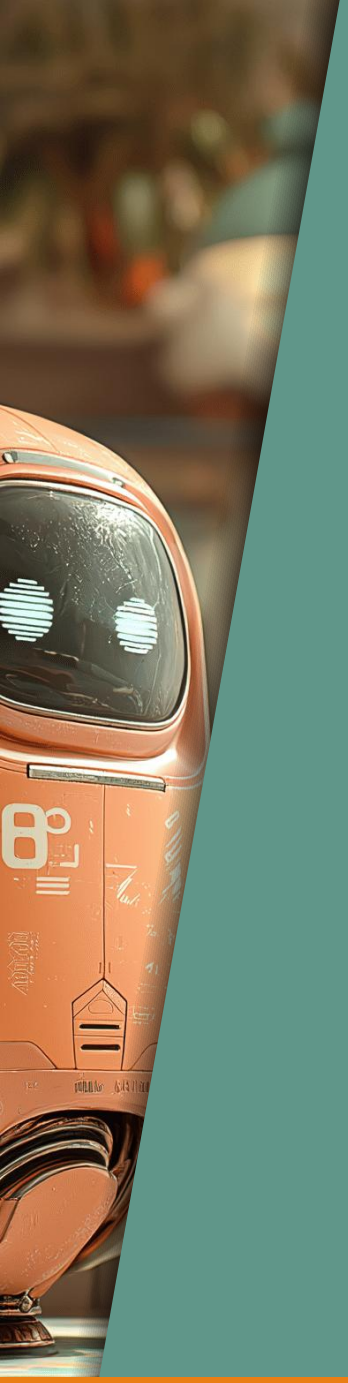
ZGC – Référence

	F	R		Adresse (44 bits)

Adresse : jusqu’à 16 To

Flag Finalizer

Flag Remapped : l’adresse est bonne



ZGC – Référence

	F	R	M1	M0	Adresse (44 bits)

Adresse : jusqu’à 16 To

Flag Finalizer

Flag Remapped : l’adresse est bonne

Flags Mark : faut récupérer la bonne adresse

ZGC

- Auto-tune

ZGC

- Auto-tune
- 🔄 Le thread applicatif peut faire la mise à jour

ZGC

- Auto-tune
- Le thread applicatif peut faire la mise à jour
- 🐘 Références de 64 octets

ZGC

- Auto-tune
- Le thread applicatif peut faire la mise à jour
- Références de 64 octets
- ⏸️ Allocation stalls

Shenandoah GC

- 💡 Idée: stocker des informations dans les entêtes d’objets

Shenandoah GC

Idée: stocker des informations dans les entêtes d'objets

```
var conf = new Conference("Devoxx", 2026);
```

Shenandoah GC

Idée: stocker des informations dans les entêtes d'objets

```
var conf = new Conference("Devoxx", 2026);
```

Shenandoah GC – Entête

Shenandoah GC – Entête

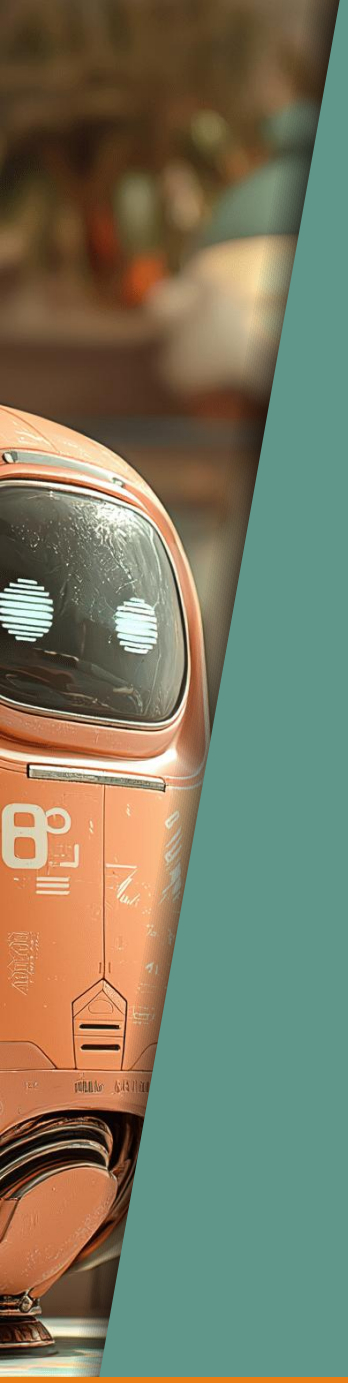
	Klass (32 ou 64 bits)

Shenandoah GC – Entête

	GC Age		Klass (32 ou 64 bits)

Shenandoah GC – Entête

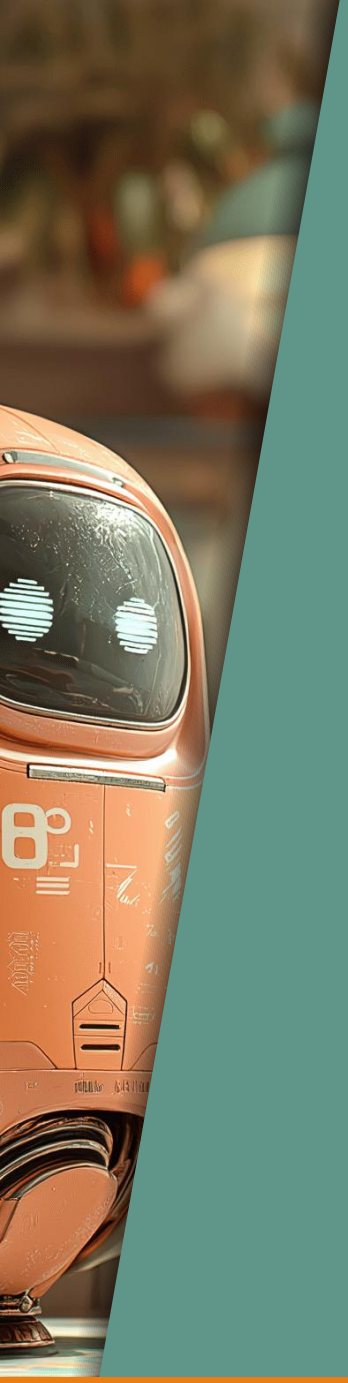
	Identity hash		GC Age		Klass (32 ou 64 bits)



Shenandoah GC – Entête

	Identity hash		GC Age		T	T	Klass (32 ou 64 bits)

Tags : 01=unlocked, 00/10=locked, 11=GC

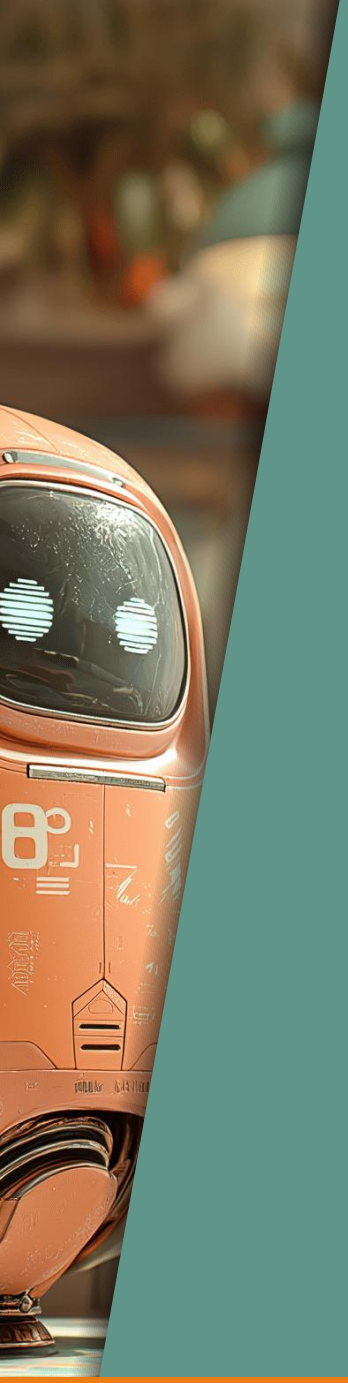


Shenandoah GC – Entête

	Identity hash		GC Age		T	T	Klass (32 ou 64 bits)

Tags : 01=unlocked, 00/10=locked, 11=GC

	1	1	Klass (32 ou 64 bits)



Shenandoah GC – Entête

	Identity hash		GC Age		T	T	Klass (32 ou 64 bits)

Tags : 01=unlocked, 00/10=locked, 11=GC

Nouvelle adresse	1	1	Klass (32 ou 64 bits)

Shenandoah GC

- Très configurable

Shenandoah GC

- Très configurable
- 📜 Pas de table de correspondance

Shenandoah GC

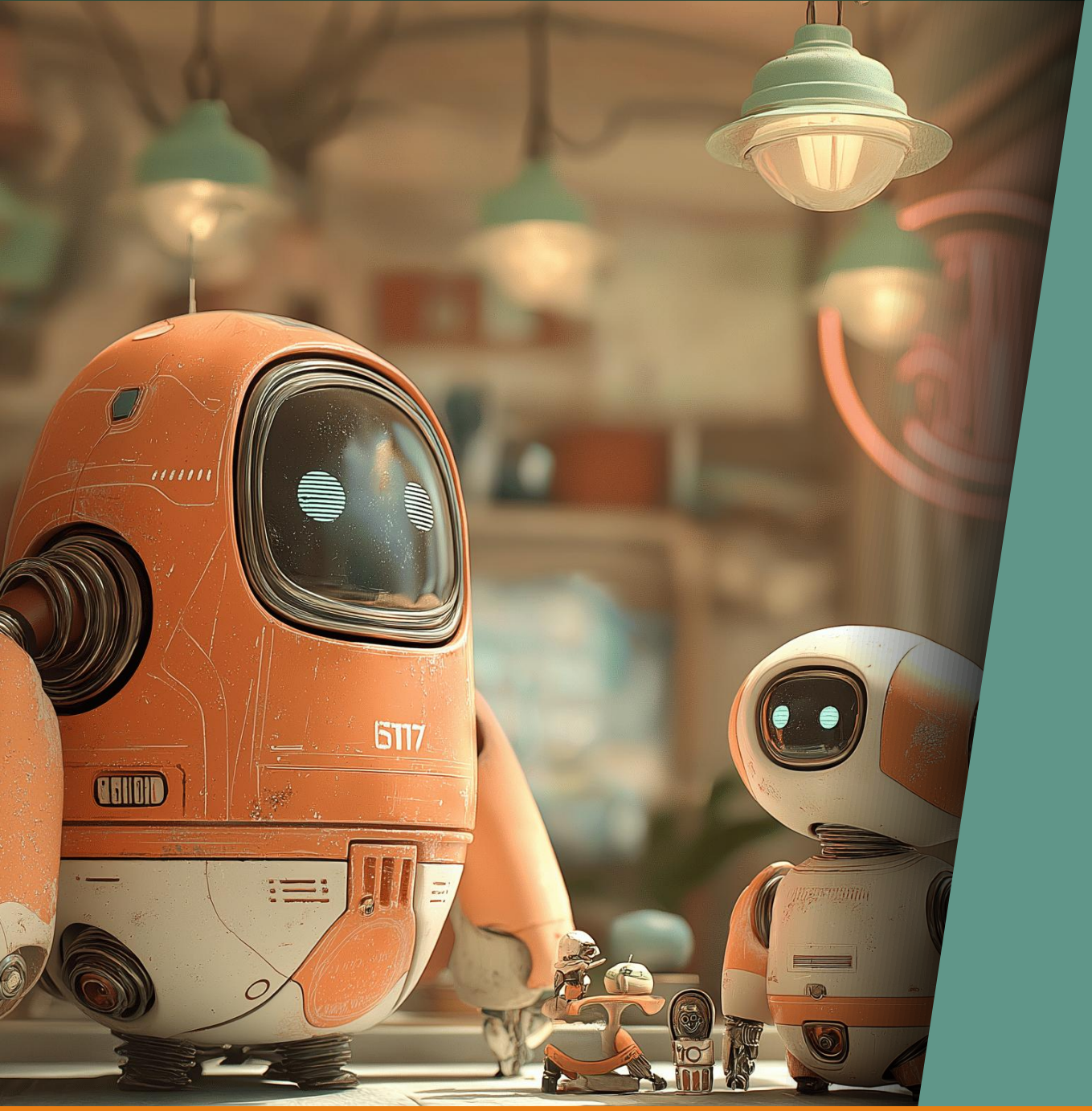
- Très configurable
- Pas de table de correspondance
- ♻ Toutes les références doivent être mise à jour avant réutilisation

Shenandoah GC

- Très configurable
- Pas de table de correspondance
- Toutes les références doivent être mise à jour avant réutilisation
- ⏸ Allocation stalls

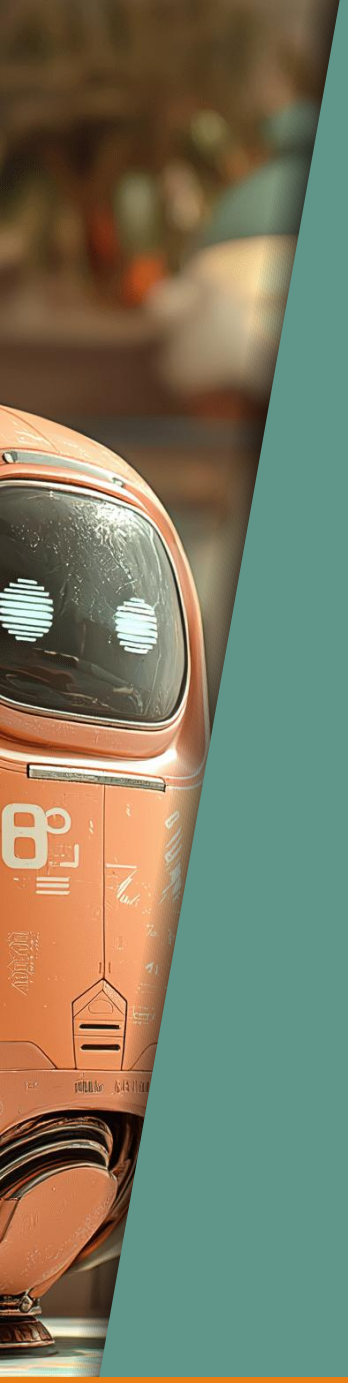
Shenandoah GC

- Très configurable
- Pas de table de correspondance
- Toutes les références doivent être mise à jour avant réutilisation
- Allocation stalls
- ⏸ Full GC



En résumé

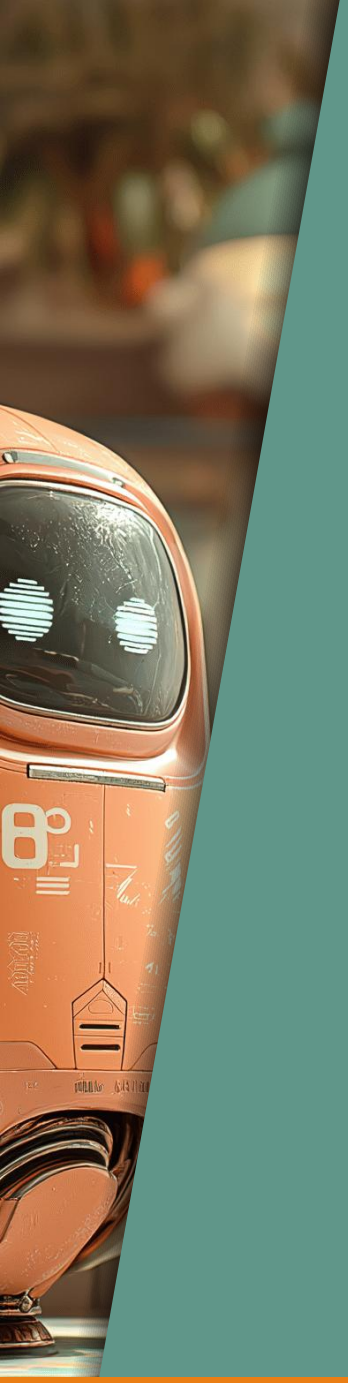
Quel GC choisir ?



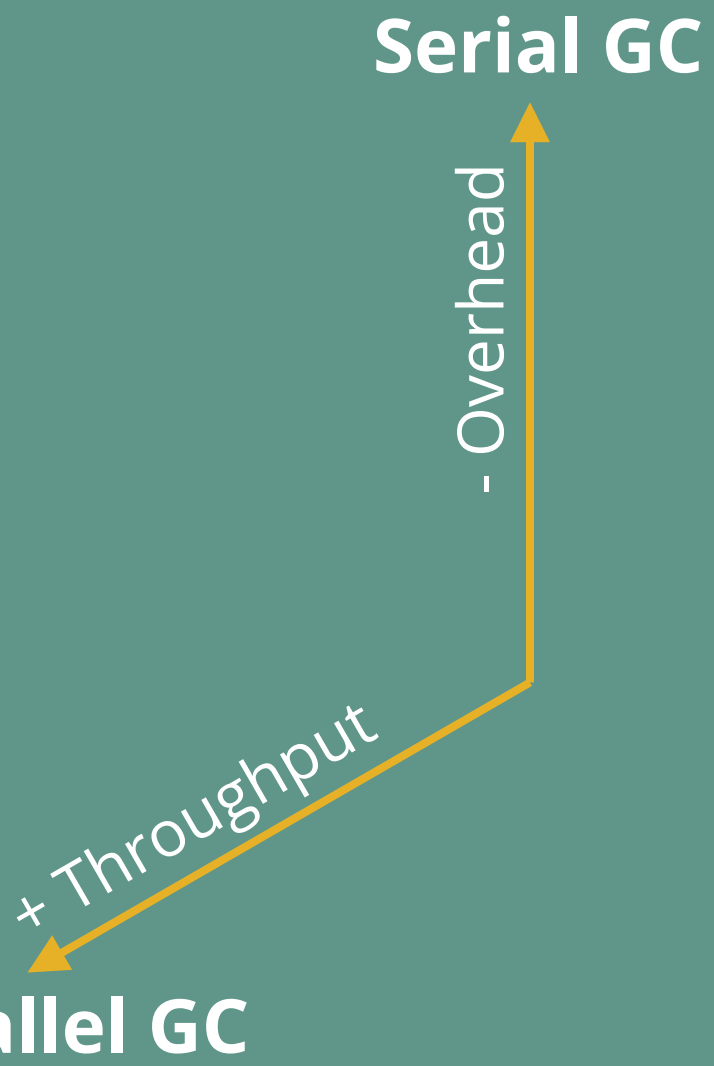
Quel GC choisir ?

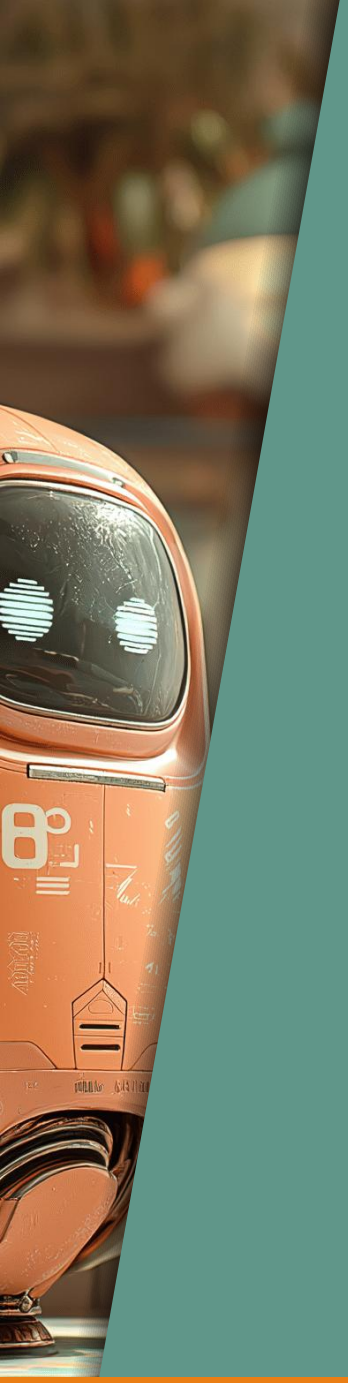
Serial GC

- Overhead

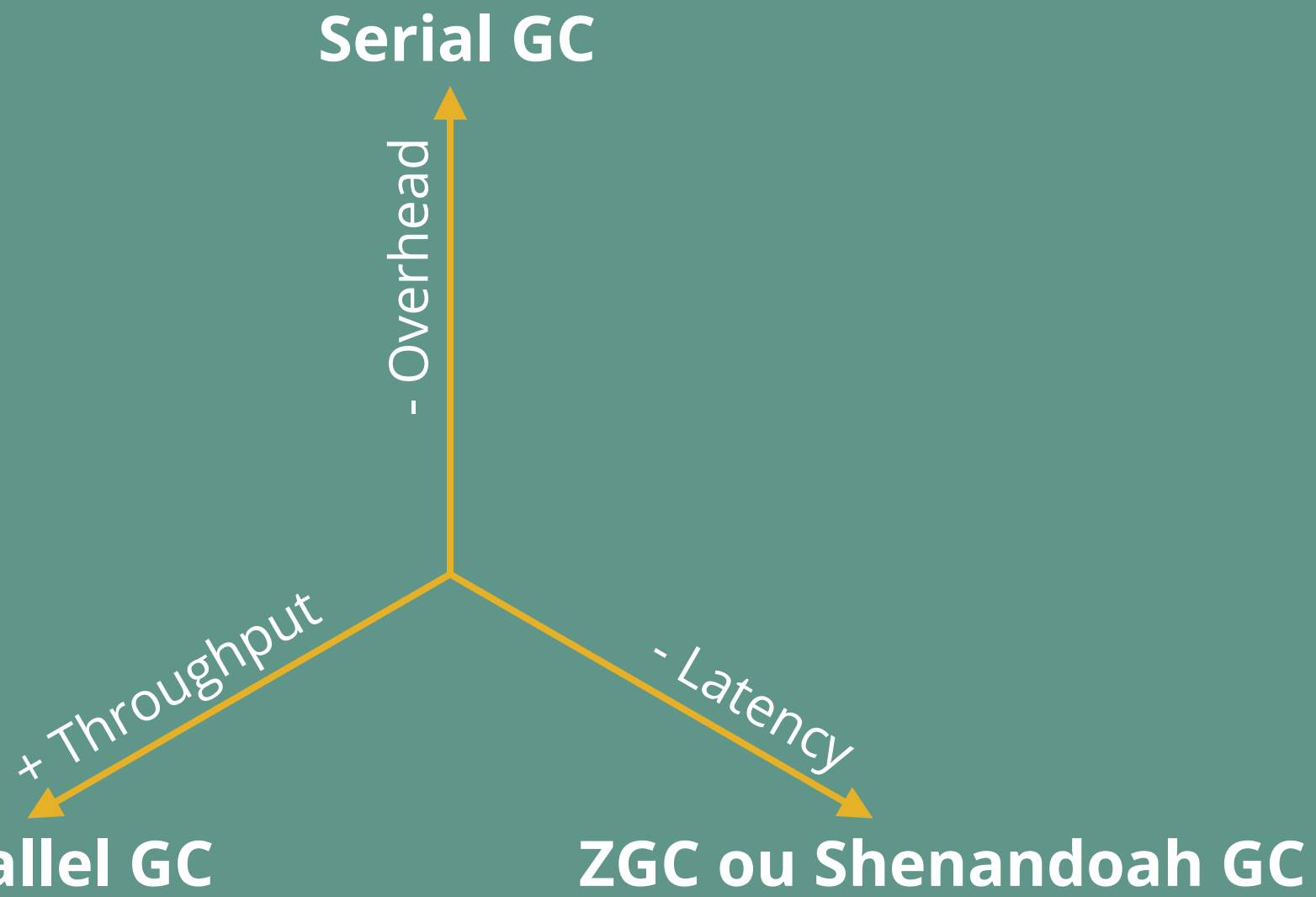


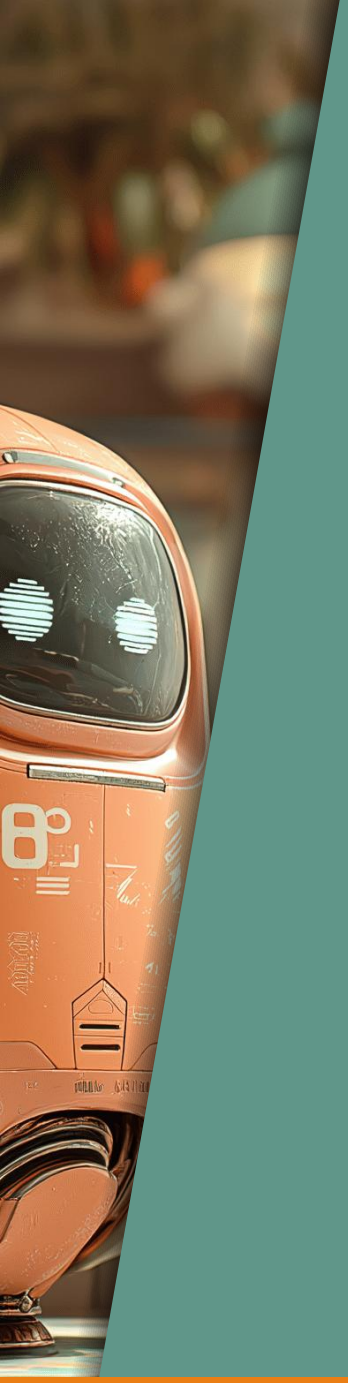
Quel GC choisir ?



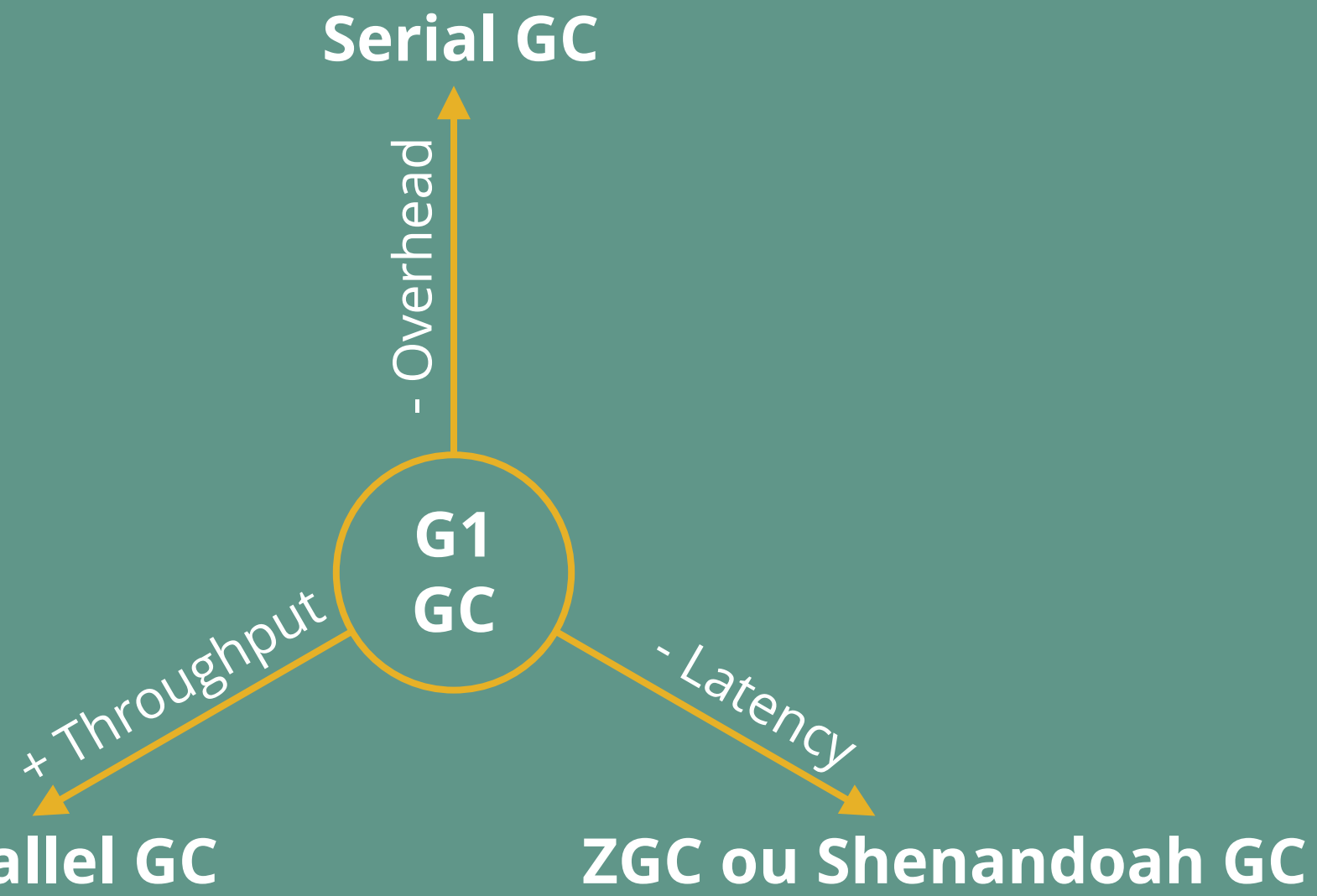


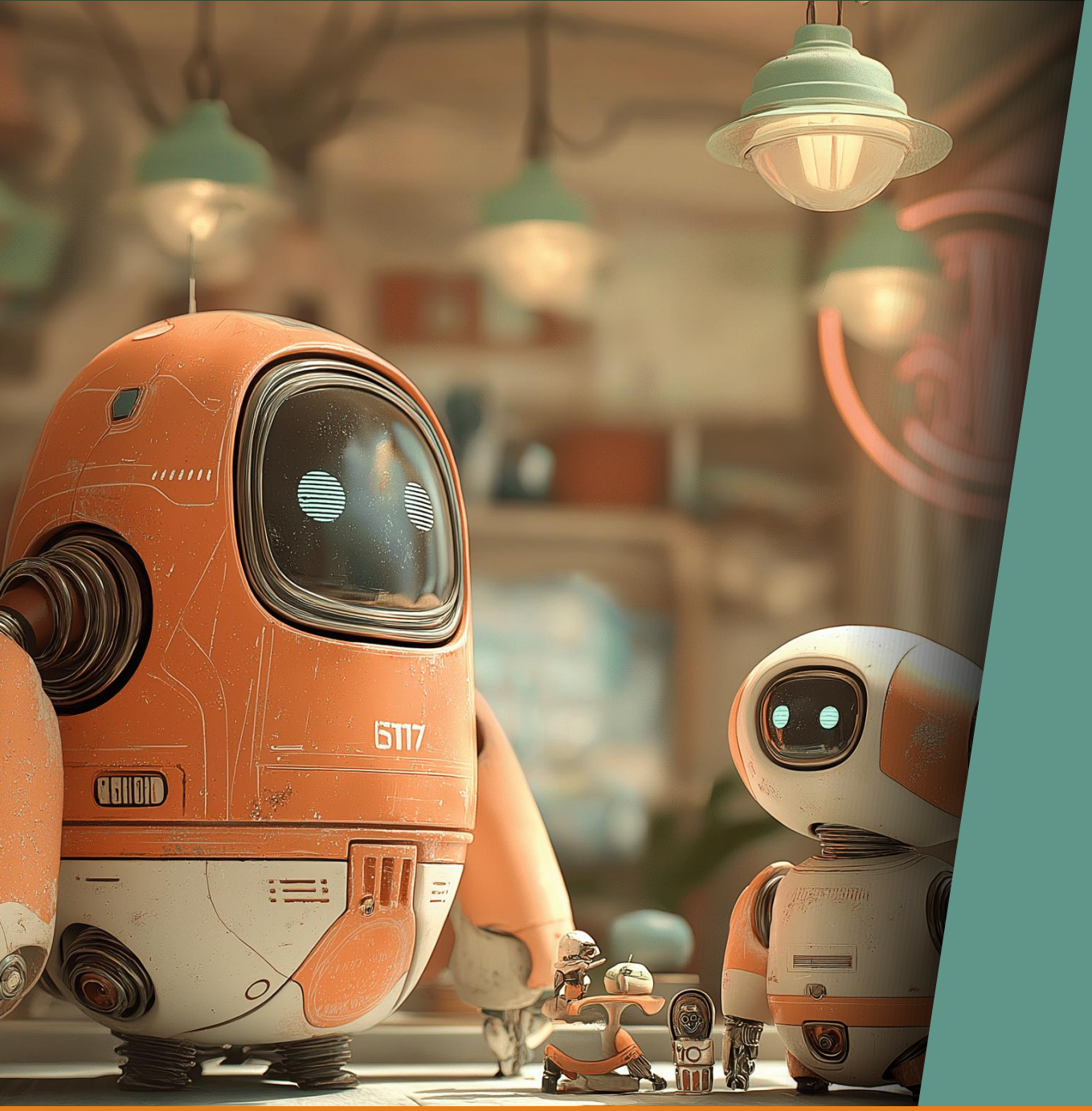
Quel GC choisir ?





Quel GC choisir ?



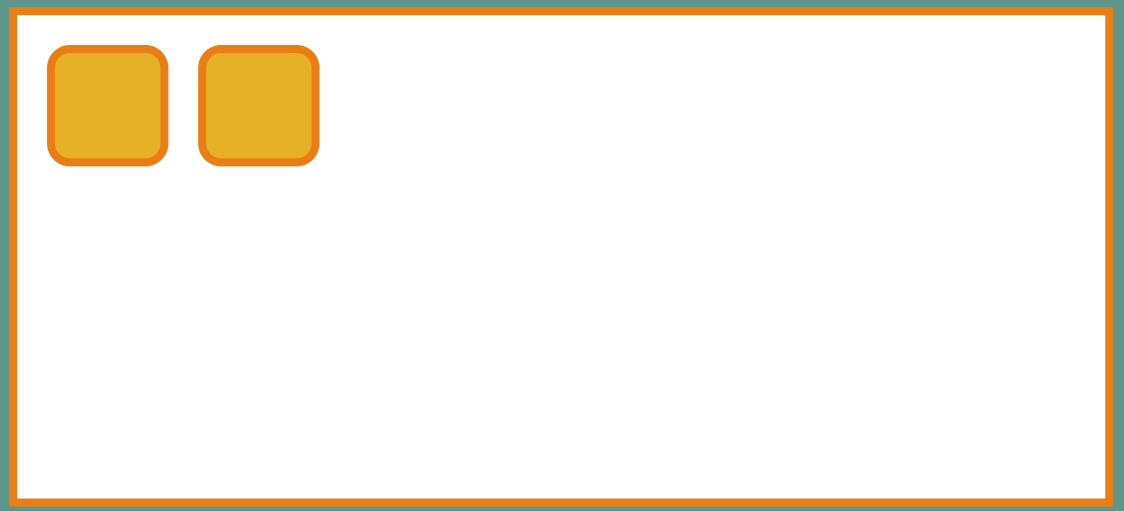


Epsilon GC ?

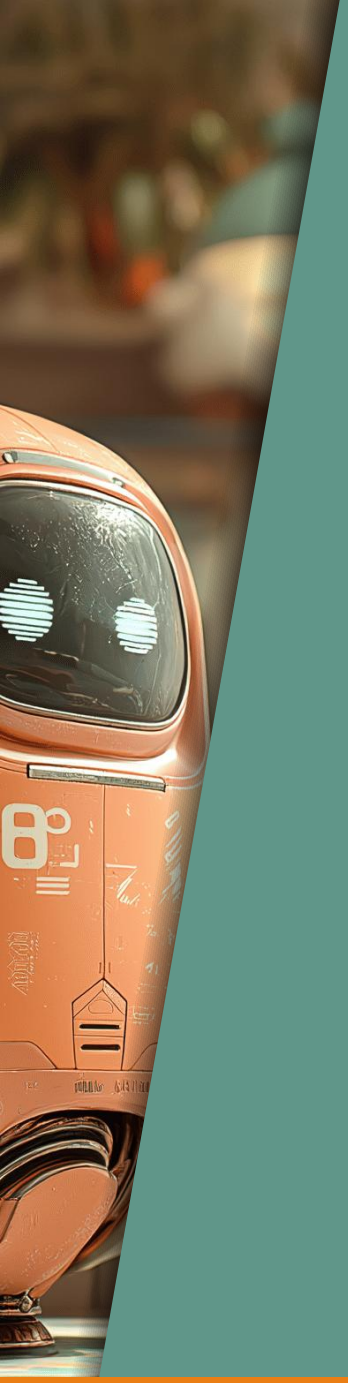
Epsilon GC

Epsilon GC

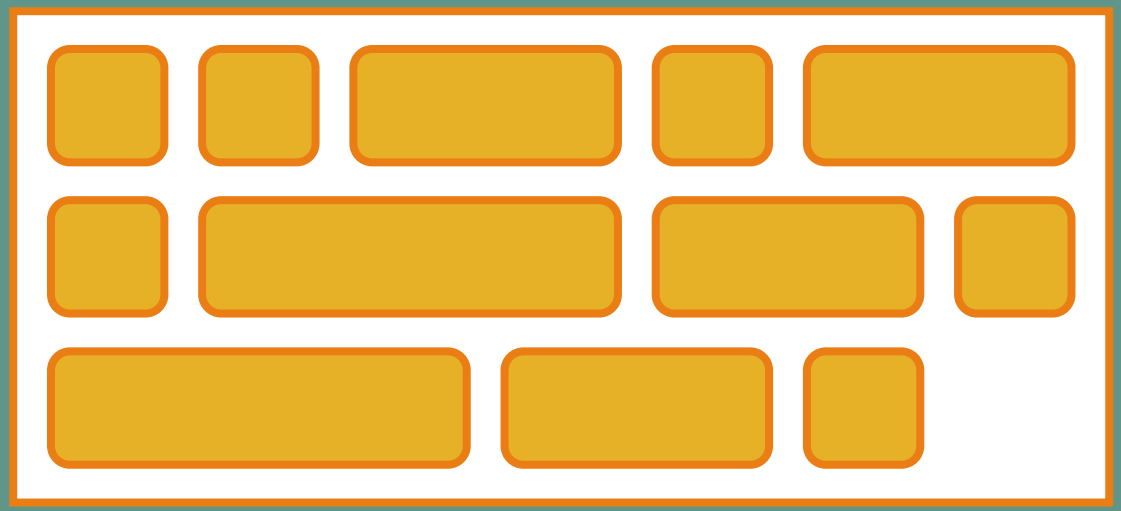
Epsilon GC



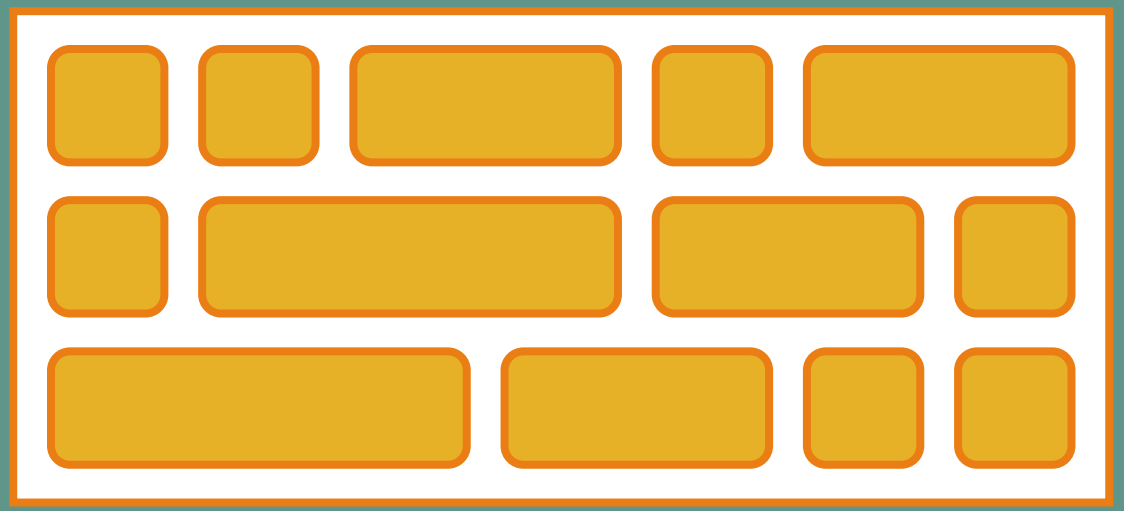
Epsilon GC

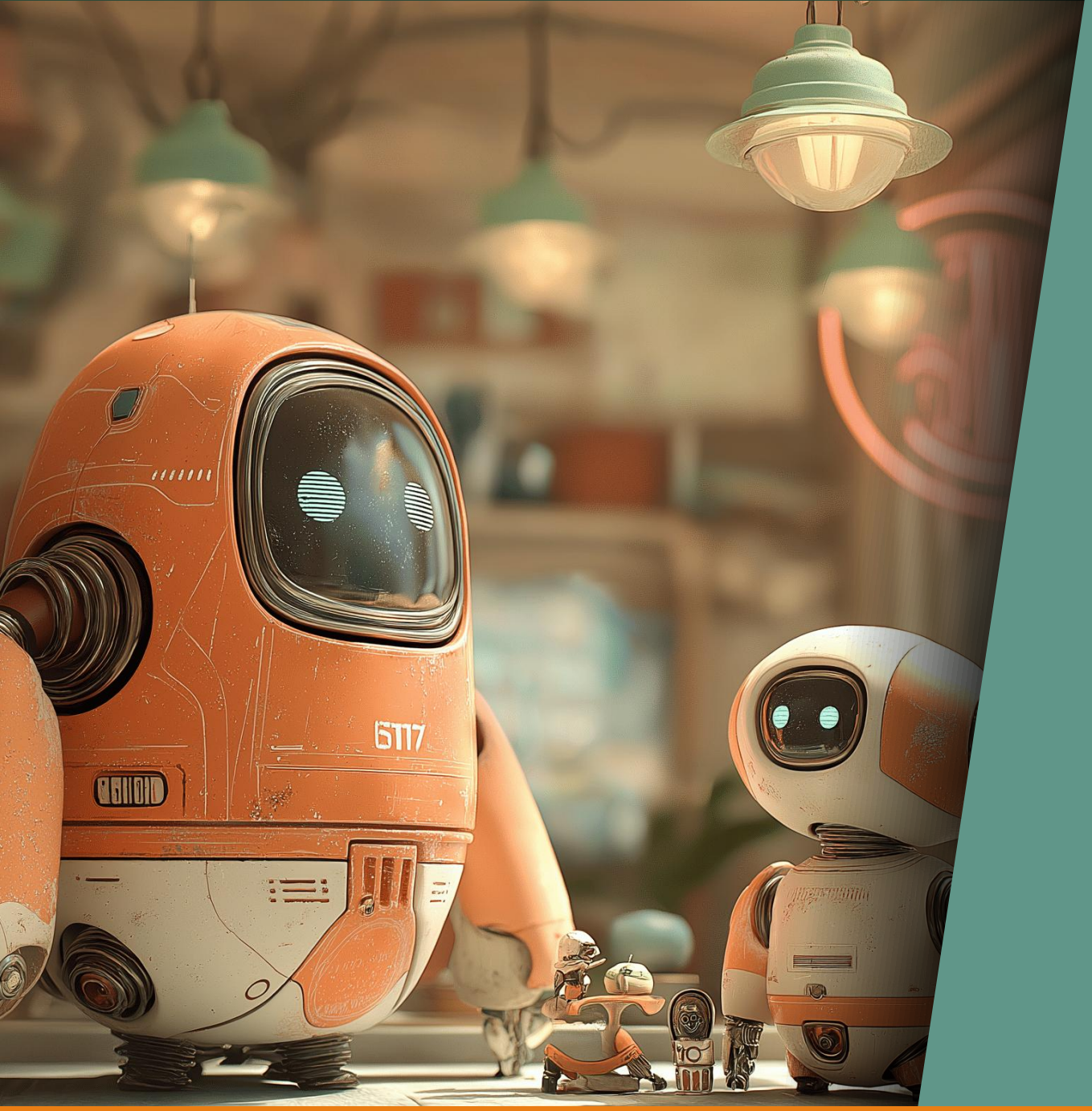


Epsilon GC



Epsilon GC





En résumé

Quel GC choisir ?

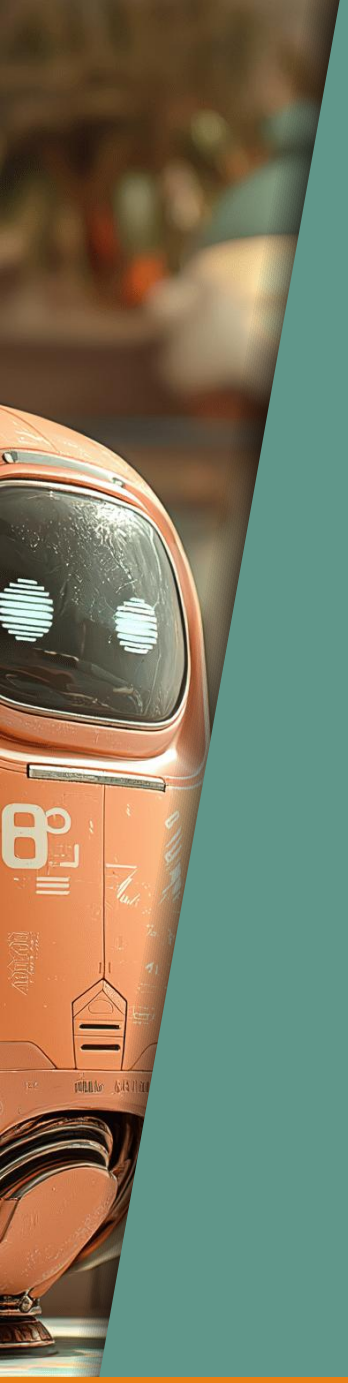
- Micro service (1 ou 2 Go) → Serial GC

Quel GC choisir ?

- Micro service (1 ou 2 Go) → Serial GC
- 💻 Serverless, CLI → Epsilon GC

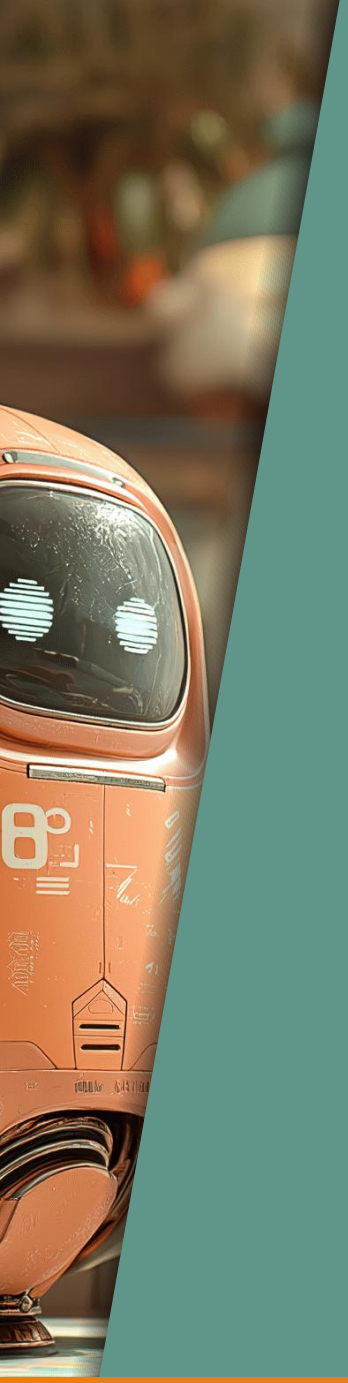
Quel GC choisir ?

- Micro service (1 ou 2 Go) → Serial GC
- Serverless, CLI → Epsilon GC
- Batch processing, file processing, Kafka streams → Parallel GC



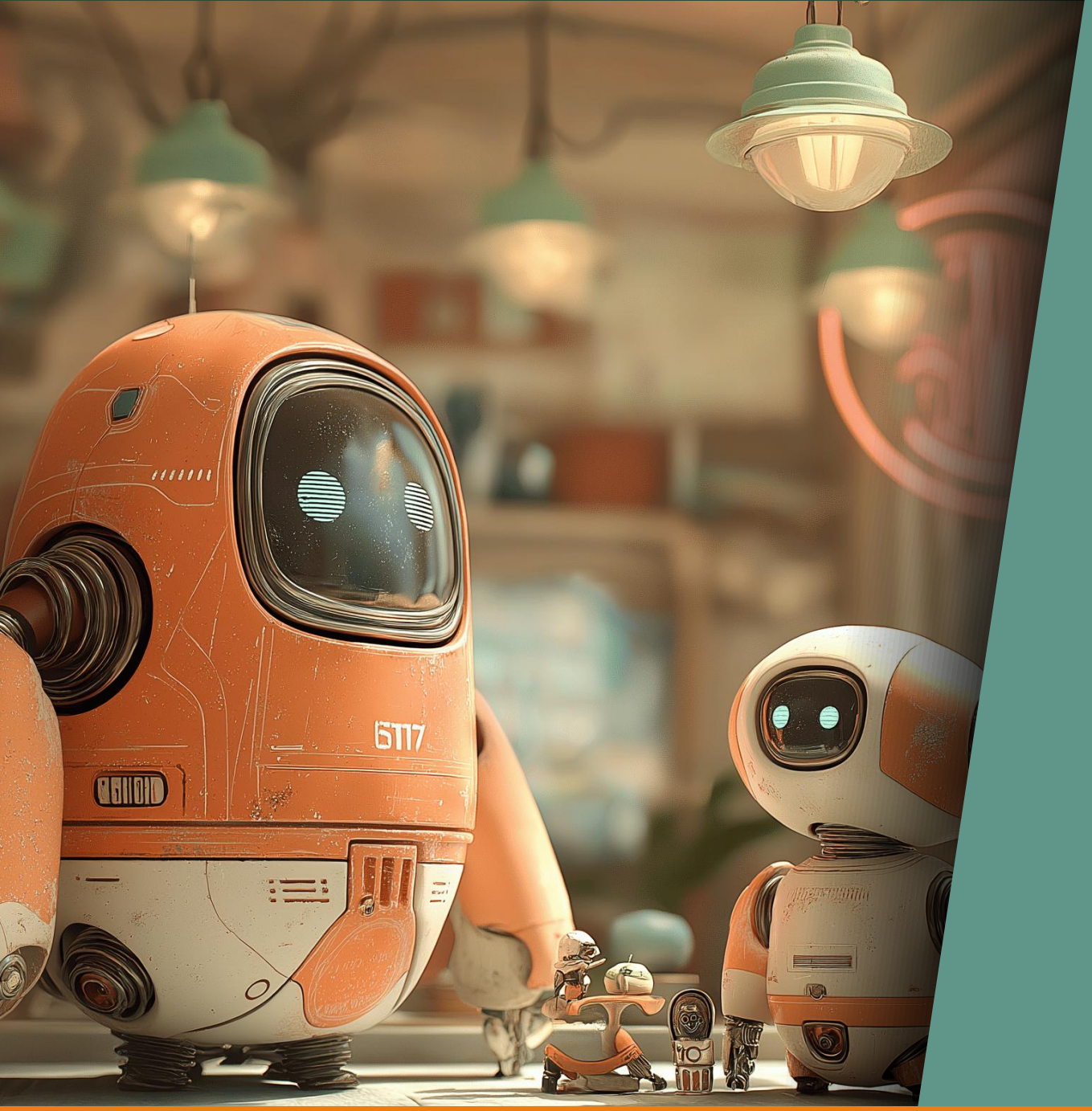
Quel GC choisir ?

- Micro service (1 ou 2 Go) → Serial GC
- Serverless, CLI → Epsilon GC
- Batch processing, file processing, Kafka streams → Parallel GC
- ⏱️ Latence est critique → ZGC ou Shenandoah GC



Quel GC choisir ?

- Micro service (1 ou 2 Go) → Serial GC
- Serverless, CLI → Epsilon GC
- Batch processing, file processing, Kafka streams → Parallel GC
- Latence est critique → ZGC ou Shenandoah GC
- 👌 Tous les autres cas → G1 GC



En natif

Java natif

Quarkus	Graal VM

Java natif

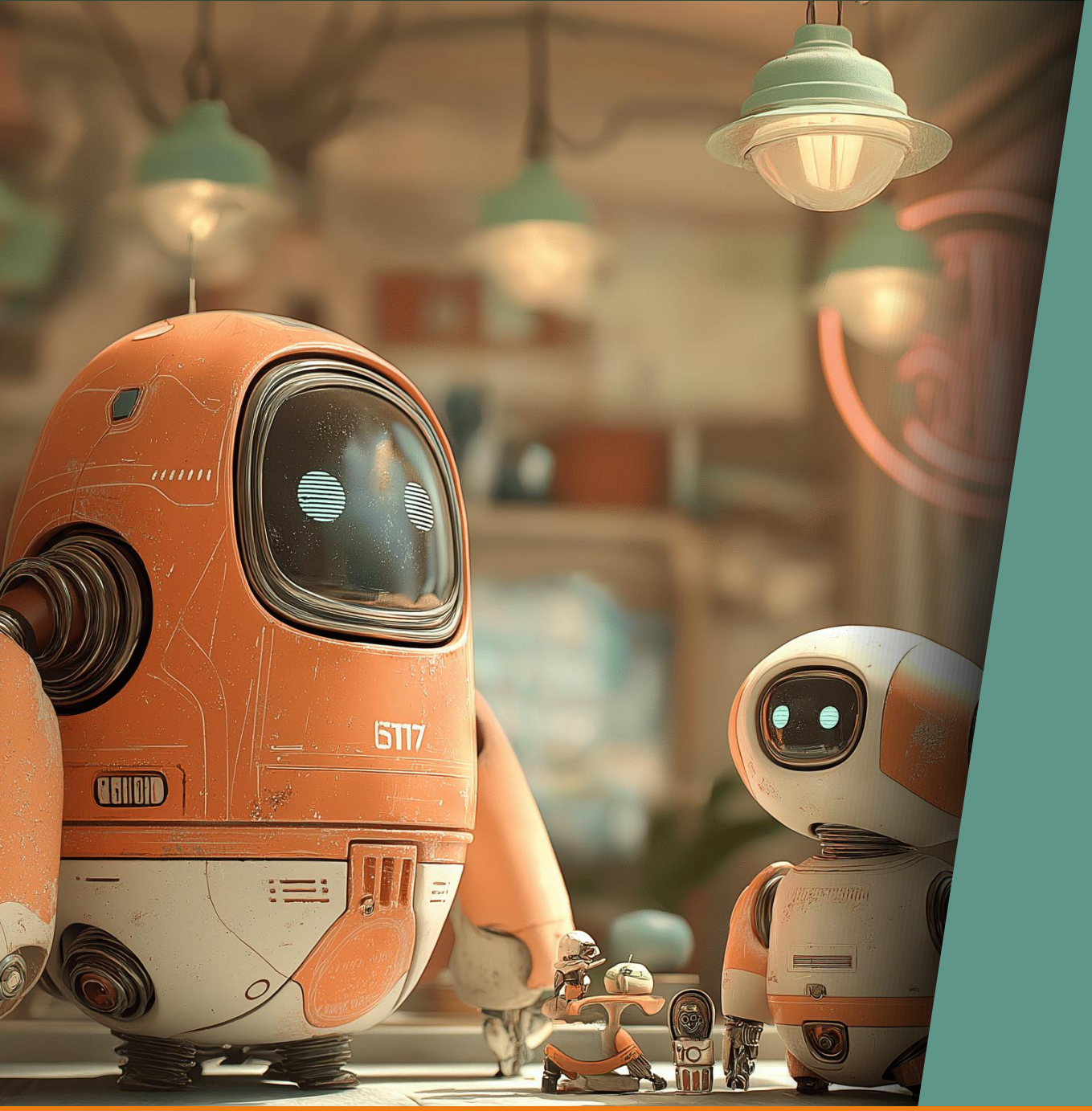
Quarkus	Graal VM
Serial GC	Serial GC

Java natif

Quarkus	Graal VM
Serial GC	Serial GC
Epsilon GC	Epsilon GC

Java natif

Quarkus	Graal VM
Serial GC	Serial GC
Epsilon GC	Epsilon GC
	G1 GC (version enterprise)



Le futur

Le futur

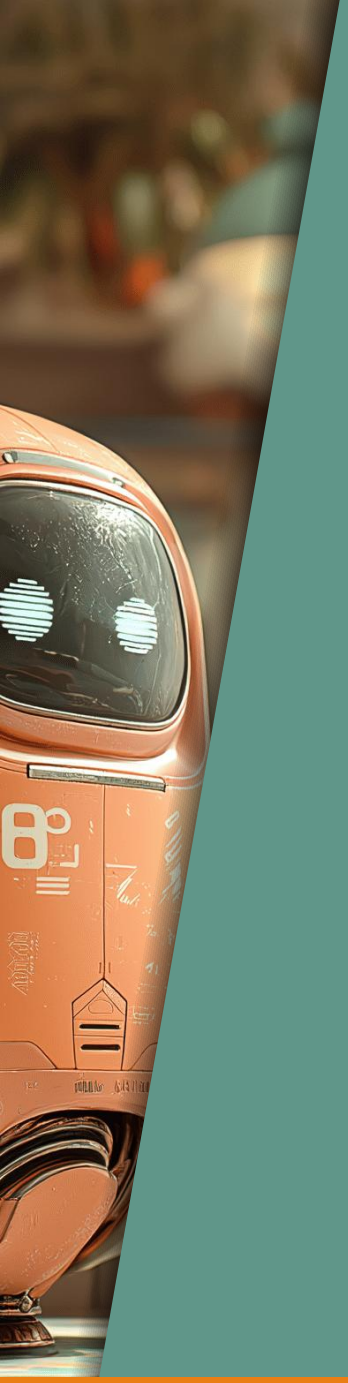
- 💪 Amélioration du throughput de G1 (JEP-522, JDK 26)

Le futur

- Amélioration du throughput de G1 (JEP-522, JDK 26)
- 👌 G1 GC remplace Serial GC par défaut (JEP-523)

Le futur

- Amélioration du throughput de G1 (JEP-522, JDK 26)
- G1 GC remplace Serial GC par défaut (JEP-523)
- 🔮 Bootstrap plus rapide de ZGC



Le futur

- Amélioration du throughput de G1 (JEP-522, JDK 26)
- G1 GC remplace Serial GC par défaut (JEP-523)
- Bootstrap plus rapide de ZGC
- 🔮 Dimensionnement automatique de la Heap (G1 + Serial GC)

Performances

Pour améliorer les perfs

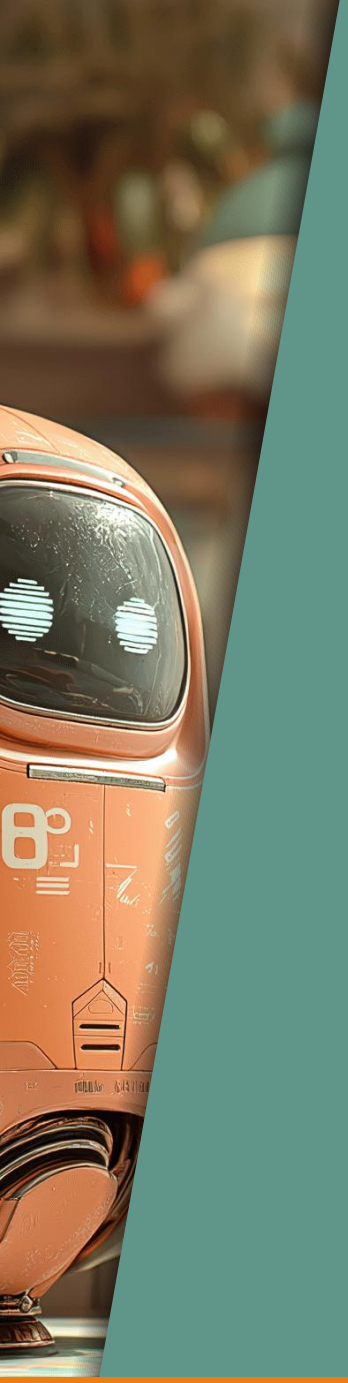
- 👌 Choisir le GC adapté à son besoin

Pour améliorer les perfs

- Choisir le GC adapté à son besoin
- 👌 Mesurer

Pour améliorer les perfs

- Choisir le GC adapté à son besoin
- Mesurer
- 👌 Avoir une JVM récente



Flags magiques

Flags magiques

- 👌 -Xms = -Xmx

Flags magiques

- -Xms = -Xmx
- 👌 -XX:+UseLargePages

Flags magiques

-Xms = -Xmx

-XX:+UseLargePages

👌 -XX:+AlwaysPreTouch

MERCI POUR VOTRE ÉCOUTE

Antoine DESSAIGNE

Slides

Feedback